

project0: 安装 e² studio

e2studio 软件

瑞萨电子灵活配置软件包 (FSP) 是用于嵌入式系统设计的高质量增强型软件包, 支持瑞萨电子 RA 产品家族 Arm 微控制器, 提供用户友好的界面且可灵活扩展, 确保从入门级到高性能的整个 RA 微控制器的软件兼容性。 在全新 Arm® TrustZone® 和其他高级安全功能的支持下, FSP 提供一种快速、通用的方式, 使用量产级驱动程序、Azure RTOS、FreeRTOS™ 和其他中间件协议栈来构建安全、互联的物联网设备。

下载地址: <https://www.renesas.cn/zh/software-tool/flexible-software-package-fsp>



概览

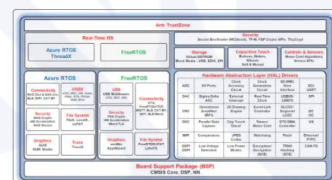
描述 特性 发布信息 目标设备

瑞萨电子灵活配置软件包 (FSP) 是用于嵌入式系统设计的高质量增强型软件包, 支持瑞萨电子 RA 产品家族 Arm 微控制器, 提供用户友好的界面且可灵活扩展, 确保从入门级到高性能的整个 RA 微控制器的软件兼容性。 在全新 Arm® TrustZone® 和其他高级安全功能的支持下, FSP 提供一种快速、通用的方式, 使用量产级驱动程序、Azure RTOS、FreeRTOS™ 和其他中间件协议栈来构建安全、互联的物联网设备。

📥 下载最新FSP (v5.9.0):

FSP 平台安装程序 (包含 e² studio 集成开发环境、工具链和 FSP 软件包):

- FSP Windows 平台安装程序
- FSP Linux 平台安装程序
- FSP macOS 平台安装程序



输入自己的信息即可下载。



填写表格以完成您的灵活配置软件包 (FSP) 下载

瑞萨电子RA灵活配置软件包 (FSP) 提供了一种使用瑞萨RA系列Arm微控制器 (MCU) 构建安全连接的物联网 (IoT) 设备的快速通用方法。FSP提供生产就绪的外围设备驱动程序、Azure RTOS、FreeRTOS和中间件协议栈，以利用FSP生态系统。



电子邮件
a m 这不是我

国家/地区
中国

姓
姓

名
名

公司
公司

地址
龙

城市
Shenzhen Shi

省/自治区
Guangdong 广东省

邮编
518000

联系电话
1

Operating System
Windows

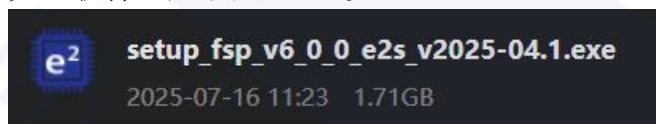
Installer
e² studio

实时操作系统 (RTOS) 选择
☐ Azure RTOS ☐ FreeRTOS ☒ 不使用实时操作系统

☒ 我同意接受瑞萨电子网站政策条款和隐私条款。

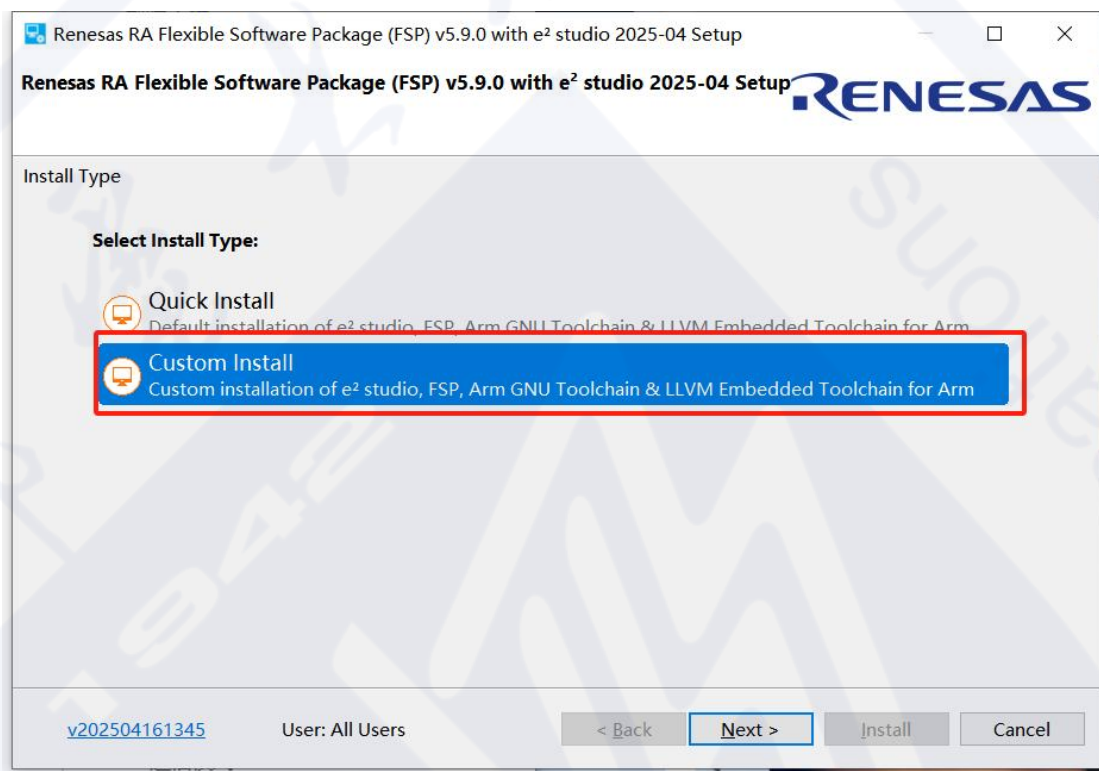
提交

完整软件包应该是 1.6G 以上。

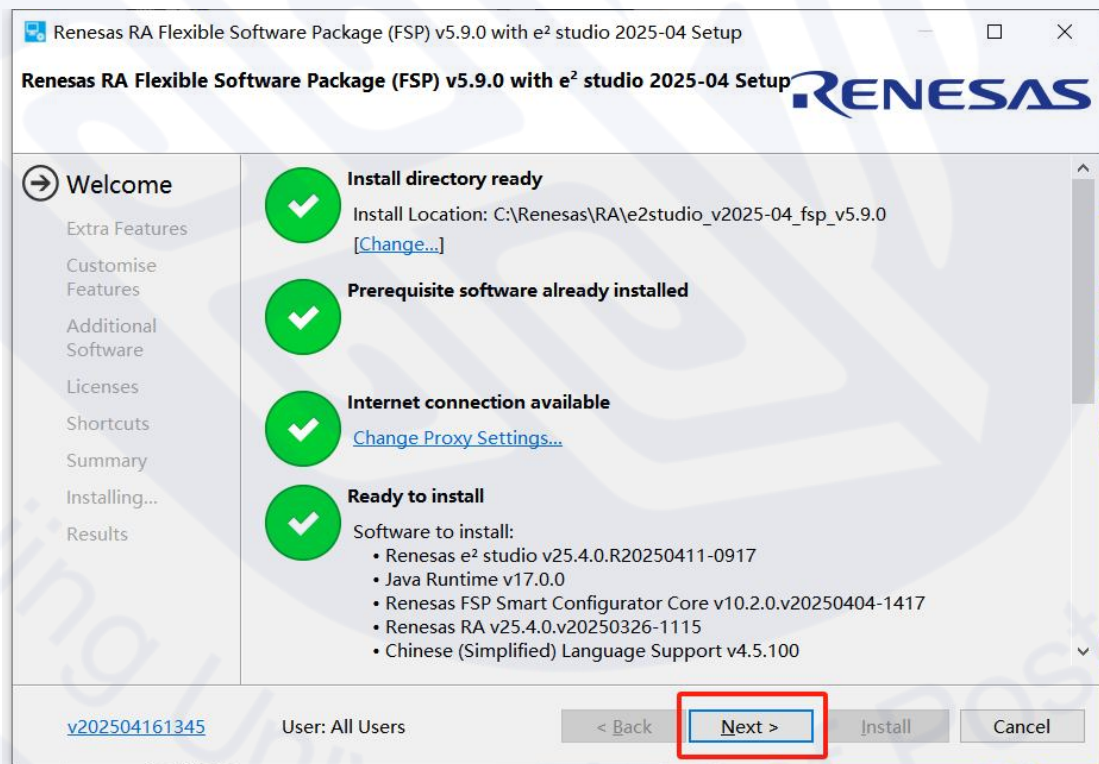


安装

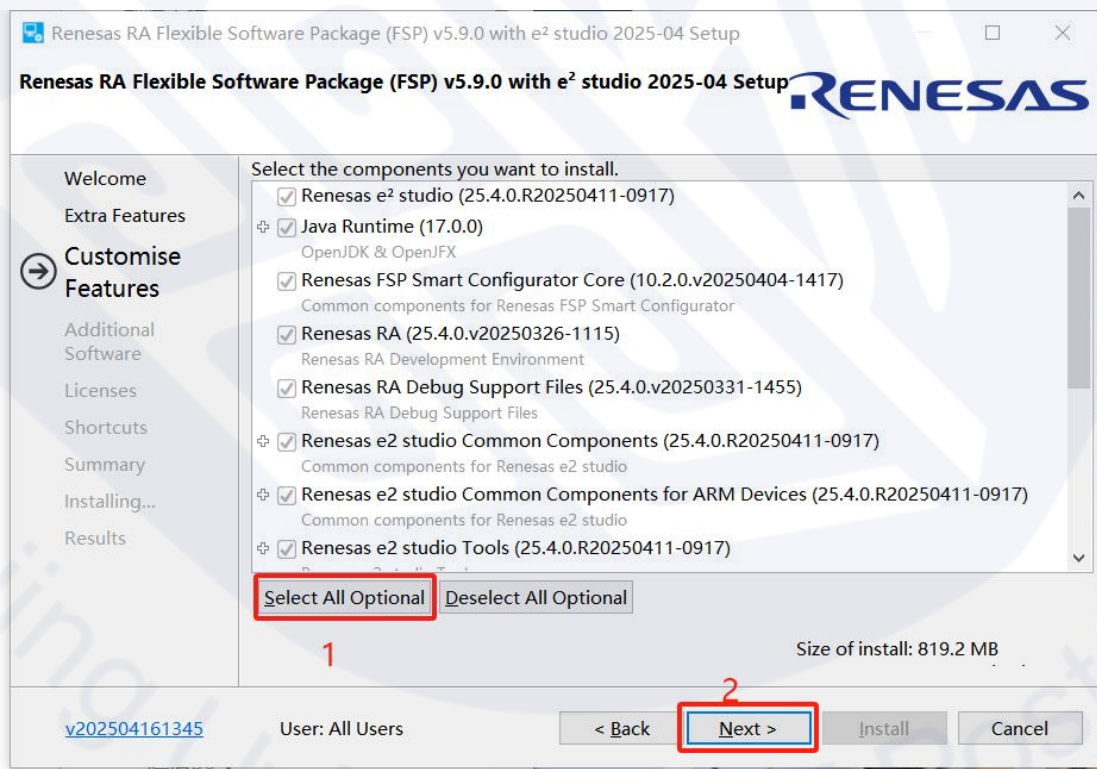
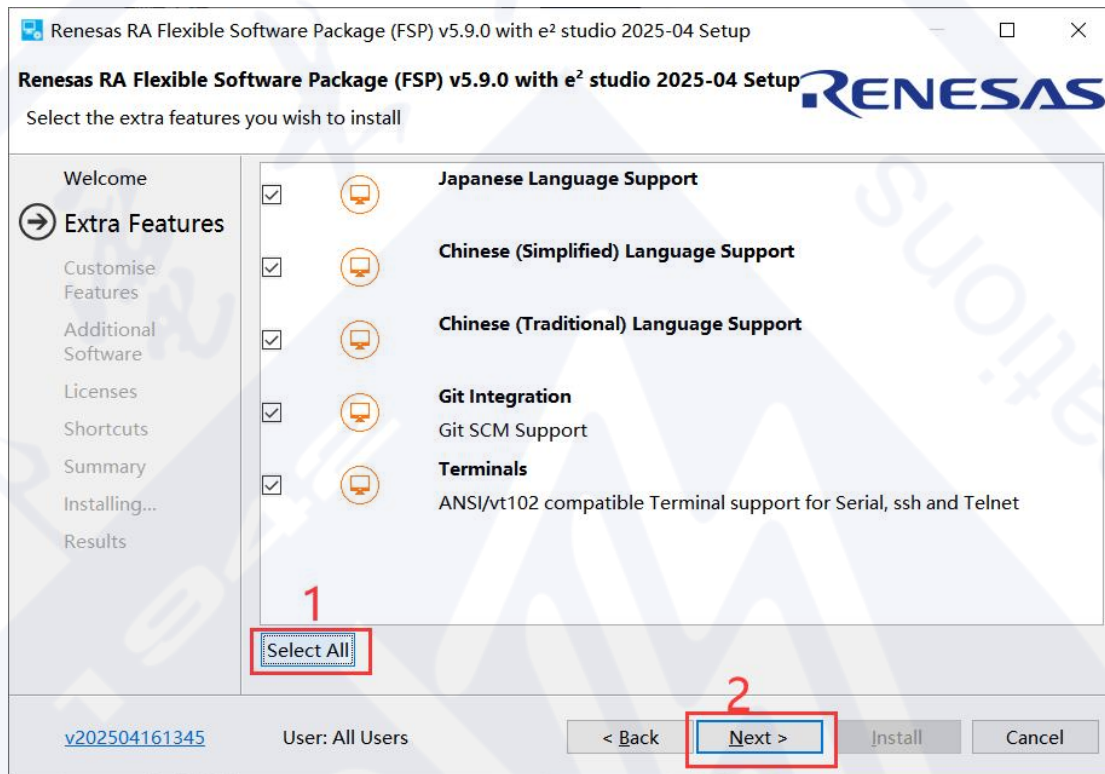
这里选择自定义安装。

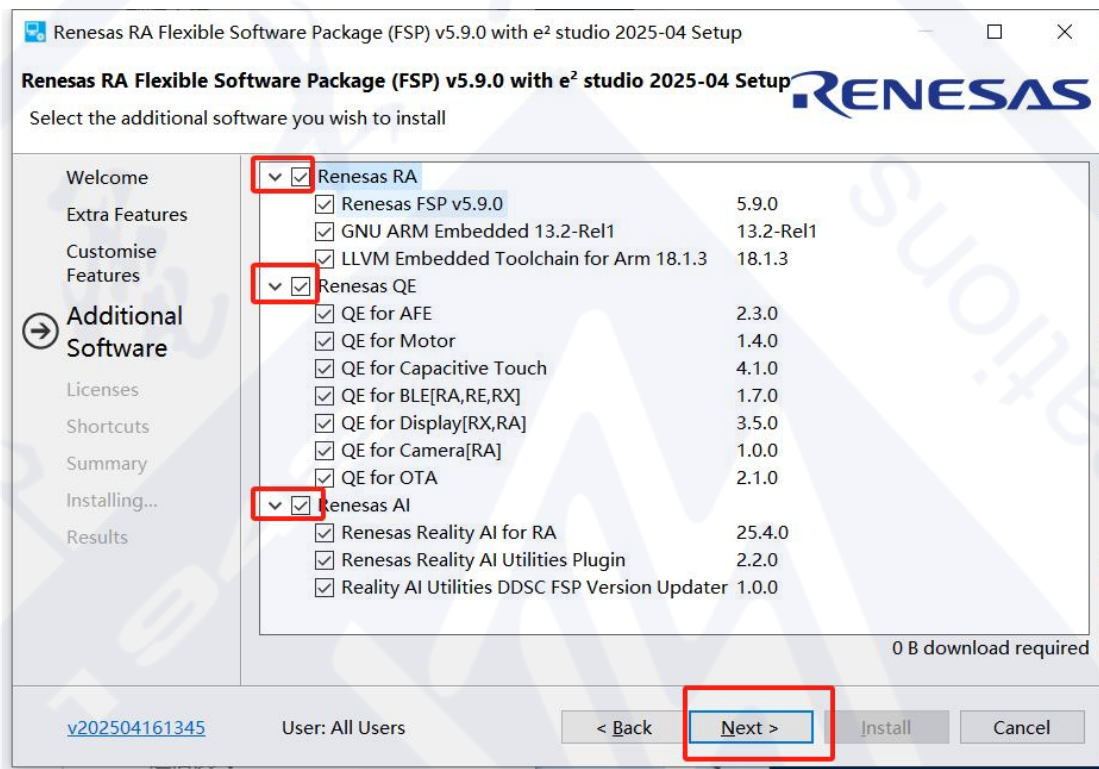


检测安装和配置开发环境，没问题点击下一步。

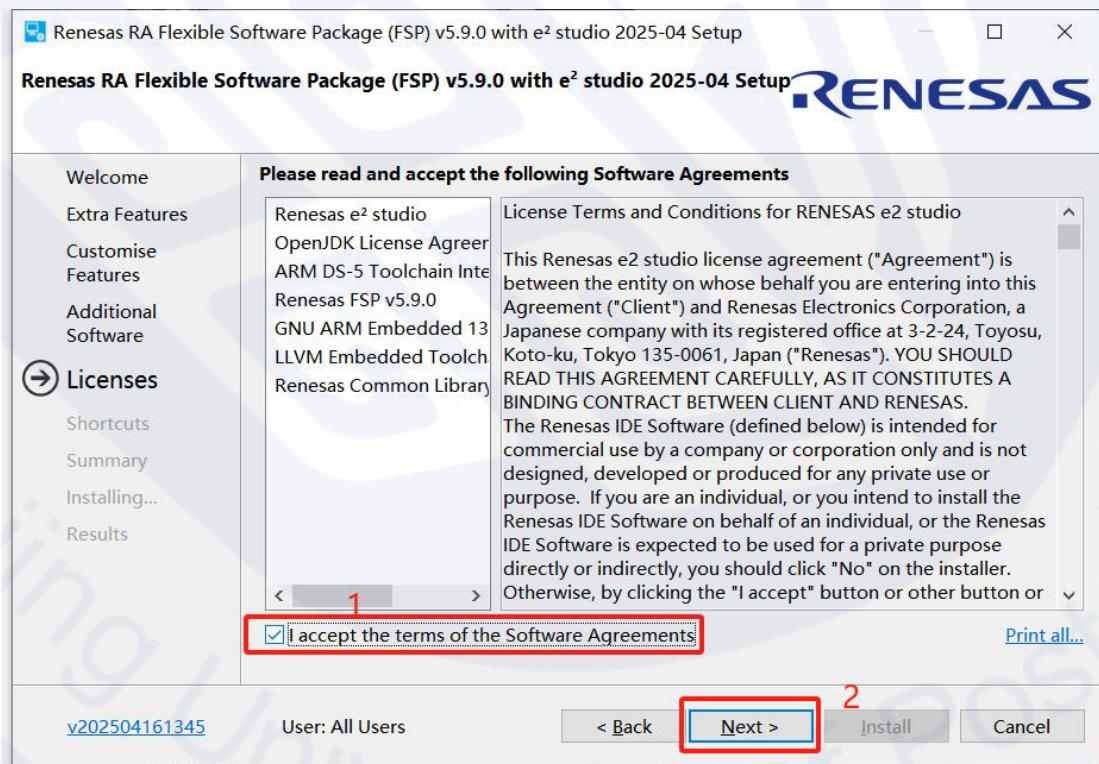


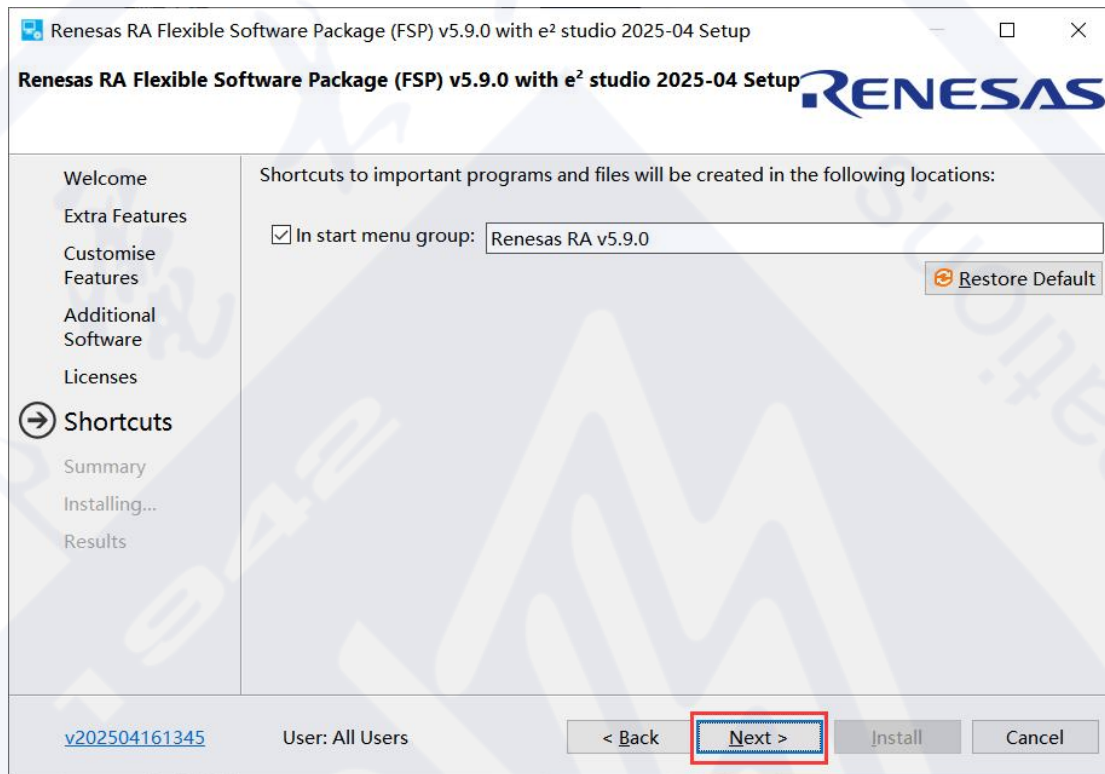
选择语言和一些组件，若不知道可以全选上。



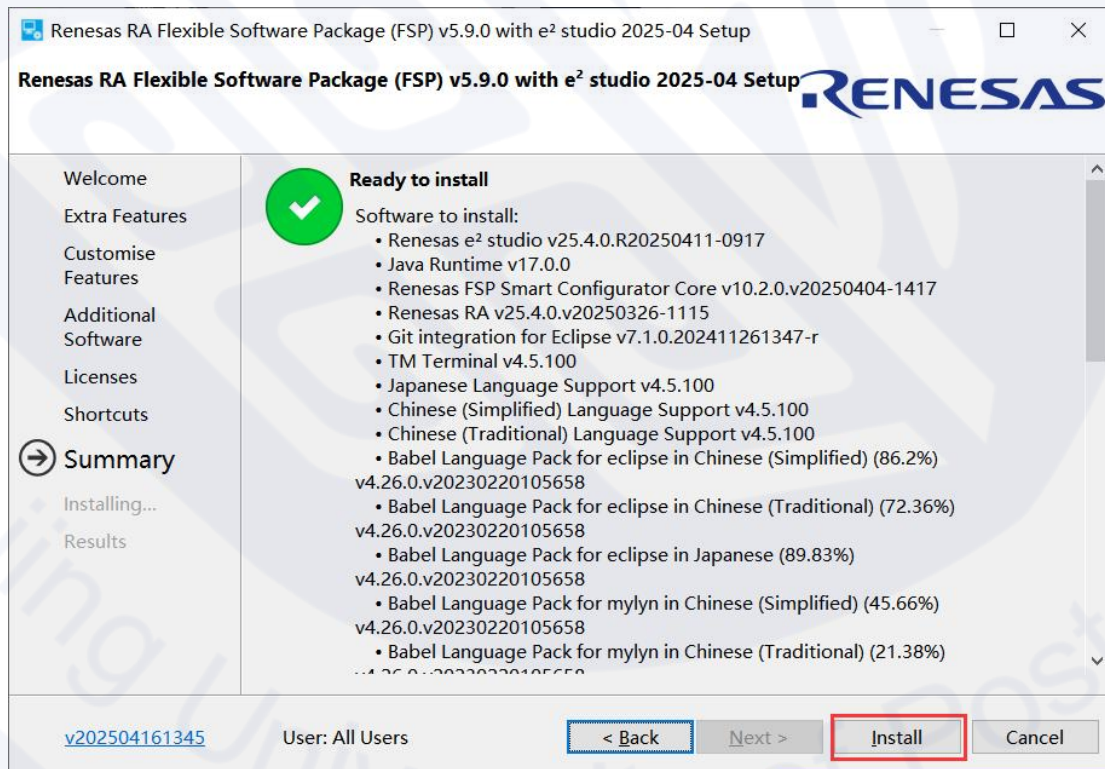


同意款项。

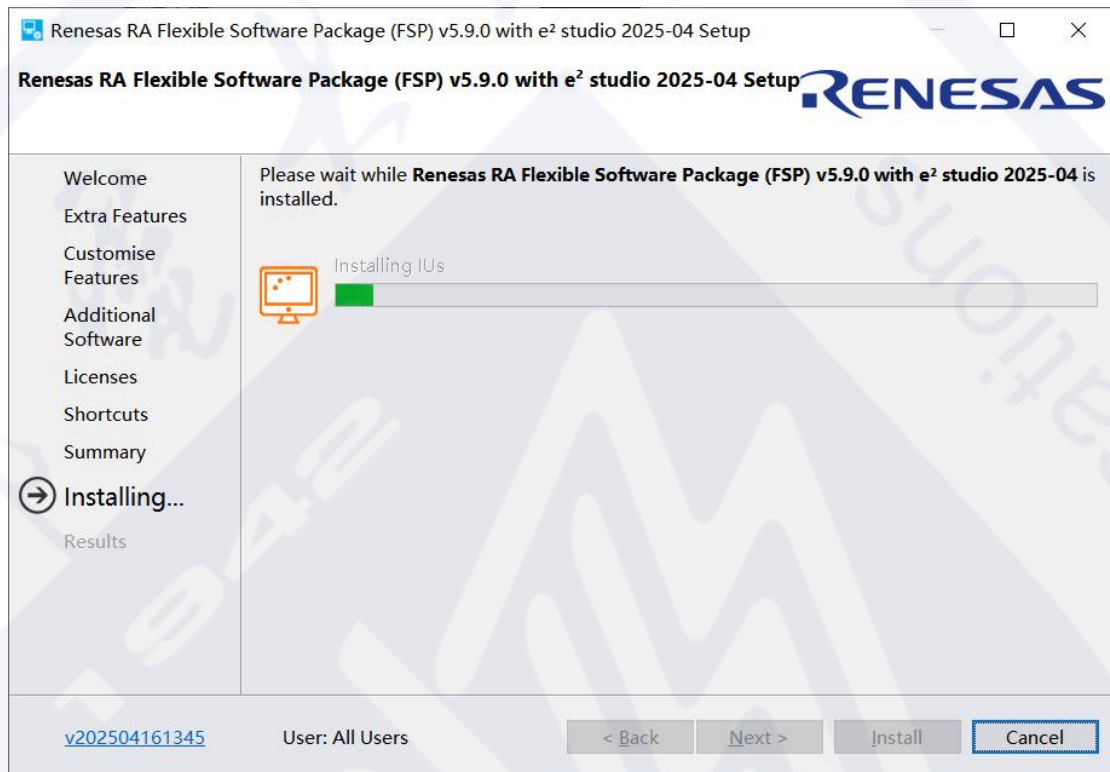




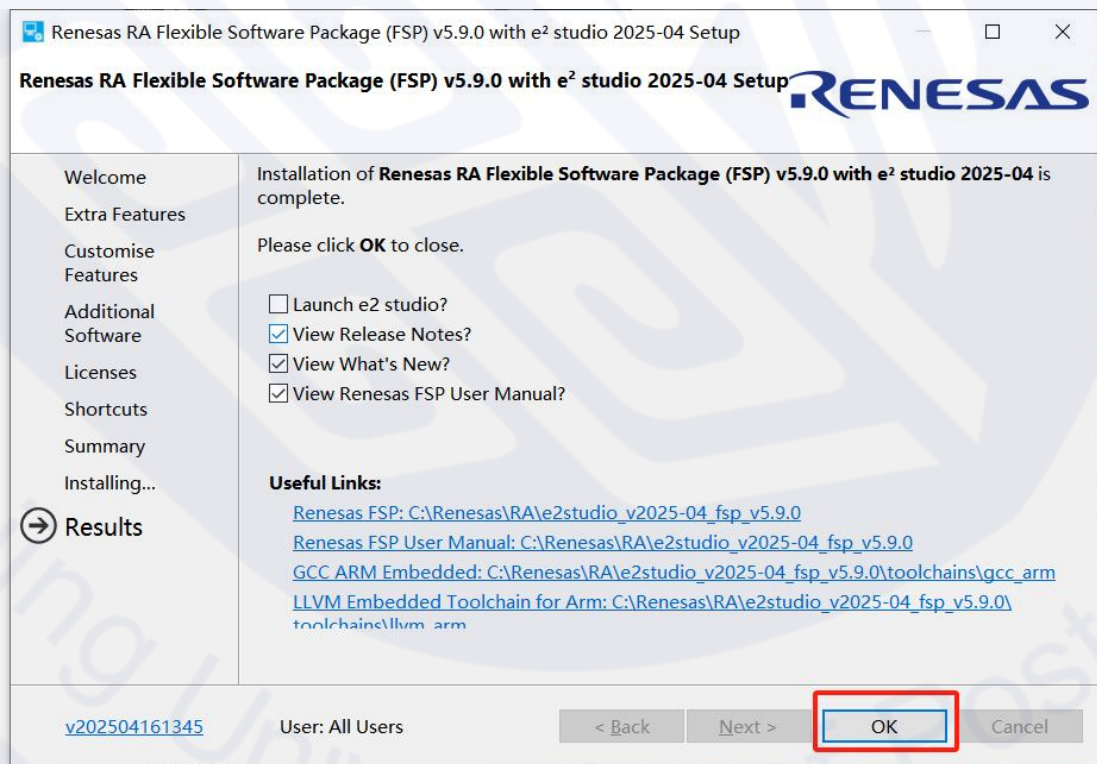
开始安装。



安装中。



安装完成。



project1: 开发板测试和 Programmer 烧录程序

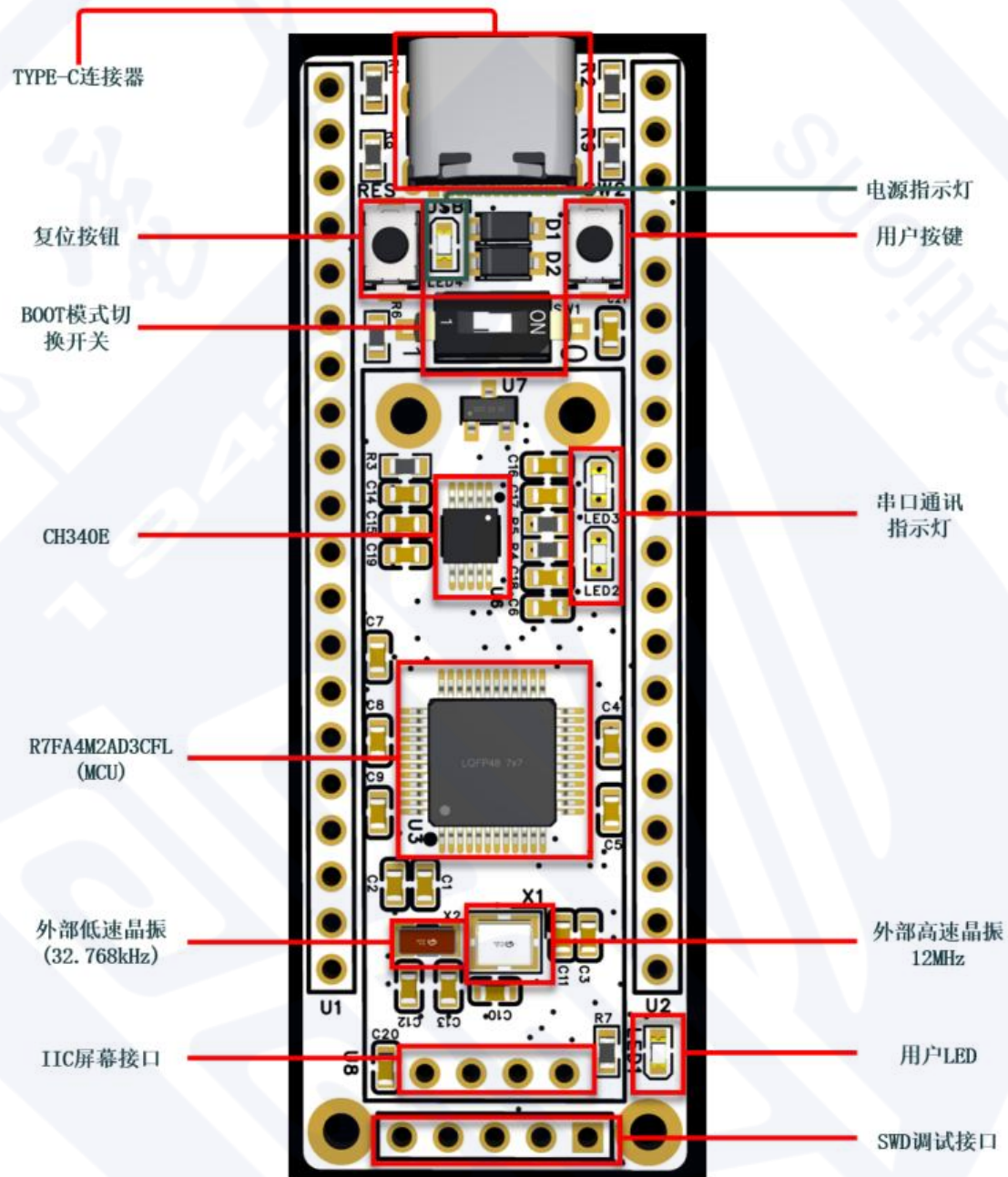
概述

本章帮助用户在 板卡到手后的“点亮”与基础功能验证：OLED 显示、LED 闪烁、按键切换与串口打印，之后使用 Renesas Flash Programmer（RFP）给板子烧录代码。

硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



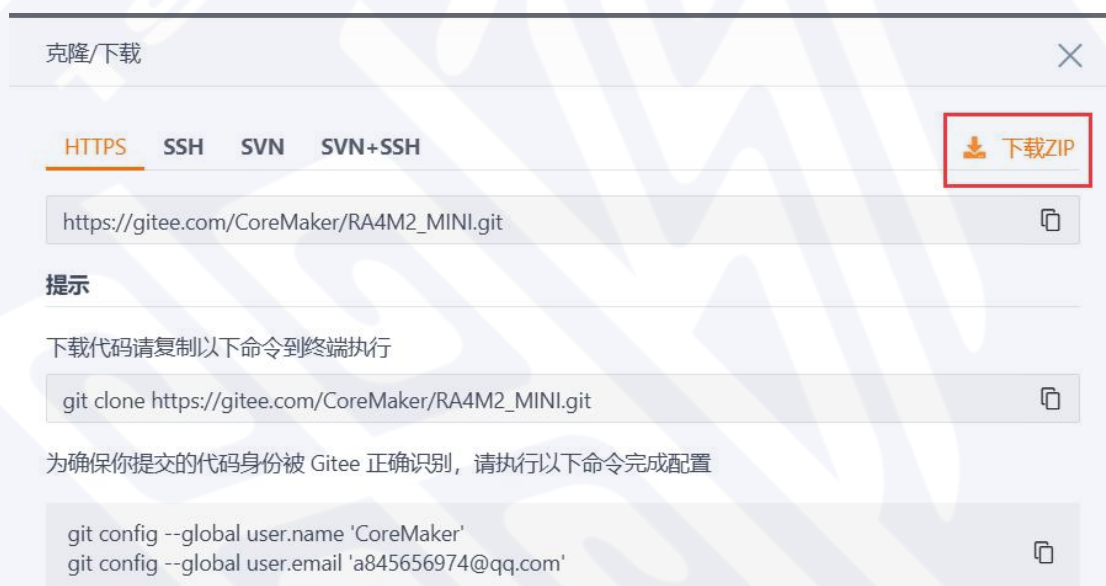
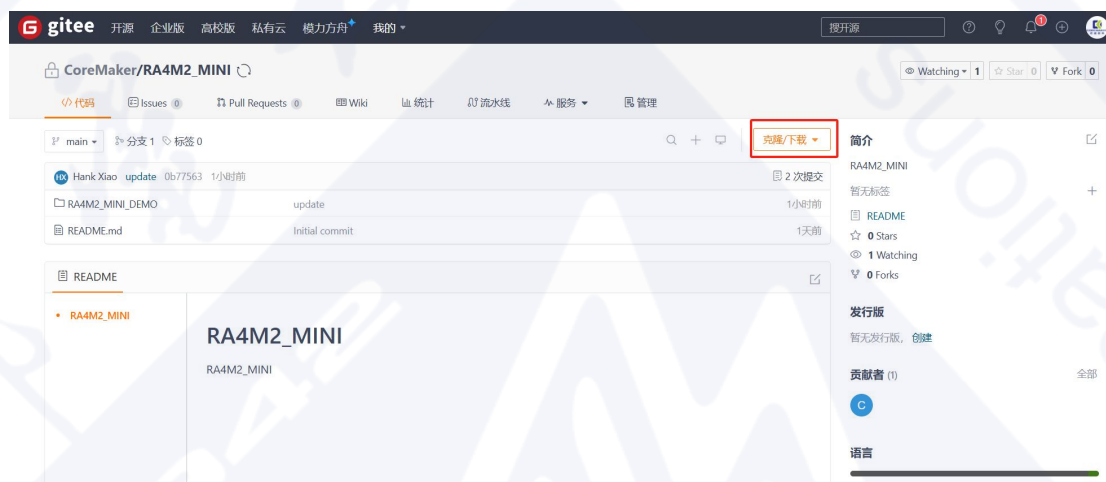
参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

获取示例程序

- 访问 Gitee 仓库: https://gitee.com/CoreMaker/RA4M2_MINI
- 点击 克隆/下载 → Download ZIP, 保存并解压。



Programmer

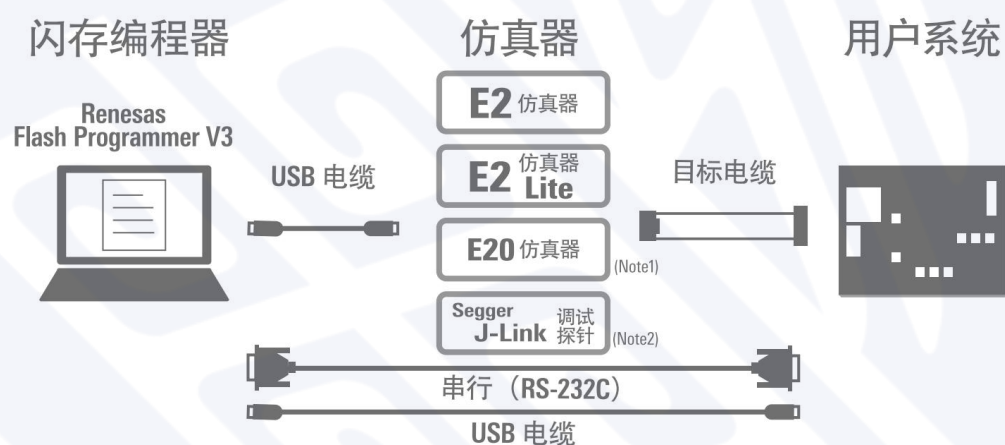
Renesas Flash Programmer 是一款支持开发和批量生产阶段写入的软件, 具有简单的 GUI, 即使是初学者也能简单操作进行写入。运行环境支持 Windows、Linux、macOS。这里使用 Renesas Flash Programmer 软件进行下载。下载链接:

<https://www.renesas.cn/zh/software-tool/renesas-flash-programmer-programming-gui>

主要特性

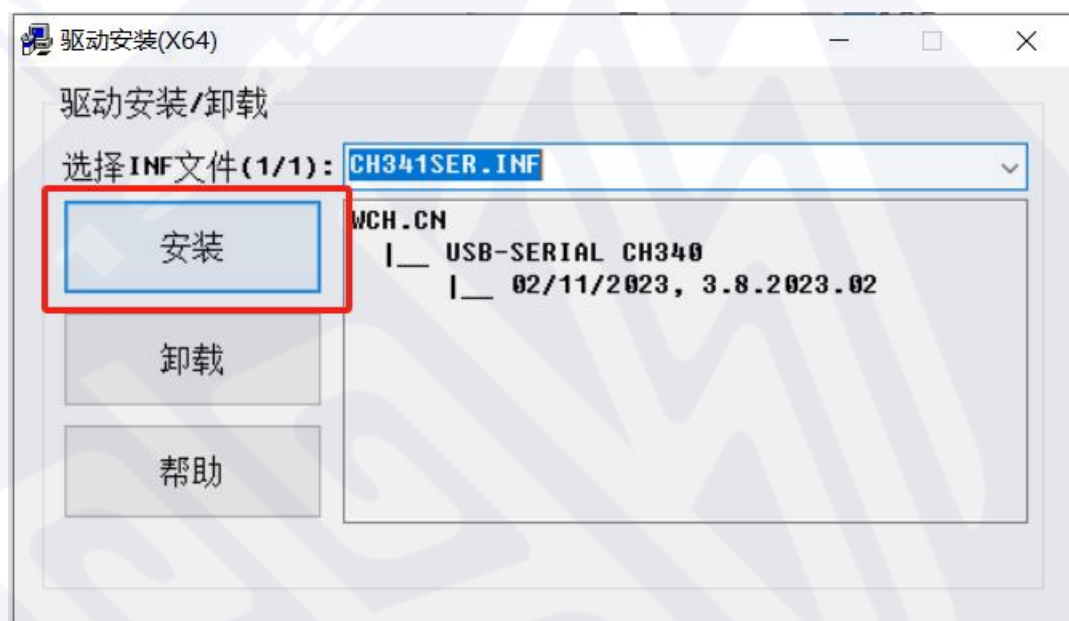
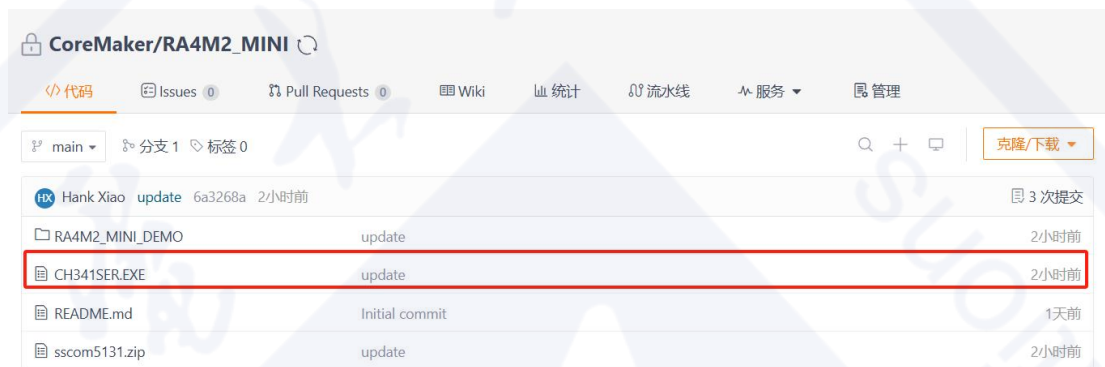
- 支持的编程文件格式：Intel HEX（用于 HEX 和 HCUHEX 文件）、Motorola S（用于 HEX 和 HCUHEX 文件）、二进制、RPI（用于 RPI 文件）
- 通信接口支持 USB、CSI、UART、FINE、SWD
- 校验计算
- 空白检查、擦除、写入、验证、读取、校验和、闪存选项的设置和获取
- 顺序执行多个命令（用于擦除、编程、验证等）
- 同时选择多个编程文件

系统配置

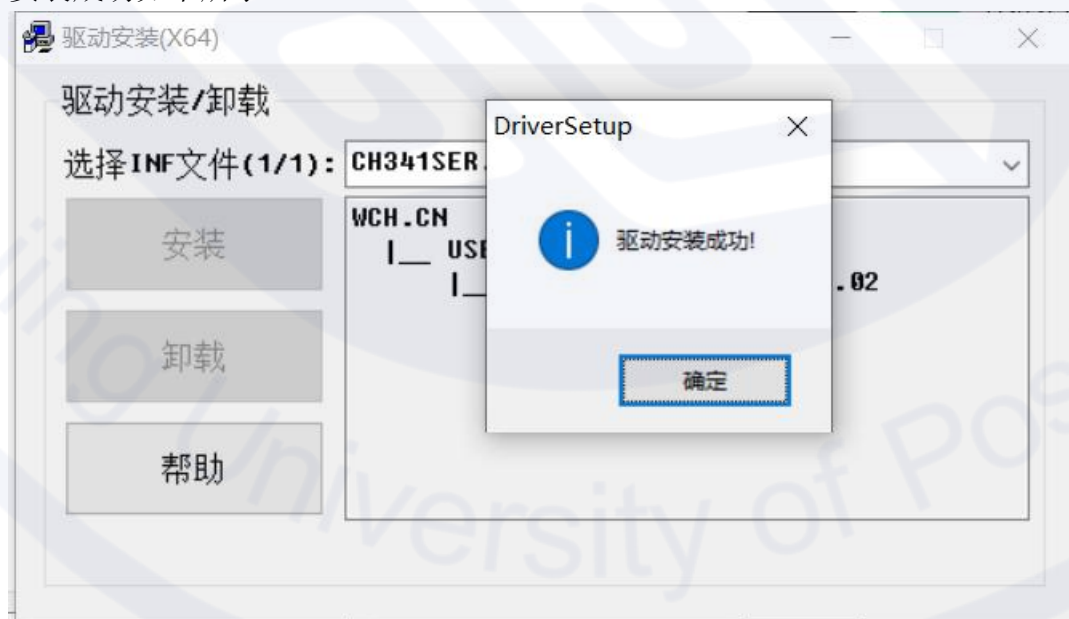


安装 CH340 驱动

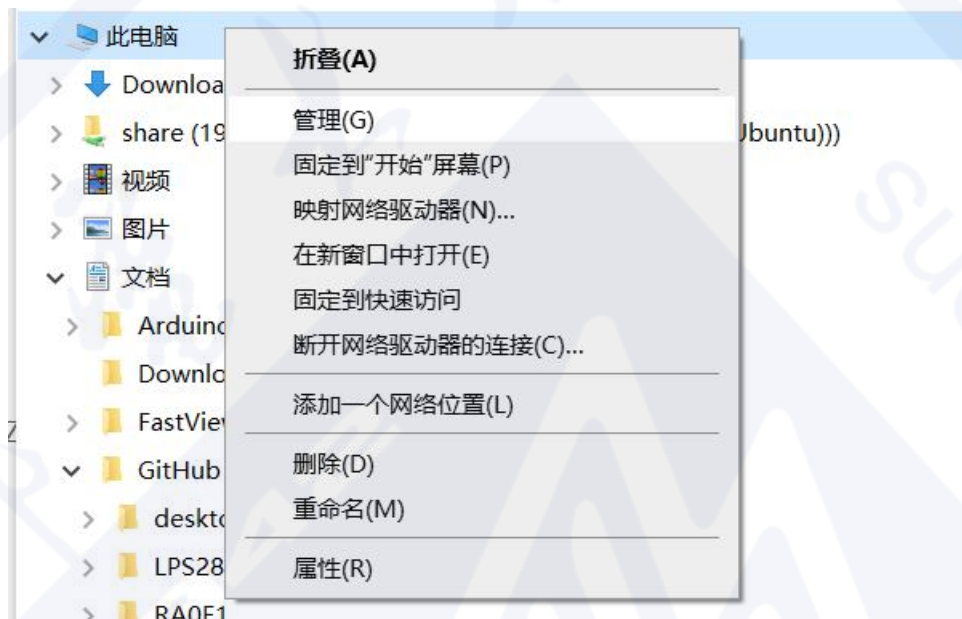
它把电脑的 USB 信号转换成 UART (TTL 串口) 信号，让开发板或模块能把日志、调试信息通过虚拟 COM 口发给电脑，也支持电脑向板子发送控制指令。没有驱动，操作系统无法识别这颗芯片，设备管理器里只会显示“未知设备”或带叹号的 USB 设备。



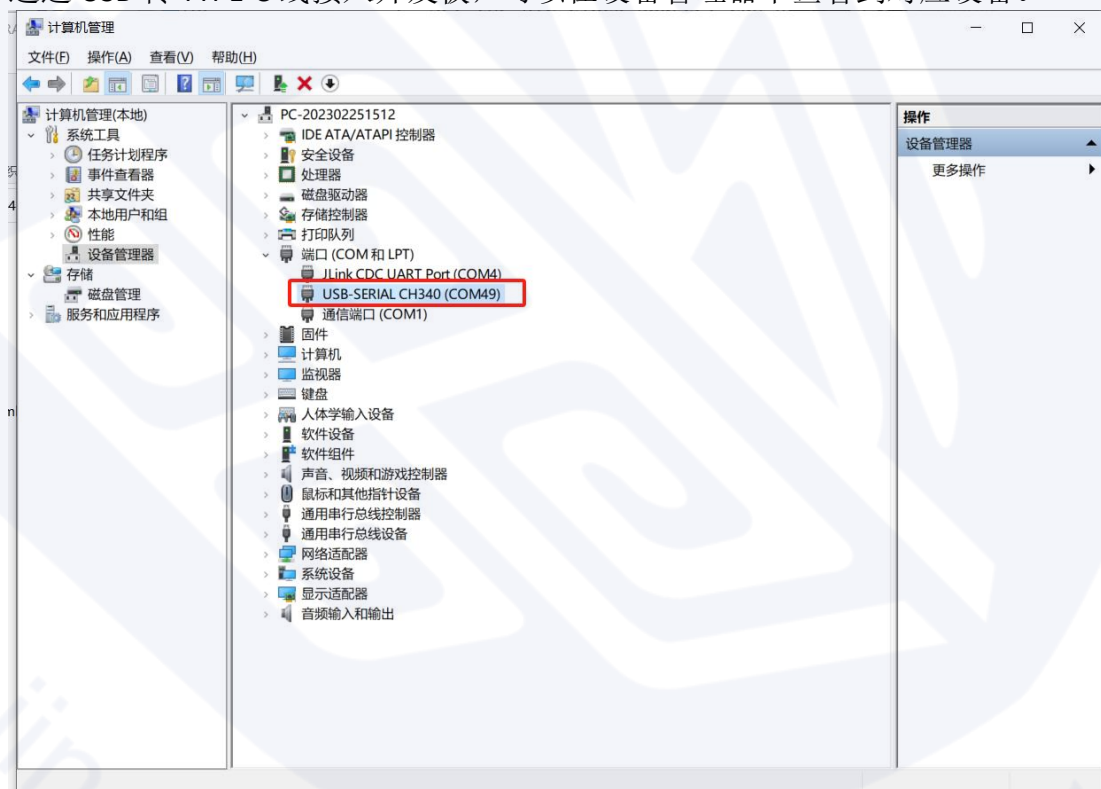
安装成功如下所示。



打开我的电脑的管理。



通过 USB 转 TYPE-C 线接入开发板，可以在设备管理器中查看到对应设备。



BOOT 设置

操作模式的切换取决于复位过程中的 MD 引脚状态：

MD = 1：复位后进入单片模式。

MD = 0：复位后进入 SCI 引导模式。

3. Operating Modes

3.1 Overview

Table 3.1 shows the selection of operating modes by the mode-setting pin. For details, see [section 3.2. Details of Operating Modes](#). Operation starts with the on-chip flash memory enabled, regardless of the mode in which operation started.

Table 3.1 Selection of operating modes by the mode-setting pin

Mode-setting pin (MD)	Operating mode	On-chip Flash
1	Single-chip mode	Enable
0	SCI / USB boot mode	Enable

串口烧录

串行编程模式包括：
通过 SCI9 的引导模式（Boot mode with SCI9）。

44.13 Boot Mode

There are two serial programming modes; the boot mode (for the SCI interface) with SCI9 and the boot mode (for the USB interface) with USBFS. The available interfaces and connection times to the tool vary depending on the clock source connected to the MCU. [Table 44.27](#) lists the I/O pins used in boot mode. [Table 44.28](#) lists the available communication interface and the connection time depending on the clock sources in the boot mode.

Table 44.27 I/O pins used in boot mode

Pin name	I/O	Mode to be used	Use
MD	Input	Boot mode (for the SCI interface) Boot mode (for the USB interface)	Selection of operating mode
P110/RXD9	Input	Boot mode (for the SCI interface)	For host communication (to receive data through SCI)
P109/TXD9	Output		For host communication (to transmit data through SCI)
USB_DP, USB_DM	I/O	Boot mode (for the USB interface)	Data input/output of USB
USB_VBUS	Input		Detection of connection and disconnection of USB cables

Table 44.28 Tool connection time depending on the clock source

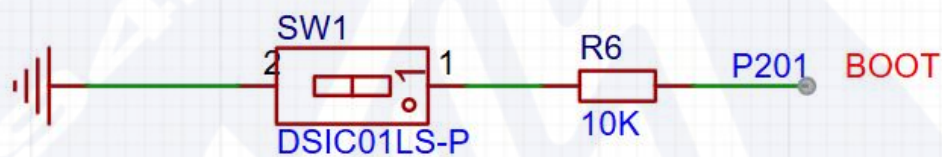
Main clock oscillator	Sub-clock oscillator	Available interface	Tool connection time*2
Connected	Don't care	SCI/USB	Up to 1 second
Unconnected	Connected*1	SCI/USB	Up to 2 seconds
Unconnected	Unconnected	SCI	Up to 3 seconds

Note 1. The drive capability of the sub-clock oscillator is set to standard by SOMCR.SODRV bit. Note that if you use the crystal resonator corresponding to the low drive capability on your board, the crystal resonator may not oscillate in the boot mode.

Note 2. Tool connection time means that it takes for communication to be established between MCU and the host. For details, see the boot firmware application note.

boot 模式可以通过拨码进行设置。

BOOT切换按键

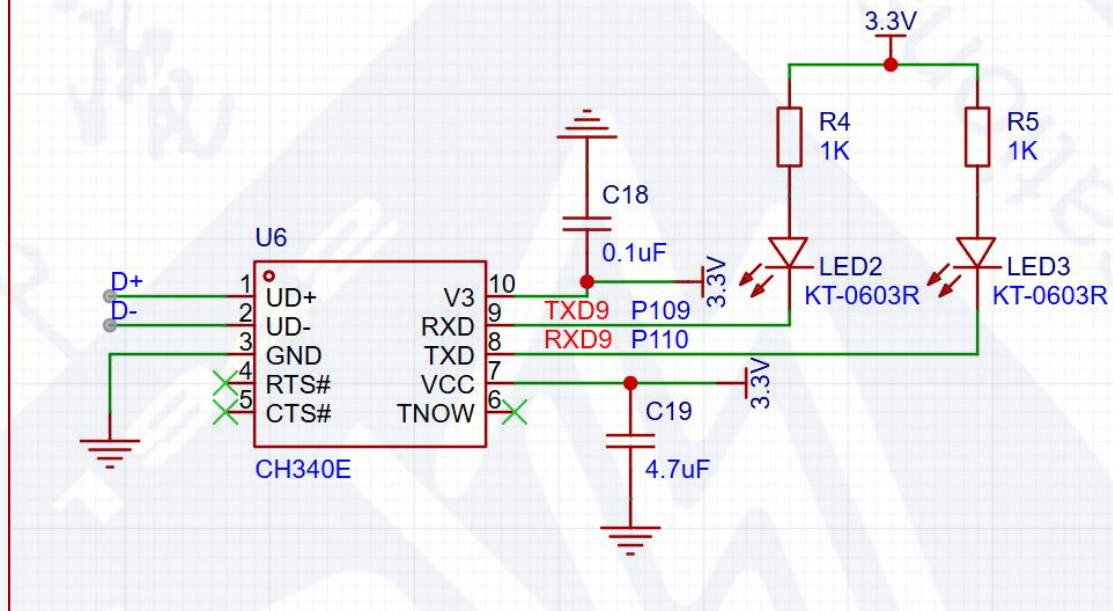


切换如下模式进入烧录。



板载已经将 P109,P110 给连接出来。

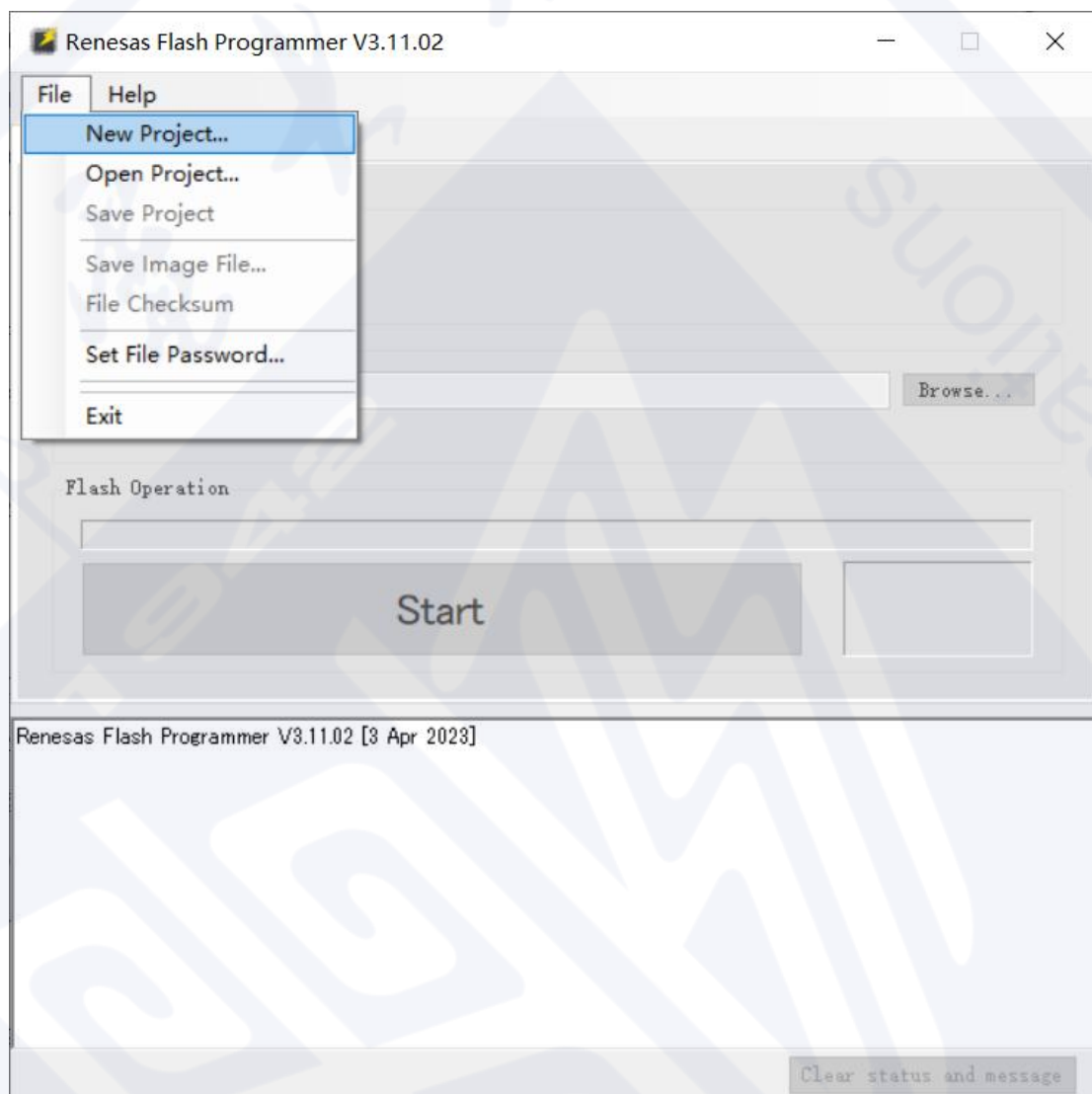
串口CH340E



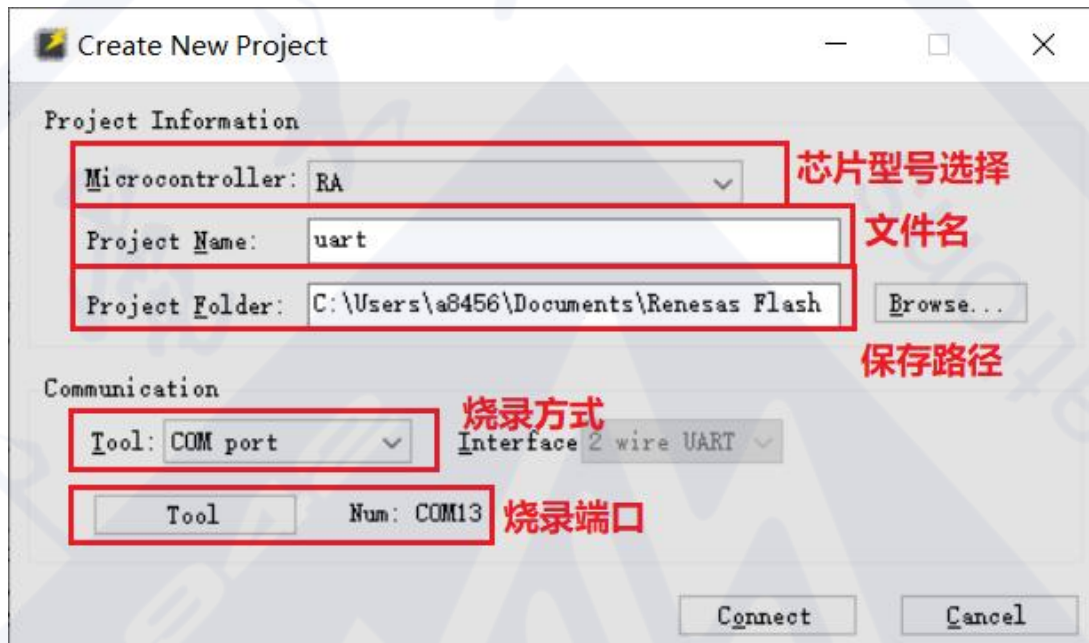
打开 Renesas Flash Programmer 进行烧录。



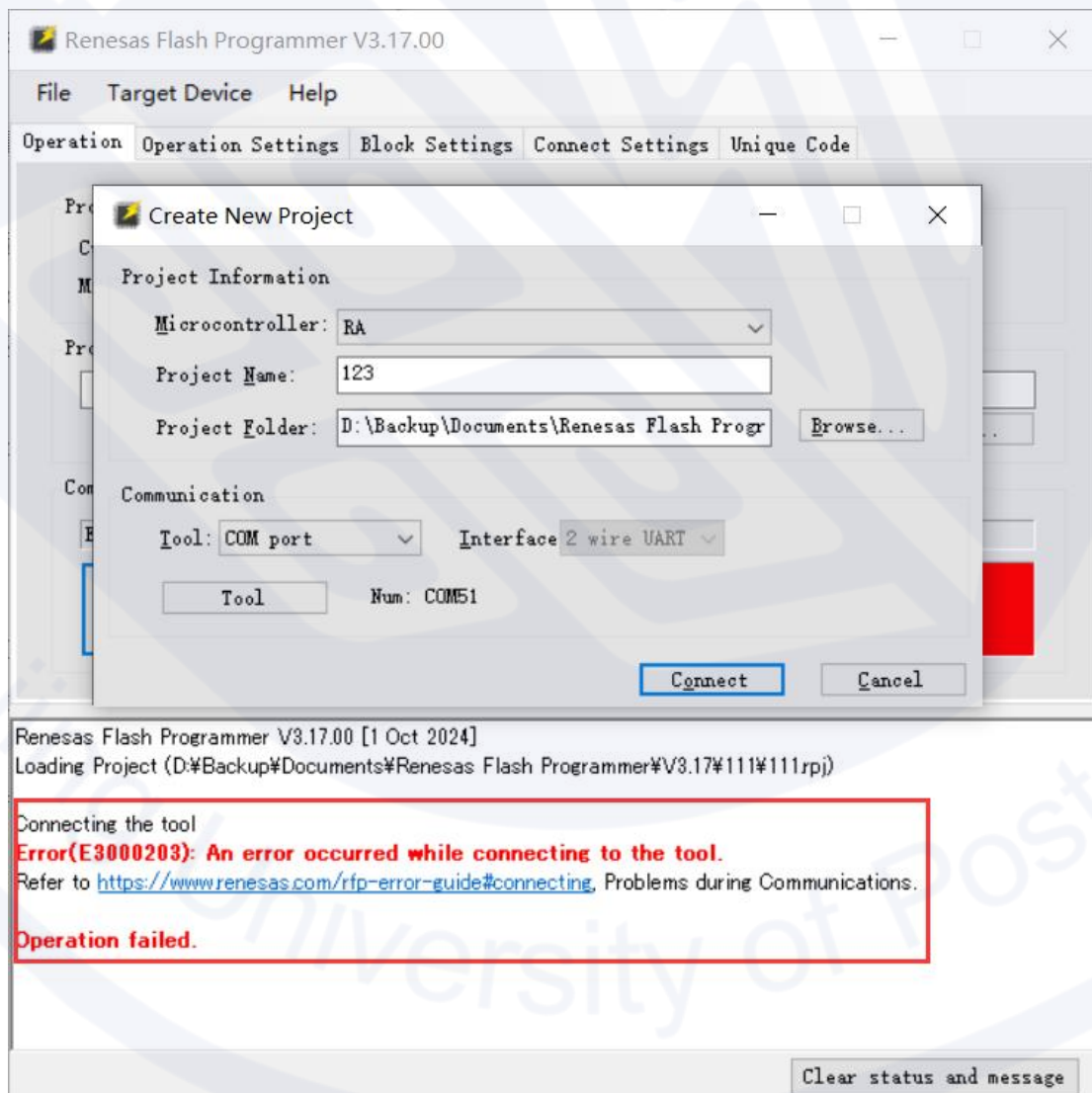
新建一个工程。



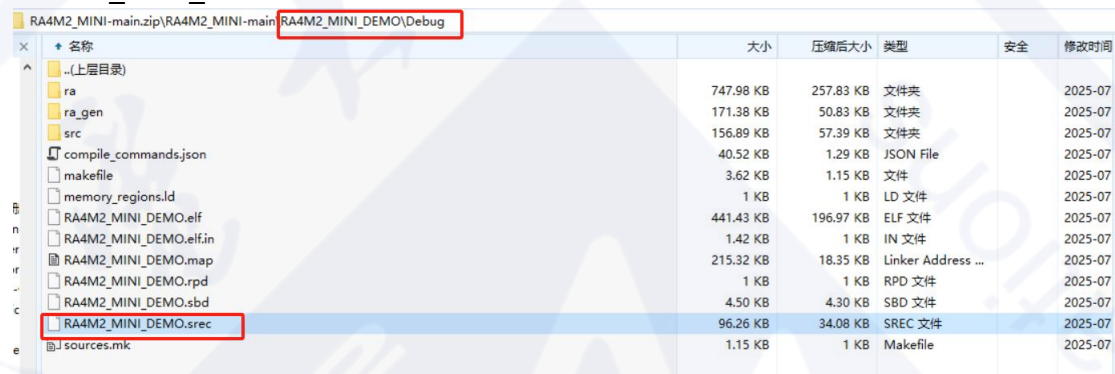
烧录设置。



若出现下列报错,需要按一下复位按键或者断电在上电,在重新设置即可。

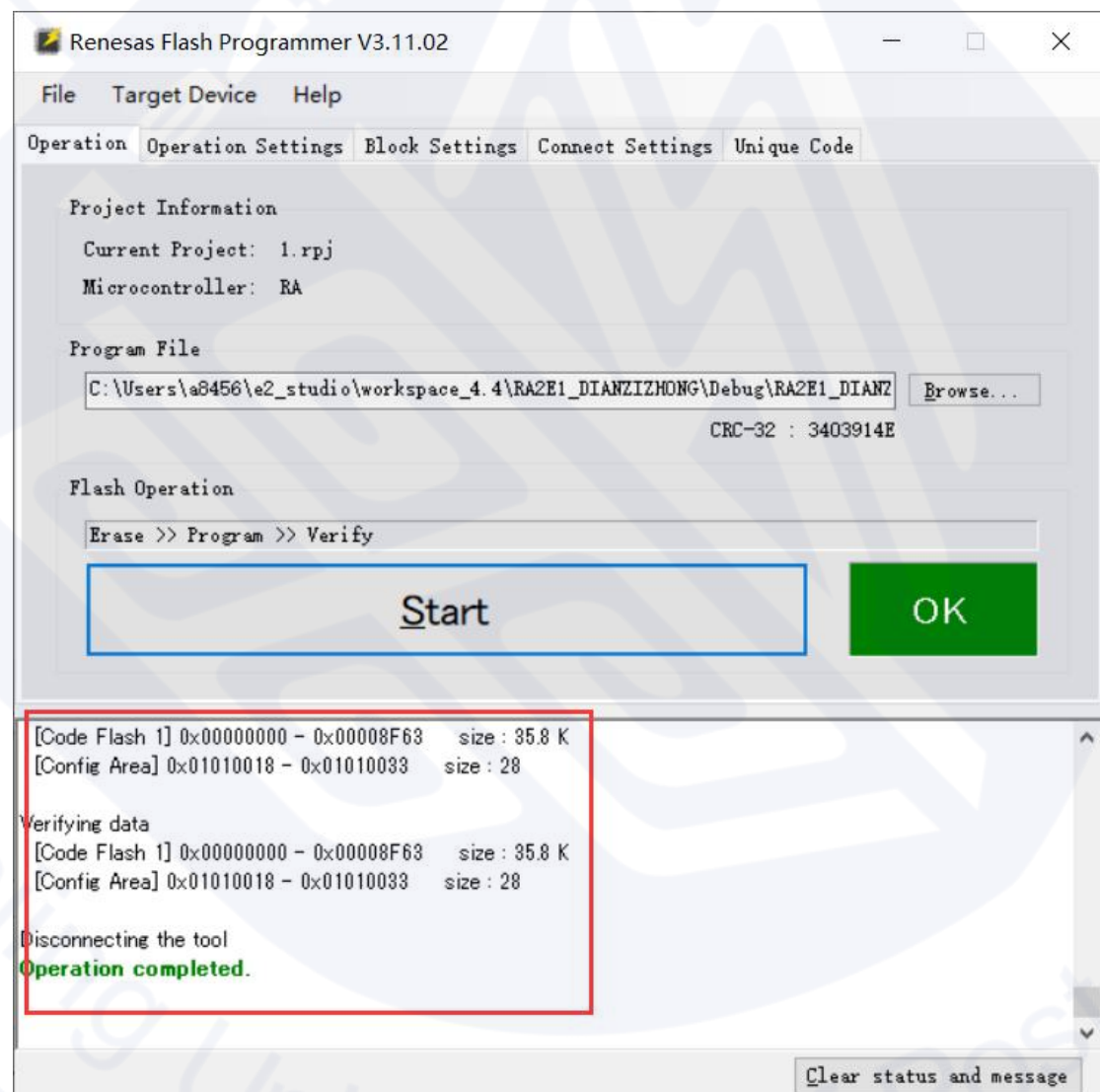


烧录文件可以选 hex，也可以选 srec，选择刚刚下载文件中的 RA4M2_MINI_DEMO.srec。



名称	大小	压缩后大小	类型	安全	修改时间
..(上层目录)					
ra	747.98 KB	257.83 KB	文件夹		2025-07
ra_gen	171.38 KB	50.83 KB	文件夹		2025-07
src	156.89 KB	57.39 KB	文件夹		2025-07
compile_commands.json	40.52 KB	1.29 KB	JSON File		2025-07
makefile	3.62 KB	1.15 KB	文件		2025-07
memory_regions.ld	1 KB	1 KB	LD 文件		2025-07
RA4M2_MINI_DEMO.elf	441.43 KB	196.97 KB	ELF 文件		2025-07
RA4M2_MINI_DEMO.elf.in	1.42 KB	1 KB	IN 文件		2025-07
RA4M2_MINI_DEMO.map	215.32 KB	18.35 KB	Linker Address ...		2025-07
RA4M2_MINI_DEMO.rpd	1 KB	1 KB	RPD 文件		2025-07
RA4M2_MINI_DEMO.sbd	4.50 KB	4.30 KB	SBD 文件		2025-07
RA4M2_MINI_DEMO.srec	96.26 KB	34.08 KB	SREC 文件		2025-07
sources.mk	1.15 KB	1 KB	Makefile		2025-07

烧录成功如下所示。



之后给板子断电，拨回如下状态，在进行上电。



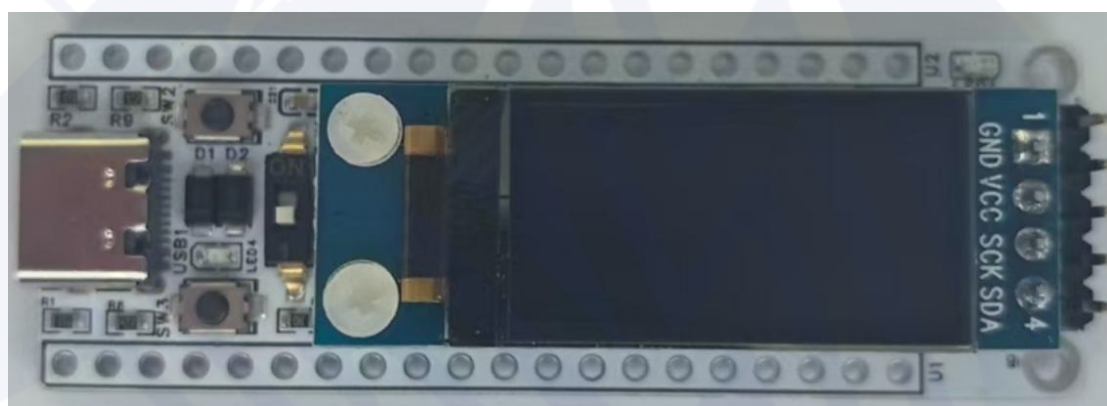
显示如下所示。



project2: 导入程序到 E2studio

概述

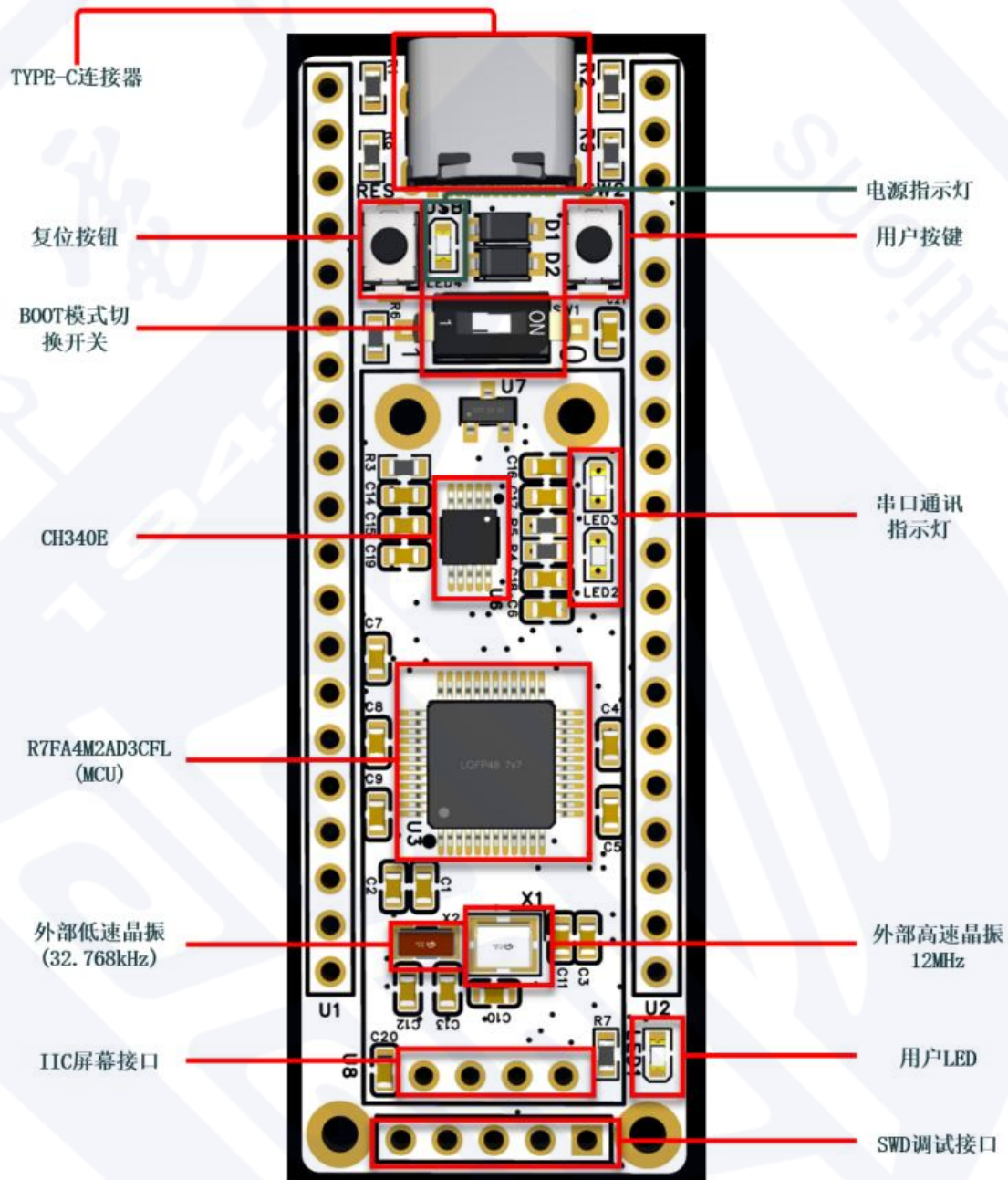
项目导入可以在本地搭建一个可随意实验的“沙盒工程”。这样用户不只是烧录现成固件，而是能够 先动手改代码、深度了解开发流程，为后续定制或量产奠定基础。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



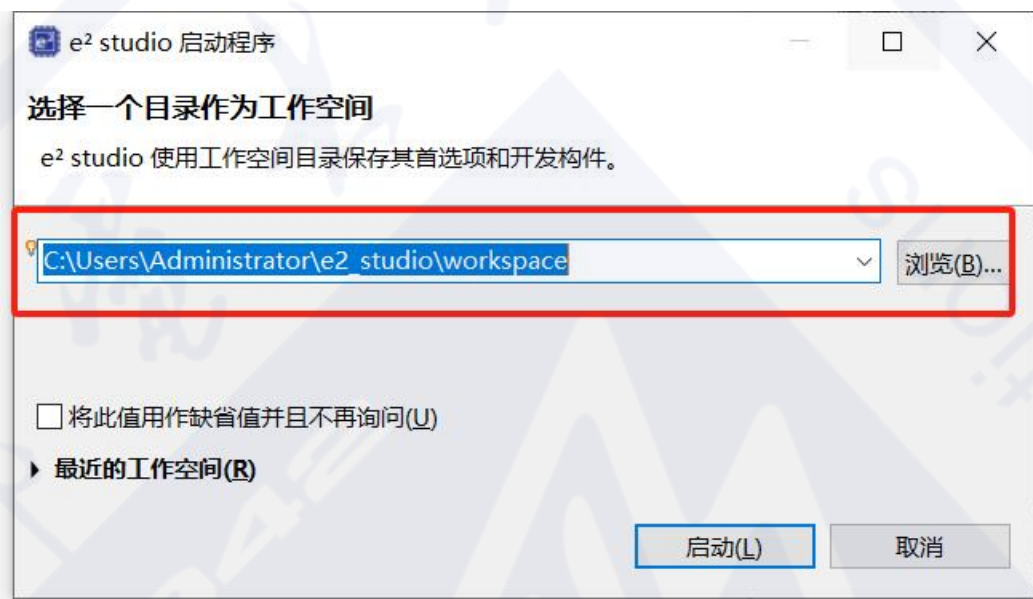
参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

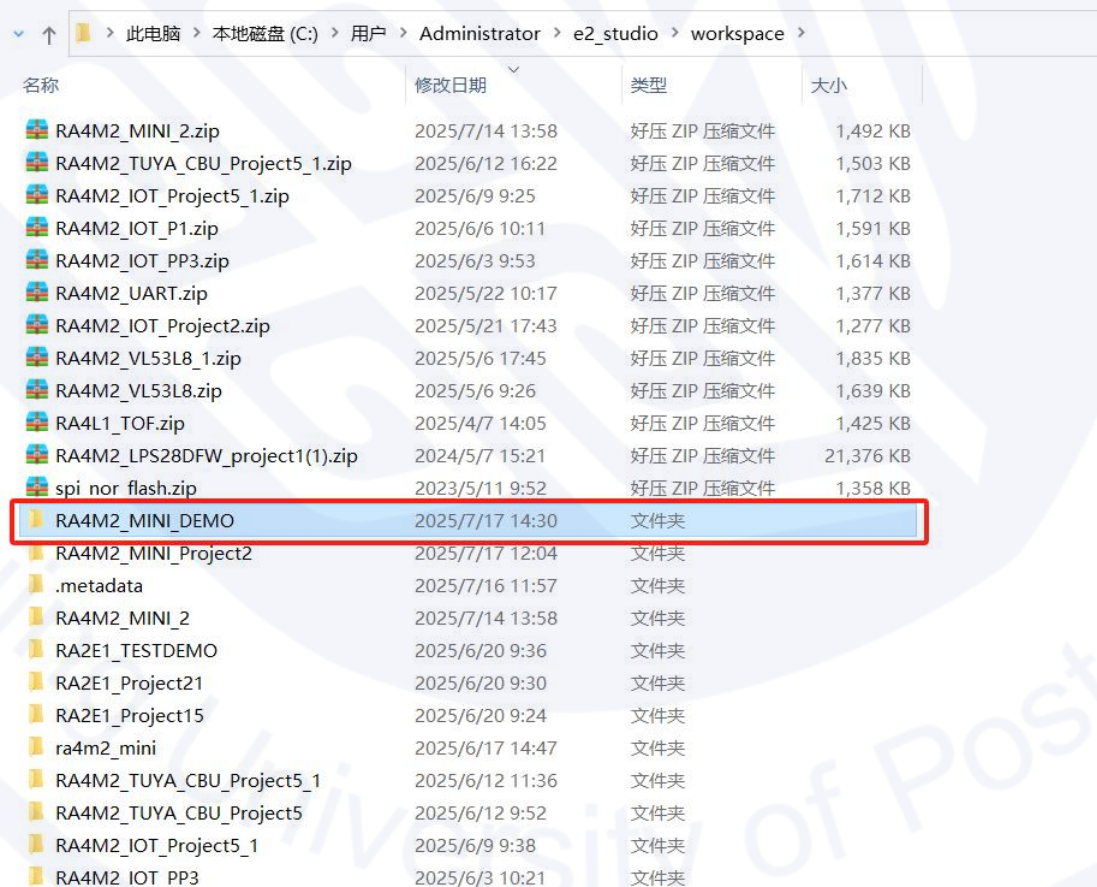
打开 E2studio

路径可以不改，要记得这个路径，需要注意的是不可以带中文。

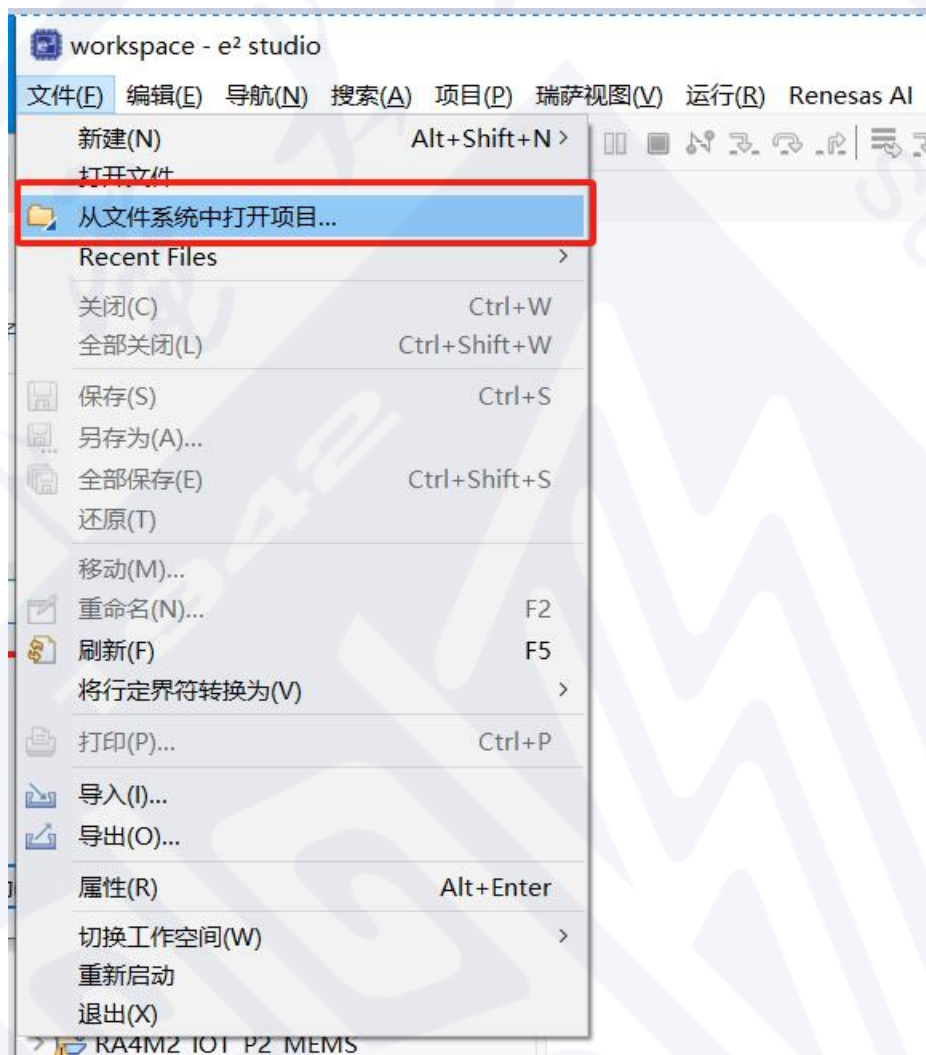


导入工程

把之前下载的程序放入该路径 RA4M2_MINI_DEMO。



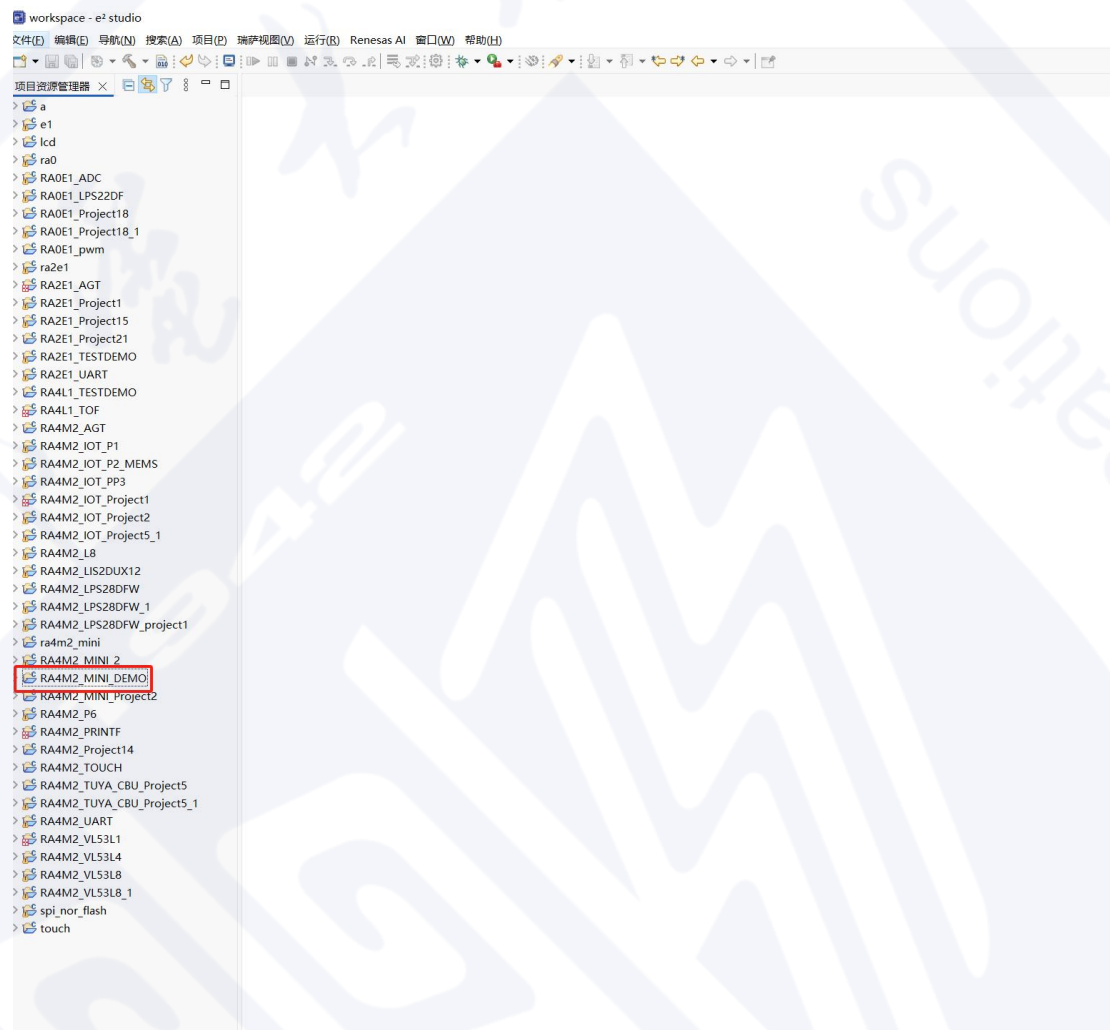
选择文件->从文件系统中打开项目...



选中刚刚导入的文件夹。



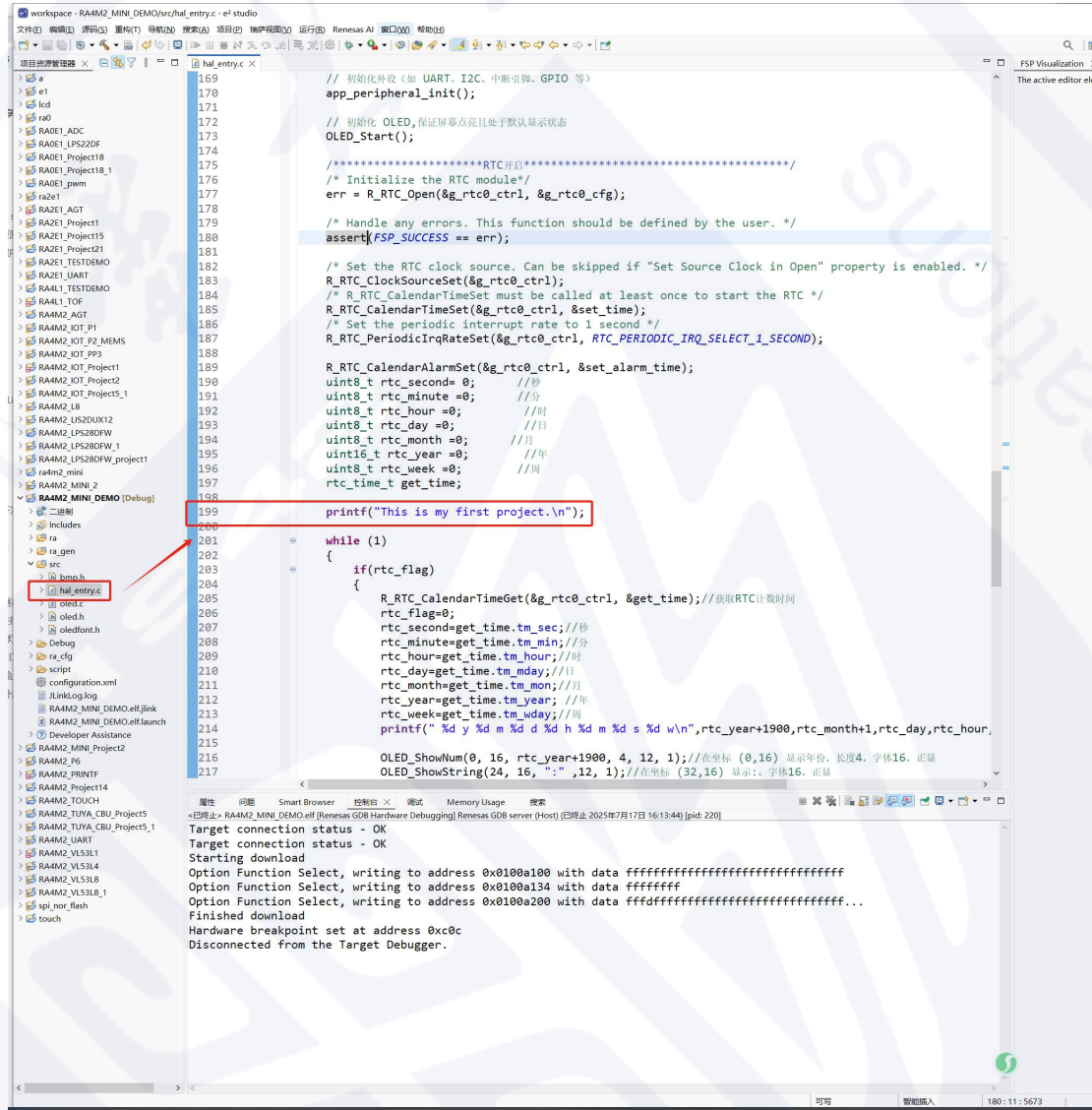
这时候左边项目资源管理器会出现对应工程。



修改代码

在 `hal_entry.c` 添加如下代码，当然也可以自己设置自己的输出，不要使用中文。

```
printf("This is my first project.\n");
```

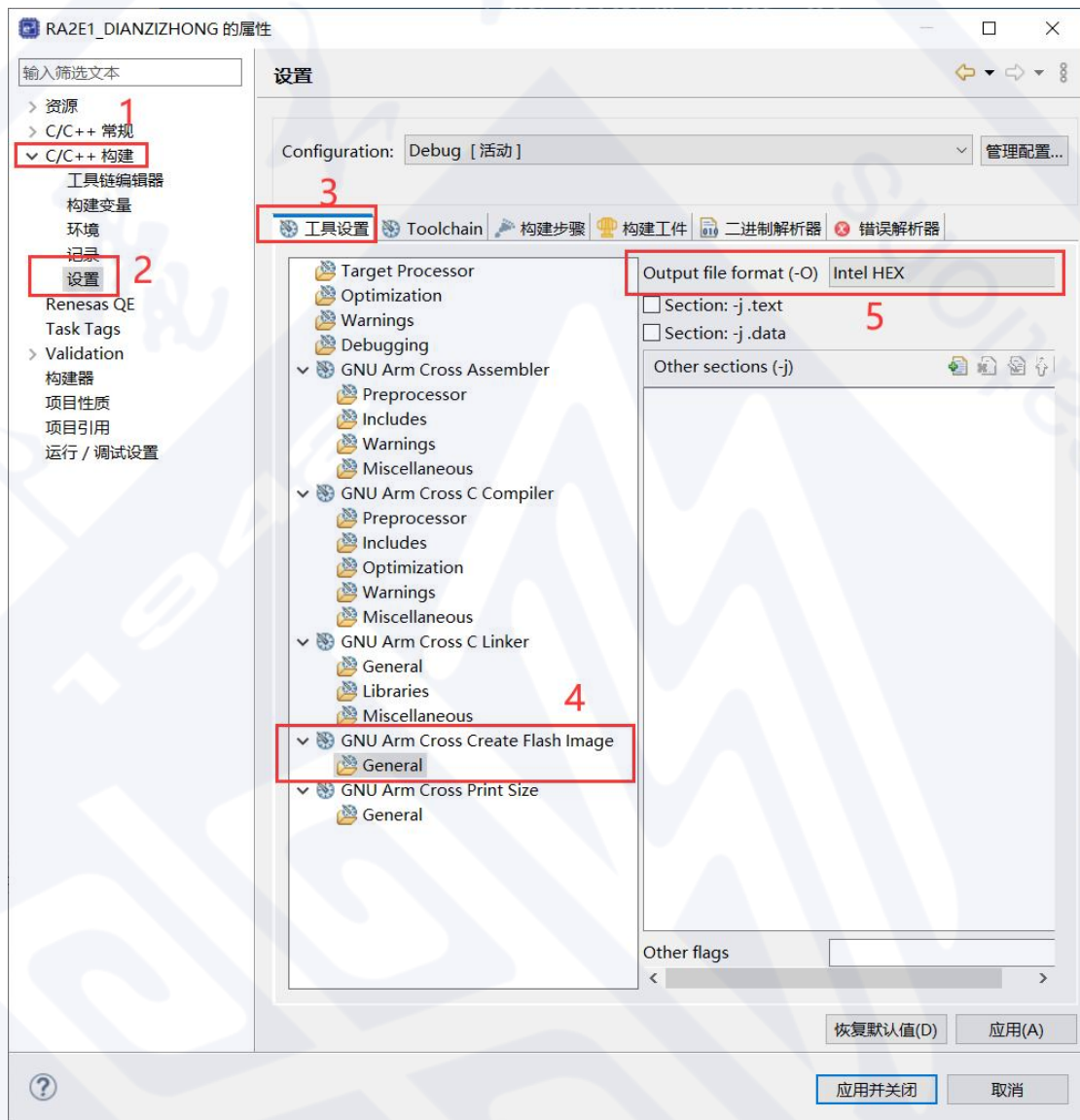


hex 文件准备

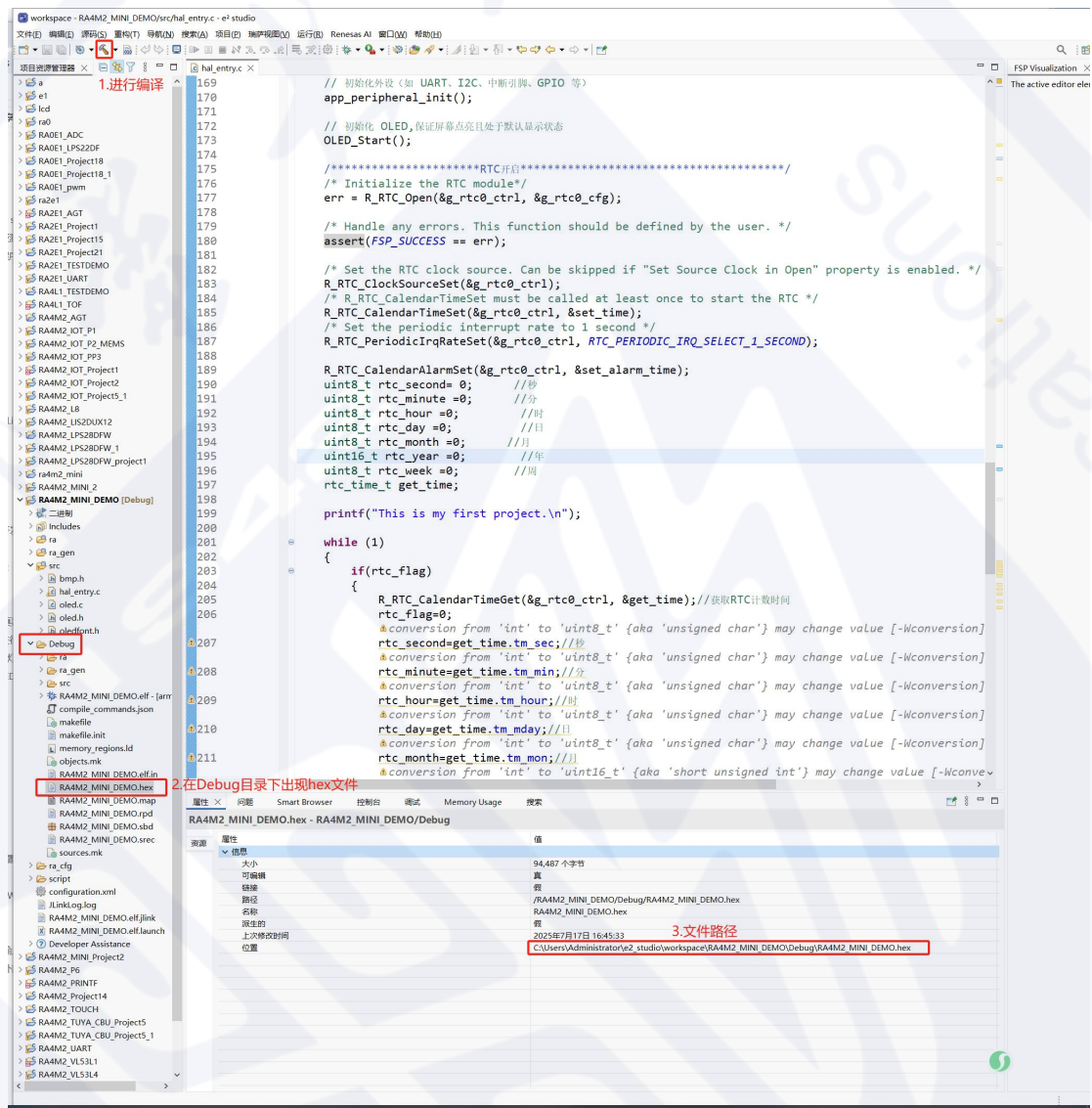
hex 文件格式是一种可以直接烧录到单片机并被其执行的格式。可以通过多种方法生成 Hex 文件，例如使用各种编译器将 C 程序或汇编程序转化为 hex 文件。要在 e2studio 中设置输出 hex 文件，请按照以下步骤操作：选择项目->属性，然后按照以下配置进行设置。



之后选择 C/C++ 构建 -> 设置 -> 工具设置 -> GNU ARM Cross Create Flash Image -> General, 在 Output file format 中选择 Intel HEX 进行输出 HEX 文件。



对程序进行重新编译,就会出现 hex 文件,文件一般在该项目的 Debug 文件夹中。



烧录

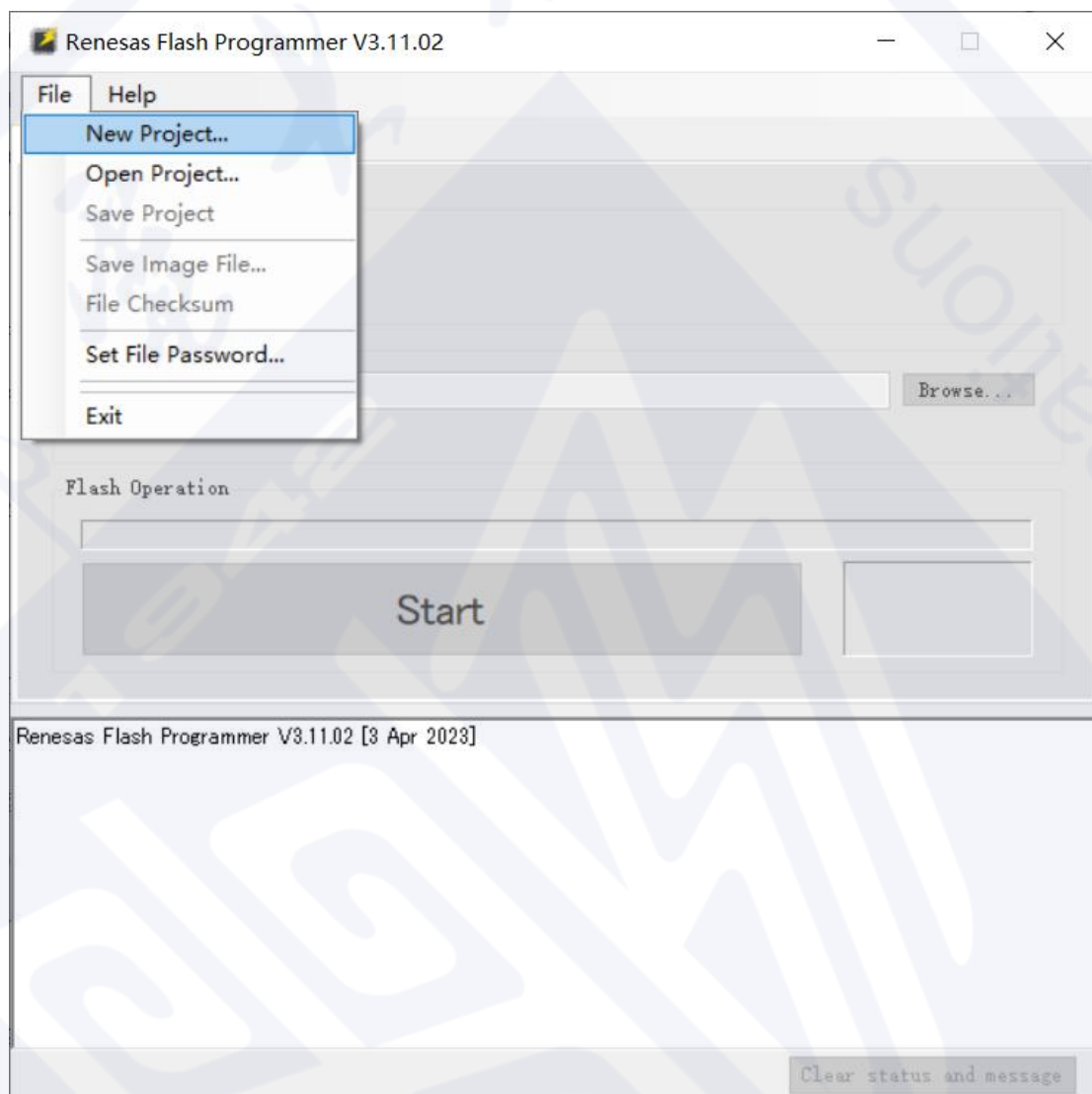
切换如下模式进入烧录。



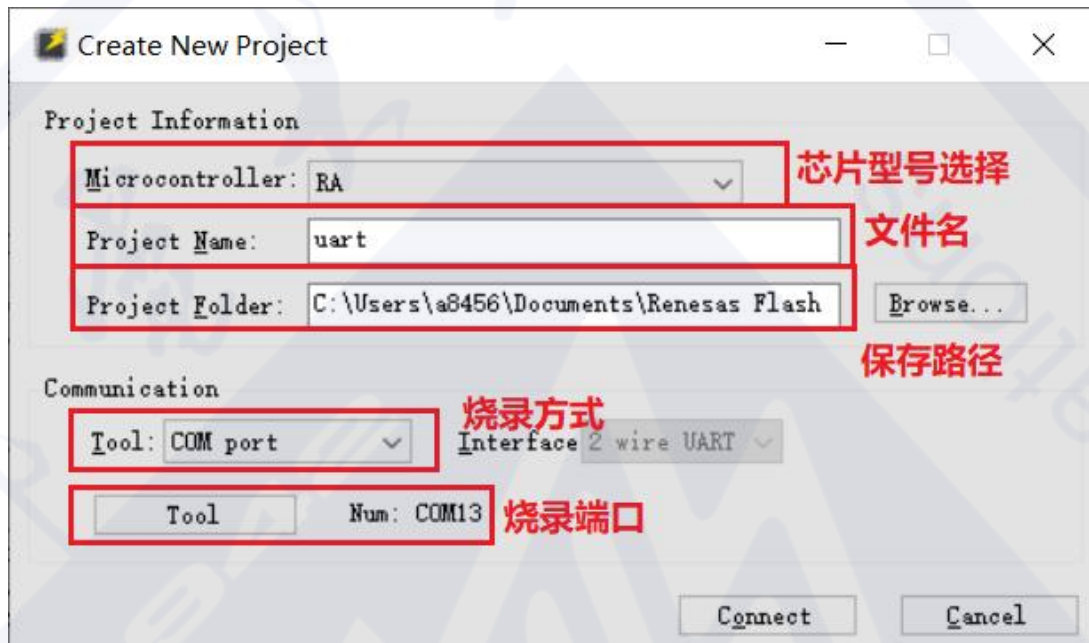
打开 Renesas Flash Programmer 进行烧录。



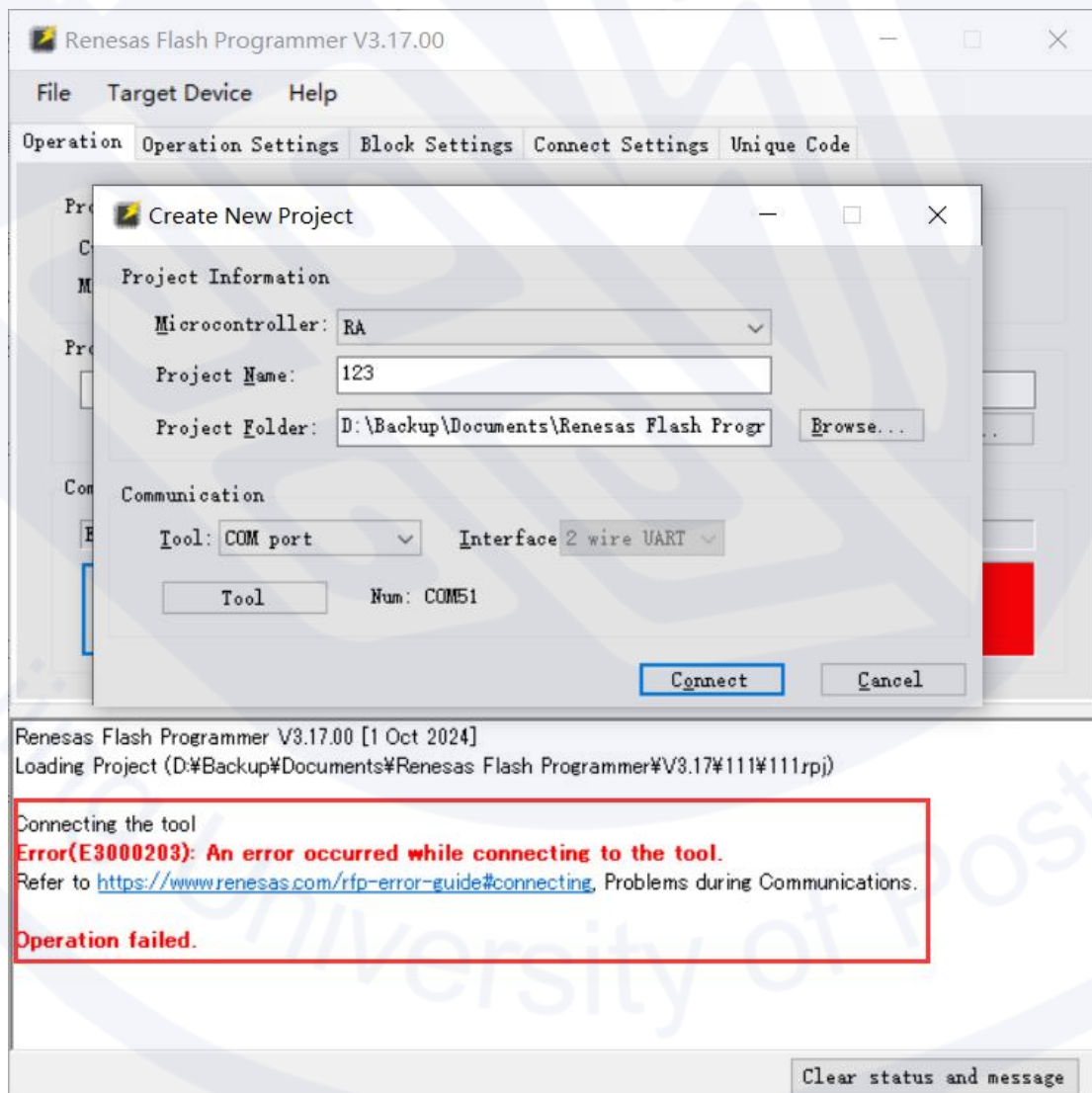
新建一个工程。



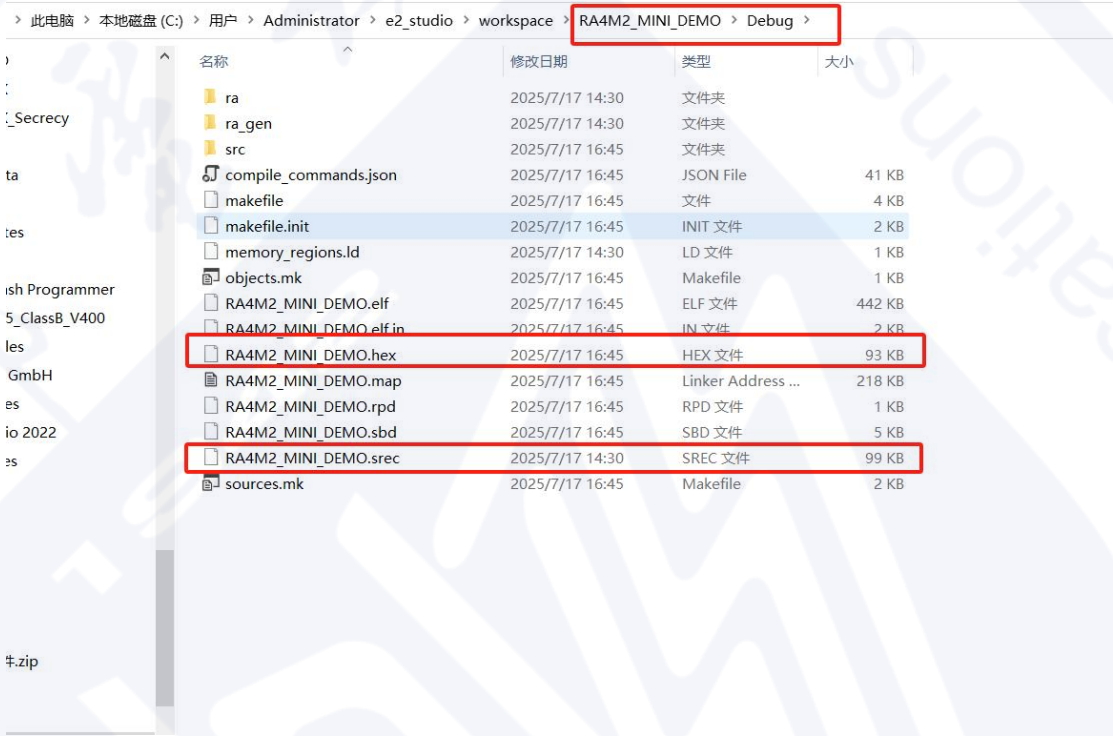
烧录设置。



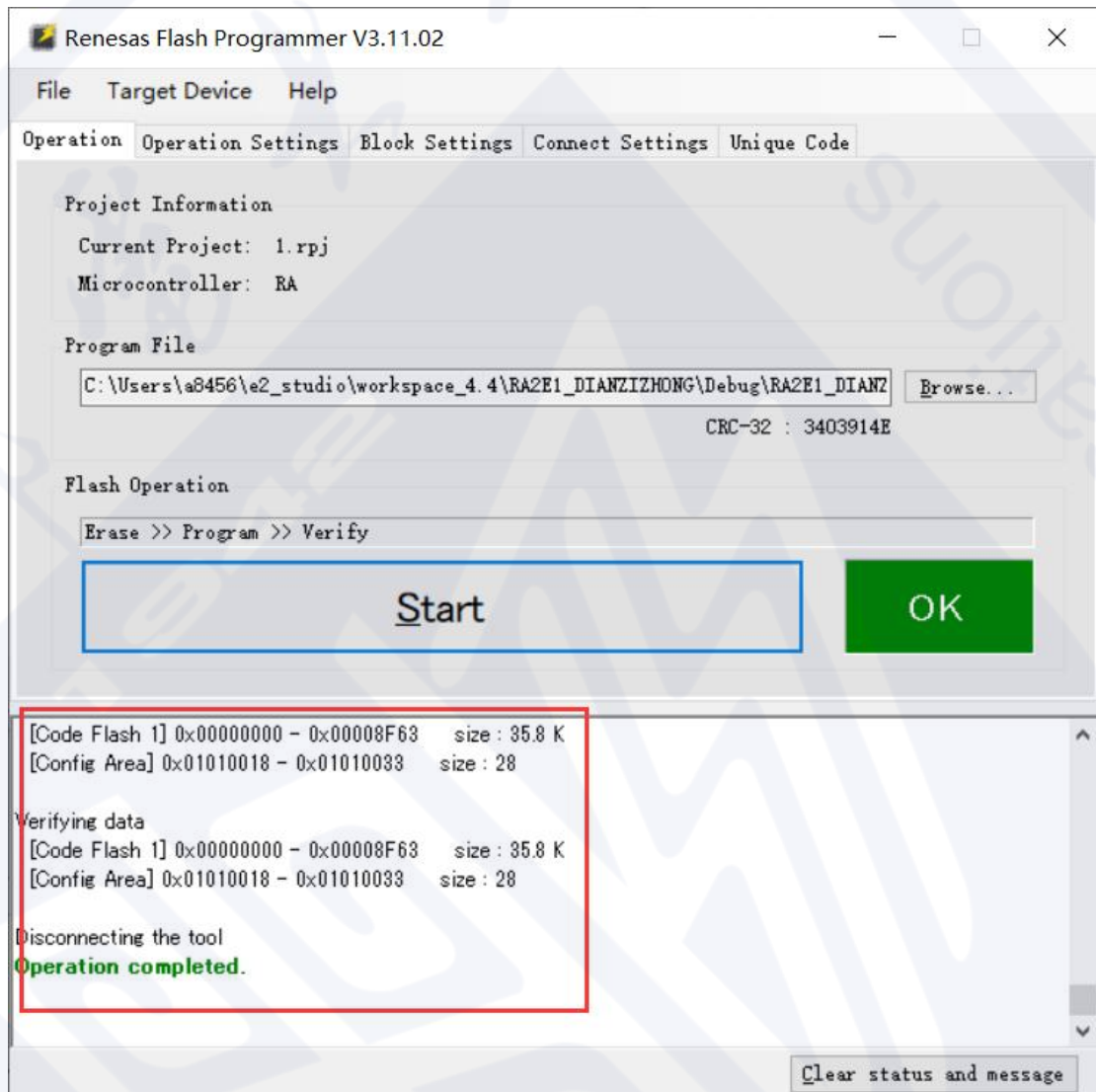
若出现下列报错,需要按一下复位按键或者断电在上电,在重新设置即可。



烧录文件可以选 hex，也可以选 srec，选择刚刚下载文件中的 RA4M2_MINI_DEMO.srec。



烧录成功如下所示。



演示

之后给板子断电，拨回如下状态，在进行上电。



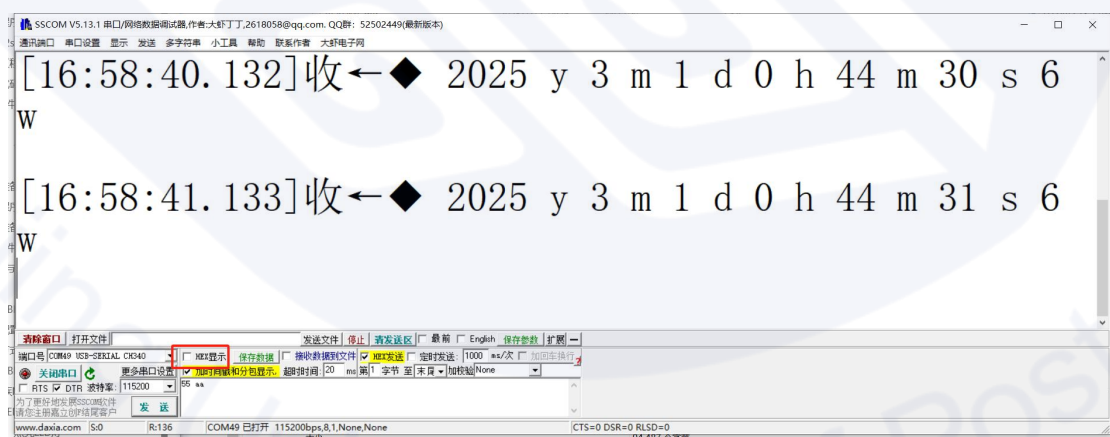
打开 SSCOM.



端口号选中带 CH340 的选项，波特率选择 115200，然后打开串口。



取消 HEX 显示。



然后点击复位按钮。



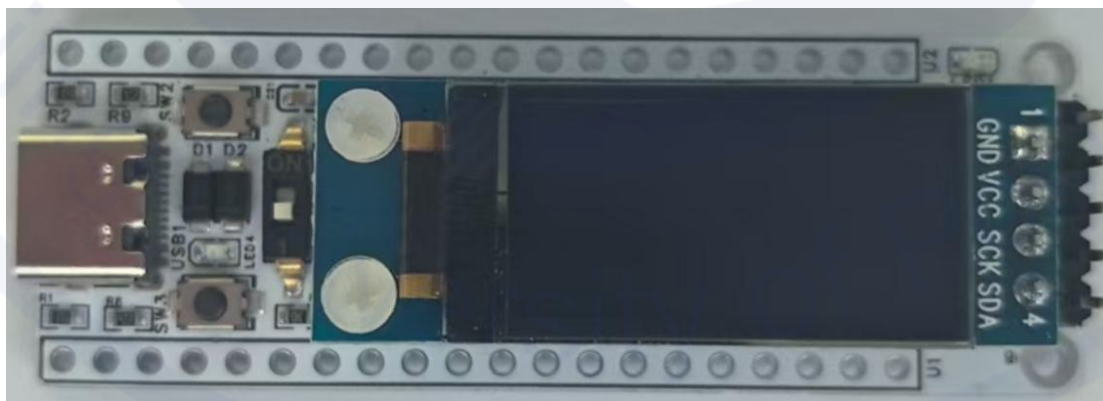
在 `sscom` 串口工具中可以查看到刚刚添加的代码。



project3: 使用 J-Link 或 EZ-CUBE3 烧录程序

概述

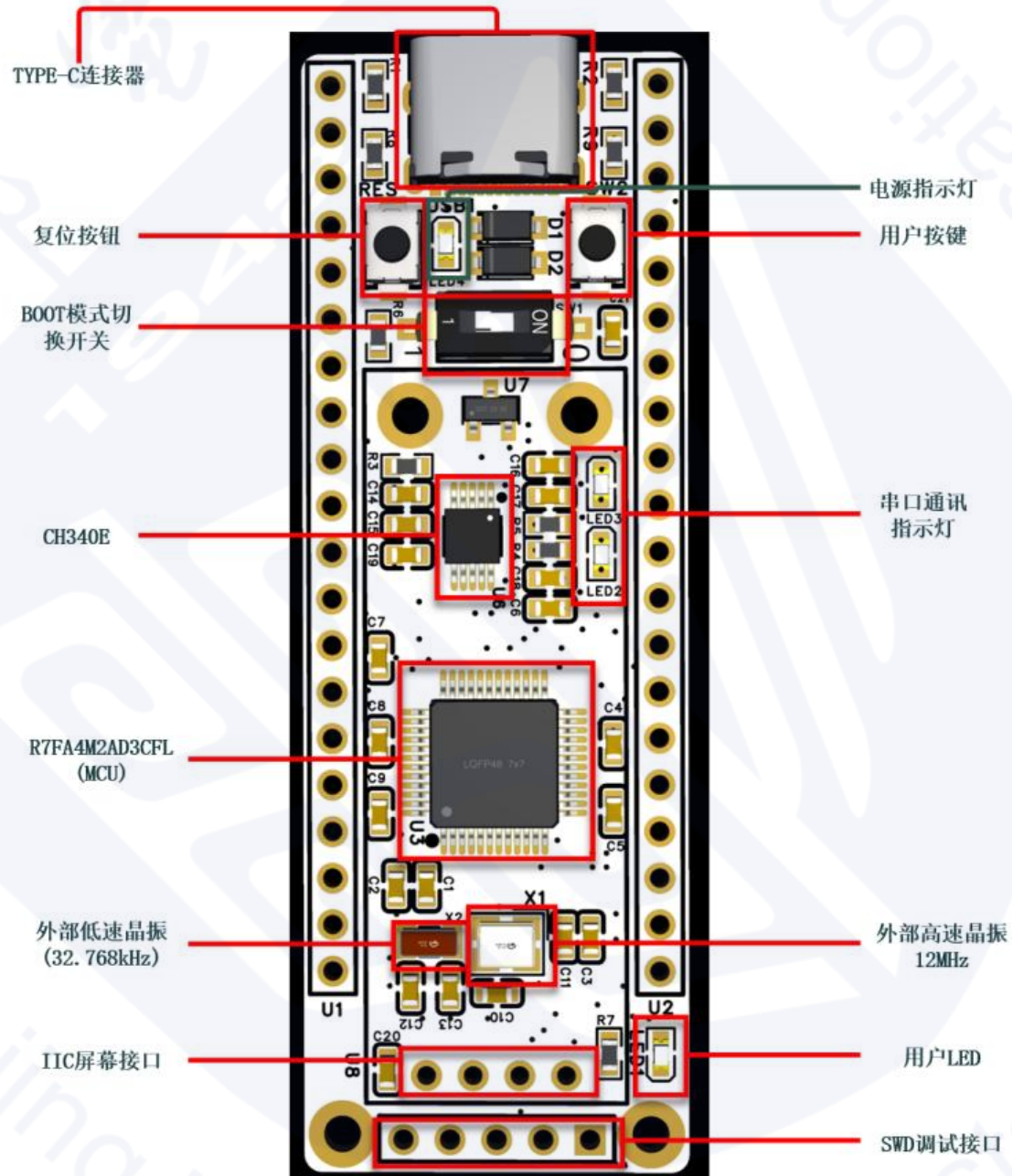
这一节主要讲解如何使用 J-Link 或 EZ-CUBE3 对瑞萨 RA 芯片进行烧录。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



参考程序

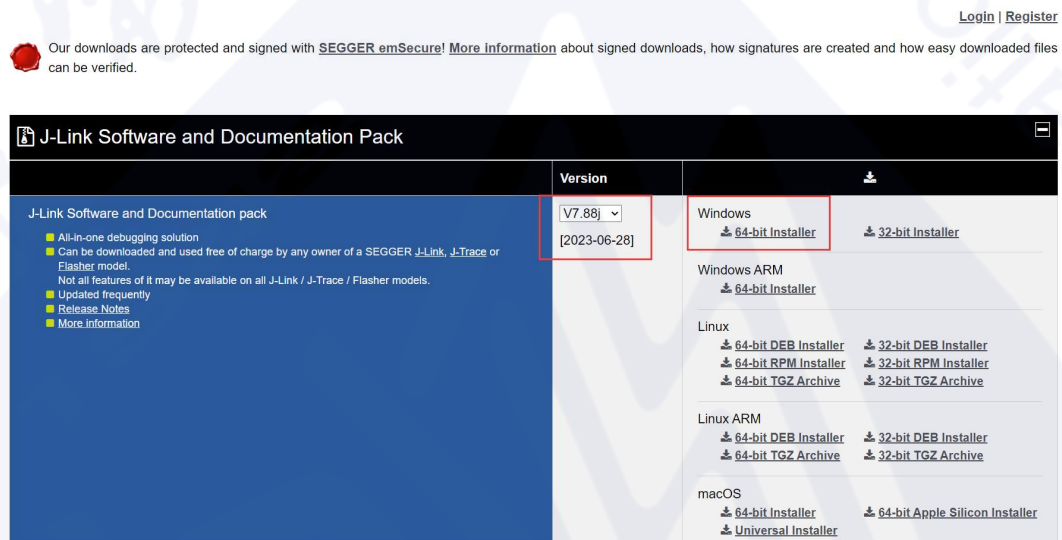
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

软件准备

由于瑞萨芯片是最近才出的芯片，所以旧版的 J-Link 软件不一定支持，所以需要下载较新的软件，同时 J-Link 需要 V9 以上。

下载地址：<https://www.segger.com/downloads/jlink/>



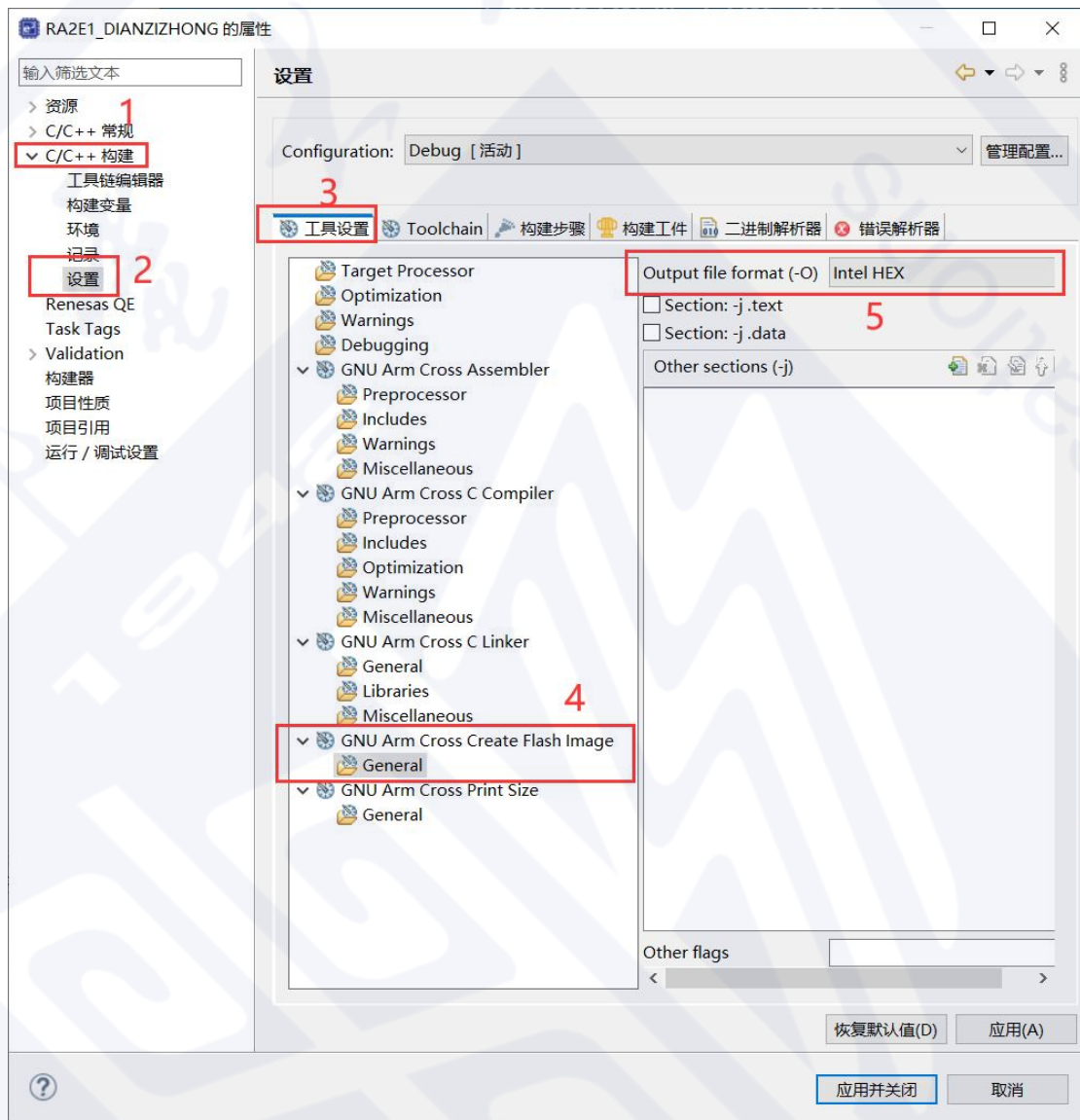
下载完成之后进行安装即可。

hex 文件准备

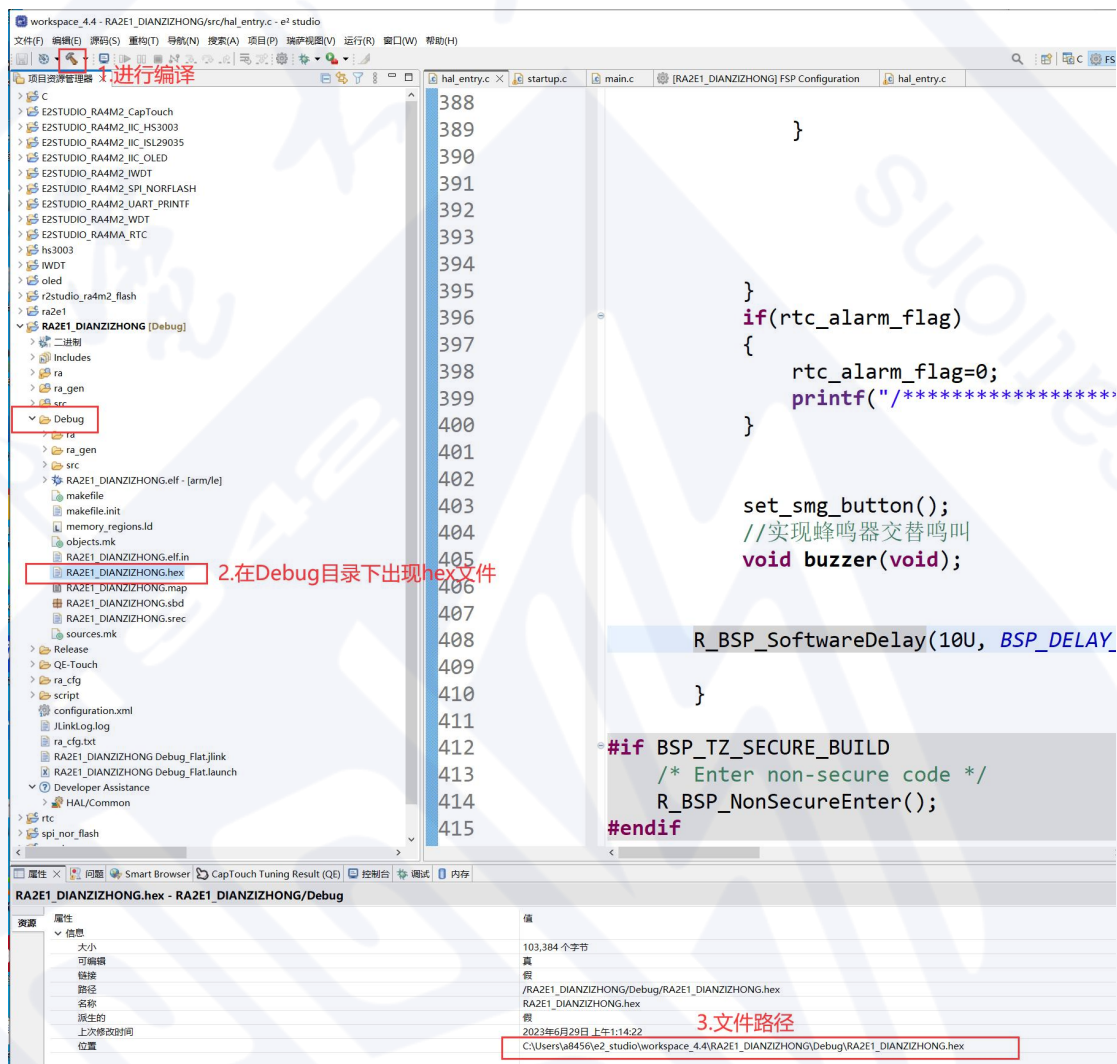
hex 的全称是 Intel HEX，此类文件通常用于传输将被存于 ROM 或者 EPROM 中的程序和数据。是由一行行符合 Intel HEX 文件格式的文本所构成的 ASCII 文本文件。大多数 EPROM 编程器或模拟器使用 Intel HEX 文件。HEX 文件记录由对应机器语言码和/或常量数据的十六进制编码数字组成。



之后选择 C/C++ 构建 -> 设置 -> 工具设置 -> GNU ARM Cross Create Flash Image->General, 在 Output file format 中选择 Intel HEX 进行输出 HEX 文件。

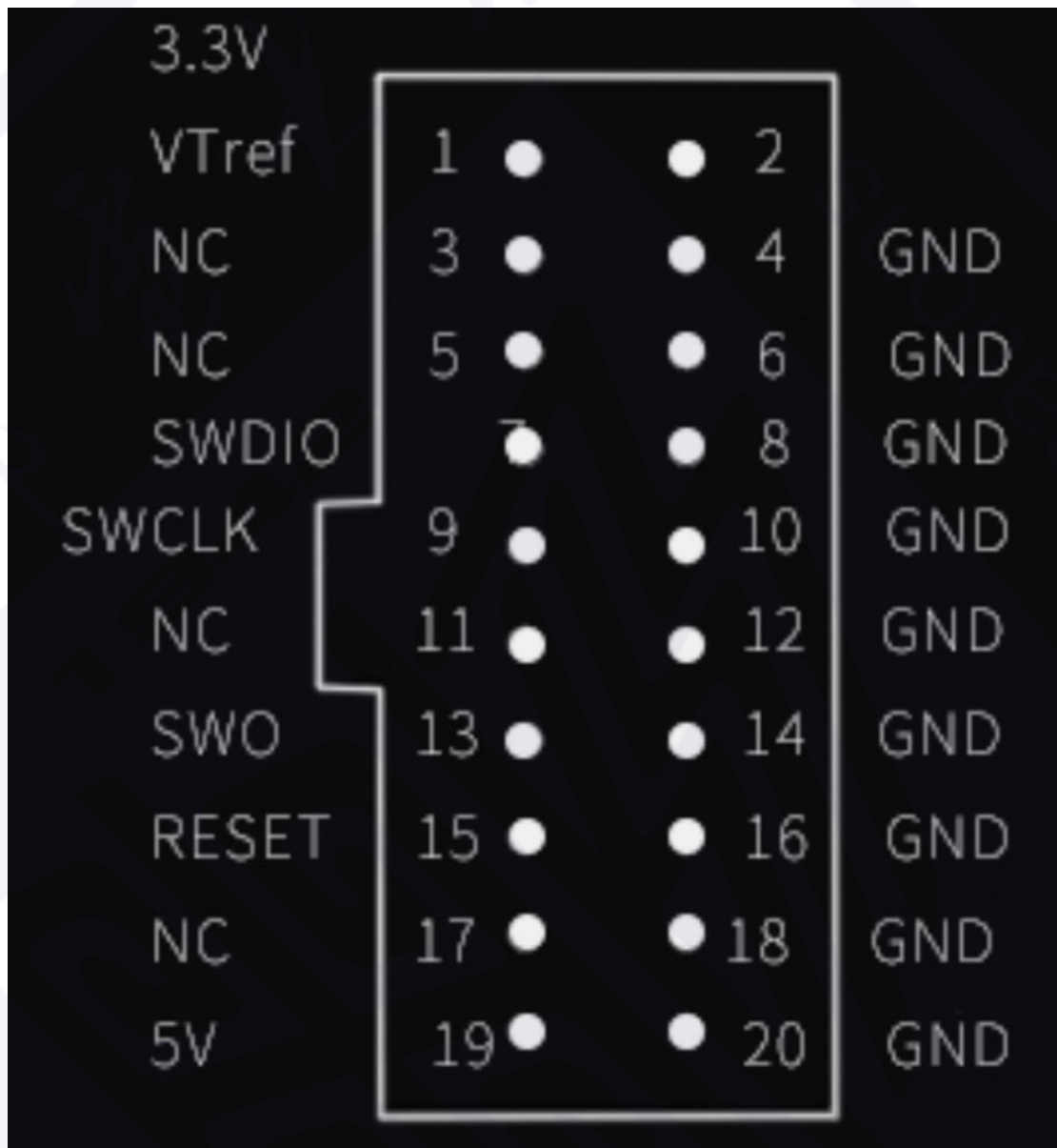


对程序进行重新编译,就会出现 hex 文件,文件一般在该项目的 Debug 文件夹中。



J-Link 与瑞萨开发板进行 SWD 方式接线

J-Link 接口如下所示。



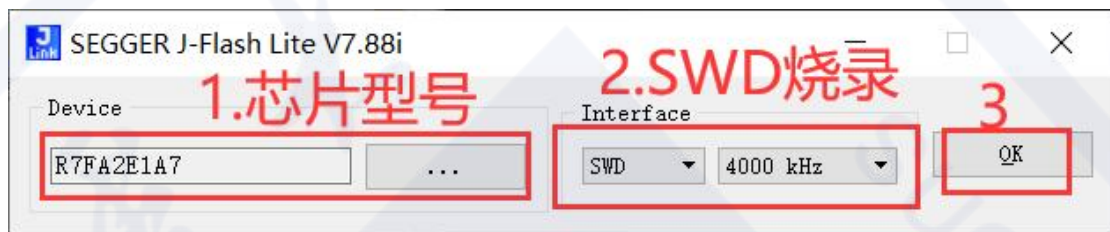
J-Link	开发板
2pin->3.3V	VCC
4pin->GND	GND
7pin->SWDIO	SWDIO
9pin->SWCLK	SWCLK

烧录

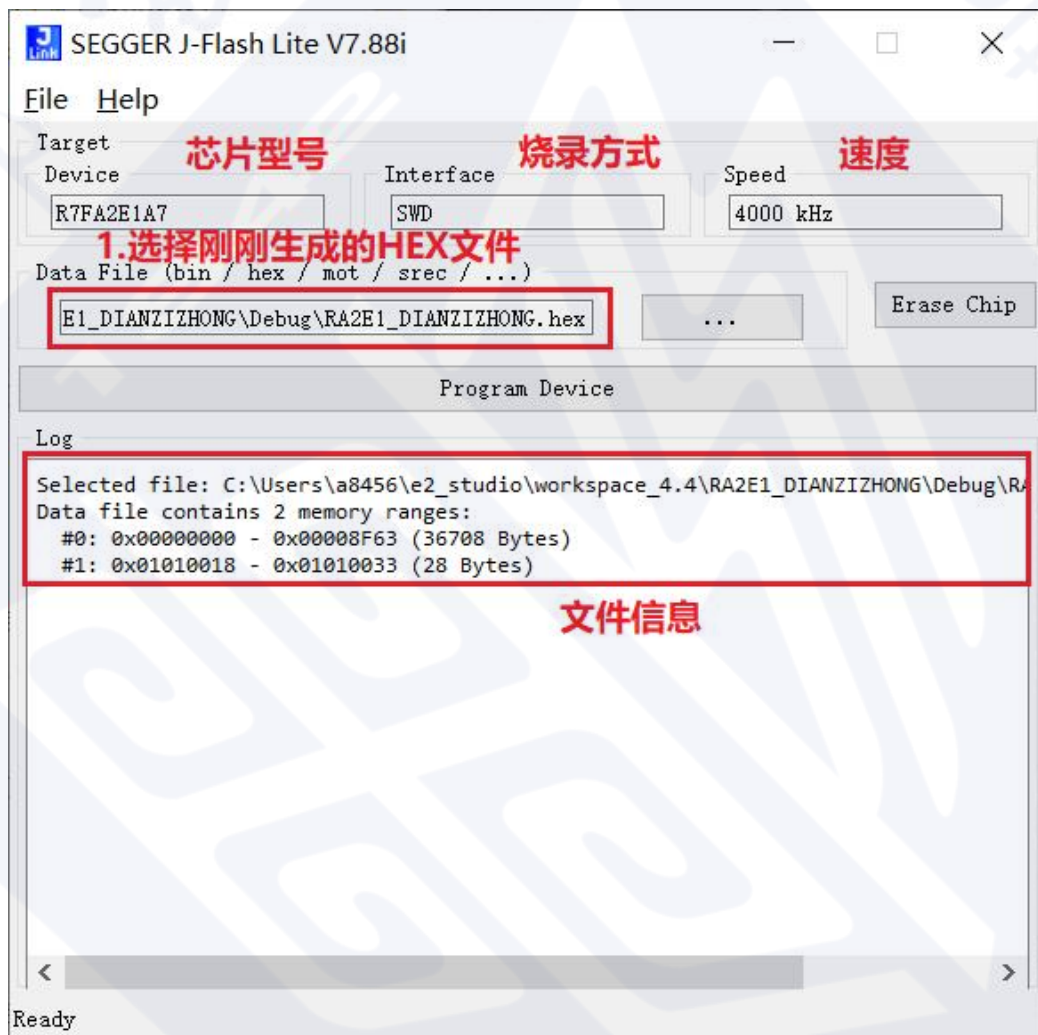
打开 J-Flash Lite 进行烧录。



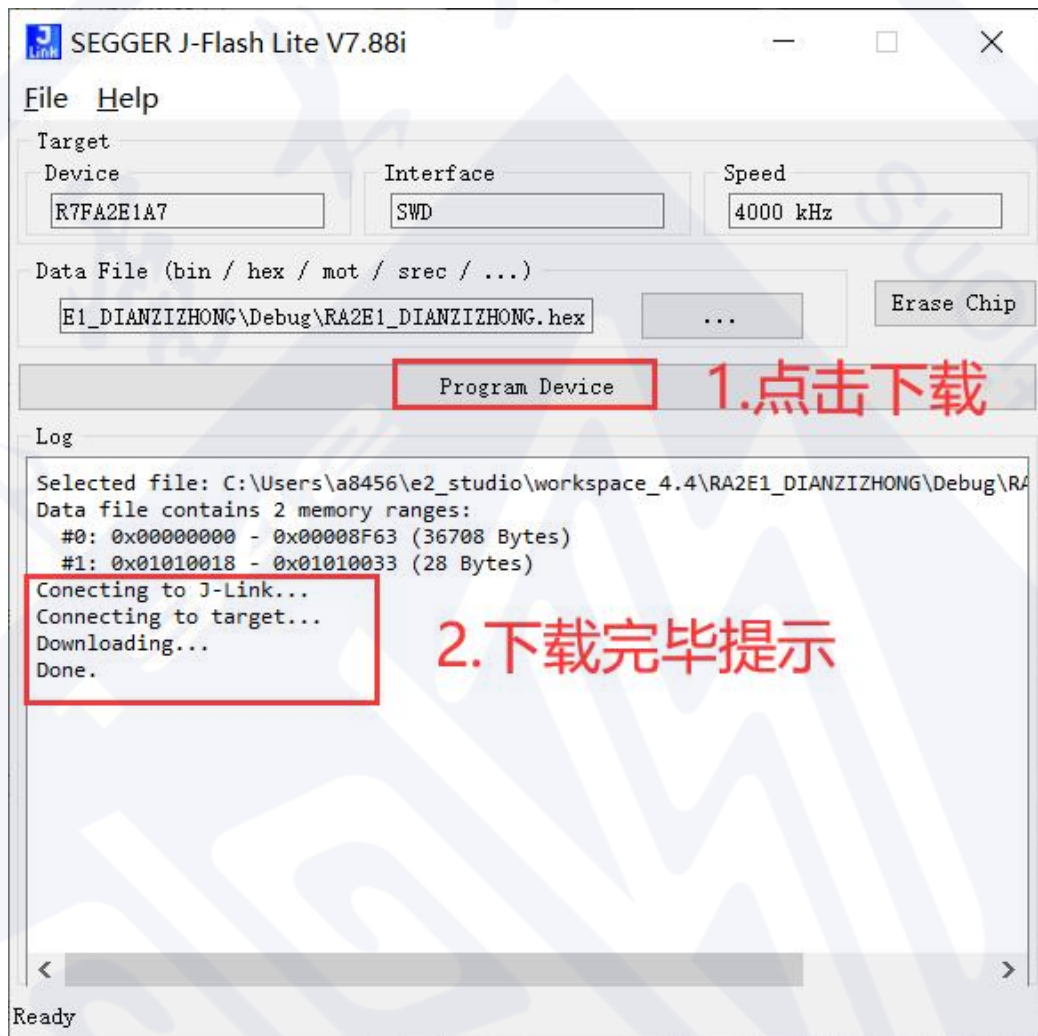
将芯片型号以及烧录方式选择好。



选择烧录文件。



HEX 文件下载。



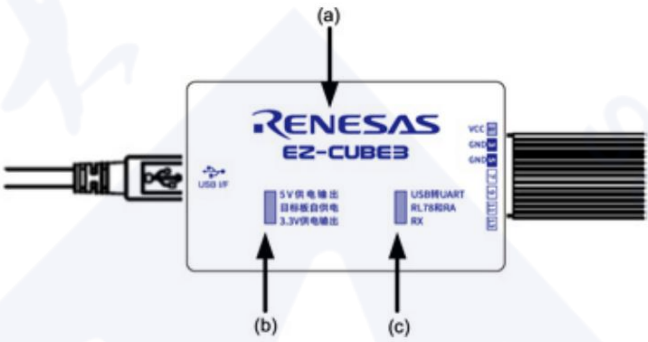
之后对开发板进行断电或者复位即可查看效果。

E2-CUBE3 烧录

EZ-CUBE3 (CYRNEZCUBE03) 是具有 Flash 存储器编程功能的片上调试仿真器，可以用于调试 MCU 程序或将程序烧写到 MCU 的内置 Flash 中。

EZ-CUBE3 支持 RA、RL78 和 RX 产品家族 MCU。

各部件名称



(a) LOGO	在主体表面上清楚地说明了 RENESAS EZ-CUBE3
(b) SW1	目标板供电模式选择开关 5V供电输出: EZ-CUBE3向目标板供电5V/200mA 目标板自供电: 目标板自供电 (2.7 ~ 5.5V) * 3.3V供电输出: EZ-CUBE3向目标板供电3.3V/200mA * 此模式下 EZ-CUBE3 的运行电压由目标板决定
(c) SW2	EZ-CUBE3调试模式选择开关 USB转UART RL78和RA: 调试RL78族和RA族MCU RX: 调试RX族MCU

引脚配置

下表列出了引脚的使用情况。

表 5.1 引脚的使用情况（RA）

引脚序号	EZ-CUBE3 接口引脚定义	连接到 RA 族 MCU 时用到的引脚	
		SWD	SWD+串口
1	VCC	√	√
2	SWDIO	√	√
3	GND	√	√
4	SWCLK/MD	√	√
5	GND	√	√
6	RA TXD9		√
7	—		
8	RA RXD9		√
9	—		
10	RESET	√	√
11	—		
12	—		
13	—		
14	—		

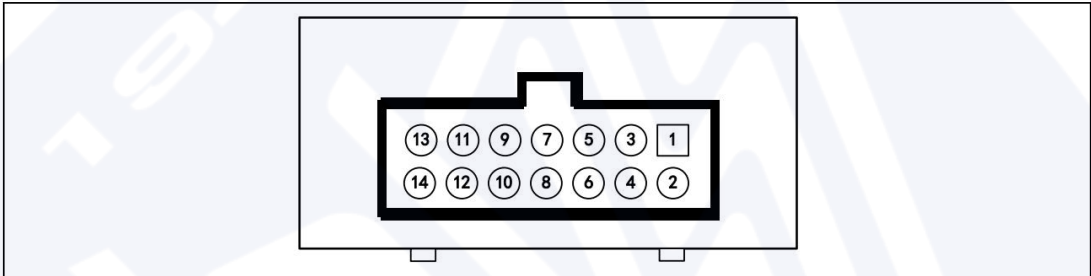


图 5.2 14 引脚接口中引脚的序号和排列（EZ-CUBE3 侧视图）

接线方式

RA0E1 开发板	EZ-CUBE3
3V3	VCC(1)
GND	GND(3)
NRST	RESET(10)
SWDIO	P108/SWDIO(2)
SWCLK	P300/SWCLK(4)

EZ-CUBE3 仿真器开关设置

对于烧录瑞萨 RA，拨动开关配置到如下所示。

根据系统要求
选择供电模式

☐

5V供电输出

☐

目标板自供电

☒

3.3V供电输出

☐

USB转UART

☒

RL78和RA

☐

RX

调试 RA 族 MCU

对 RA 族 MCU 进行 Flash 编程

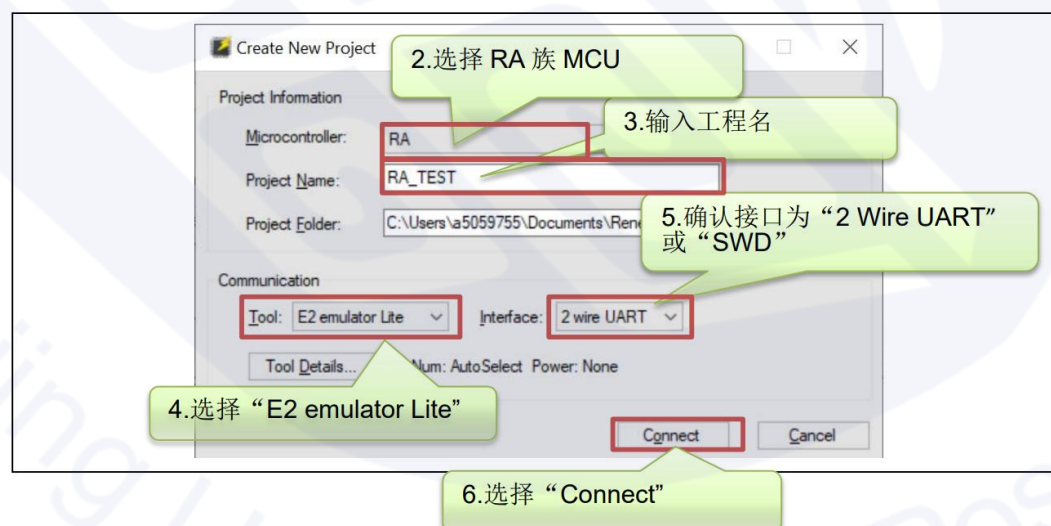
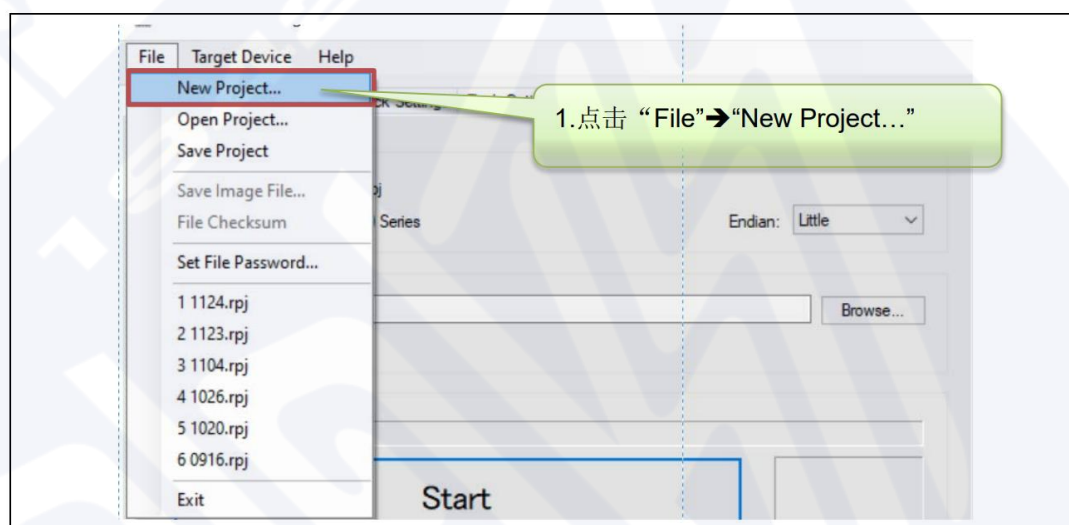
本节将介绍使用 EZ-CUBE3 对 RA 微控制器的 Flash 进行编程时的系统配置以及启动/关闭顺序。

这里使用 Renesas Flash Programmer 软件进行下载。

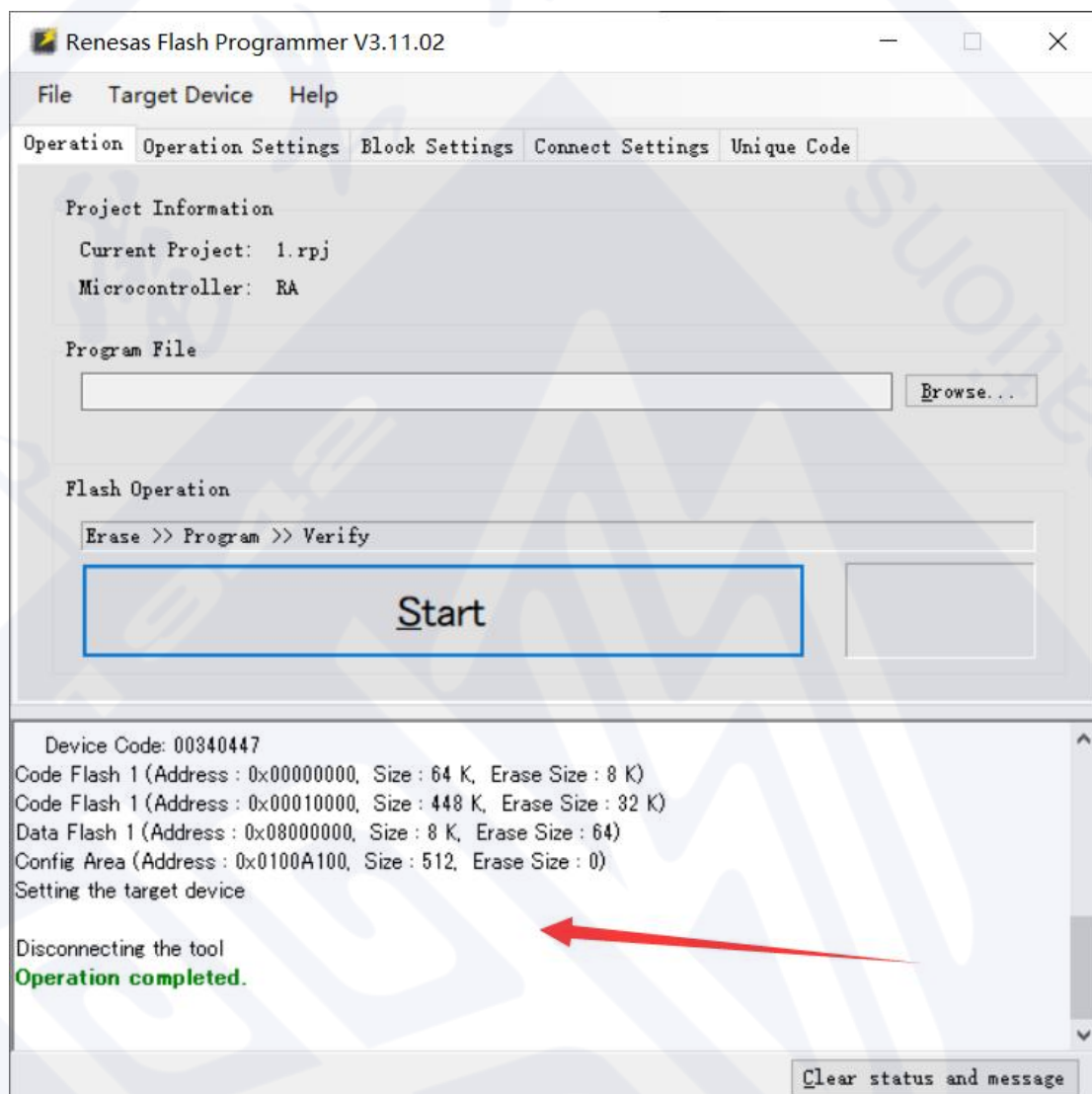
下载链接：

<https://www.renesas.cn/zh/software-tool/renesas-flash-programmer-programming-gui>

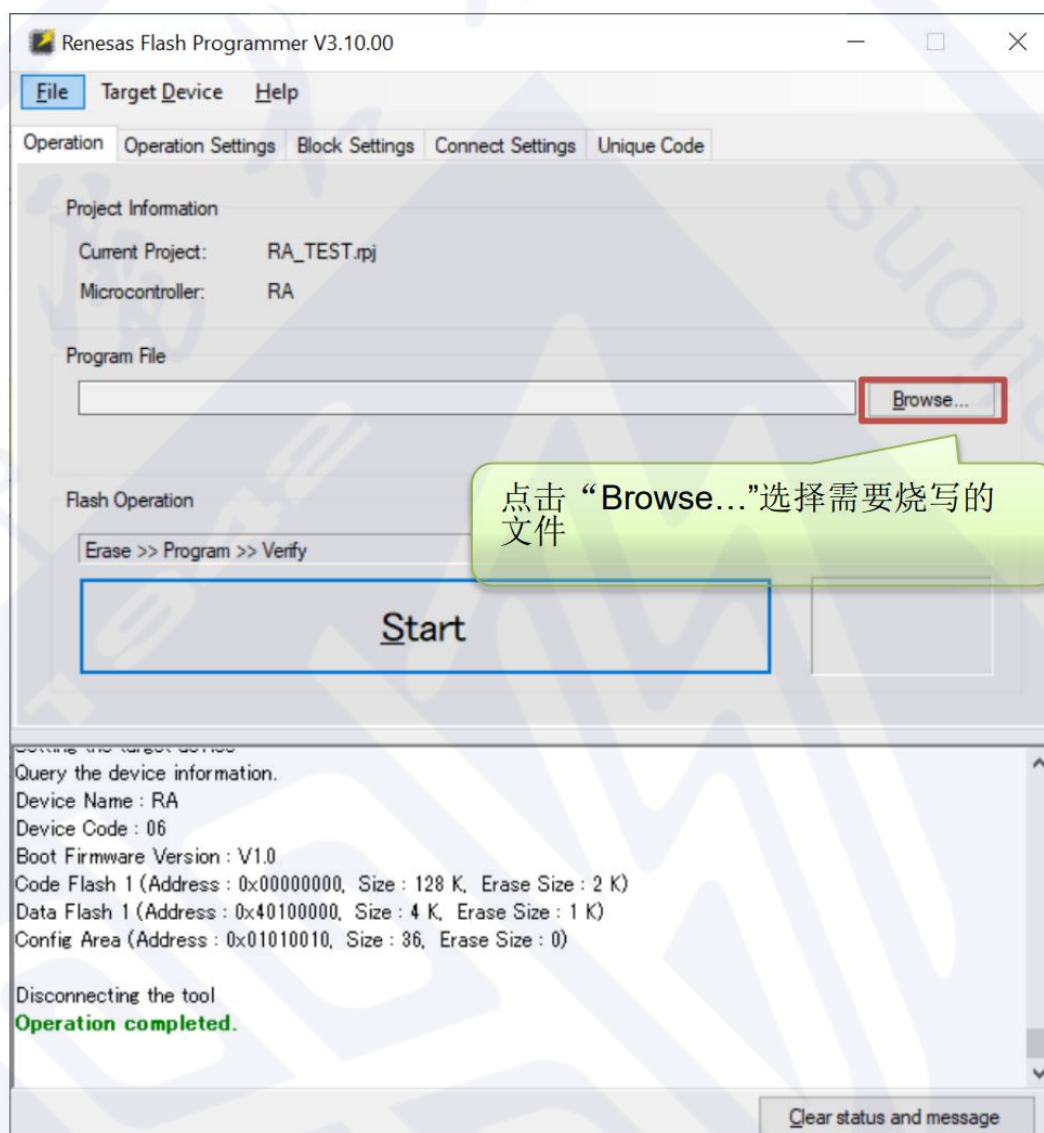
创建 RA 新工程



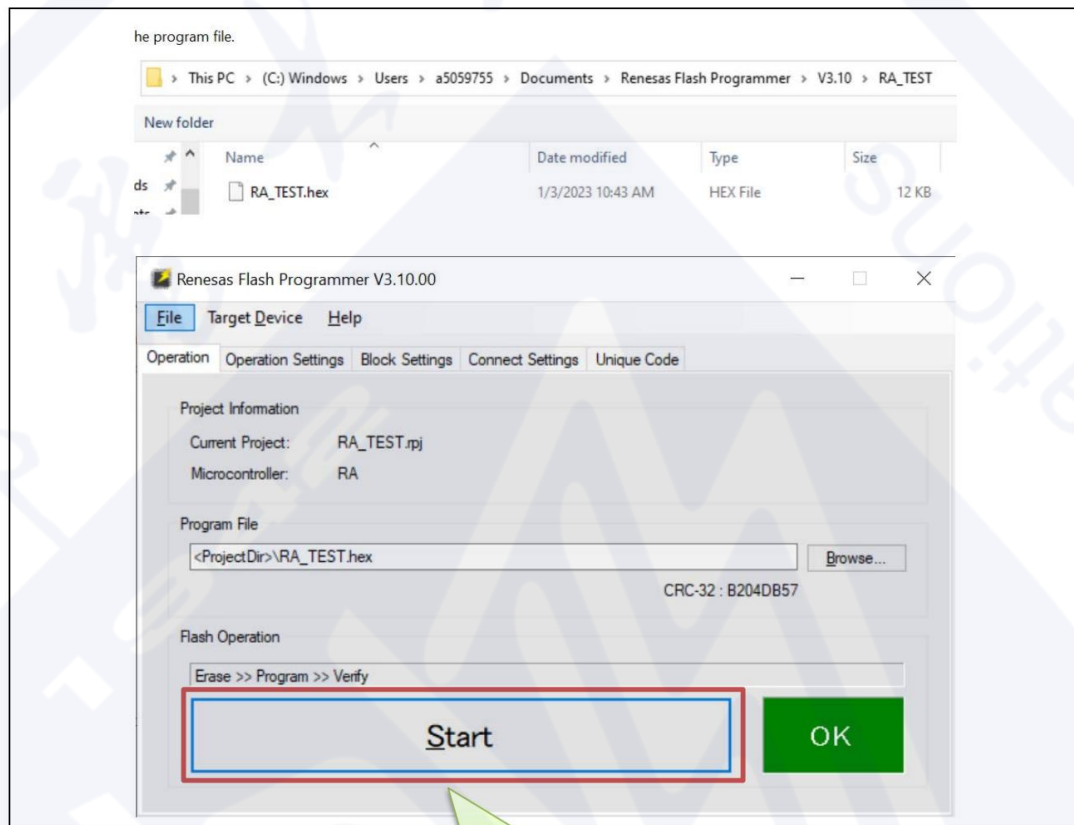
Renesas Flash Programmer 软件与 EZ-CUBE3 以及目标板正确连接后，会显示如下信息。



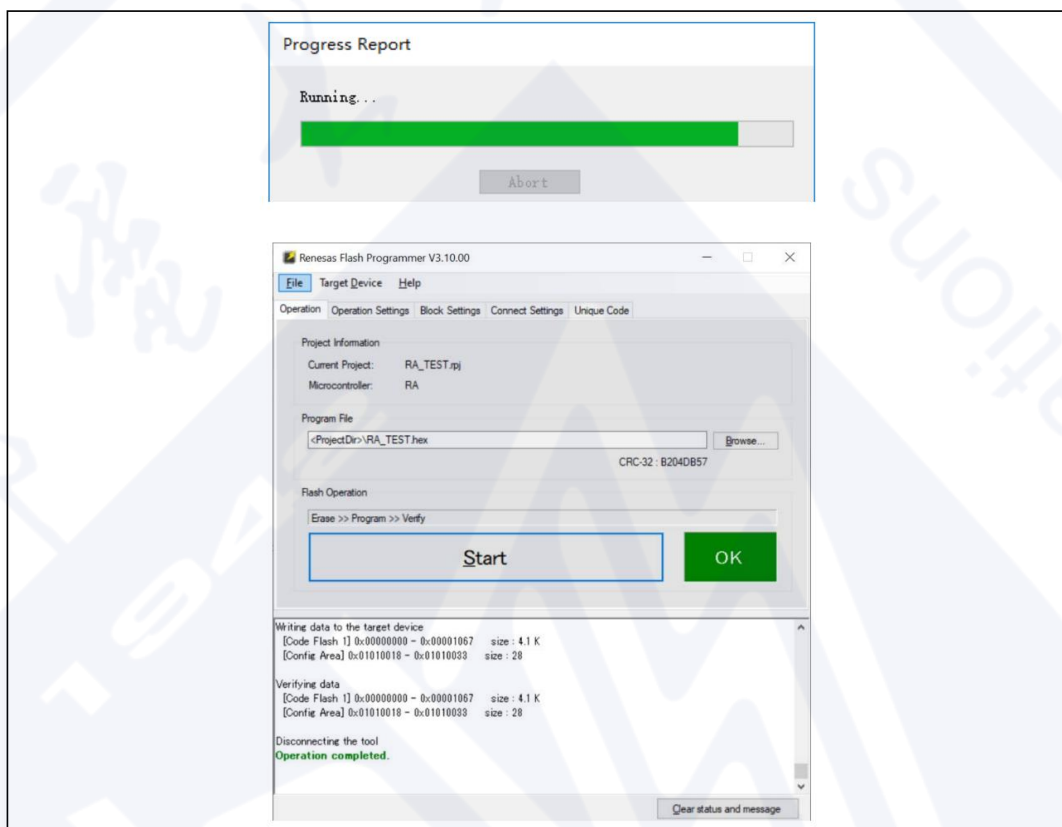
选择需要烧录的文件。



选择需要烧写的文件后，点击“Start”开始对目标板进行编程操作。



编程操作结束后软件会显示编程结果。
若出现烧录失败，建议把复位也接上。



蓝色 LED 指示灯的状态信息

蓝色 LED 指示灯每 1 秒闪烁一次，表明 EZ-CUBE3 上电，固件运行正常，等待设备连接。

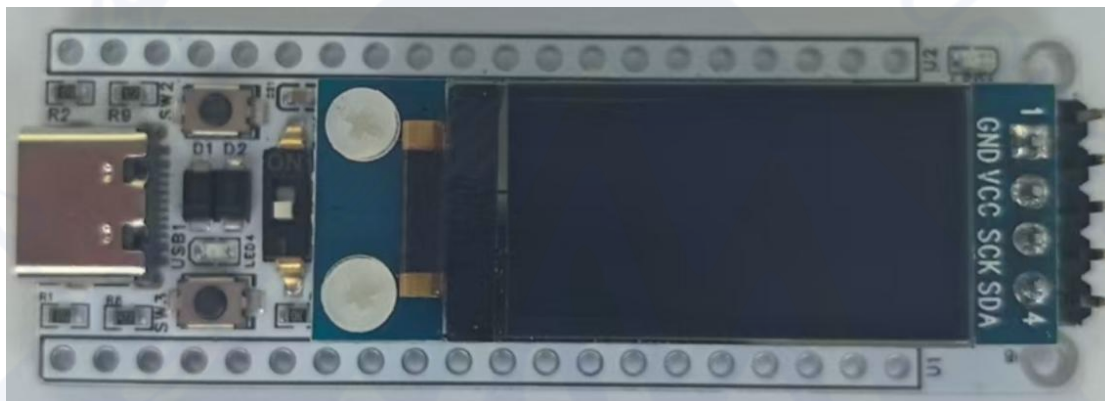
蓝色 LED 指示灯保持常亮，表示连接目标板成功。

蓝色 LED 指示灯快速闪烁，表示连接过程中出错，或出现通讯错误。此时，请断开与 PC 的连接后，重新连接。

project4: 点亮 LED 灯

概述

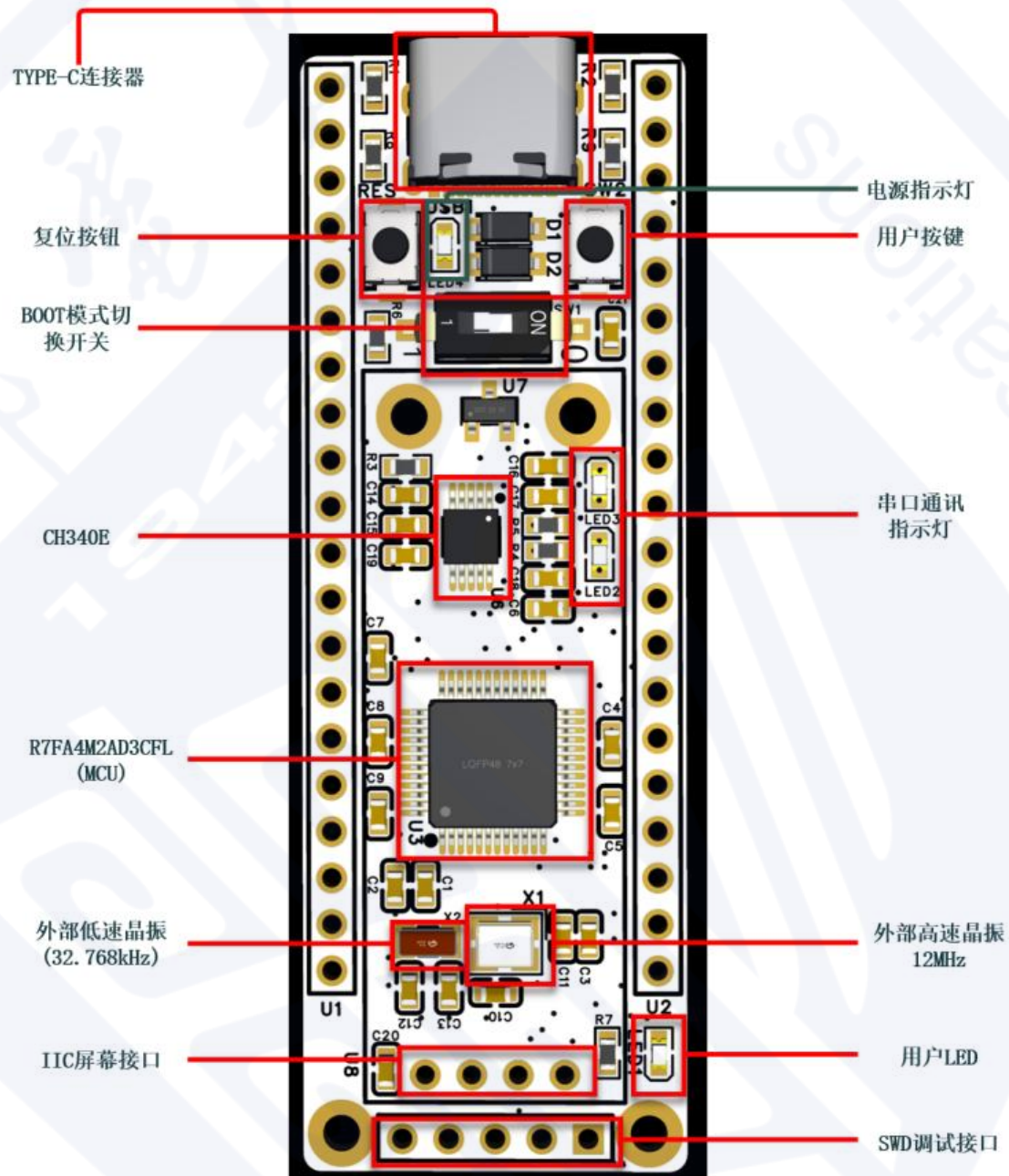
点亮 LED 是 RA4M2 系列开发的“Hello World”。通过创建一个最小工程，让板载 LED 亮起来。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

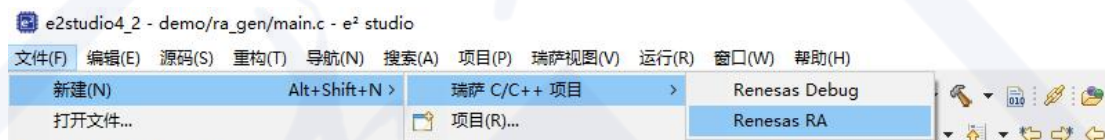


参考程序

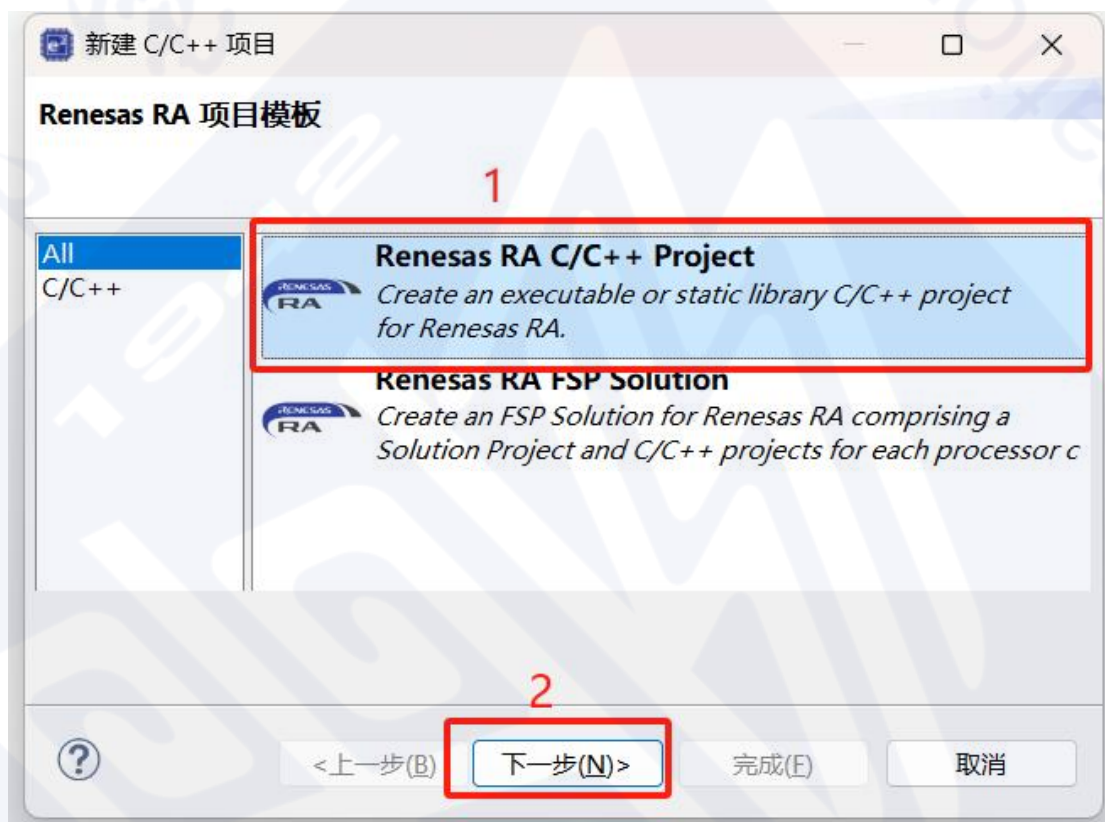
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

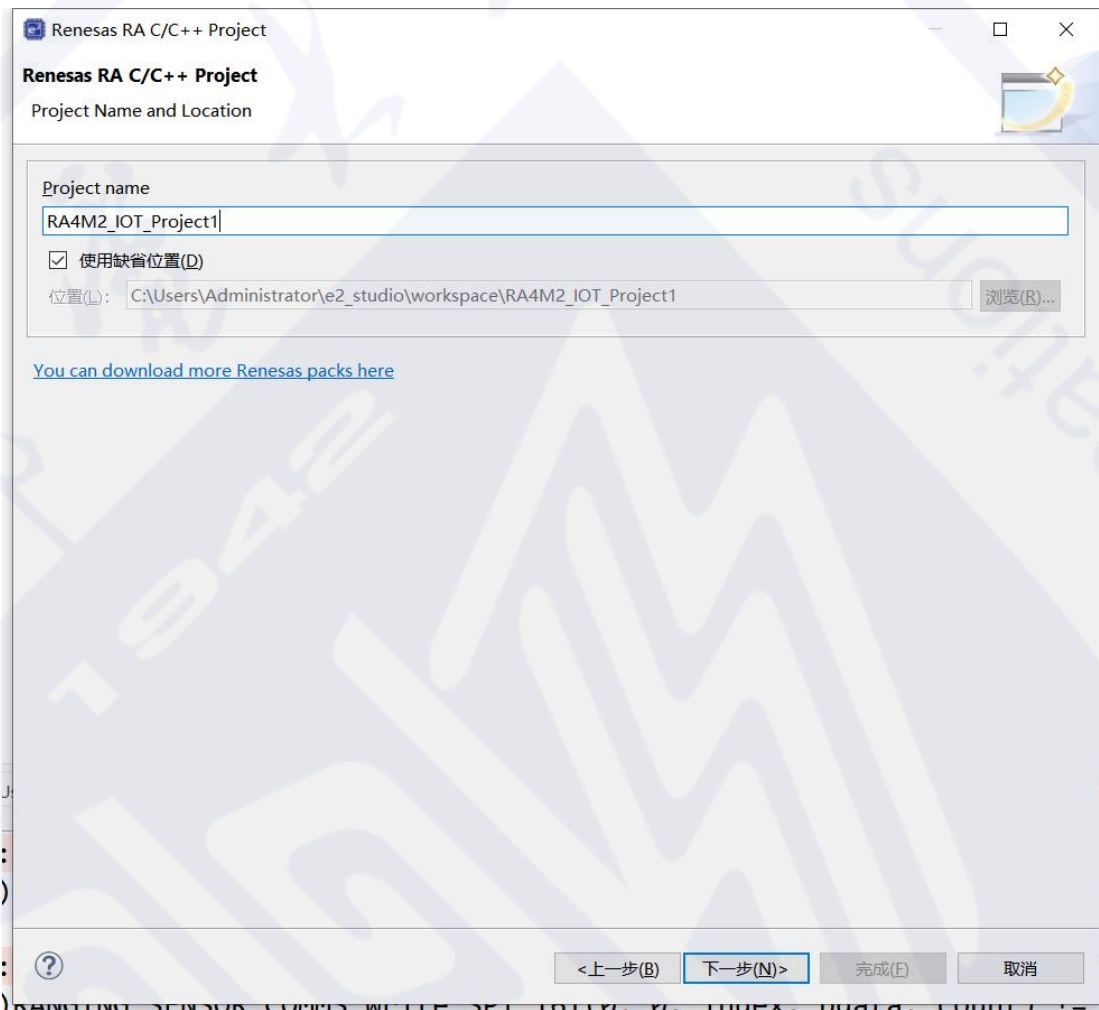
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

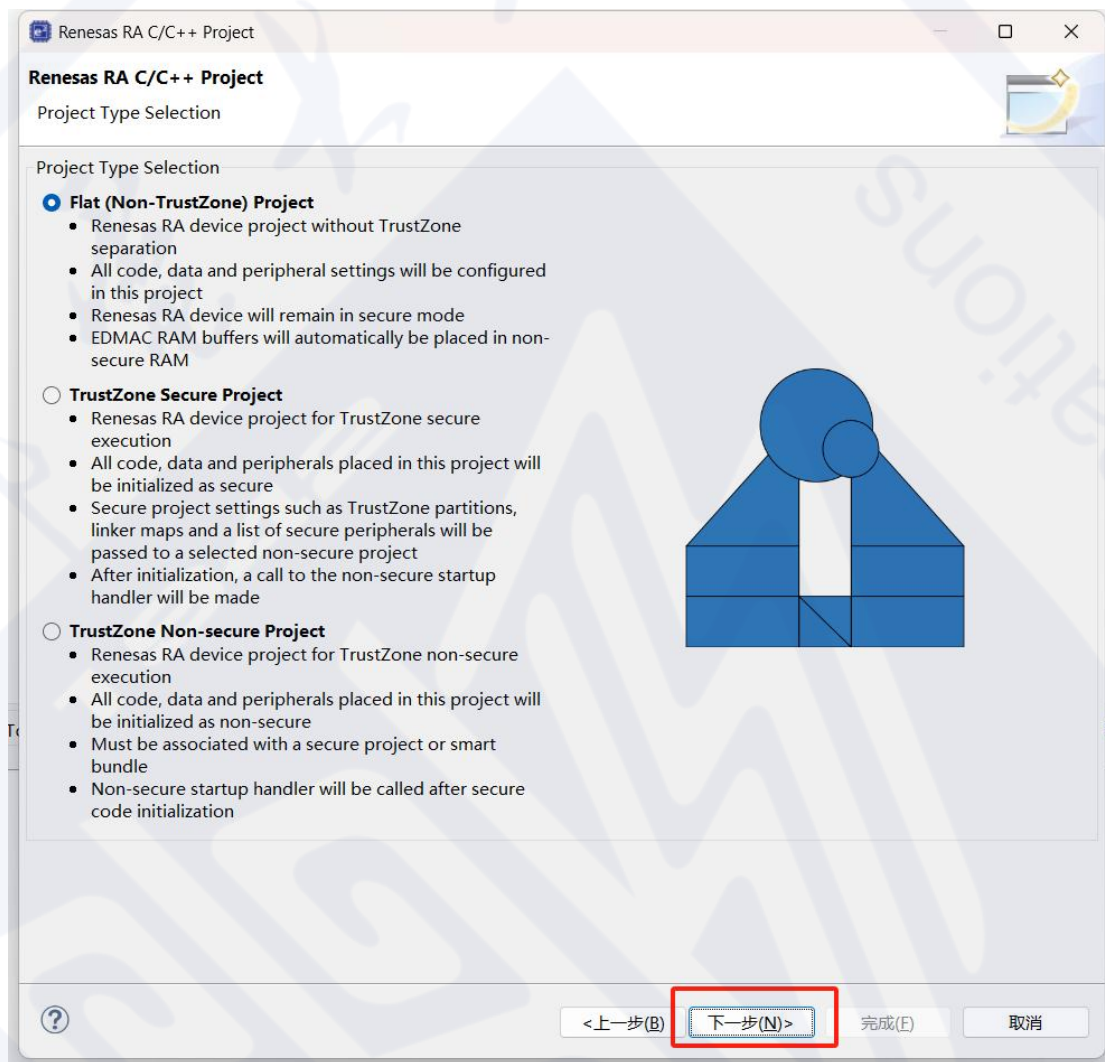
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

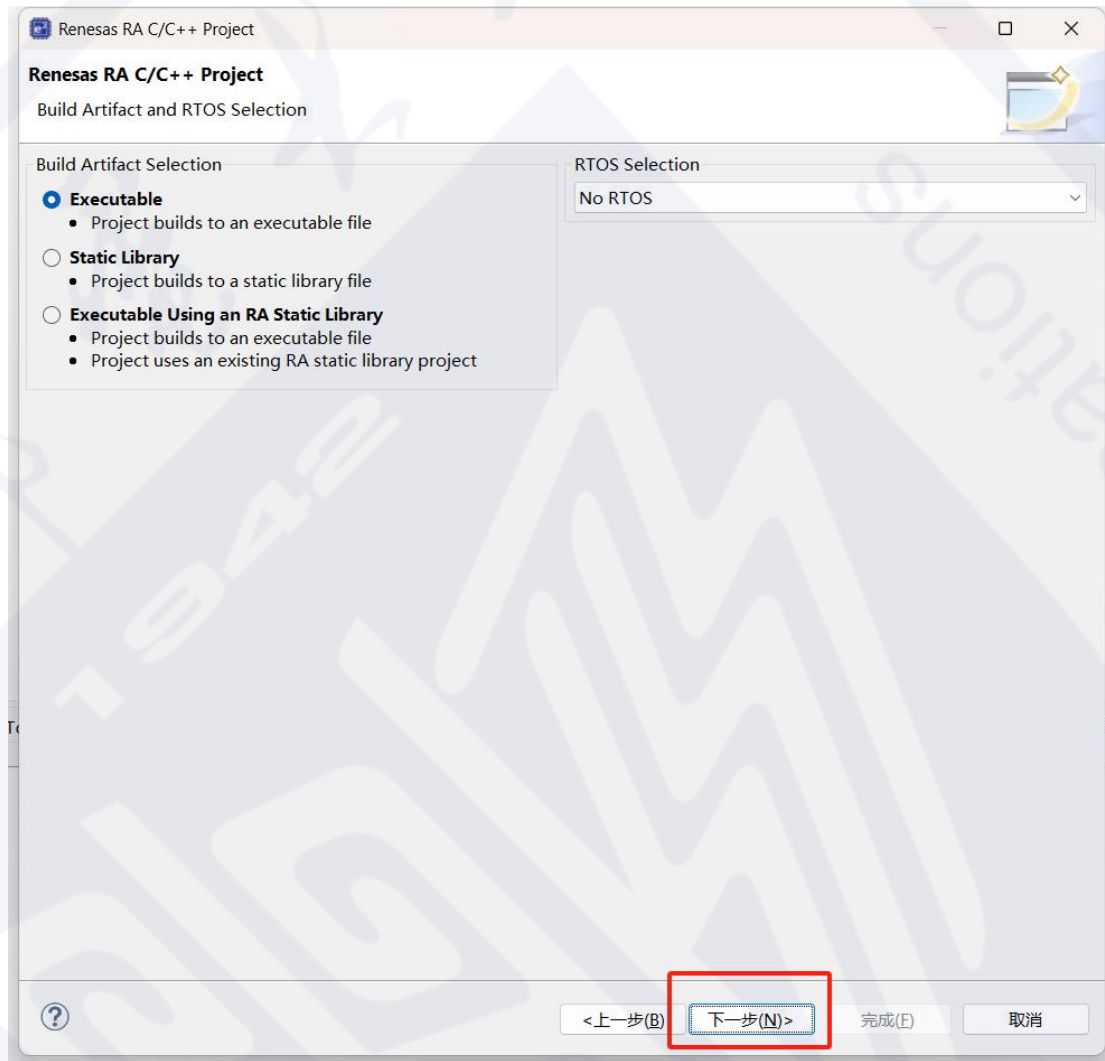
?

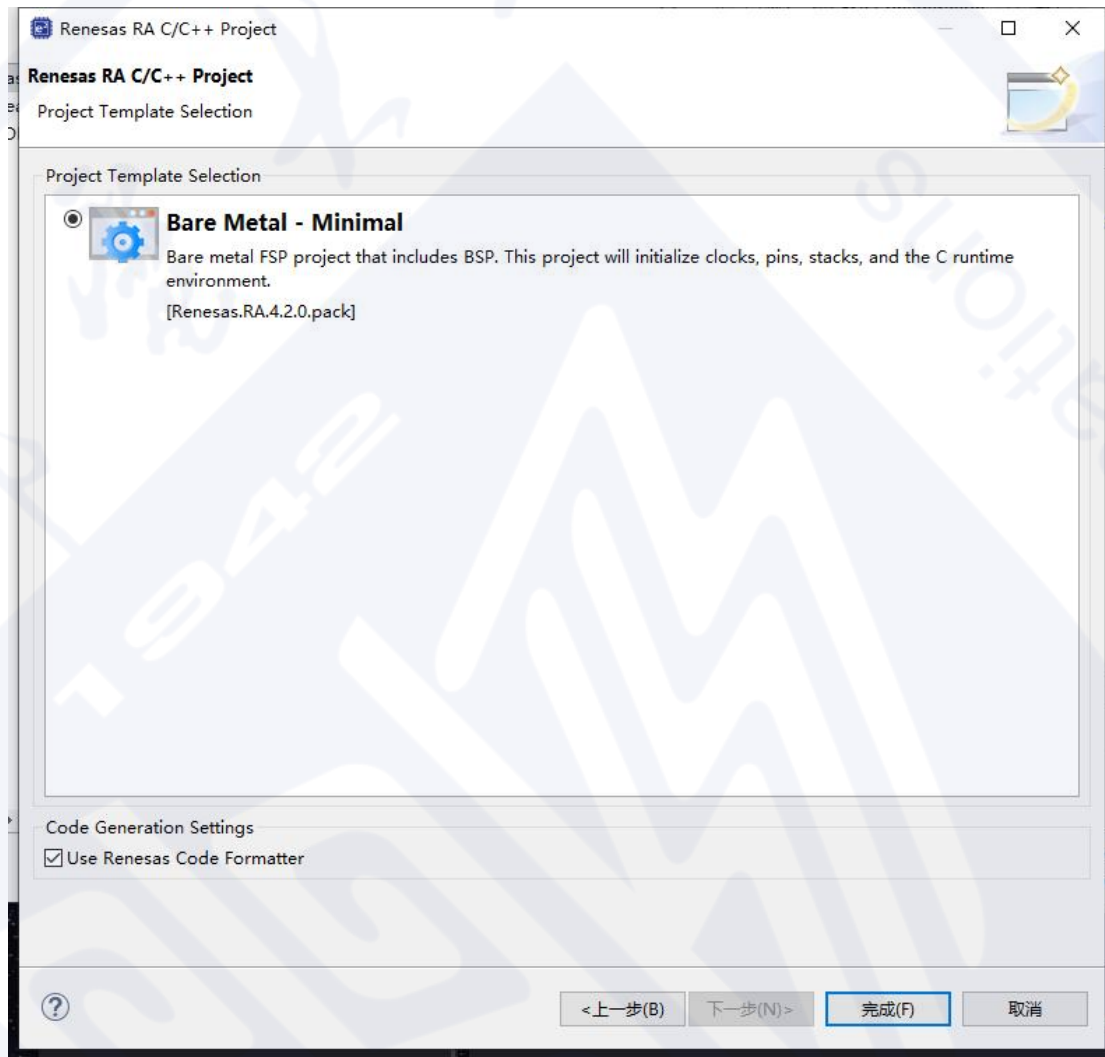
< 上一步(B)

下一步(N) >

完成(E)

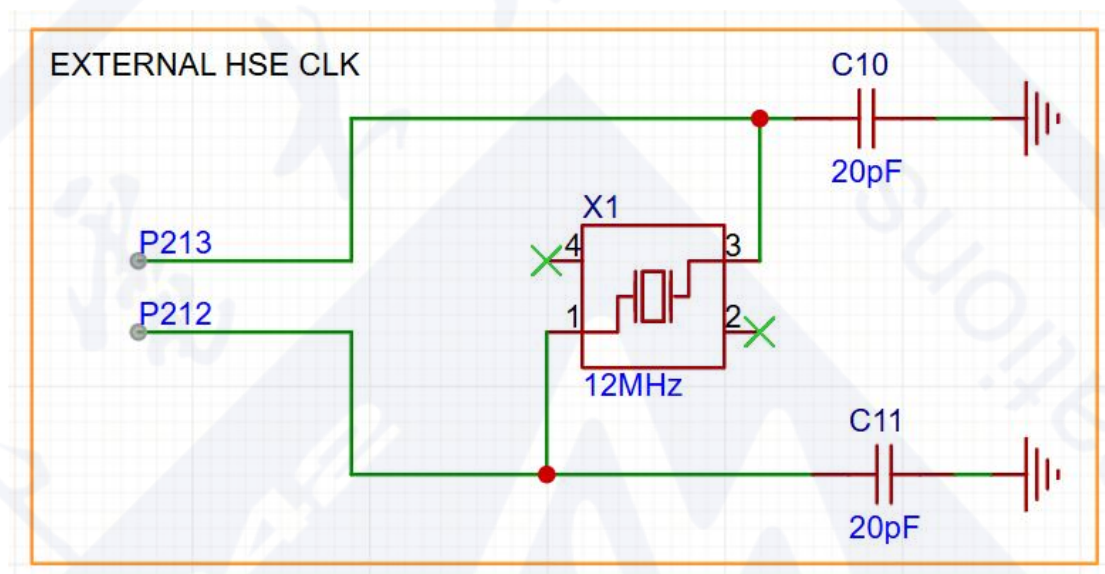
取消



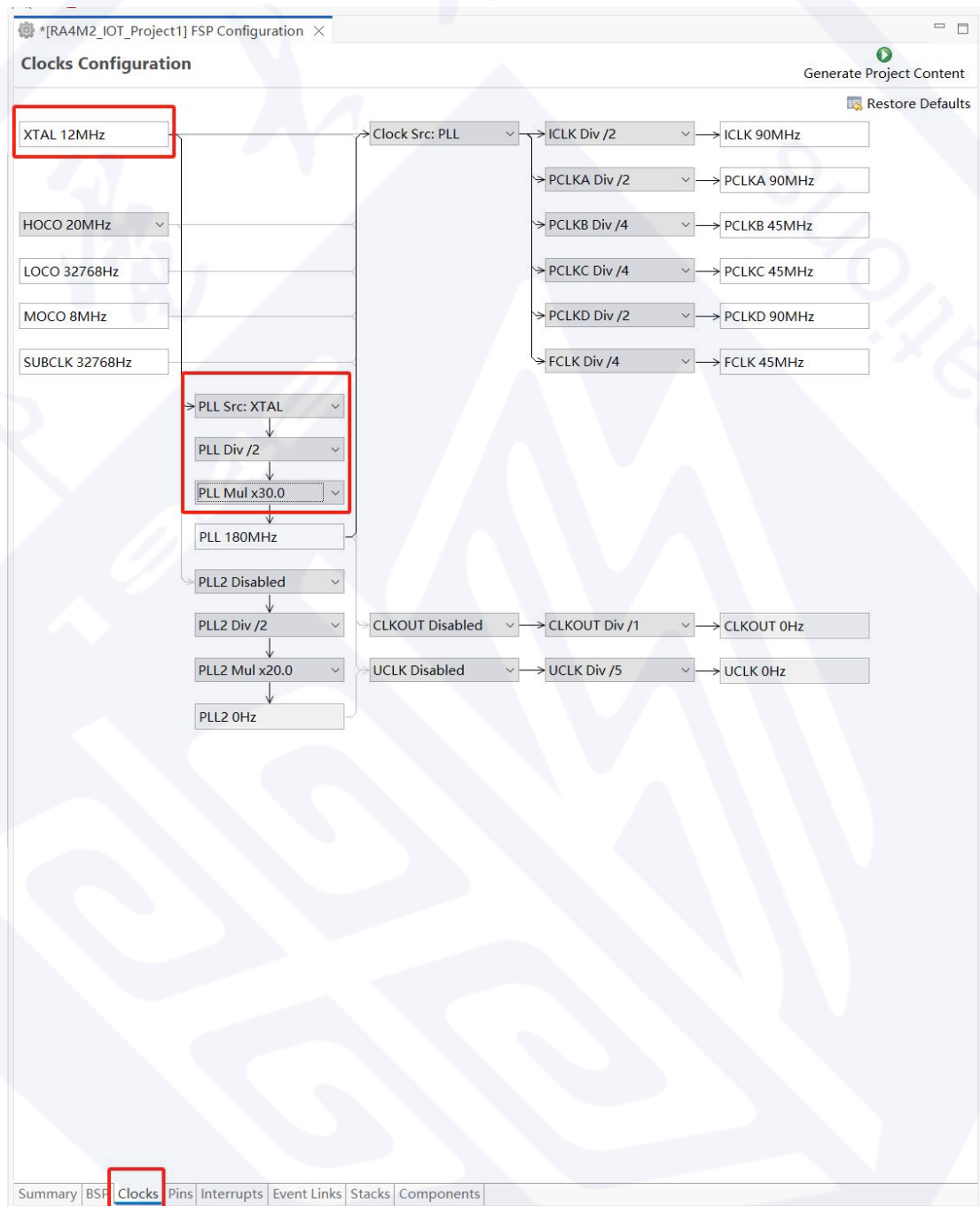


时钟设置

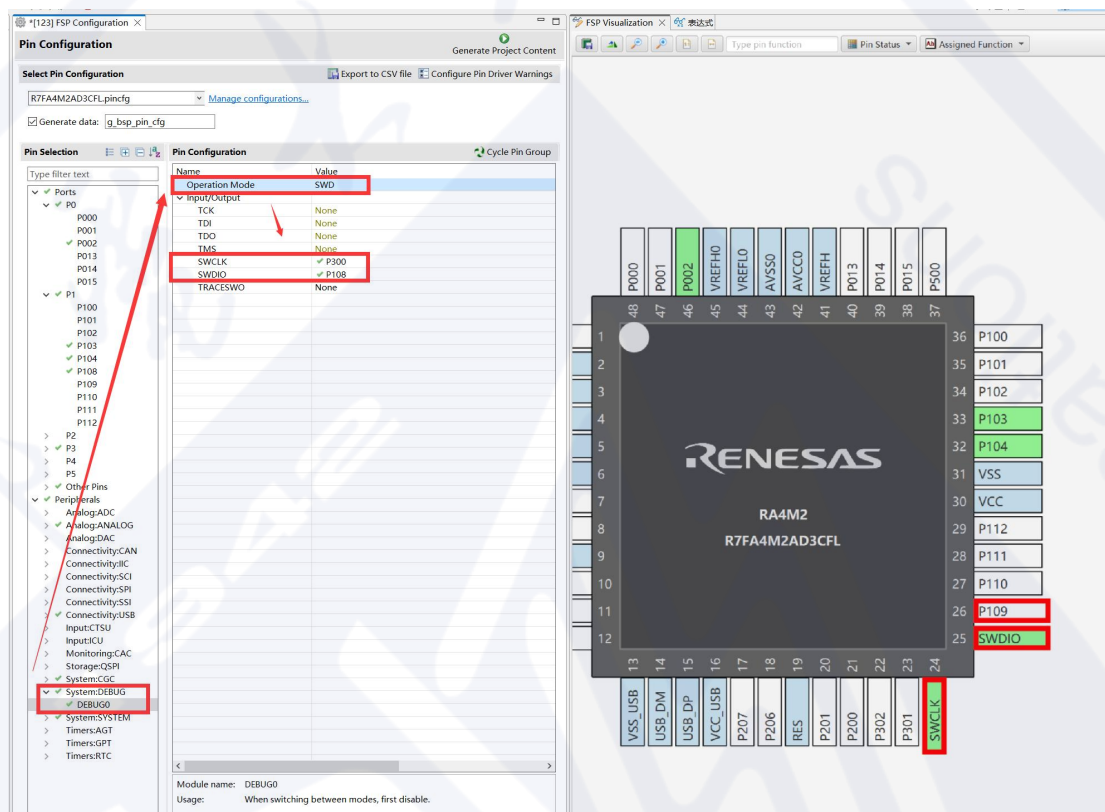
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

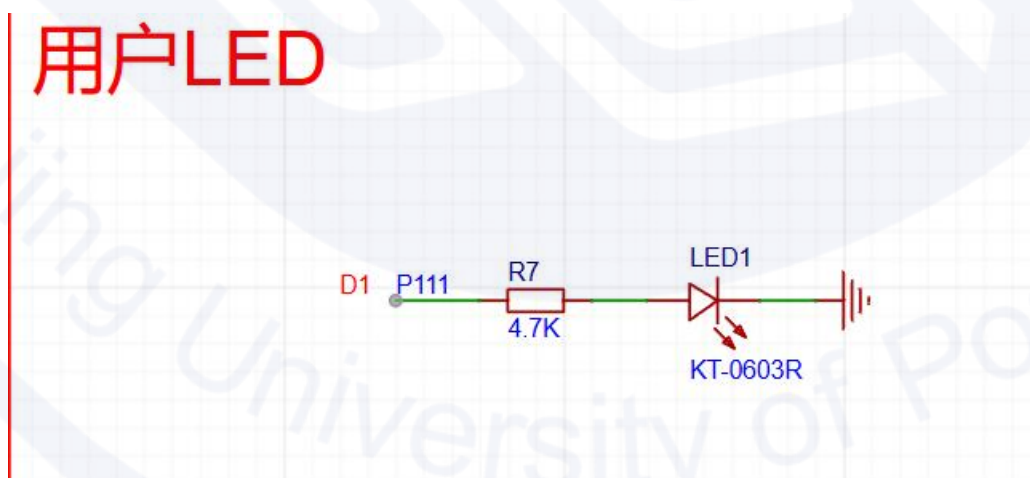


SWD 调试口设置



GPIO 口配置

由下图我们可以得知，板子上有 1 个 LED 灯，同时需要给高电平才可以点亮，故以 P111 管脚为例，配置方式如下所示。



在 e2studio 中可以直接设置 P111 为输出口。

*[RA4M2_MINI_Project2] FSP Configuration X

Pin Configuration

Generate Project Content

Select Pin Configuration [Export to CSV file](#) [Configure Pin Driver Warnings](#)

R7FA4M2AD3CFL.pincfg [Manage configurations...](#) ☒ Generate data: g_bsp_pin_cfg

Pin Selection

Type filter text

- Ports
 - P0
 - P1
 - P100
 - P101
 - P102
 - P103
 - P104
 - P108
 - P109
 - P110
 - P111**
 - P112
 - P2
 - P3
 - P4
 - P5
 - Other Pins
- Peripherals
 - Analog:ADC
 - Analog:ANALOG
 - Analog:DAC
 - Connectivity:CAN
 - Connectivity:ILC
 - Connectivity:SCI
 - Connectivity:SPI
 - Connectivity:SSI
 - Connectivity:USB
 - Input:CTSU
 - Input:ICU
 - Monitoring:CAC
 - Storage:QSPI
 - System:CGC
 - System:DEBUG
 - DEBUG0
 - System:SYSTEM
 - Timers:AGT
 - Timers:GPT
 - Timers:RTC

Pin Configuration

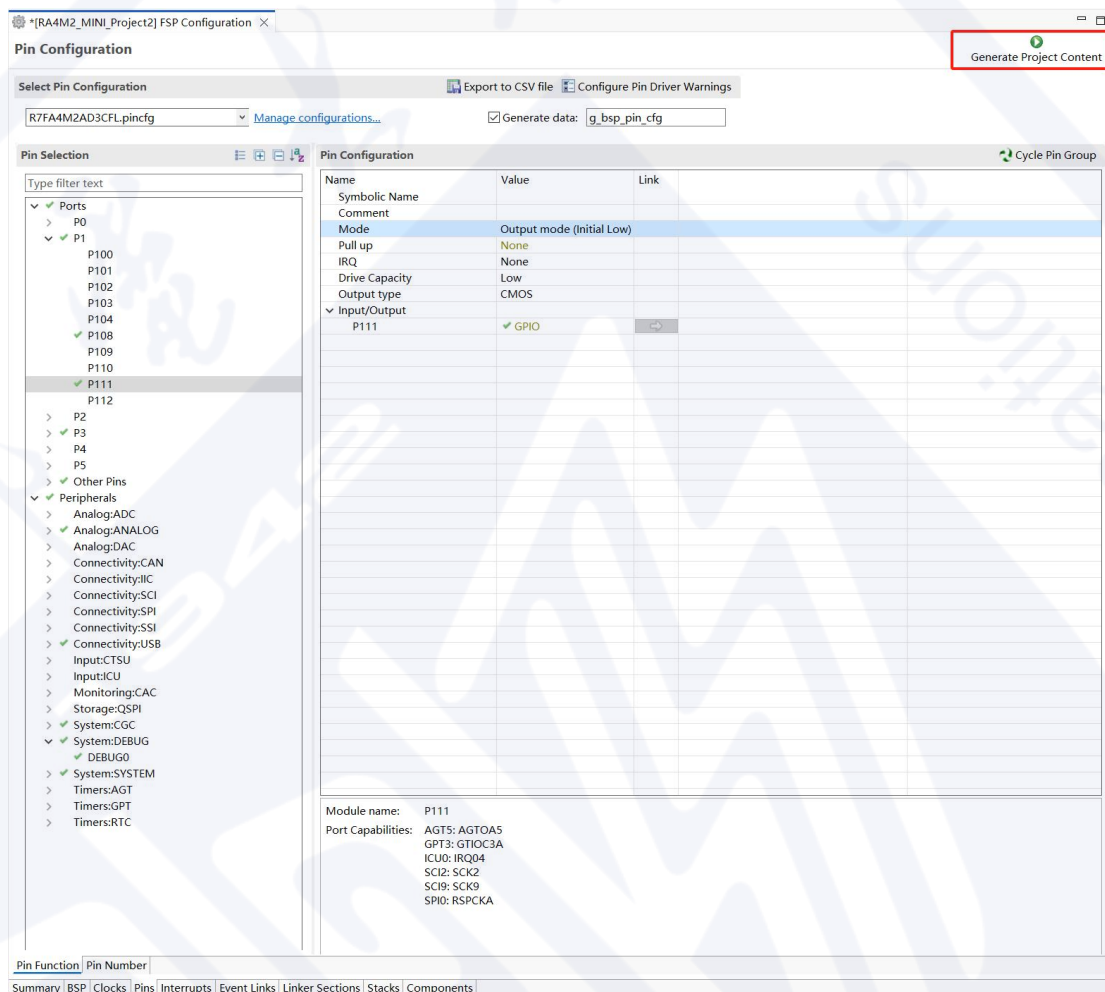
Name	Symbolic Name	Value	Link
Comment			
Mode		Output mode (initial Low)	
Pull up		None	
IRQ		None	
Drive Capacity		Low	
Output type		CMOS	
Input/Output			
P111		GPIO	

Module name: P111
Port Capabilities: AGTS: AGTOA5
GPT3: GTIOC3A
ICU0: IRQ04
SCI2: SCK2
SCI9: SCK9
SPI0: RSPCKA

Pin Function | Pin Number

Summary | BSP | Clocks | **Pins** | Interrupts | Event Links | Linker Sections | Stacks | Components

同步到工程。



R_IOPORT_PinWrite() 函数原型

◆ R_IOPORT_PinWrite()

```
fsp_err_t R_IOPORT_PinWrite ( ioport_ctrl_t *const p_ctrl,
                              bsp_io_port_pin_t pin,
                              bsp_io_level_t level
                              )
```

Sets a pin's output either high or low. Implements `ioport_api_t::pinWrite`.

Return values

FSP_SUCCESS	Pin written to
FSP_ERR_INVALID_ARGUMENT	The pin and/or level not valid
FSP_ERR_NOT_OPEN	The module has not been opened
FSP_ERR_ASSERTION	NULL pointer

Note

This function is re-entrant for different pins. This function makes use of the PCNTR3 register to atomically modify the level on the specified pin on a port.

故可以用 R_IOPORT_PinWrite()函数进行配置，配置高电平如下所示。

```
//点亮 LED
```

```
R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
```



```
1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This f
8
9  void hal_entry(void)
10 {
11     /* TODO: add your own code here */
12
13
14
15     // 点亮LED
16     R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
17
18     #if BSP_TZ_SECURE_BUILD
19         /* Enter non-secure code */
20         R_BSP_NonSecureEnter();
21     #endif
22 }
23
24 /* * This function is called at various points during the startup process. This implementation uses the event that
25
26 void R_BSP_WarmStart(bsp_warm_start_event_t event)
27 {
28     if (BSP_WARM_START_RESET == event)
29     {
30         #if BSP_FEATURE_FLASH_LP_VERSION != 0
31             /* Enable reading from data flash. */
32             R_FACI_LP->DFCTL = 1U;
33
34             /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the enable here, before clock
35             * C runtime initialization, should negate the need for a delay since the initialization will typically t
36
37         #endif
38     }
39
40     if (BSP_WARM_START_POST_C == event)
41     {
42         /* C runtime environment and system clocks are setup. */
43
44         /* Configure pins. */
45         R_IOPORT_Open (&IOPORT_CFG_CTRL, &IOPORT_CFG_NAME);
46
47         #if BSP_CFG_SDRAM_ENABLED
48             /* Setup SDRAM and initialize it. Must configure pins first. */
49             R_BSP_SdramInit(true);
50         #endif
51     }
52 }
53
54 #if BSP_TZ_SECURE_BUILD
55 FSP_CPP_HEADER
56 BSP_CMSE_NONSECURE_ENTRY void template_nonsecure_callable ();
57
58 /* Trustzone Secure Projects require at least one nonsecure callable function in order to build (Remove this if i
59 BSP_CMSE_NONSECURE_ENTRY void template_nonsecure_callable ()
60 {
61 }
62 FSP_CPP_FOOTER
63
64 #endif
```

代码

在 hal_entry()中添加如下呼吸灯代码。

```
while(1)
{
    //熄灭 LED
    R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_LOW);
    R_BSP_SoftwareDelay (500, BSP_DELAY_UNITS_MILLISECONDS);
    //点亮 LED
```



```

R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);

R_BSP_SoftwareDelay (500, BSP_DELAY_UNITS_MILLISECONDS);

}

```



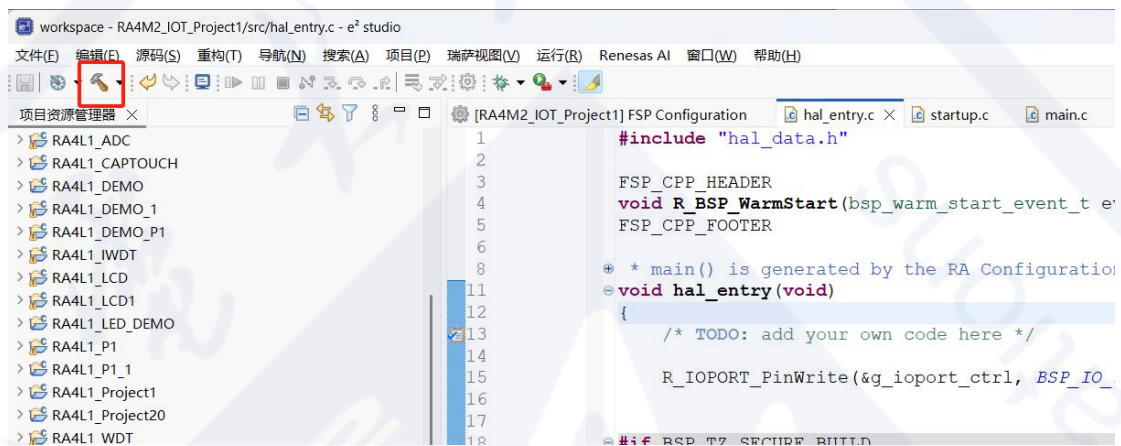
```

1  *hal_entry.c x startup.c main.c [RA4M2_MINI_Project2] FSP Configuration
2  RA4M2_MINI_Project2/src/hal_entry.c ta.h"
3
4  FSP_CPP_HEADER
5  void R_BSP_WarmStart(bsp_warm_start_event_t event);
6  FSP_CPP_FOOTER
7
8
9
10
11  /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This f
12  void hal_entry(void)
13  {
14      /* TODO: add your own code here */
15
16      //点亮LED
17      R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
18
19      while(1)
20      {
21          //熄灭LED
22          R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_LOW);
23          R_BSP_SoftwareDelay (500, BSP_DELAY_UNITS_MILLISECONDS);
24
25          //点亮LED
26          R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
27          R_BSP_SoftwareDelay (500, BSP_DELAY_UNITS_MILLISECONDS);
28      }
29
30  #if BSP_TZ_SECURE_BUILD
31      /* Enter non-secure code */
32      R_BSP_NonSecureEnter();
33  #endif
34
35  /* * This function is called at various points during the startup process. This implementation uses the event that
36  void R_BSP_WarmStart(bsp_warm_start_event_t event)
37  {
38      if (BSP_WARM_START_RESET == event)
39      {
40          #if BSP_FEATURE_FLASH_LP_VERSION != 0
41              /* Enable reading from data flash. */
42              R_FACI_LP->DFLCTL = 1U;
43
44              /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the enable here, before clock
45              * C runtime initialization, should negate the need for a delay since the initialization will typically t
46          #endif
47      }
48
49      if (BSP_WARM_START_POST_C == event)
50      {
51          /* C runtime environment and system clocks are setup. */
52
53          /* Configure pins. */
54          R_IOPORT_Open (&IOPORT_CFG_CTRL, &IOPORT_CFG_NAME);
55
56          #if BSP_CFG_SDRAM_ENABLED
57              /* Setup SDRAM and initialize it. Must configure pins first. */
58              R_BSP_SdramInit(true);
59          #endif
60      }
61
62  #if BSP_TZ_SECURE_BUILD
63
64  }
65
66  }
67
68  }
69
70
71

```

编译

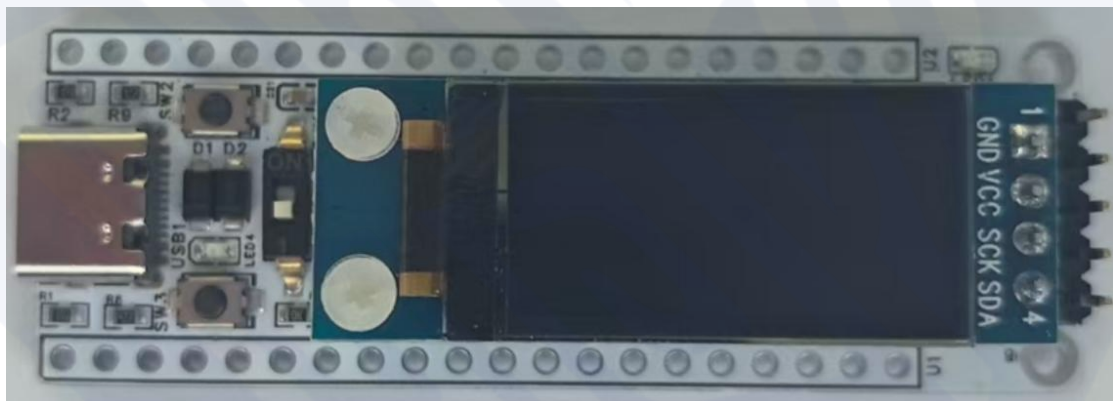
点击小锤子进行编译。



project5: GPIO 输入检测

概述

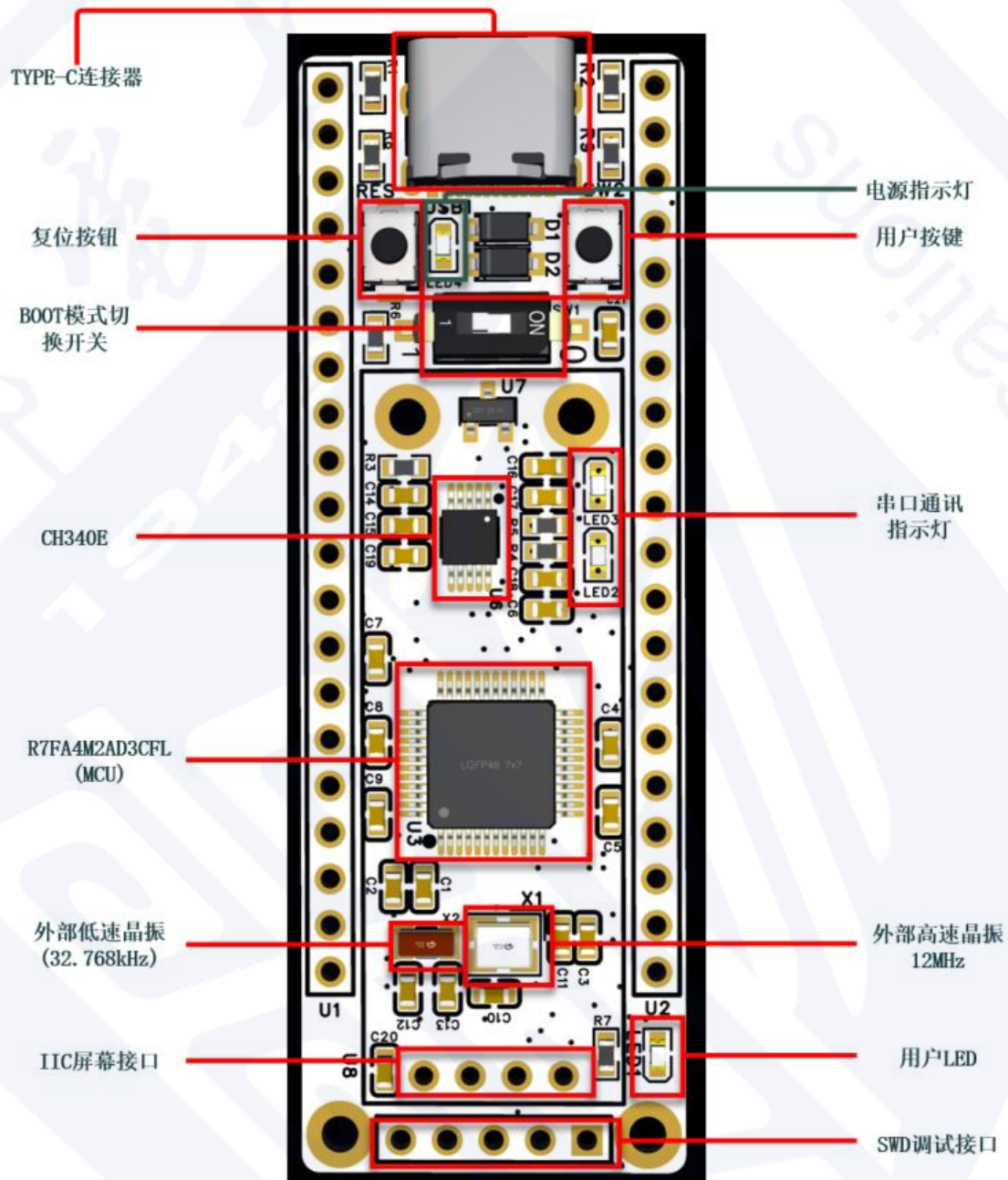
本篇文章主要介绍如何使用 e2studio 对瑞萨单片机进行 GPIO 输入检测。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

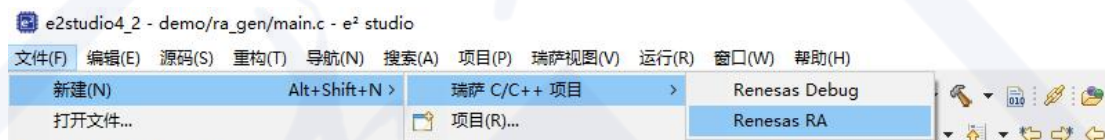


参考程序

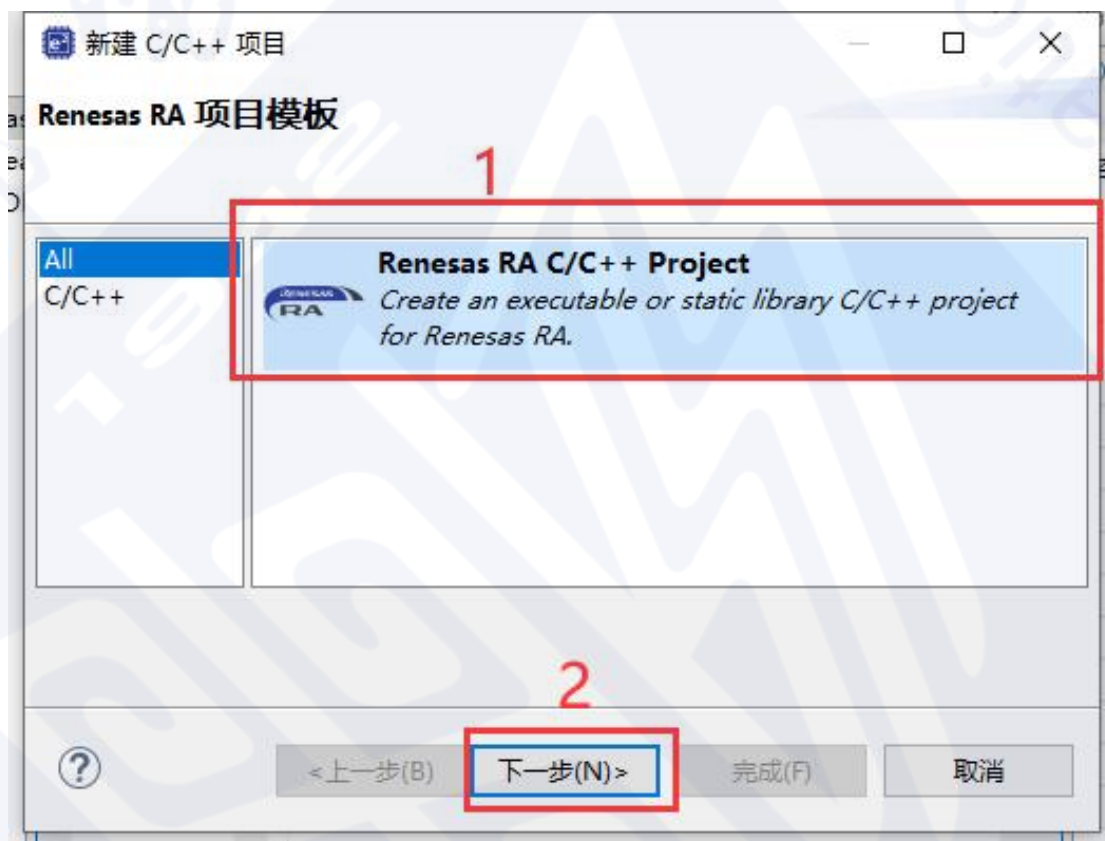
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

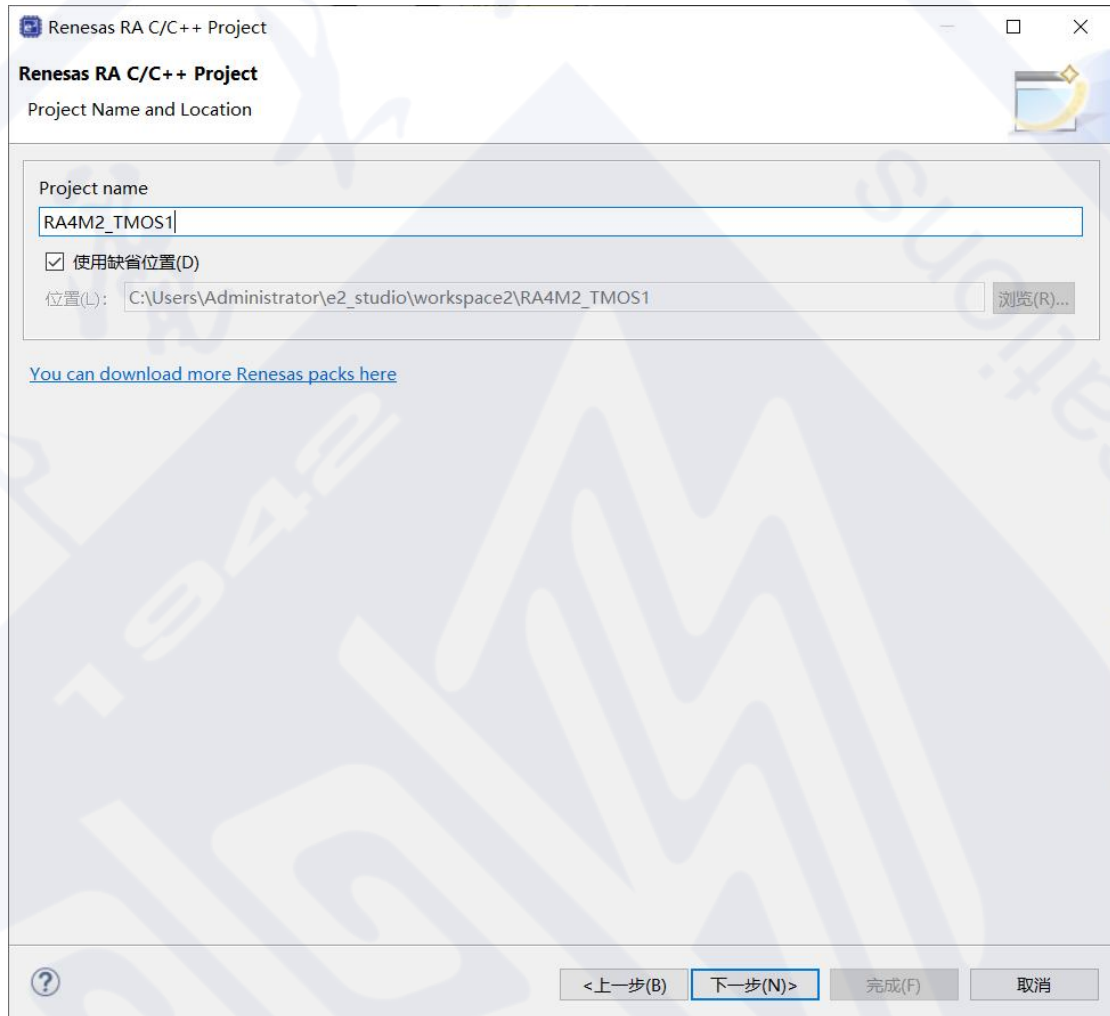
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

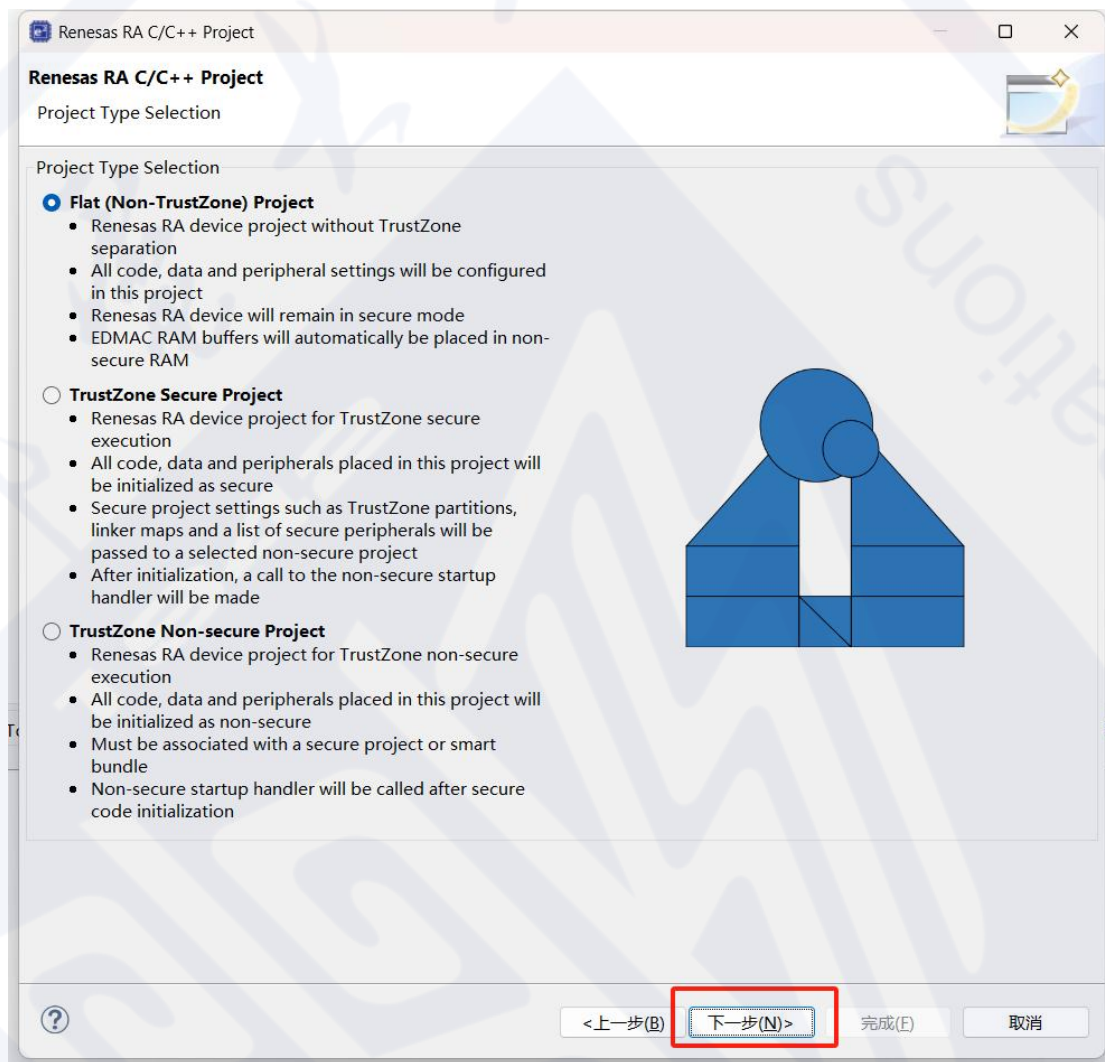
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

< 上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

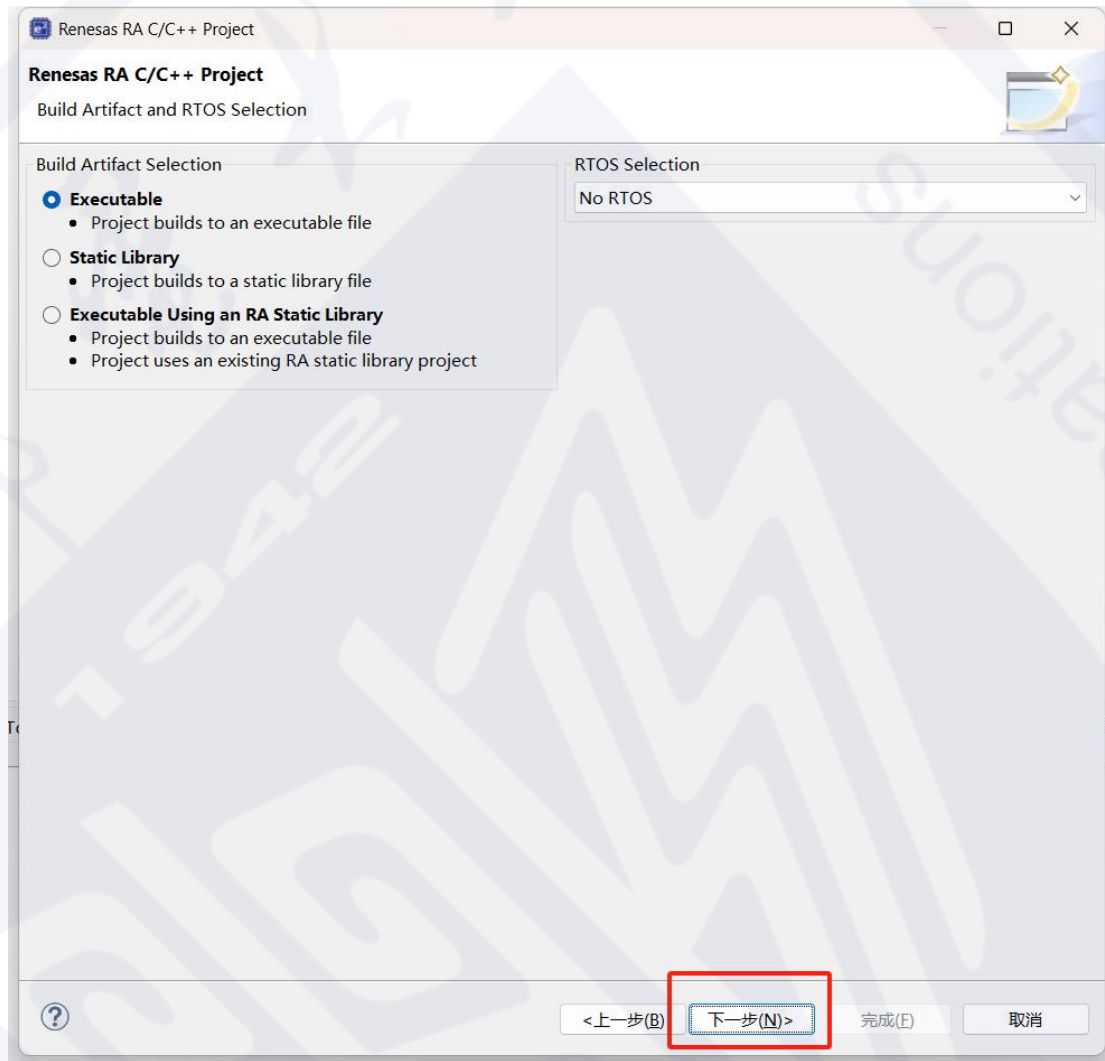
?

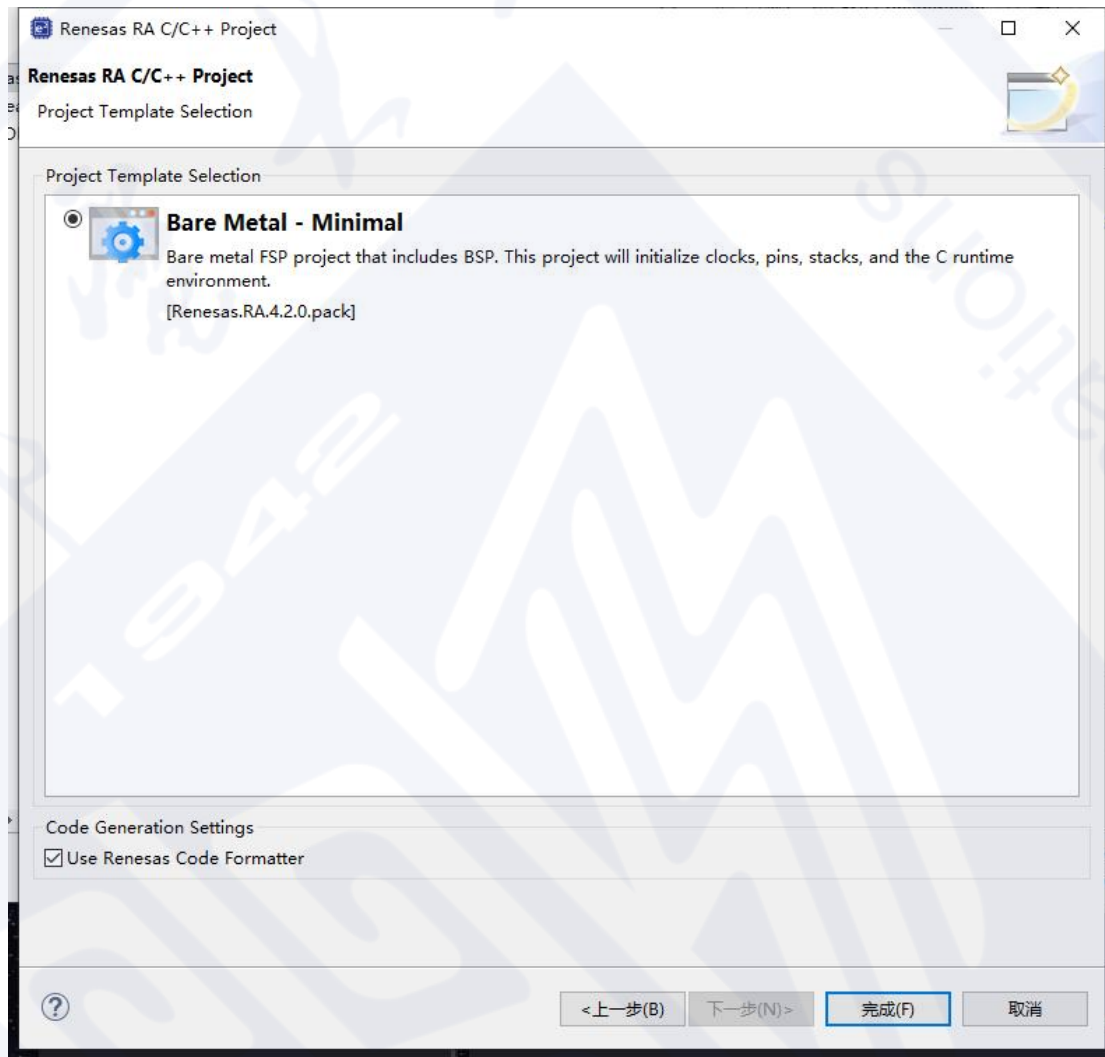
< 上一步(B)

下一步(N) >

完成(E)

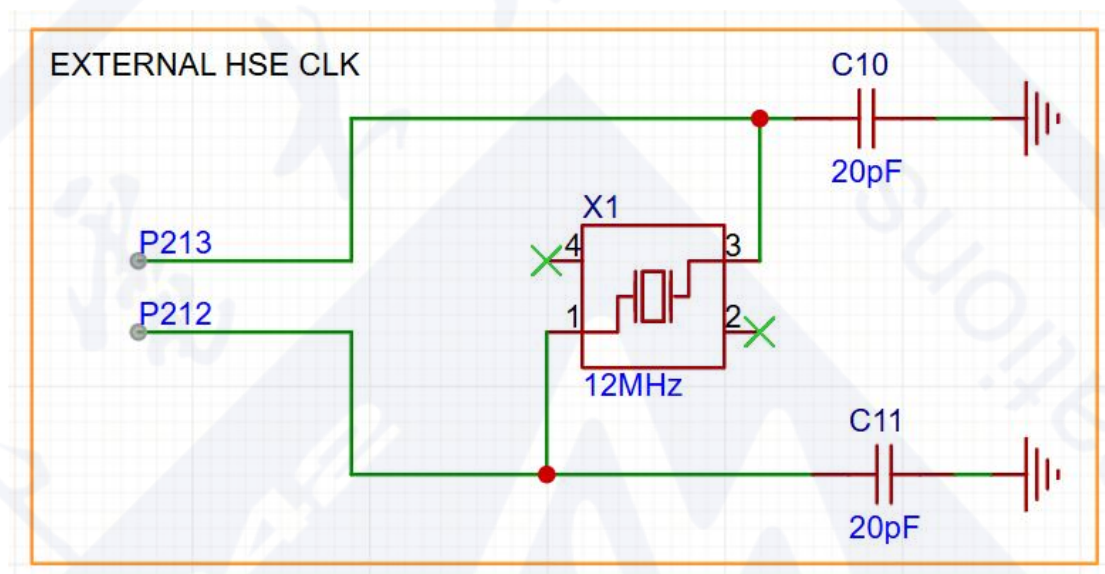
取消



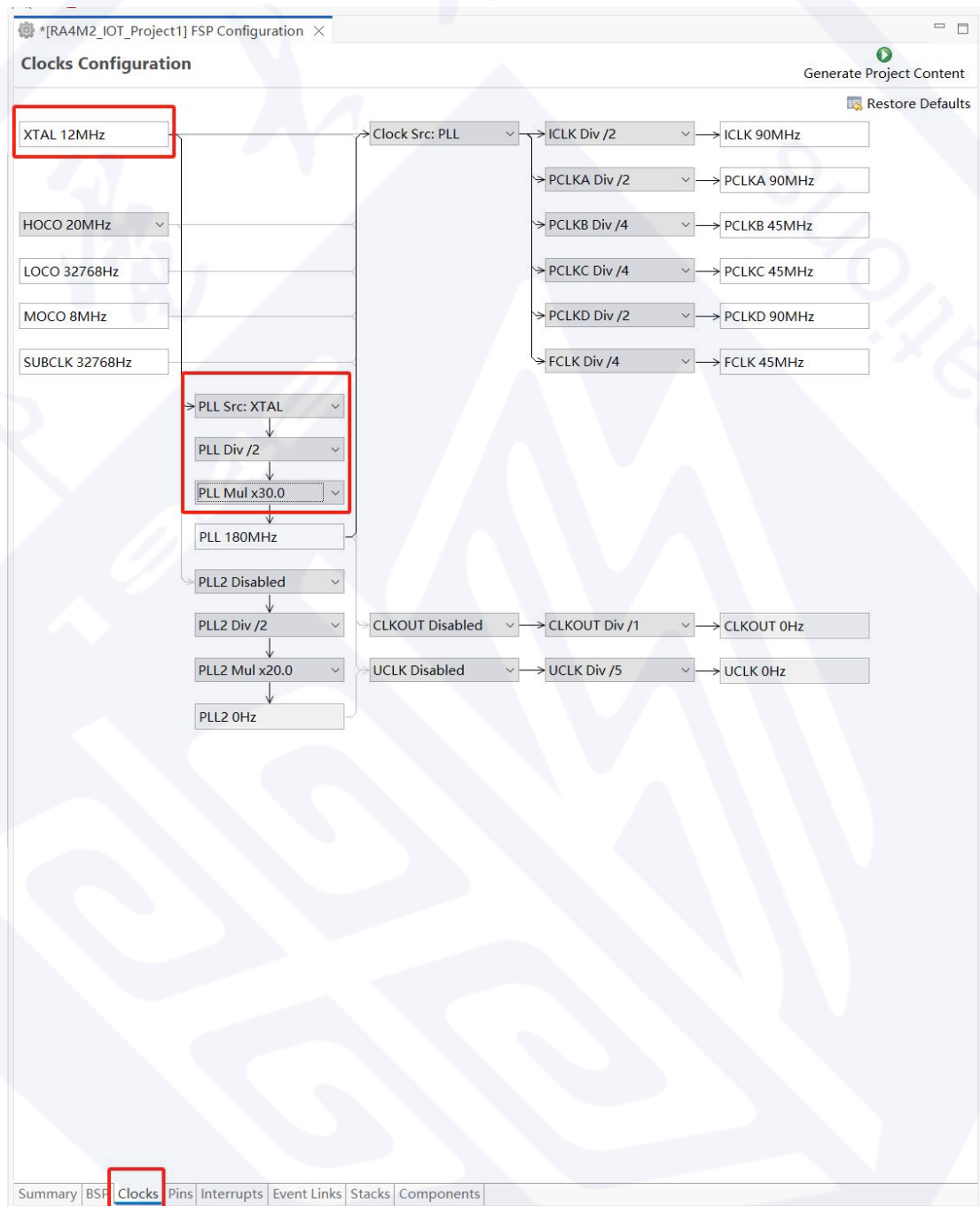


时钟设置

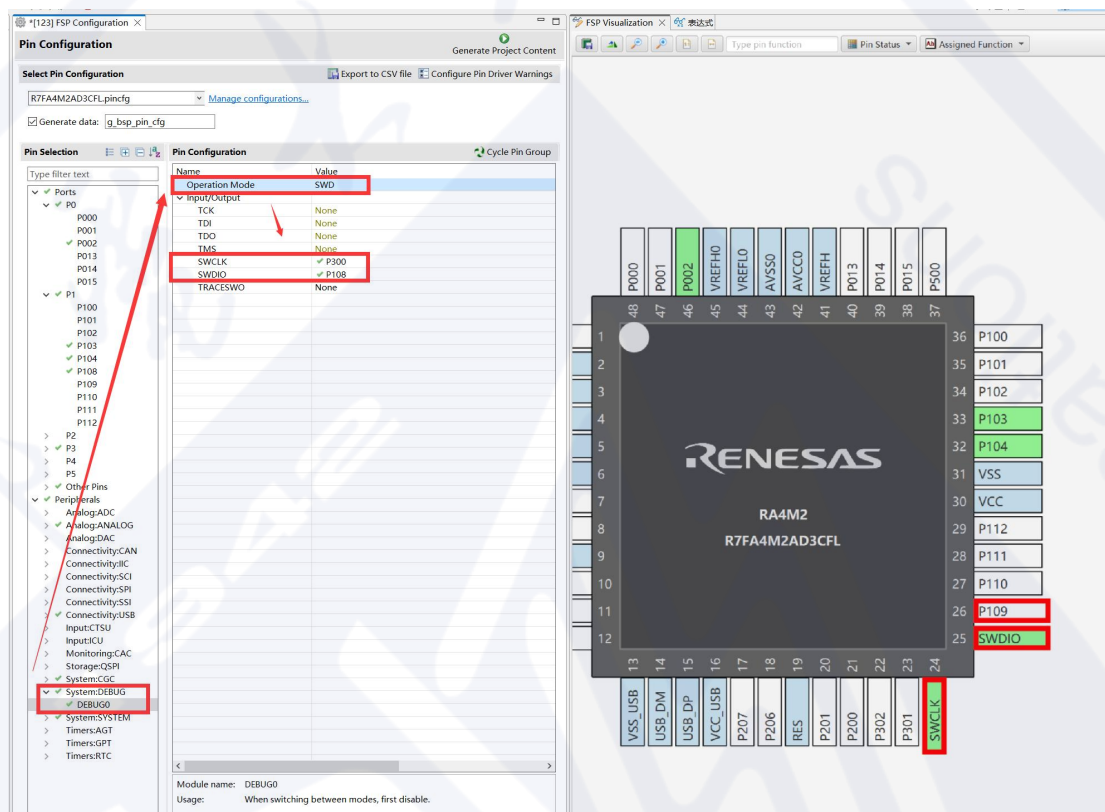
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。



SWD 调试口设置



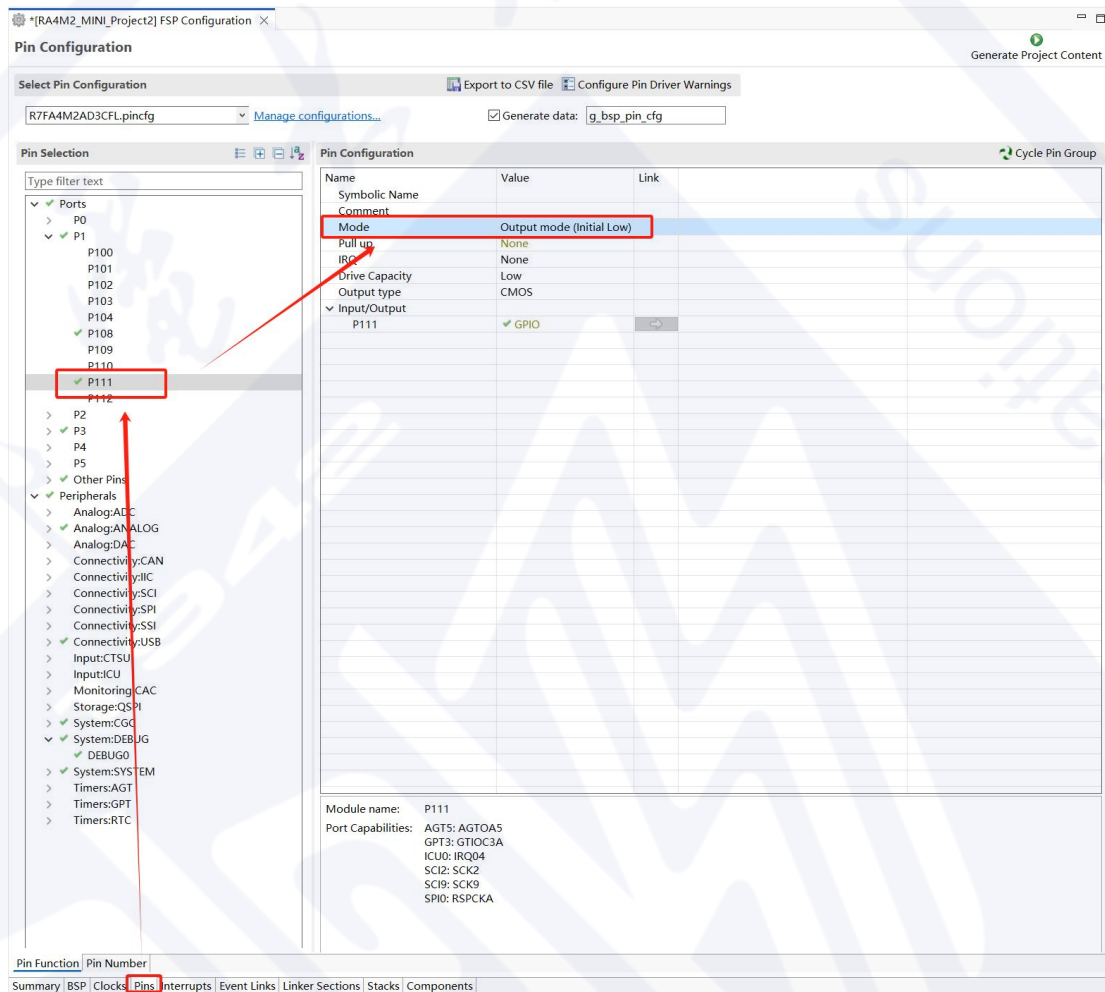
GPIO 口配置

由下图我们可以得知，板子上有 1 个 LED 灯，同时需要给高电平才可以点亮，故以 P111 管脚为例，配置方式如下所示。

用户LED

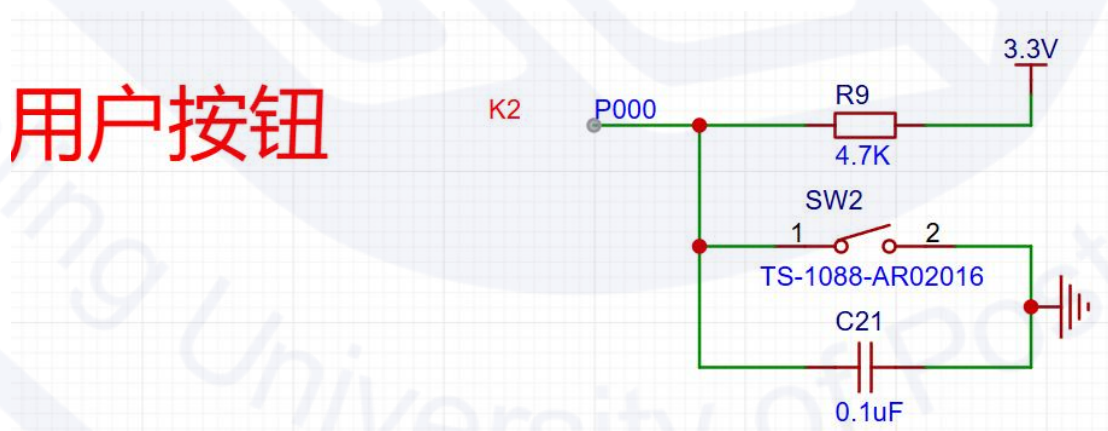


在 e2studio 中可以直接设置 P111 为输出口。



按键口配置

由下图我们可以得知，按键在 P000 管脚，并且有一个上拉。



在 e2studio 中可以直接设置 P000 为输入口。

hal_entry.c startup.c main.c *[RA4M2_MINI_Project2] FSP Configuration

Pin Configuration Generate Project Content

Select Pin Configuration Export to CSV file Configure Pin Driver Warnings

R7FA4M2AD3CFL-pincfg Manage configurations... ☒ Generate data: g_bsp_pin_cfg

Pin Selection Cycle Pin Group

Type filter text

- Ports
 - P000
 - P001
 - P002
 - P003
 - P004
 - P005
 - P006
 - P007
 - P008
 - P009
 - P010
 - P011
 - P012
- P1
 - P100
 - P101
 - P102
 - P103
 - P104
 - P105
 - P106
 - P107
 - P108
 - P109
 - P110
 - P111
 - P112
- P2
- P3
- P4
- P5
- Other Pins
- Peripherals
 - Analog:ADC
 - Analog:ANALOG
 - Analog:IAC
 - Connectivity:CAN
 - Connectivity:J1C
 - Connectivity:SCI
 - Connectivity:SPI
 - Connectivity:SSI
 - Connectivity:USB
 - Input:CTSU
 - Input:ICU
 - Monitoring:CAC
 - Storage:SPI
 - System:CSC
 - System:DEBUG
 - System:SYSTEM
 - Timers:AGT
 - Timers:GRT
 - Timers:RTC

Pin Configuration

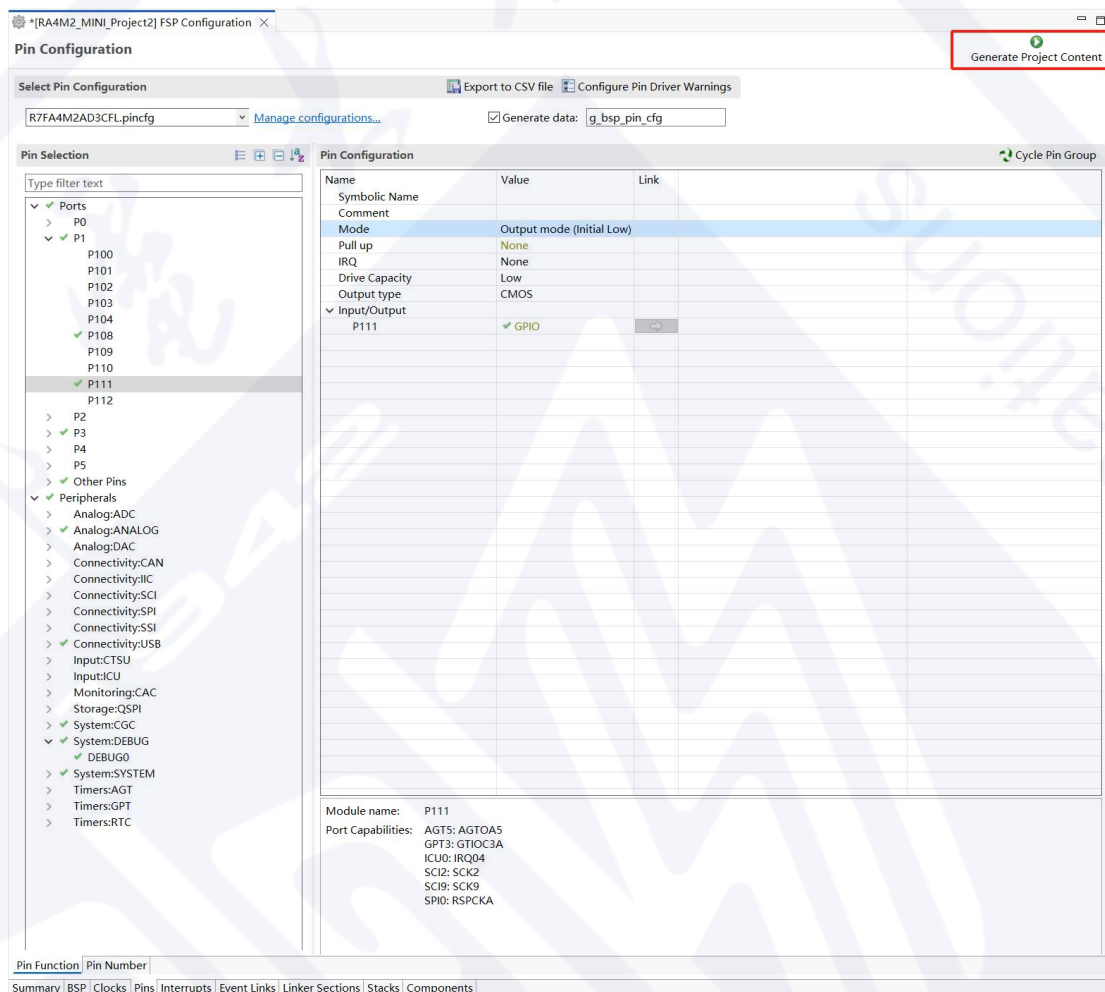
Name	Value	Link
Symbolic Name		
Comment		
Mode	Input mode	
Pull up	None	
IRQ	None	
Output type	CMOS	
Input/Output		
P000	GPIO	

Module name: P000
Port Capabilities: ADC0: AN000
ICU0: IRQ06

Pin Function Pin Number

Summary BSP Clocks Pins Interrupts Event Links Linker Sections Stacks Components

同步到工程。



R_IOPORT_PinRead() 函数原型

◆ R_IOPORT_PinRead()

```
fsp_err_t R_IOPORT_PinRead ( ioport_ctrl_t *const p_ctrl,
                             bsp_io_port_pin_t pin,
                             bsp_io_level_t * p_pin_value
                             )
```

Reads the level on a pin. Implements `ioport_api_t::pinRead`.

Return values

FSP_SUCCESS Pin read
FSP_ERR_ASSERTION NULL pointer
FSP_ERR_NOT_OPEN The module has not been opened

Note

This function is re-entrant for different pins.

故可以用 R_IOPORT_PinRead() 函数进行读取 IO 口电平状态，该函数只能读取一个端口的电平。

```
bsp_io_level_t p_port_value_pin_000;
R_IOPORT_PinRead(&g_ioport_ctrl1, BSP_IO_PORT_00_PIN_00, &p_port_value_pin_000);
```



```
if(p_port_value_pin_000)
    R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_LOW);
else//按下按键 LED 亮
    R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
```

代码

在 `hal_entry()` 中添加如下。

```
while(1)
{
    bsp_io_level_t p_port_value_pin_000;
    R_IOPORT_PinRead(&g_ioport_ctrl, BSP_IO_PORT_00_PIN_00, &p_port_value_pin_000);
    if(p_port_value_pin_000)
        R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_LOW);
    else//按下按键 LED 亮
        R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
}
```

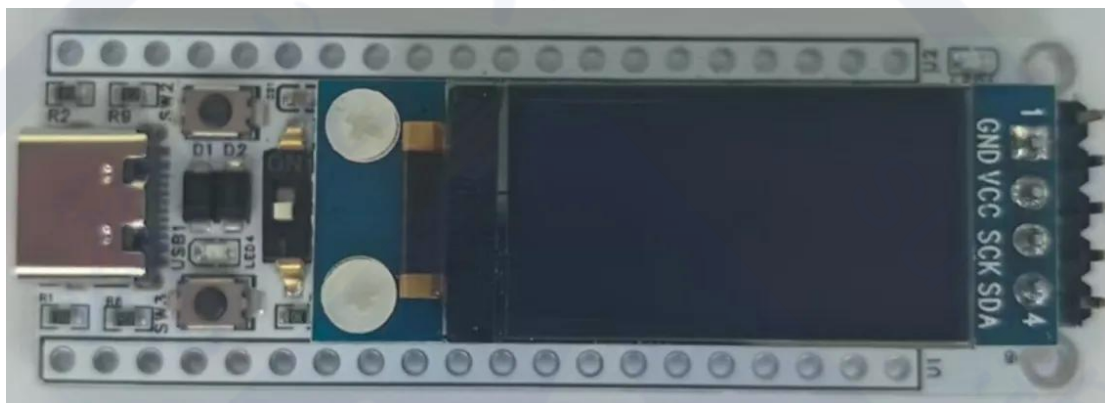


```
1 #include "hal_data.h"
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7
8 /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. */
9
10 void hal_entry(void)
11 {
12     /* TODO: add your own code here */
13
14     while(1)
15     {
16         bsp_io_level_t p_port_value_pin_000;
17         R_IOPORT_PinRead(&g_ioport_ctrl, BSP_IO_PORT_00_PIN_00, &p_port_value_pin_000);
18         if(p_port_value_pin_000)
19             R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_LOW);
20         else //按下按钮LED亮
21             R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
22     }
23
24     #if BSP_TZ_SECURE_BUILD
25         /* Enter non-secure code */
26         R_BSP_NonSecureEnter();
27     #endif
28
29     /* This function is called at various points during the startup process. This implementation uses the
30      * void R_BSP_WarmStart(bsp_warm_start_event_t event)
31      */
32     {
33         if (BSP_WARM_START_RESET == event)
34         {
35             #if BSP_FEATURE_FLASH_LP_VERSION != 0
36                 /* Enable reading from data flash. */
37                 R_FACI_LP->DFLCTL = 1U;
38
39                 /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the enable here, but
40                  * C runtime initialization, should negate the need for a delay since the initialization will
41                  */
42             #endif
43         }
44
45         if (BSP_WARM_START_POST_C == event)
46         {
47             /* C runtime environment and system clocks are setup. */
48         }
49     }
50 }
```

project6: GPIO 外部中断（IRQ）配置

概述

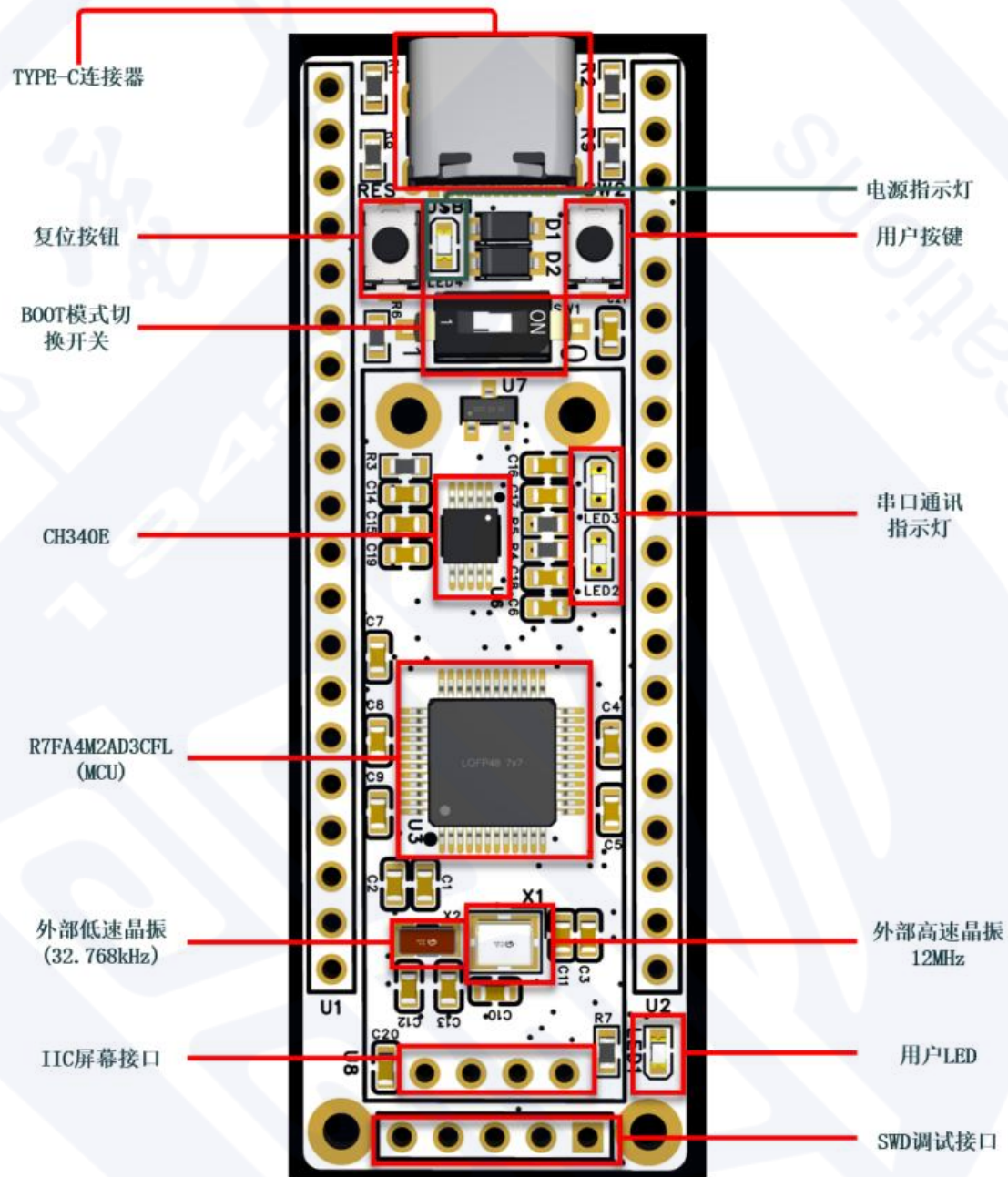
GPIO（通用输入/输出）引脚作为嵌入式系统的基本接口，可以用于信号输入、输出以及事件检测。GPIO 外部中断（IRQ）是一种常见的硬件中断配置，通过在特定引脚上检测信号的变化（例如上升沿或下降沿）来触发中断响应。这种机制在按钮按压、传感器信号输入等场景中尤为常用，可实现低延迟的事件捕获与处理。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

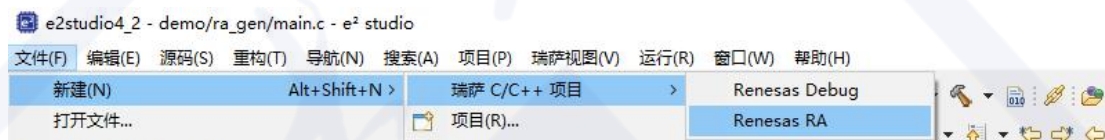


参考程序

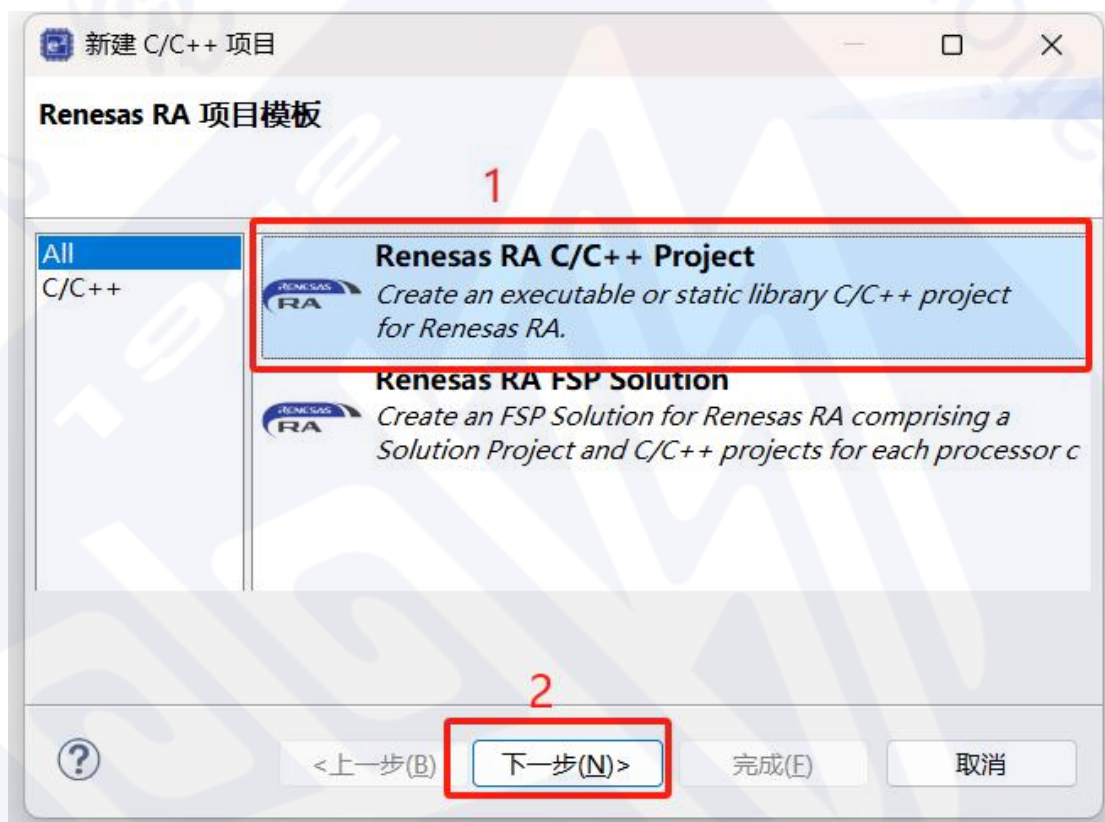
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

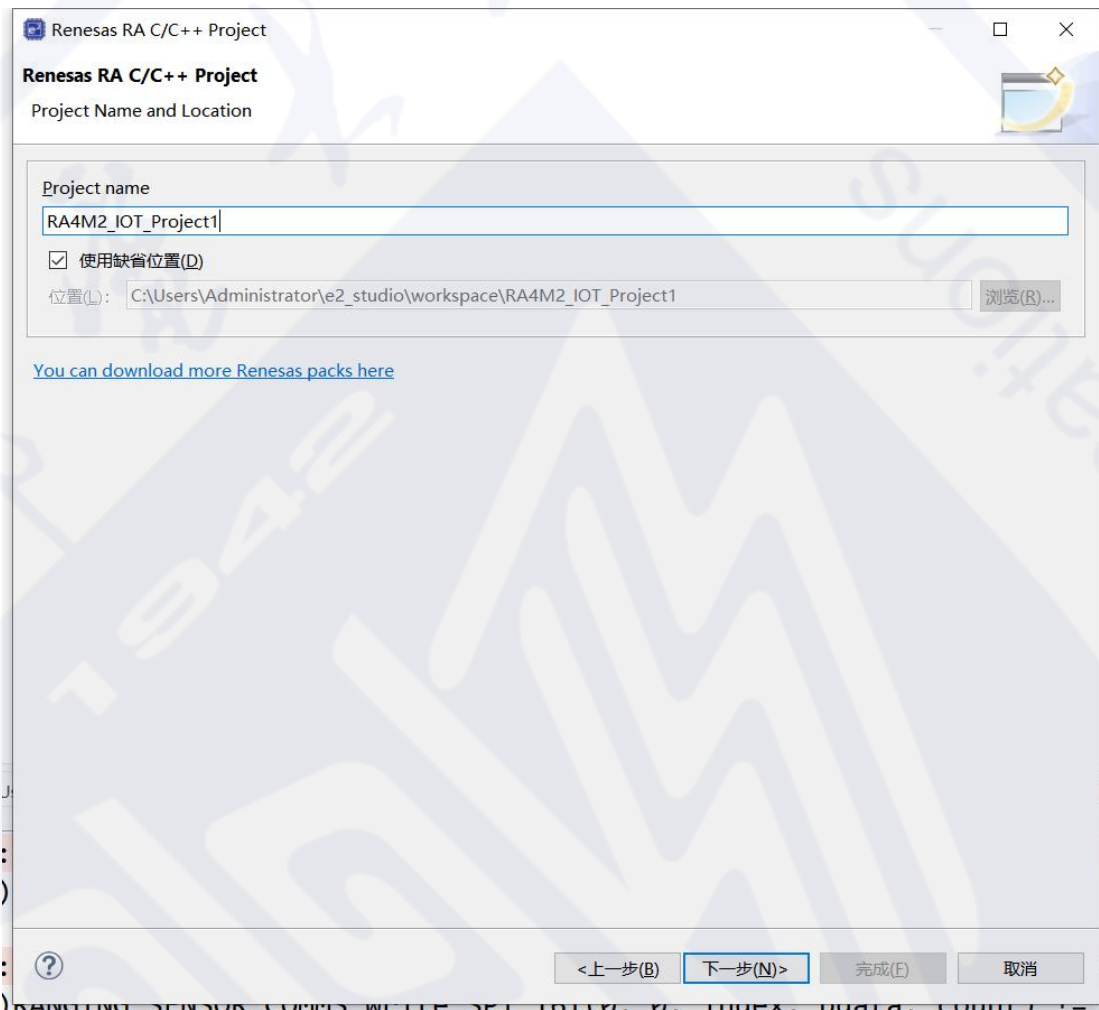
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

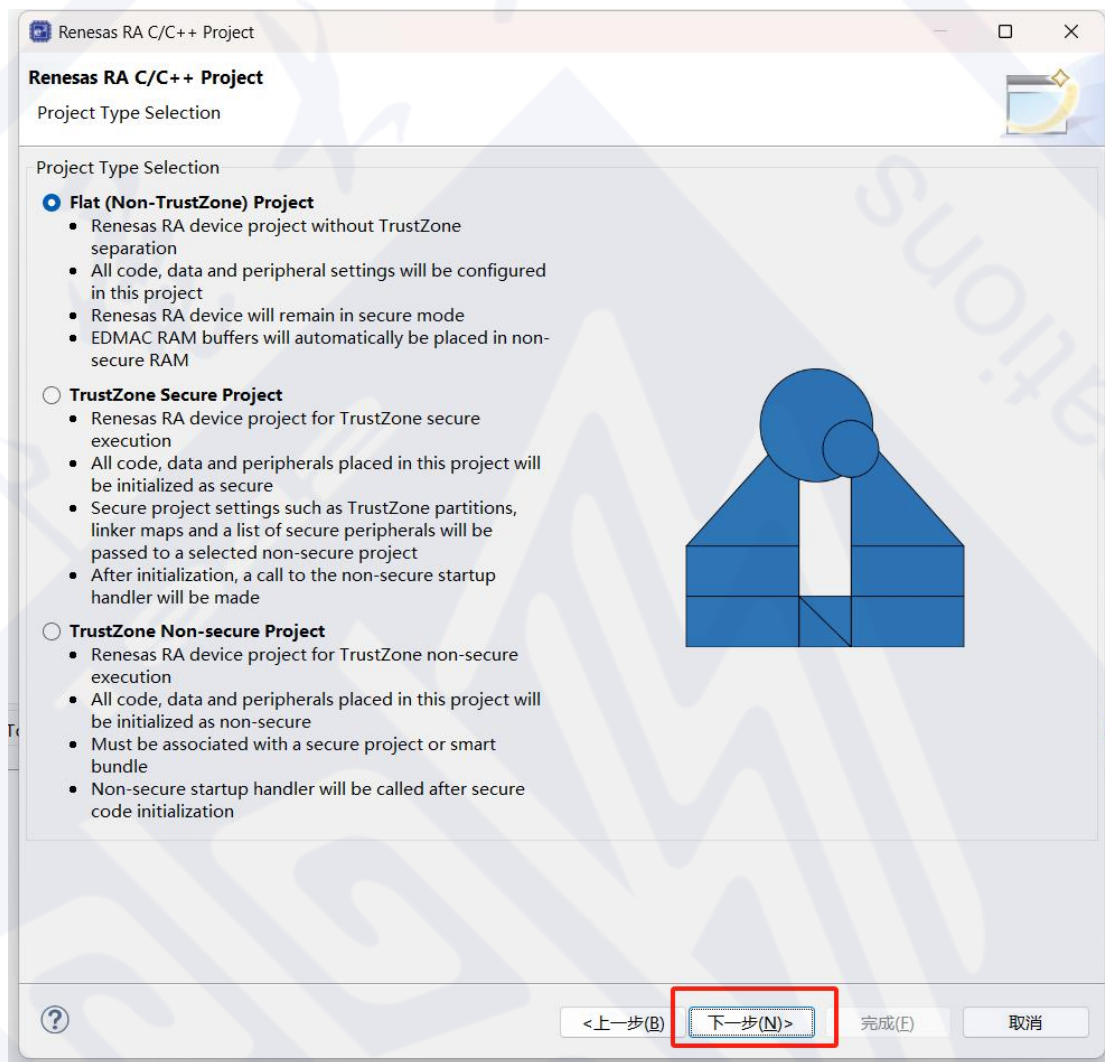
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

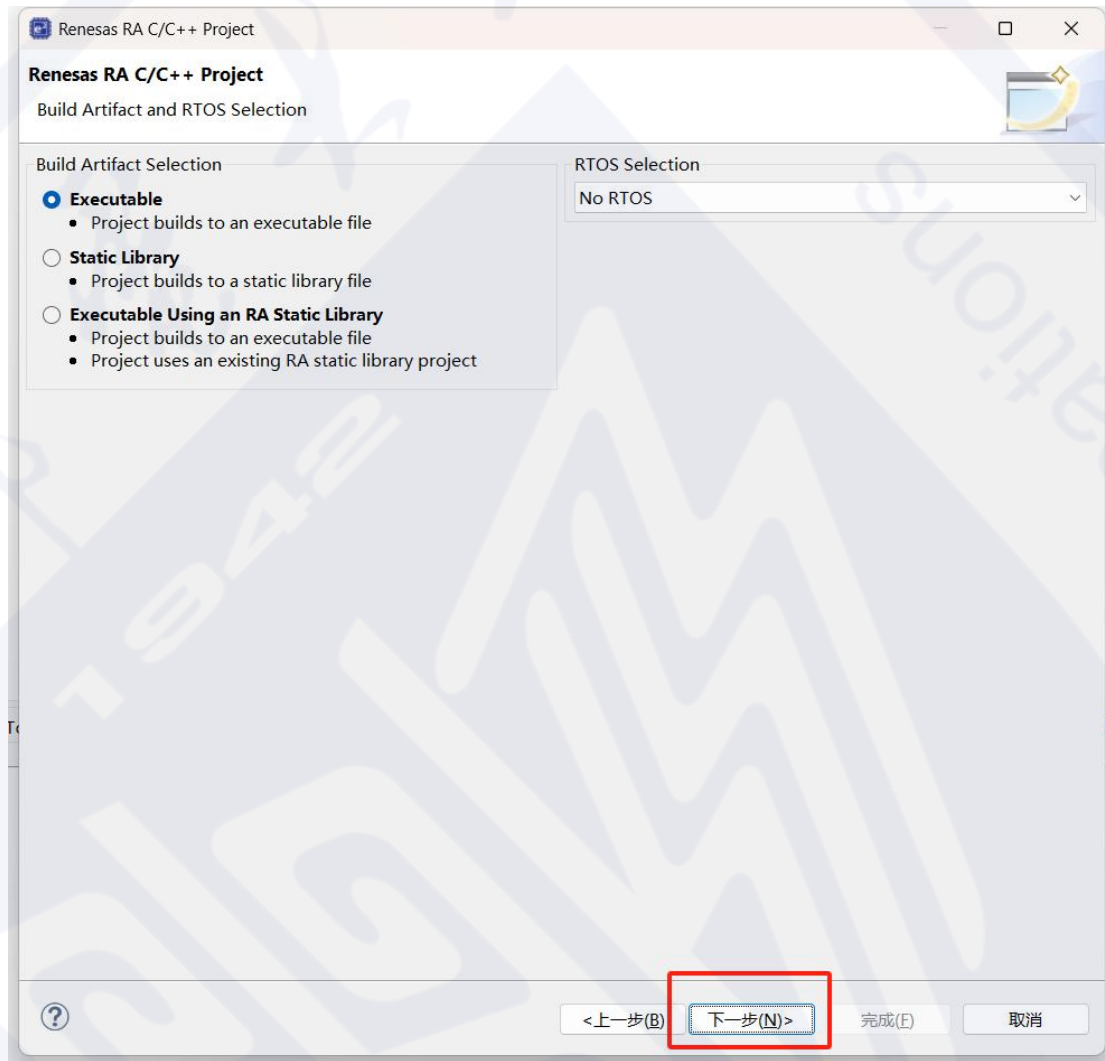
?

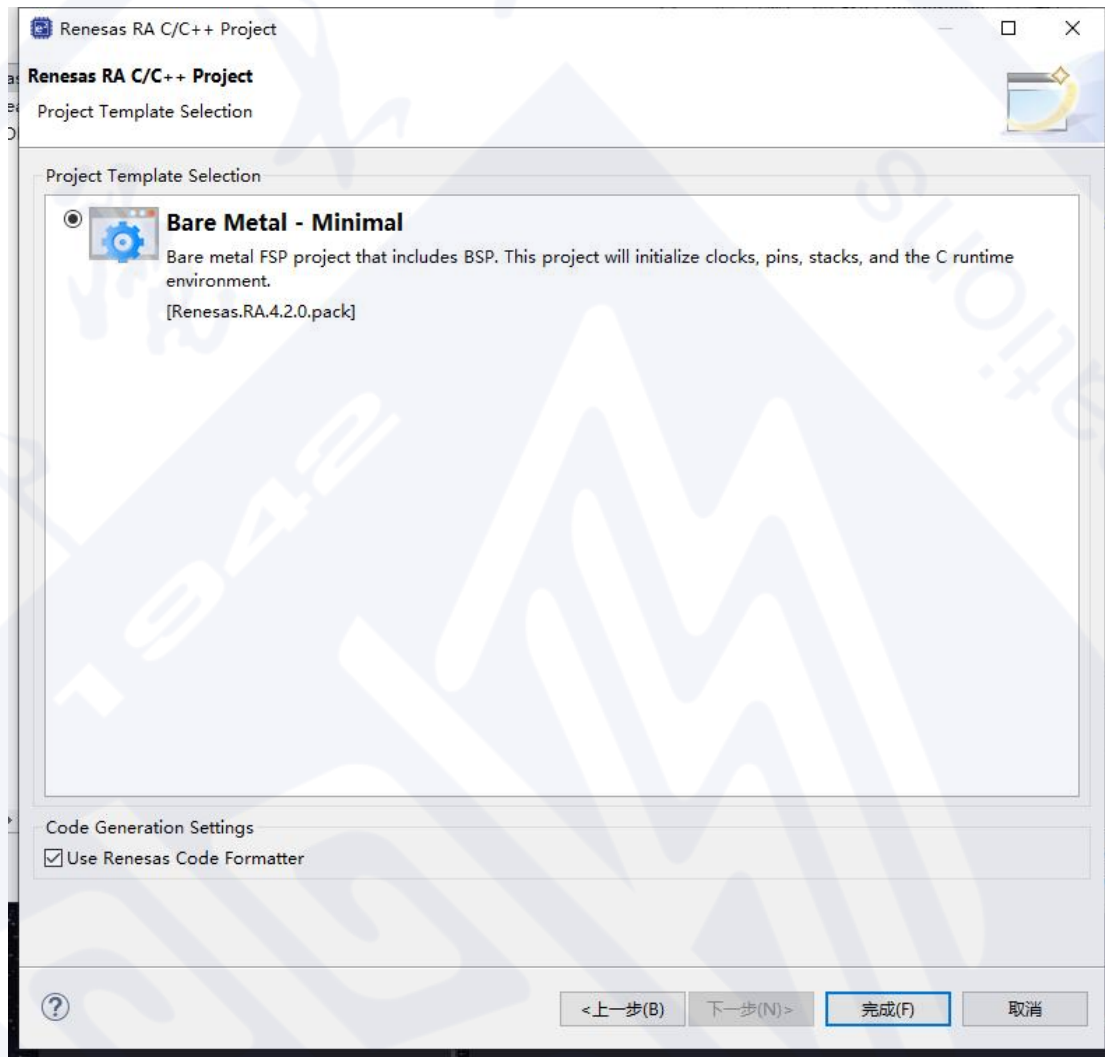
< 上一步(B)

下一步(N) >

完成(E)

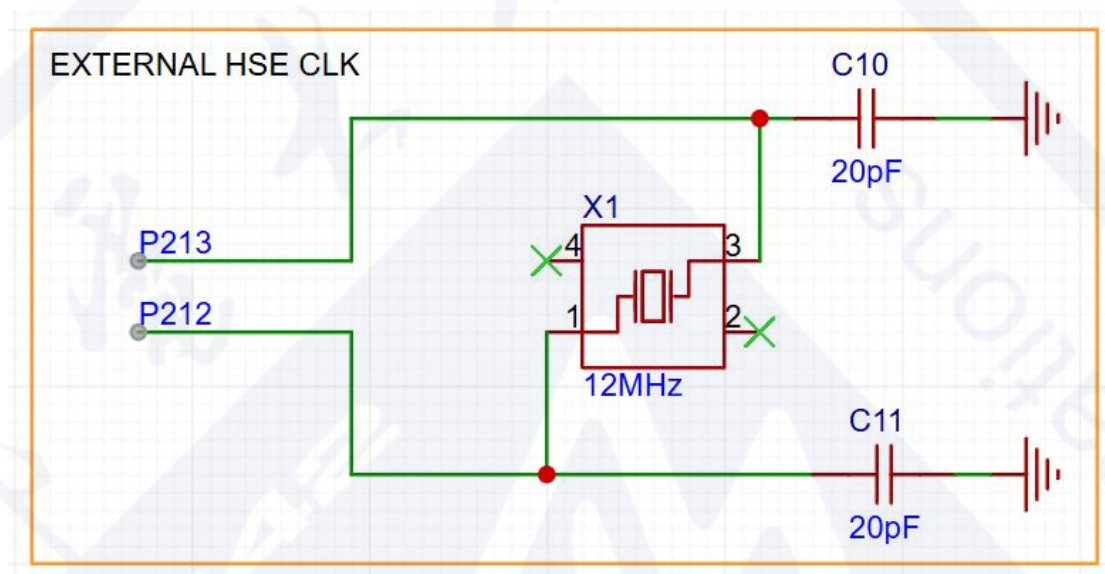
取消



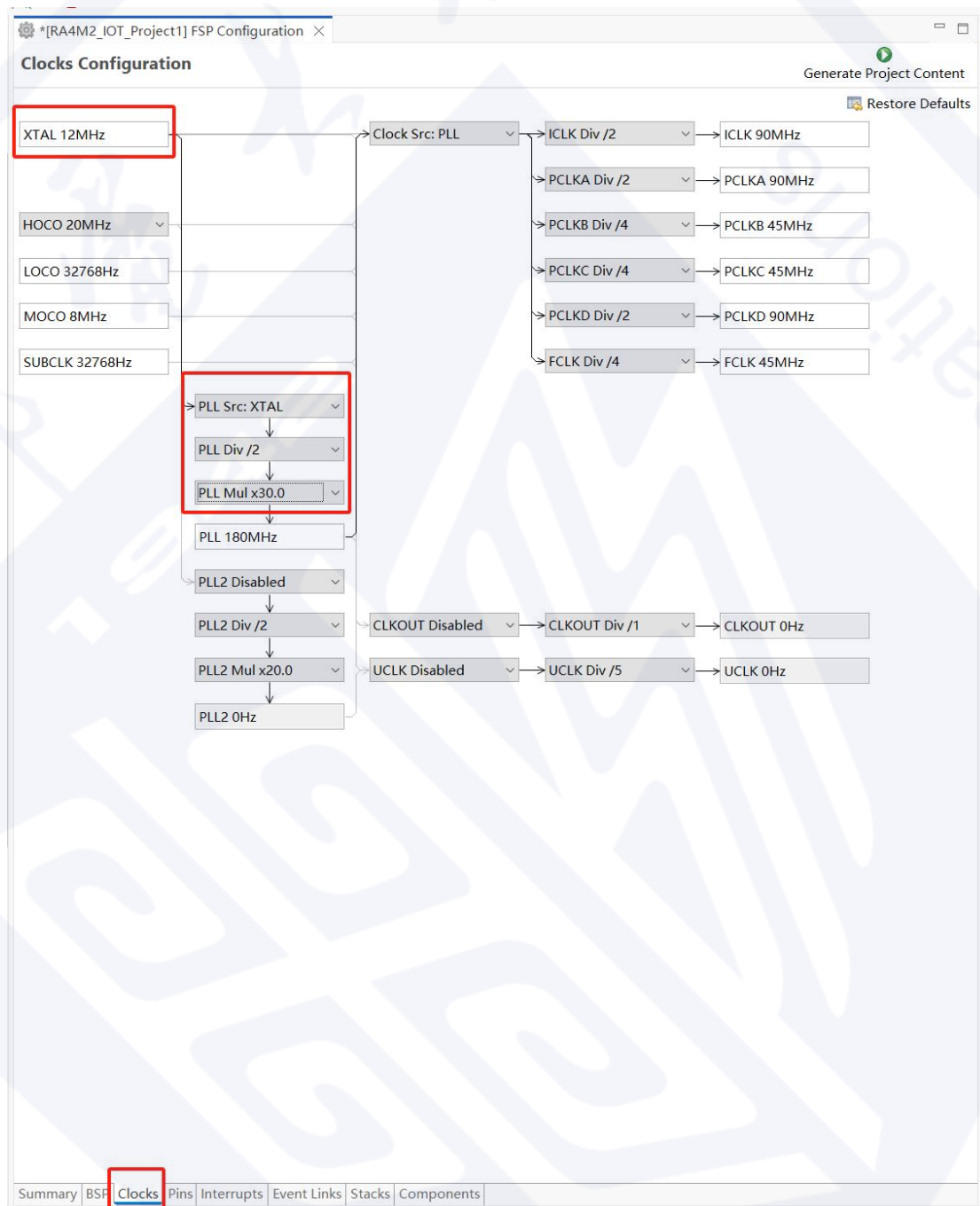


时钟设置

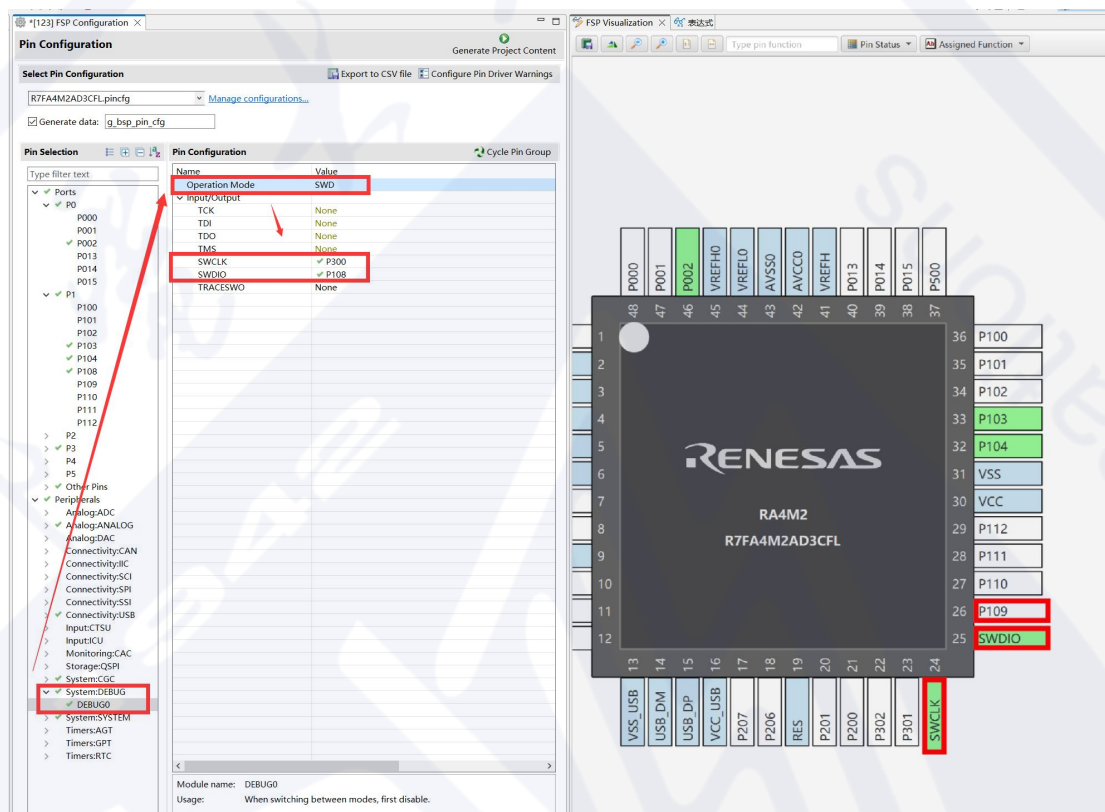
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。



SWD 调试口设置



GPIO 口配置

由下图我们可以得知，板子上有 1 个 LED 灯，同时需要给高电平才可以点亮，故以 P111 管脚为例，配置方式如下所示。

用户LED



在 e2studio 中可以直接设置 P111 为输出口。

*[RA4M2_MINI_Project2] ESP Configuration X

Pin Configuration

Generate Project Content

Select Pin Configuration [Export to CSV file](#) [Configure Pin Driver Warnings](#)

R7FA4M2AD3CFL.pincfg [Manage configurations...](#) ☒ Generate data: g_bsp_pin_cfg

Pin Selection

Type filter text

- Ports
 - P0
 - P1
 - P100
 - P101
 - P102
 - P103
 - P104
 - P108
 - P109
 - P110
 - P111**
 - P112
 - P2
 - P3
 - P4
 - P5
 - Other Pins
- Peripherals
 - Analog:ADC
 - Analog:ANALOG
 - Analog:DAC
 - Connectivity:CAN
 - Connectivity:ILC
 - Connectivity:SCI
 - Connectivity:SPI
 - Connectivity:SSI
 - Connectivity:USB
 - Input:CTSU
 - Input:ICU
 - Monitoring:CAC
 - Storage:QSPI
 - System:CGC
 - System:DEBUG
 - DEBUG0
 - System:SYSTEM
 - Timers:AGT
 - Timers:GPT
 - Timers:RTC

Pin Configuration

Name	Symbolic Name	Value	Link
Comment			
Mode		Output mode (initial Low)	
Pull up		None	
IRQ		None	
Drive Capacity		Low	
Output type		CMOS	
Input/Output			
P111		GPIO	

Module name: P111

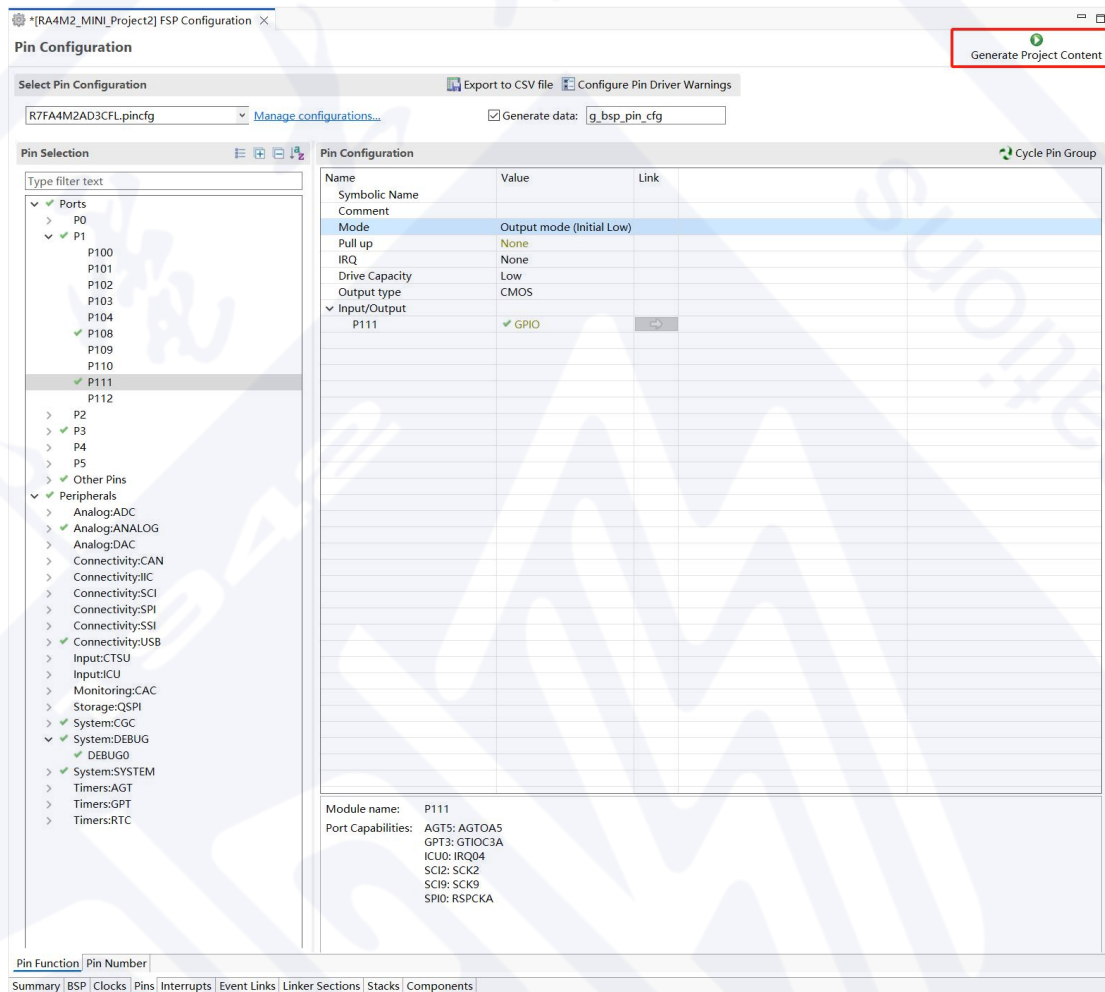
Port Capabilities:

- AGTS: AGTOA5
- GPT3: GTIOC3A
- ICU0: IRQ04
- SCI2: SCK2
- SCI9: SCK9
- SPI0: RSPCKA

Pin Function | Pin Number

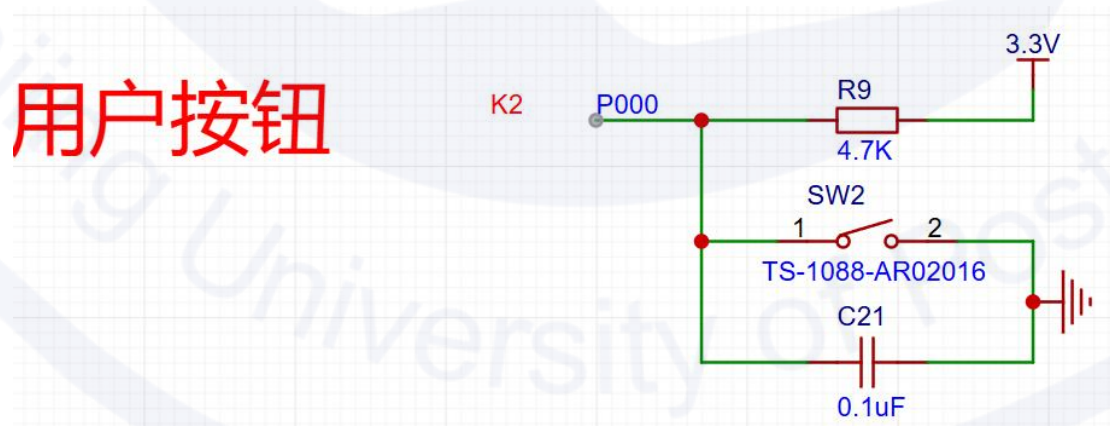
Summary | BSP | Clocks | **Pins** | Interrupts | Event Links | Linker Sections | Stacks | Components

同步到工程。

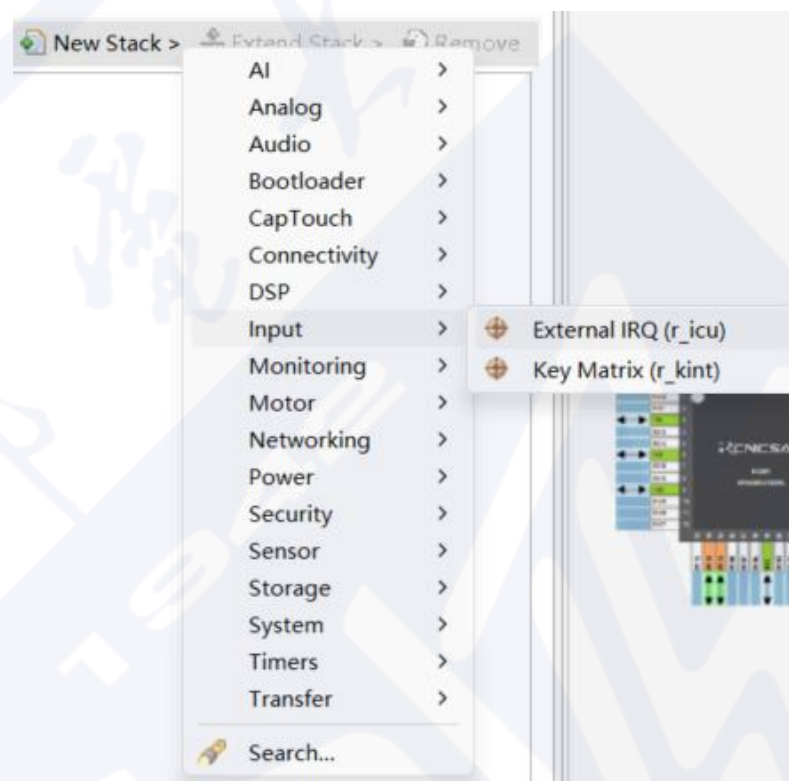


按键口配置

由下图我们可以得知，按键在 P000 管脚，并且有一个上拉。



在“New Stack”下选择 Input > External IRQ (r_icu)。



模块配置如下所示。

- Name: `g_external_irq6`, 这是该外部中断的名称。
- Channel: 选择了 6 通道。
- Trigger: 触发方式设置为 Rising (上升沿触发), 即信号上升时触发中断。
- Digital Filtering: 未启用数字滤波 (Not Supported)。
- Digital Filtering Sample Clock: 由于数字滤波未启用, 因此该项也未支持。
- Callback: 指定了回调函数 `external_irq6_callback`。当中断触发时, 将调用此函数处理具体逻辑。
- Pin Interrupt Priority: 设置为 Priority 2, 表示该中断的优先级为 2。
- IRQ06: 映射到引脚 P000, 即该中断信号通过引脚 P000 触发。

属性 ×	问题	Smart Browser	控制台	调试	Memory Usage	搜索
g_external_irq6 External IRQ (r_icu)						
Settings	属性					值
API Info	▼ Common					
	Parameter Checking					Default (BSP)
	▼ Module g_external_irq6 External IRQ (r_icu)					
	Name					g_external_irq6 名称
	Channel					6 通道
	Trigger					Rising 触发方式
	Digital Filtering					Disabled
	Filter Source					PCLK filter
	Digital Filtering Sample Clock (Only valid when Digital Filtering is					PCLK / 64
	Callback					external_irq6_callback 回调函数
	Pin Interrupt Priority					Priority 12 优先级
	▼ Pins					
	IRQ06					P000 引脚

*[RA4M2_MINI_Project2] FSP Configuration × hal_entry.c

Pin Configuration

Generate Project Content

Select Pin Configuration: R7FA4M2AD3CFL.pincfg [Manage configurations...](#) ☒ Generate data: g_bsp_pin_cfg

Pin Selection

Type filter text

- ✓ Ports
 - ✓ P0
 - ✓ P000
 - ✓ P001
 - ✓ P002
 - ✓ P013
 - ✓ P014
 - ✓ P015
 - > P1
 - > P2
 - > P3
 - > P4
 - > P5
 - > Other Pins
- ✓ Peripherals
 - > Analog:ADC
 - > Analog:ANALOG
 - > Analog:DAC
 - > Connectivity:CAN
 - > Connectivity:ILC
 - > Connectivity:SCI
 - > Connectivity:SPI
 - > Connectivity:SSI
 - > Connectivity:USB
 - > Input:CTSU
 - ✓ Input:ICU**
 - ✓ ICU0**
 - > Monitoring:CAC
 - > Storage:QSPI
 - > System:CGC
 - > System:DEBUG
 - > System:SYSTEM
 - > Timers:AGT
 - > Timers:GPT
 - > Timers:RTC

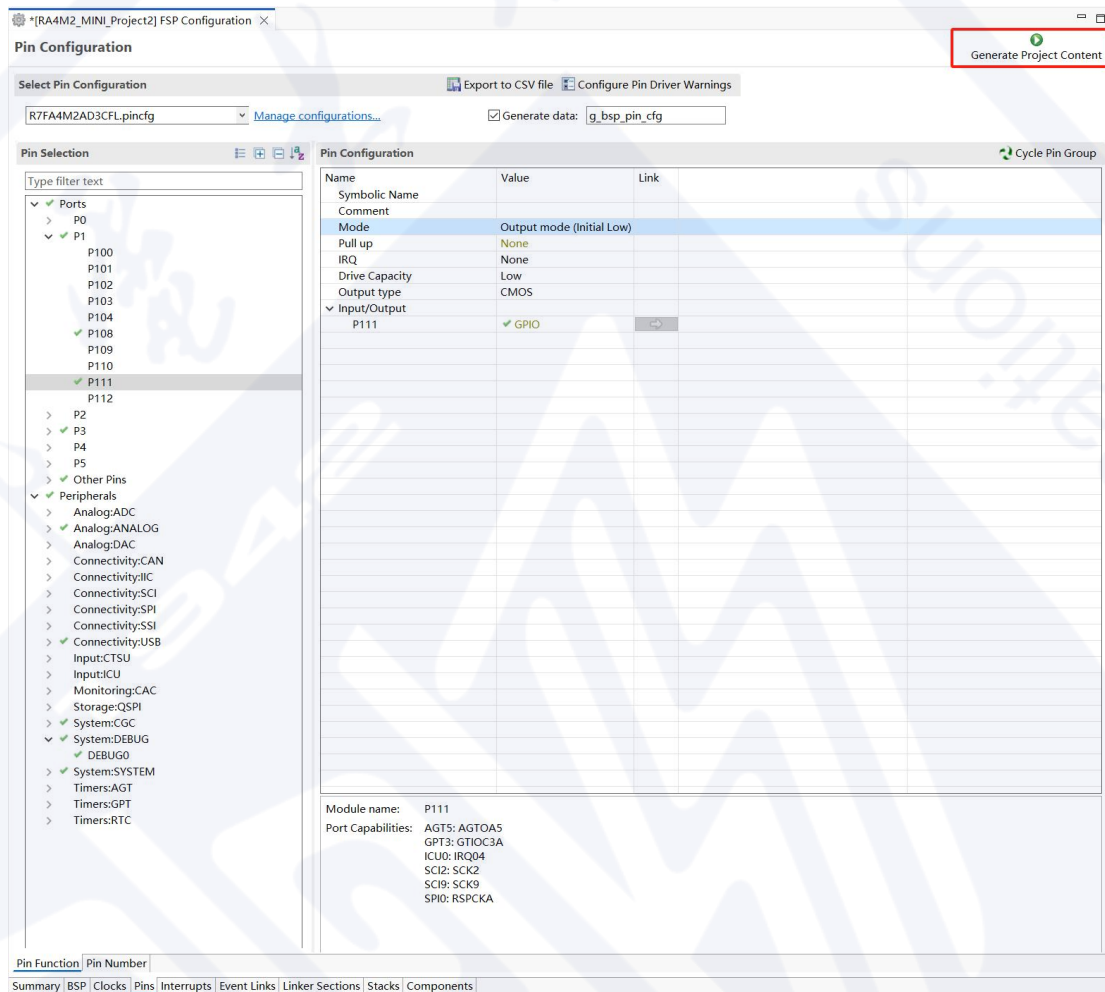
Pin Configuration [Cycle Pin Group](#)

Name	Value	Lock	Link
Operation Mode	Enabled		
✓ Input/Output			
NMI	None		
IRQ00	None		
IRQ01	None		
IRQ02	None		
IRQ03	None		
IRQ04	None		
IRQ05	None		
IRQ06	✓ P000		
IRQ07	None		
IRQ08	None		
IRQ13	None		

Module name: ICU0
Usage: To use IRQ function with output or peripheral modes, change directly in port dialog

Pin Function | Pin Number | Summary | BSP | Clocks | Pins | Interrupts | Event Links | Linker Sections | Stacks | Components

同步到工程。



中断回调函数

- `external_irq6_callback` 函数是外部中断的回调函数，当中断触发时，`icu_irq_isr` 中断服务程序会调用此函数。
- `g_external_irq6_complete` 变量在每次中断时切换状态（如果是 0 则变 1，如果是 1 则变 0）。

```
bool g_external_irq_complete =0;
/* Called from icu_irq_isr */
void external_irq6_callback (external_irq_callback_args_t * p_args)
{
    (void) p_args;

    g_external_irq_complete = !g_external_irq_complete;
}
```

```
[RA4M2_MINI_Project2] FSP Configuration  *hal_entry.c X
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7 bool g_external_irq_complete = 0;
8 /* Called from icu_irq_isr */
9 void external_irq6_callback (external_irq_callback_args_t * p_args)
10 {
11     (void) p_args;
12     g_external_irq_complete = !g_external_irq_complete;
13 }
14
15
16
17 * main() is generated by the RA Configuration editor and is used to generate threads if
18
19 void hal_entry(void)
20 {
21     /* TODO: add your own code here */
22 }
23
24
25 #if BSP_TZ_SECURE_BUILD
26 /* Enter non-secure code */
27 R_BSP_NonSecureEnter();
28 #endif
29 }
30
31 * This function is called at various points during the startup process. This implements
32
33 void R_BSP_WarmStart(bsp_warm_start_event_t event)
34 {
35     if (BSP_WARM_START_RESET == event)
36     {
37         #if BSP_FEATURE_FLASH_LP_VERSION != 0
38             /* Enable reading from data flash. */
39             R_FACI_LP->DFCTL = 1U;
40
41             /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the e
42              * C runtime initialization, should negate the need for a delay since the initial
43             #endif
44         }
45
46         if (BSP_WARM_START_POST_C == event)
47         {
48             /* C runtime environment and system clocks are setup. */
49
50             /* Configure pins. */
51             R_IOPORT_Open (&IOPORT_CFG_CTRL, &IOPORT_CFG_NAME);
52         }
53     }
54 }
55
56
57
```

主程序

- 调用 `R_ICU_ExternalIrqOpen` 函数初始化外部中断模块 `g_external_irq6`。
`g_external_irq6_ctrl` 和 `g_external_irq6_cfg` 是配置外部中断的控制结构体和配置结构体。
- 如果初始化成功，`err` 会返回 `FSP_SUCCESS`，否则程序会停止（通过 `assert` 进行错误检测）。

```
fsp_err_t err = R_ICU_ExternalIrqOpen(&g_external_irq6_ctrl, &g_external_irq6_cfg);
assert(FSP_SUCCESS == err);
```

- 启用外部中断，使其可以响应触发事件。
- `assert` 用于确认操作成功，否则程序会停止。

```
err = R_ICU_ExternalIrqEnable(&g_external_irq6_ctrl);
```

```
assert(FSP_SUCCESS == err);

[RA4M2_MINI_Project2] FSP Configuration *hal_entry.c X
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7 bool g_external_irq_complete = 0;
8 /* Called from icu_irq_isr */
9 /* no previous declaration for 'external_irq6_callback' [-Wmissing-declarations]
10 void external_irq6_callback (external_irq_callback_args_t * p_args)
11 {
12     (void) p_args;
13     g_external_irq_complete = !g_external_irq_complete;
14 }
15
16 |
17 * main() is generated by the RA Configuration editor and is used to generate threads if
20 void hal_entry(void)
21 {
22     /* TODO: add your own code here */
23
24     fsp_err_t err = R_ICU_ExternalIrqOpen(&g_external_irq6_ctrl, &g_external_irq6_cfg);
25     assert(FSP_SUCCESS == err);
26
27     err = R_ICU_ExternalIrqEnable(&g_external_irq6_ctrl);
28     assert(FSP_SUCCESS == err);
29
30 #if BSP_TZ_SECURE_BUILD
31     /* Enter non-secure code */
32     R_BSP_NonSecureEnter();
33 #endif
34 }
35
36 * This function is called at various points during the startup process. This implements
42 void R_BSP_WarmStart(bsp_warm_start_event_t event)
43 {
44     if (BSP_WARM_START_RESET == event)
45     {
46         #if BSP_FEATURE_FLASH_LP_VERSION != 0
47             /* Enable reading from data flash. */
48             R_FACI_LP->DFLCTL = 1U;
49
50             /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the e
51              * C runtime initialization, should negate the need for a delay since the initial
52             #endif
53         }
54
55         if (BSP_WARM_START_POST_C == event)
56         {
57
```

- 主循环中根据 g_external_irq_complete 的状态来控制 GPIO 引脚的高低电平。

```
while(1){
    if(g_external_irq_complete)
    {
        R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
    }
    else
    {
        R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_LOW);
    }
}
```



```
}
```

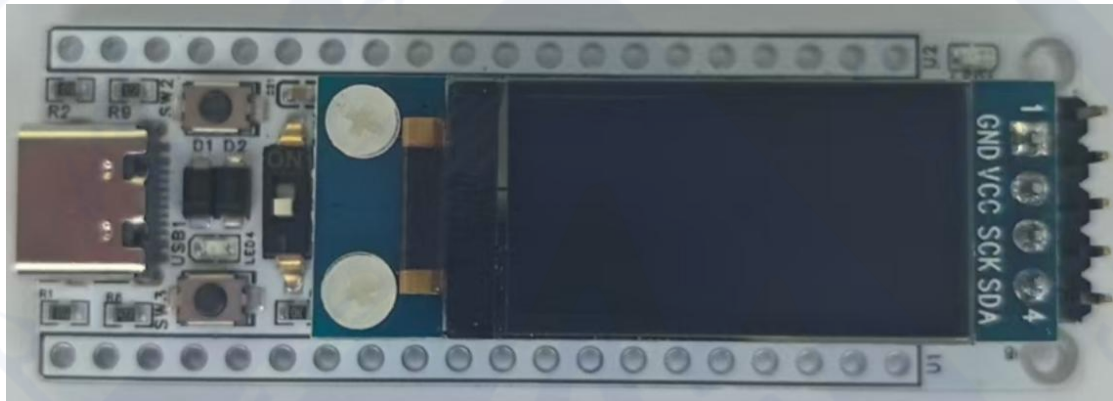
```
[RA4M2_MINI_Project2] FSP Configuration  hal_entry.c  startup.c  main.c
2
3
4 FSP_CPP_HEADER
5 void R_BSP_WarmStart(bsp_warm_start_event_t event);
6 FSP_CPP_FOOTER
7
8 bool g_external_irq_complete = 0;
9 /* Called from icu_irq_isr */
10 void external_irq6_callback (external_irq_callback_args_t * p_args)
11 {
12     (void) p_args;
13     g_external_irq_complete = !g_external_irq_complete;
14 }
15
16
17 /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This f
18 void hal_entry(void)
19 {
20     /* TODO: add your own code here */
21
22     fsp_err_t err = R_ICU_ExternalIrqOpen(&g_external_irq6_ctrl, &g_external_irq6_cfg);
23     assert(FSP_SUCCESS == err);
24
25     err = R_ICU_ExternalIrqEnable(&g_external_irq6_ctrl);
26     assert(FSP_SUCCESS == err);
27
28     while(1){
29         if(g_external_irq_complete)
30         {
31             R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_HIGH);
32         }
33         else
34         {
35             R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, BSP_IO_LEVEL_LOW);
36         }
37     }
38 }
39
40
41
42
43 #if BSP_TZ_SECURE_BUILD
44 /* Enter non-secure code */
45 R_BSP_NonSecureEnter();
46 #endif
47 }
48
49 /* * This function is called at various points during the startup process. This implementation uses the event that
50 void R_BSP_WarmStart(bsp_warm_start_event_t event)
51 {
52     if (BSP_WARM_START_RESET == event)
53     {
54         #if BSP_FEATURE_FLASH_LP_VERSION != 0
55             /* Enable reading from data flash. */
56             R_FACI_LP->DFLCTL = 1U;
57
58             /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the enable here, before clock
59              * C runtime initialization, should negate the need for a delay since the initialization will typically t
60         #endif
61     }
62
63     if (BSP_WARM_START_POST_C == event)
64     {
65         /* C runtime environment and system clocks are setup. */
66     }
67 }
68
69
70
71
72
```

project7: 定时器 GPT 输出 PWM

概述

GPT (通用 PWM 计时器) 模块是一个高精度定时器, 用于计数事件、测量外部输入信号、生成周期性中断, 或输出周期性信号/PWM 信号到 GTIOC 引脚。它广泛应用于定时

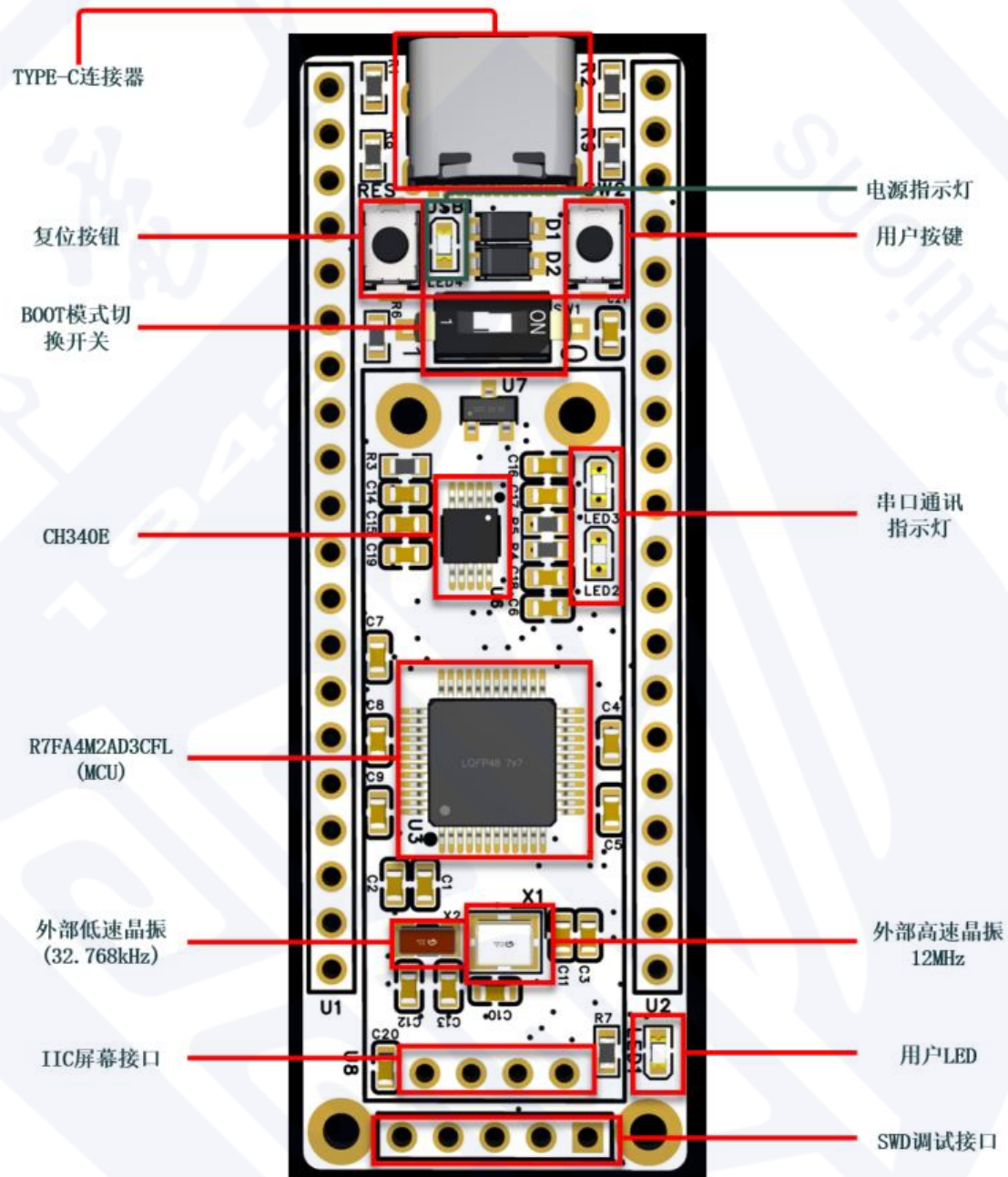
控制、信号测量、PWM 生成等应用，尤其适用于电机控制、LED 调光、PWM 伺服驱动和 ADC 触发等场景。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

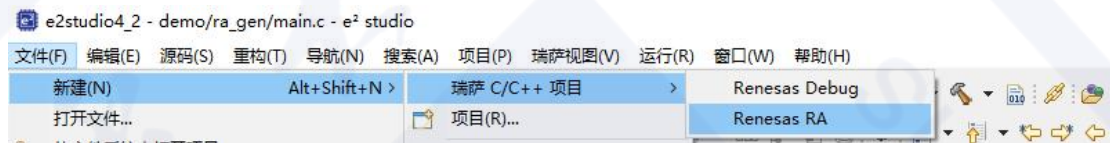


参考程序

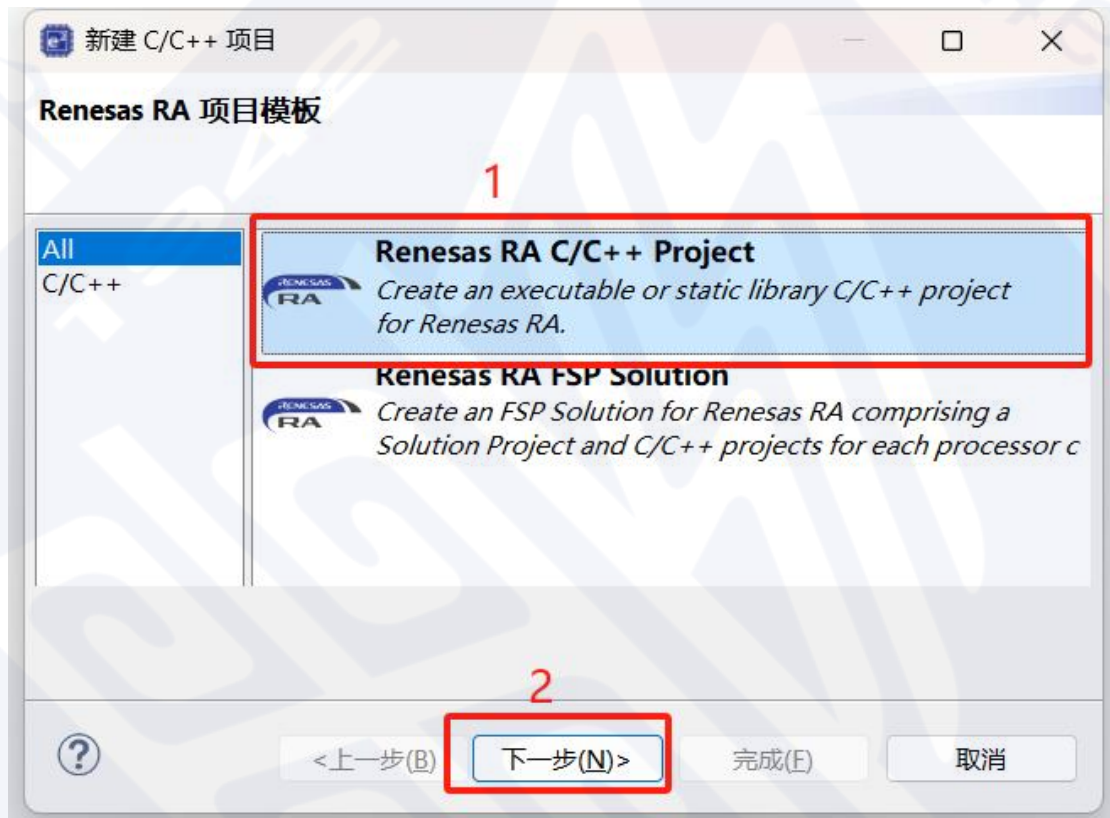
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

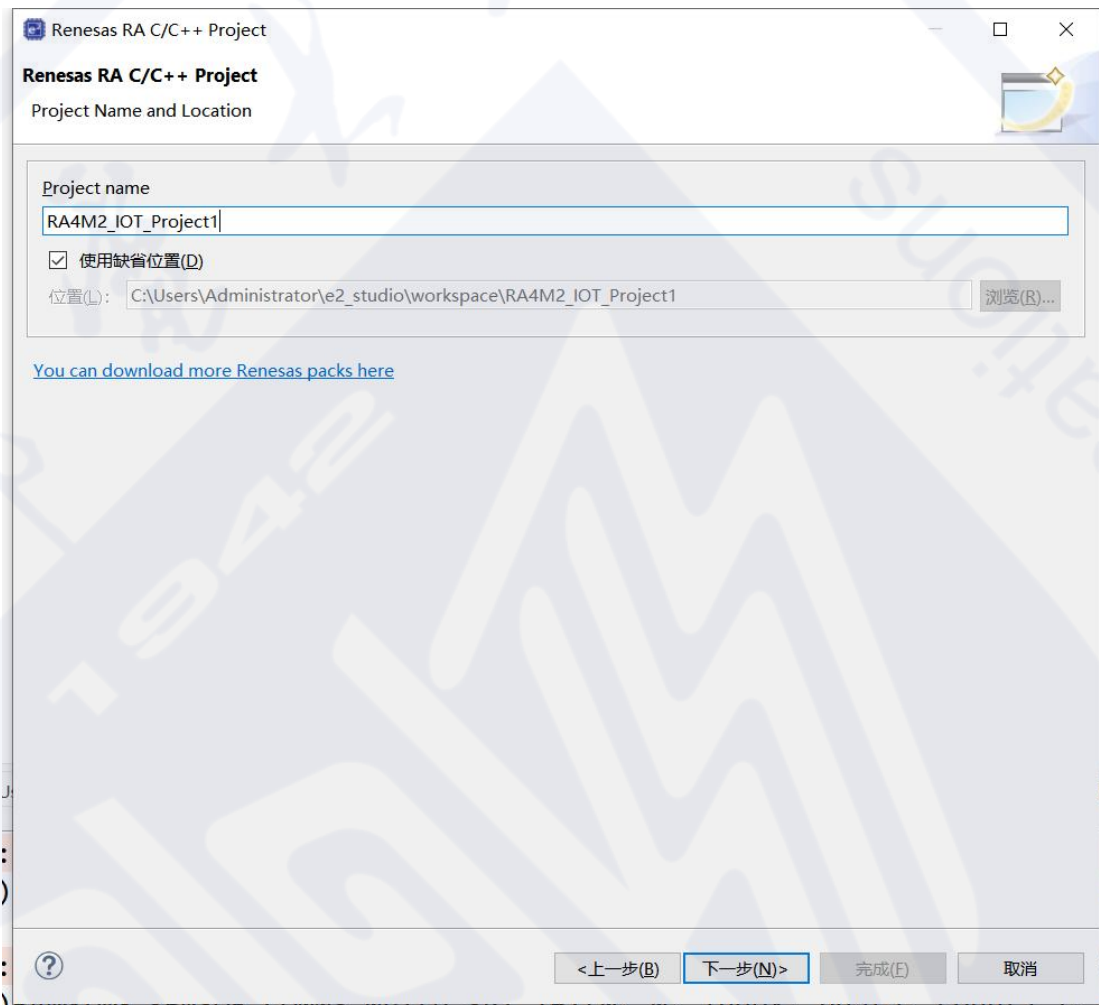
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

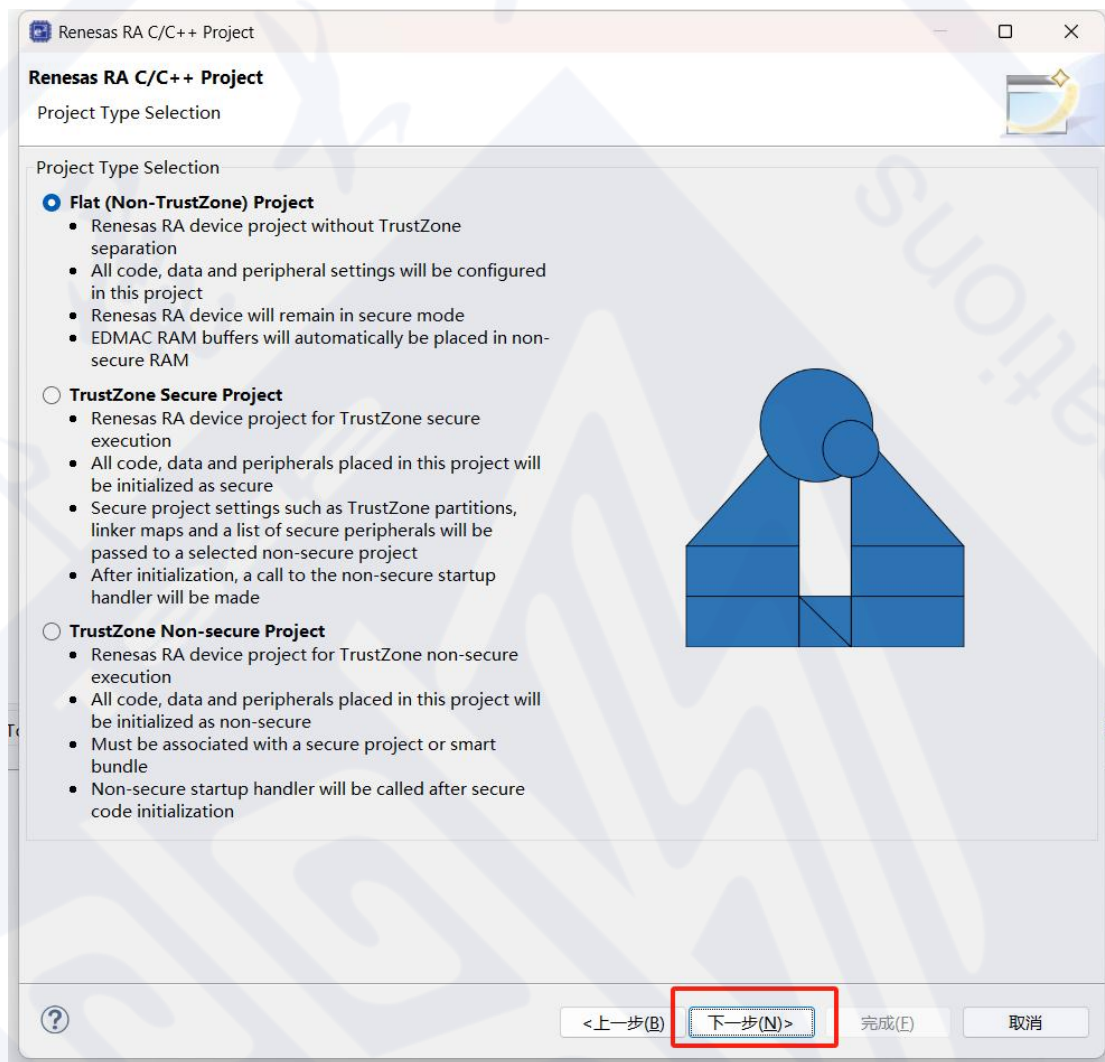
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

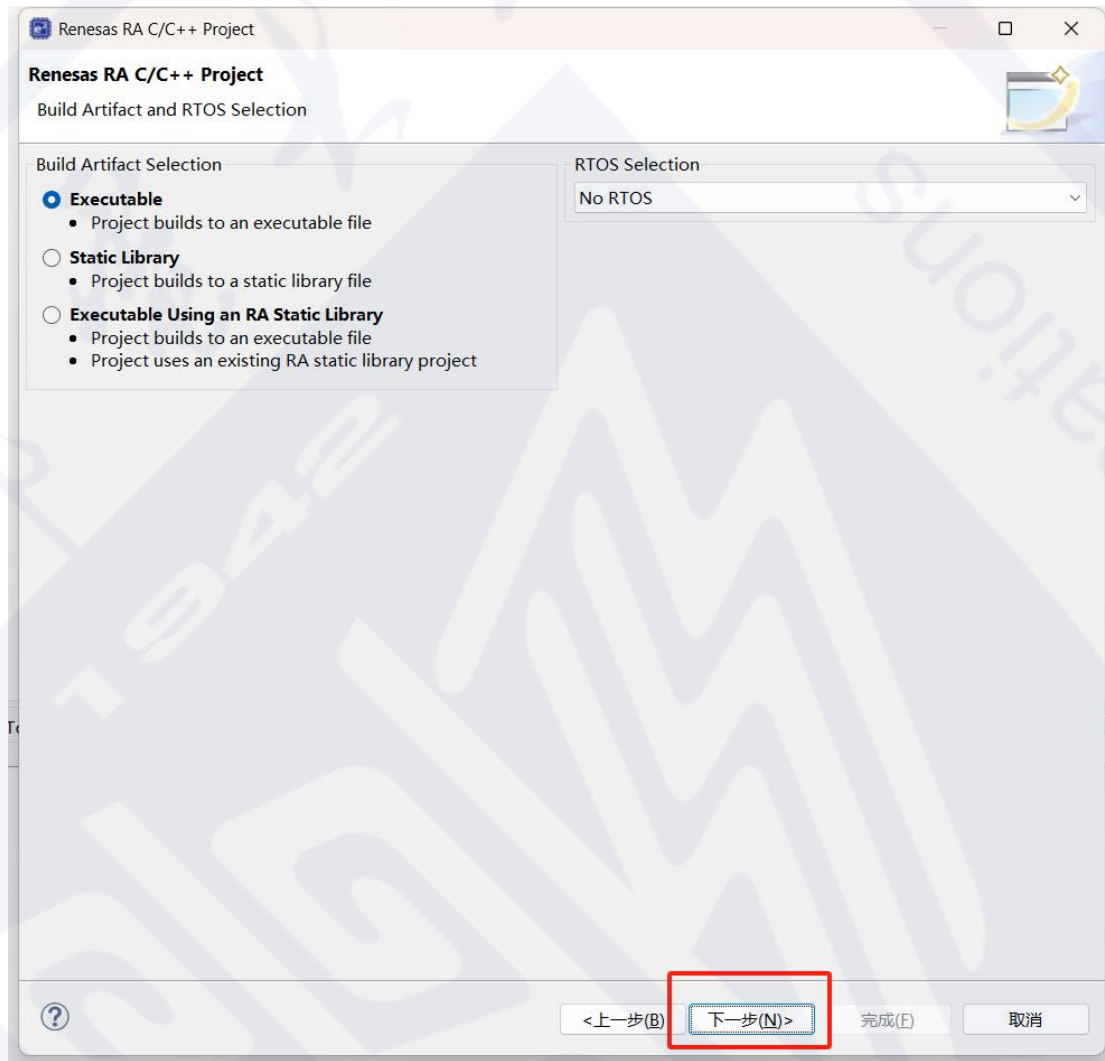
?

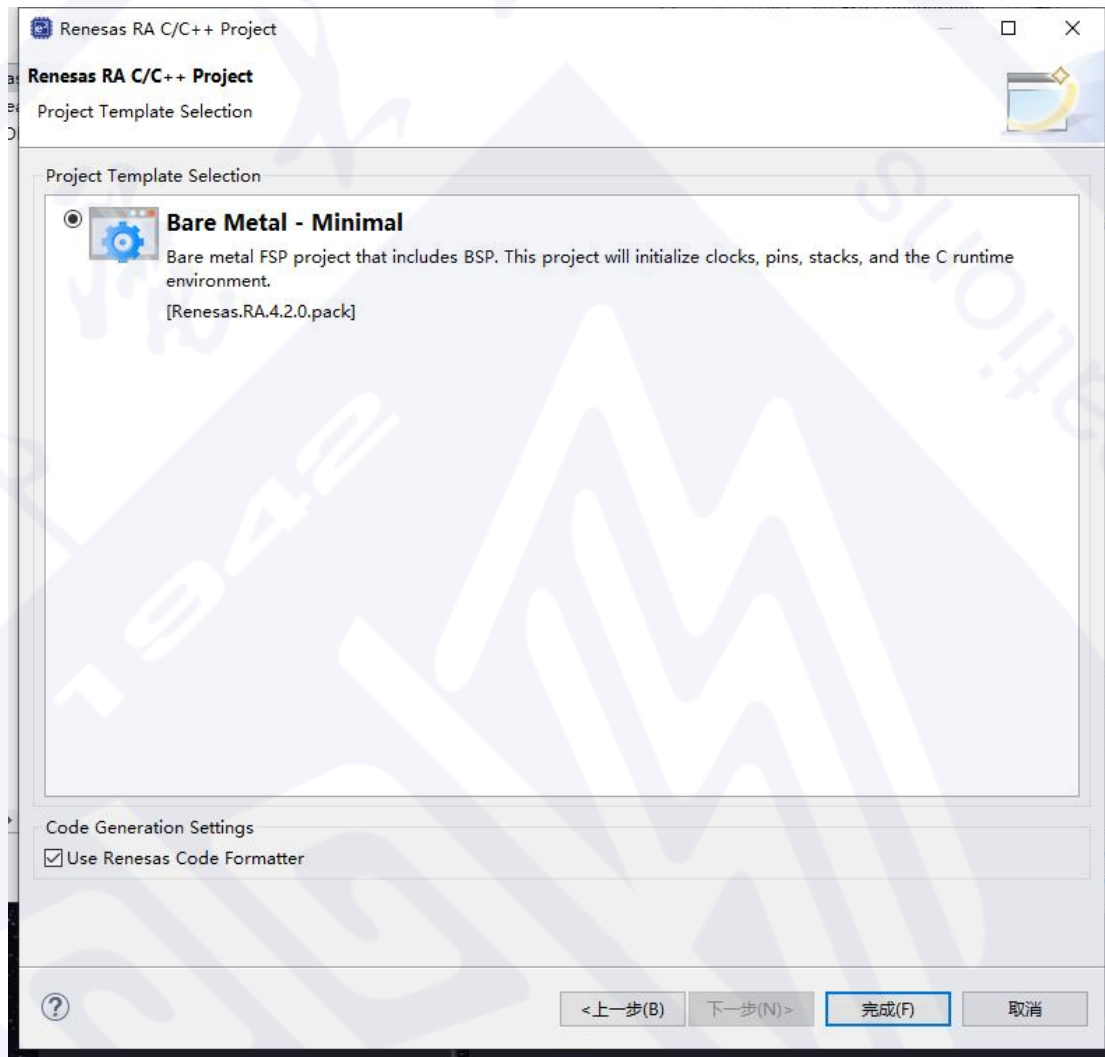
< 上一步(B)

下一步(N) >

完成(E)

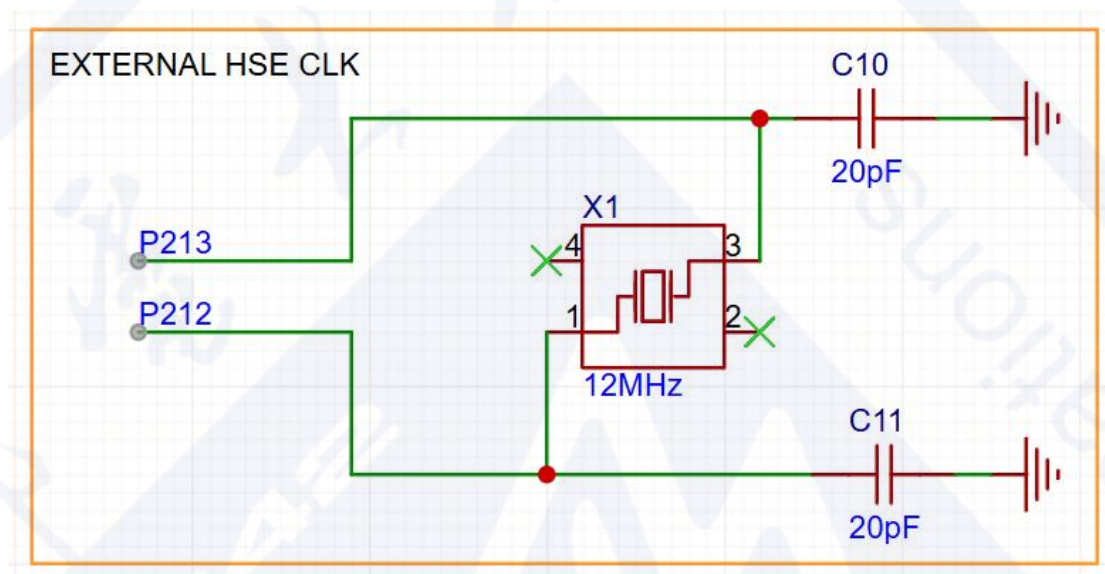
取消



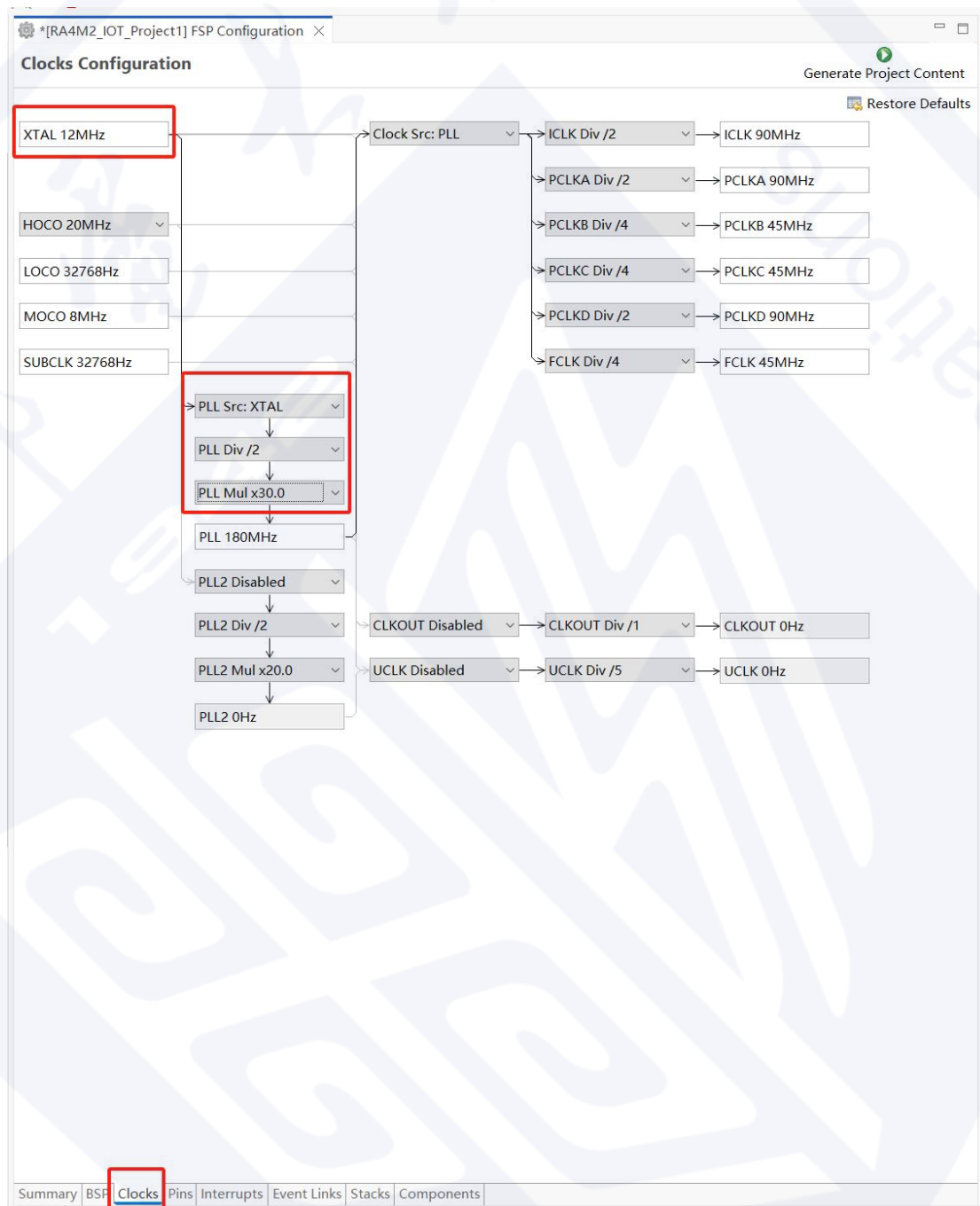


时钟设置

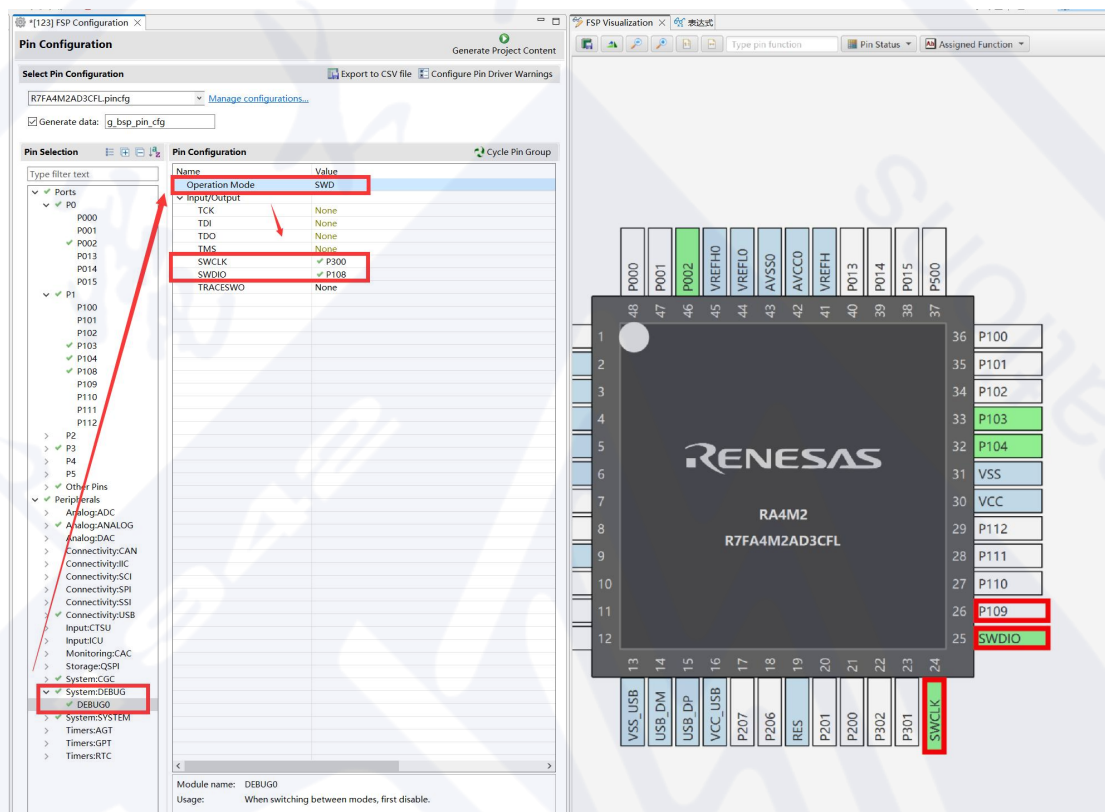
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

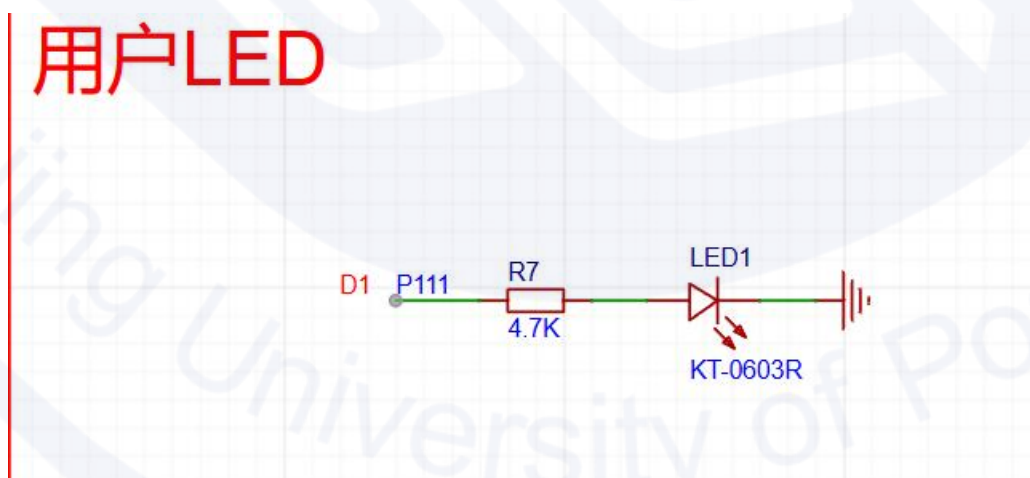


SWD 调试口设置

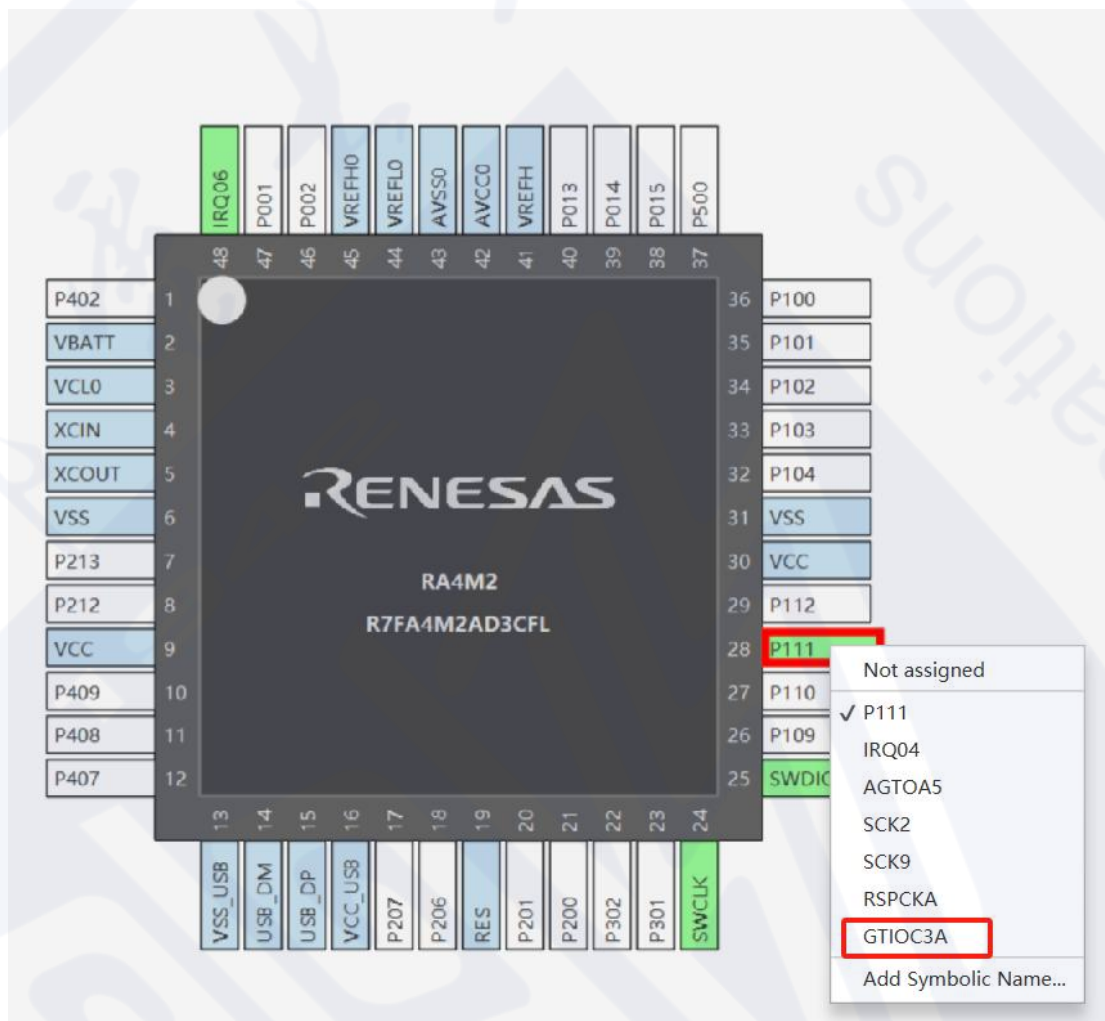


GPIO 口配置

由下图我们可以得知，板子上有 1 个 LED 灯，进行呼吸灯演示，故以 P111 管脚为例，配置方式如下所示。

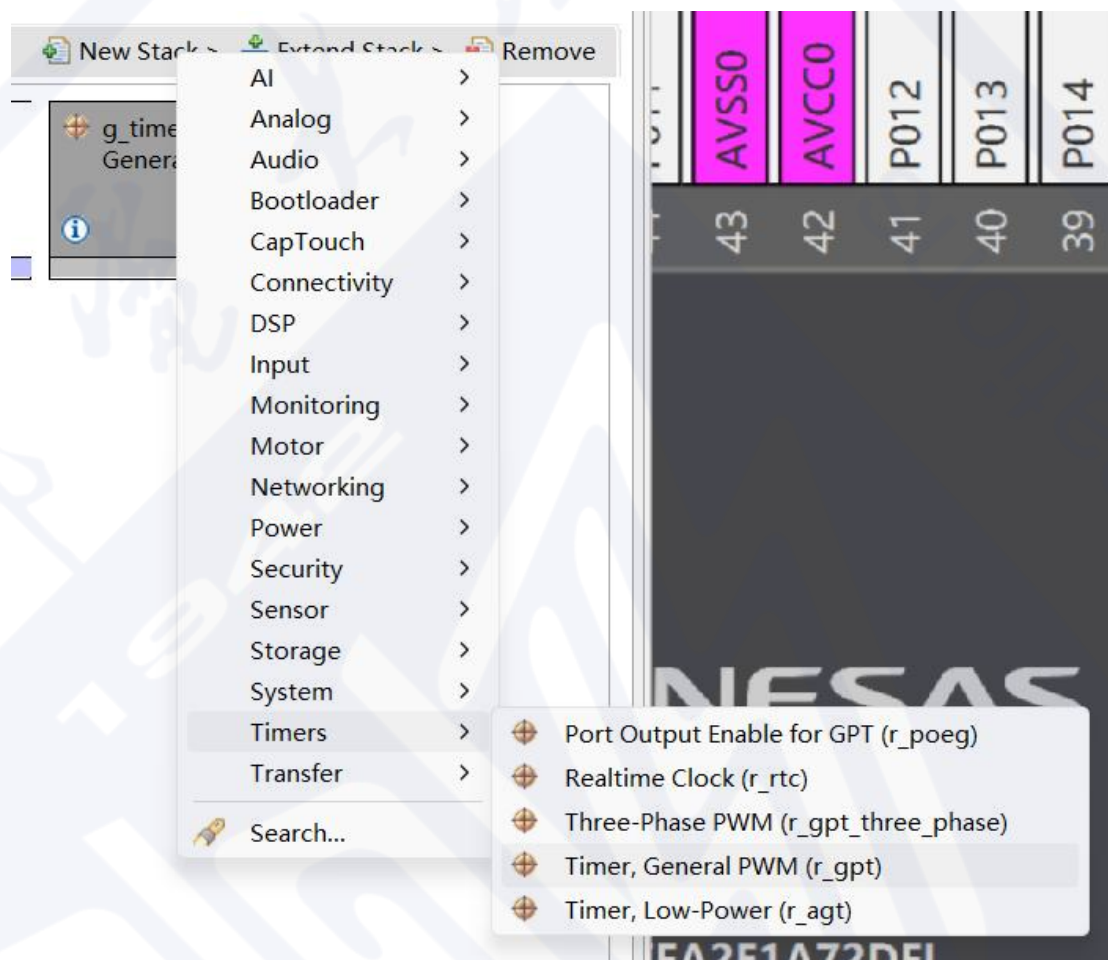


查看可视化界面可以得知，对应的是 GPT3。



GPT 定时器

操作 “New Stack > Timers > Timer, General PWM (r_gpt)” 在项目中添加 GPT 定时器的多通道操作功能，特别是用于生成 PWM 信号或单次脉冲信号。



GPT 定时器属性配置

- **Pin Output Support:** 设为 **Enabled**, 允许 PWM 信号输出到外部引脚。
- **Name:** 设置为 **g_timer3**, 这是模块的唯一标识符, 用于代码中引用此定时器实例。
- **Channel:** 设定为 **3**, 表示该定时器使用通道 **3** 进行计数和控制。
- **Mode:** 设为 **Periodic**, 表示该定时器工作在周期模式, 将在设定周期内重复运行。
- **Period:** 周期值设置为 **1**, 即每个完整计数周期的时间间隔。
- **Period Unit:** 单位选择为 **Kilohertz** (千赫兹), 表示输出信号的频率为 **1 kHz**。
- **Duty Cycle Percent:** 设置为 **50%**, 表示 PWM 输出信号的高电平时间占周期的 **50%**。
- **GTIOCA Output Enabled:** 设置为 **True**, 启用 GTIOCA (P111) 引脚的输出功能。
- **GTIOCA Stop Level:** 设为 **Pin Level Low**, 表示定时器停止时 GTIOCA 输出低电平。
- **GTIOCB Output Enabled:** 设置为 **True**, 启用 GTIOCB 引脚的输出功能。
- **GTIOCB Stop Level:** 设为 **Pin Level Low**, 表示定时器停止时 GTIOCB 输出低电平。

属性 × 问题 Smart Browser 控制台 调试 Memory Usage 搜索		
g_timer3 Timer, General PWM (r_gpt)		
Settings	属性	值
API Info	Common	
	Parameter Checking	Default (BSP)
	Pin Output Support	Enabled 启用在 GTIOCx 引脚上输出 PWM 波形的支持
	Write Protect Enable	Disabled
	Module g_timer3 Timer, General PWM (r_gpt)	
	General	
	Compare Match	
	Name	g_timer3 定时器名称
	Channel	3 通道
	Mode	Periodic 周期模式
	Period	1 1kHz周期
	Period Unit	Kilohertz 单位
	Output	
	Custom Waveform	
	Duty Cycle Percent (only applicable in PWM mode)	50 占空比
	GTIOCA Output Enabled	True 输出使能
	GTIOCA Stop Level	Pin Level Low 停止时电平
	GTIOCB Output Enabled	False 输出使能
	GTIOCB Stop Level	Pin Level Low 停止时电平
	Input	
	Interrupts	
	Extra Features	
	Pins	
	GTIOC3A	P111 输出引脚
	GTIOC3B	None

hal_entry.c startup.c main.c 0xefffffe [RA4M2_MINI_Project2] FSP Configuration ×

Pin Configuration

Select Pin Configuration

R7FA4M2AD3CFLpincfg
Manage configurations...

Export to CSV file
Configure Pin Driver Warnings

Generate data: g_bsp_pin_cfg

Pin Selection

Type filter text

P5
Other Pins
Peripherals
Analog:ADC
Analog:ANALOG
Analog:DAC
Connectivity:CAN
Connectivity:IC
Connectivity:SCI
Connectivity:SPI
Connectivity:SSI
Connectivity:USB
Input:CTSU
Input:ICU
Monitoring:CAC
Storage:QSPI
System:CGC
System:DEBUG
System:SYSTEM
Timers:AGT
Timers:GPT
GPT0
GPT1
GPT2
GPT3
GPT4
GPT5
GPT6
GPT_OP_0
GPT_POE0
GPT_POE1
GPT_POE2
GPT_POE3

Pin Configuration

Name
Value
Lock
Link

Pin Group Selection
Mixed
GTIOCA or GTIOCB

Operation Mode

Input/Output

GTIOC3A
P111

GTIOC3B
None

Module name: GPT3

Pin Function Pin Number

初始化 GPT

R_GPT_Open() 函数用于初始化通用 PWM 定时器 (GPT) 模块，并应用相应的配置。它的主要作用是启动定时器，使其按照设定的模式工作，例如周期模式 (Periodic Mode) 或 PWM 模式 (Pulse Width Modulation Mode)。

◆ R_GPT_Open()

```
fsp_err_t R_GPT_Open ( timer_ctrl_t *const p_ctrl,
                      timer_cfg_t const *const p_cfg
                      )
```

Initializes the timer module and applies configurations. Implements `timer_api_t::open`.

GPT hardware does not support one-shot functionality natively. When using one-shot mode, the timer will be stopped in an ISR after the requested period has elapsed.

The GPT implementation of the general timer can accept a `gpt_extended_cfg_t` extension parameter.

Example:

```
/* Initializes the module. */
err = R_GPT_Open(&g_timer0_ctrl, &g_timer0_cfg);
```

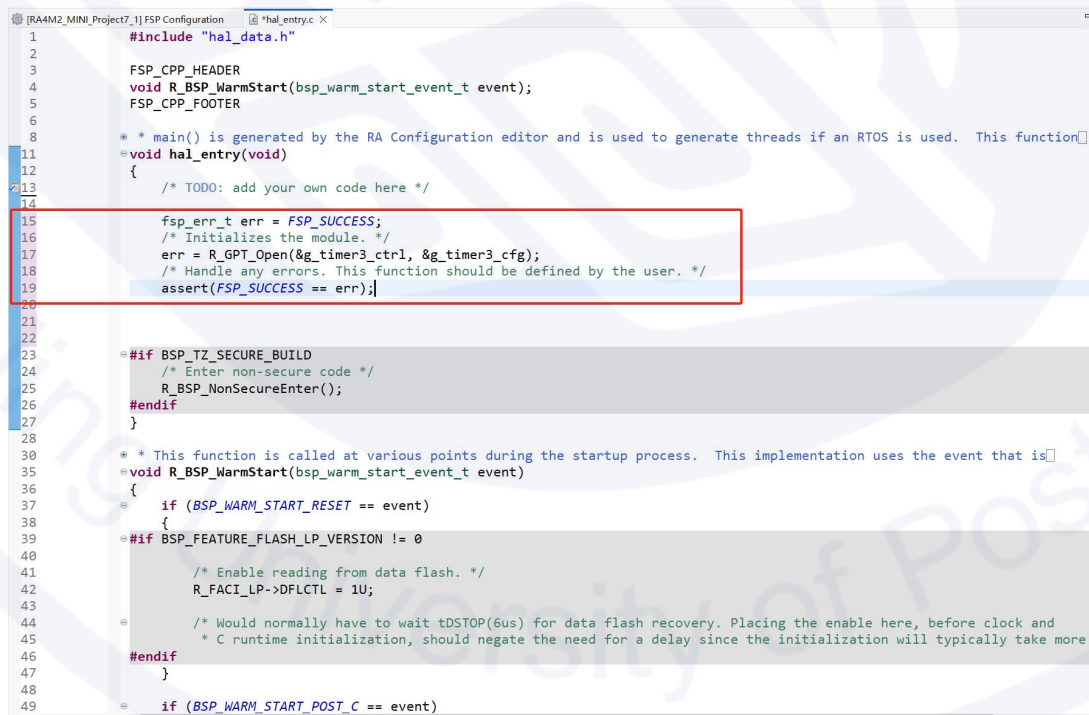
Return values

FSP_SUCCESS	Initialization was successful and timer has started.
FSP_ERR_ASSERTION	A required input pointer is NULL or the source divider is invalid.
FSP_ERR_ALREADY_OPEN	Module is already open.
FSP_ERR_IRQ_BSP_DISABLED	<code>timer_cfg_t::mode</code> is <code>TIMER_MODE_ONE_SHOT</code> or <code>timer_cfg_t::p_callback</code> is not NULL, but ISR is not enabled. ISR must be enabled to use one-shot mode or callback.
FSP_ERR_INVALID_MODE	Triangle wave PWM is only supported if <code>GPT_CFG_OUTPUT_SUPPORT_ENABLE</code> is 2. Selected channel does not support external count sources. External and event count sources not are available in this mode.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The channel requested in the <code>p_cfg</code> parameter is not available on this device.

```
fsp_err_t err = FSP_SUCCESS;

/* Initializes the module. */
err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);

/* Handle any errors. This function should be defined by the user. */
assert(FSP_SUCCESS == err);
```



```
#include "hal_data.h"

FSP_CPP_HEADER
void R_BSP_WarmStart(bsp_warm_start_event_t event);
FSP_CPP_FOOTER

/* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function
 * void hal_entry(void)
 * {
 *     /* TODO: add your own code here */
 *
 *     fsp_err_t err = FSP_SUCCESS;
 *     /* Initializes the module. */
 *     err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);
 *     /* Handle any errors. This function should be defined by the user. */
 *     assert(FSP_SUCCESS == err);
 *
 *     #if BSP_TZ_SECURE_BUILD
 *     /* Enter non-secure code */
 *     R_BSP_NonSecureEnter();
 *     #endif
 *
 *     /* This function is called at various points during the startup process. This implementation uses the event that is
 *     void R_BSP_WarmStart(bsp_warm_start_event_t event)
 *     {
 *         if (BSP_WARM_START_RESET == event)
 *         {
 *             #if BSP_FEATURE_FLASH_LP_VERSION != 0
 *
 *             /* Enable reading from data flash. */
 *             R_FACI_LP->DFLCTL = 1U;
 *
 *             /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the enable here, before clock and
 *             * C runtime initialization, should negate the need for a delay since the initialization will typically take more
 *             #endif
 *         }
 *
 *         if (BSP_WARM_START_POST_C == event)
```

启动 GPT PWM 模块

`R_GPT_Start()` 用于启动 GPT (General PWM Timer) 定时器, 使其开始按照之前的配置运行。

必须先调用 `R_GPT_Open()`, 否则 `R_GPT_Start()` 会返回 `FSP_ERR_NOT_OPEN` 错误。

在 PWM 模式下, 启动定时器后会按照设定的周期和占空比输出 PWM 波形。

◆ `R_GPT_Start()`

```
fsp_err_t R_GPT_Start ( timer_ctrl_t *const p_ctrl )
```

Starts timer. Implements `timer_api_t::start`.

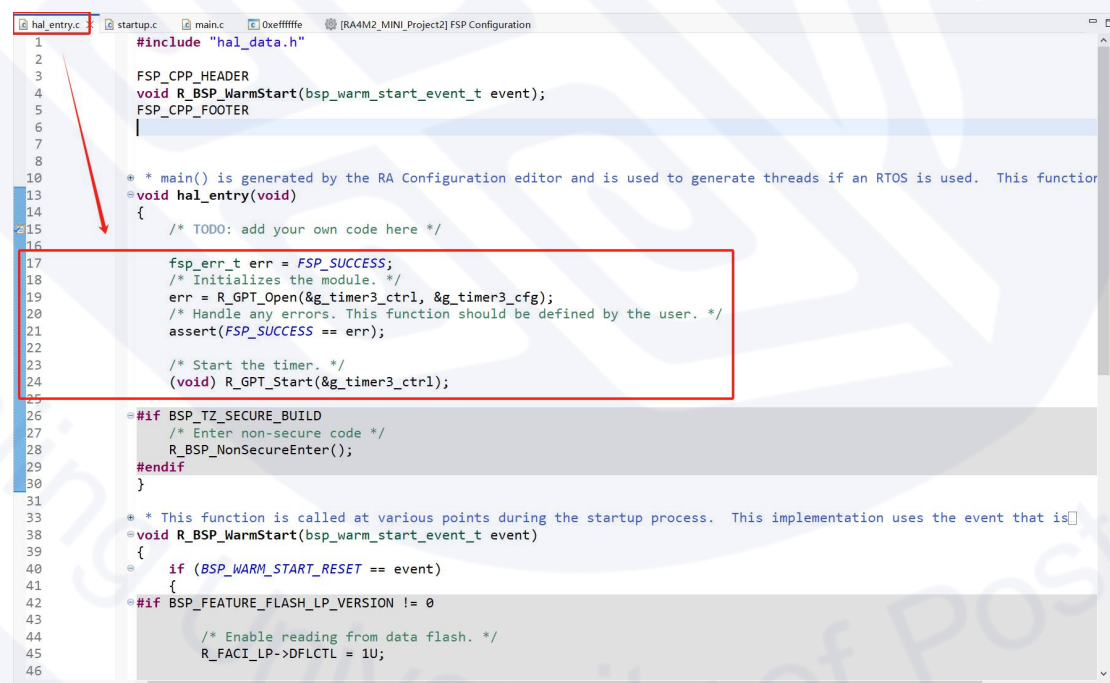
Example:

```
/* Start the timer. */  
(void) R_GPT_Start(&g_timer0_ctrl);
```

Return values

`FSP_SUCCESS` Timer successfully started.
`FSP_ERR_ASSERTION` `p_ctrl` was NULL.
`FSP_ERR_NOT_OPEN` The instance is not opened.

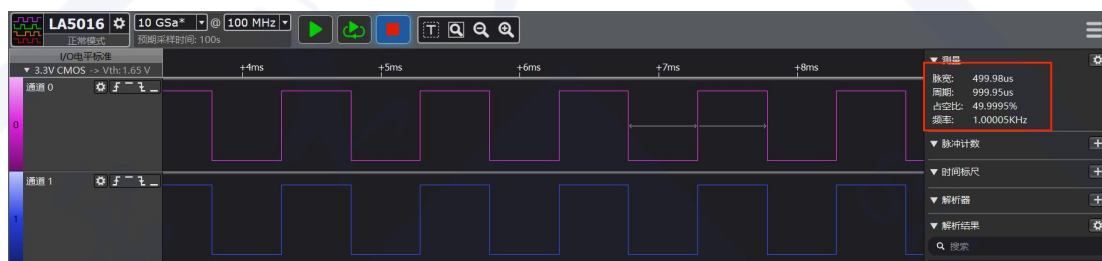
```
/* Start the timer. */  
  
(void) R_GPT_Start (&g_timer3_ctrl);
```



```
1  #include "hal_data.h"  
2  
3  FSP_CPP_HEADER  
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);  
5  FSP_CPP_FOOTER  
6  
7  
8  
9  
10  
11  
12  
13  /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function  
14  * is called at various points during the startup process. This implementation uses the event that is  
15  * defined in the RA Configuration editor. */  
16  void hal_entry(void)  
17  {  
18      /* TODO: add your own code here */  
19  
20      fsp_err_t err = FSP_SUCCESS;  
21      /* Initializes the module. */  
22      err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);  
23      /* Handle any errors. This function should be defined by the user. */  
24      assert(FSP_SUCCESS == err);  
25  
26      /* Start the timer. */  
27      (void) R_GPT_Start(&g_timer3_ctrl);  
28  
29  #if BSP_TZ_SECURE_BUILD  
30      /* Enter non-secure code */  
31      R_BSP_NonSecureEnter();  
32  #endif  
33  
34  /* * This function is called at various points during the startup process. This implementation uses the event that is  
35  * defined in the RA Configuration editor. */  
36  void R_BSP_WarmStart(bsp_warm_start_event_t event)  
37  {  
38      if (BSP_WARM_START_RESET == event)  
39      {  
40          /* Enable reading from data flash. */  
41          R_FACI_LP->DFLCTL = 1U;  
42      }  
43  }  
44  
45  
46
```

演示

输出 1k 占空比 50%的 PWM。

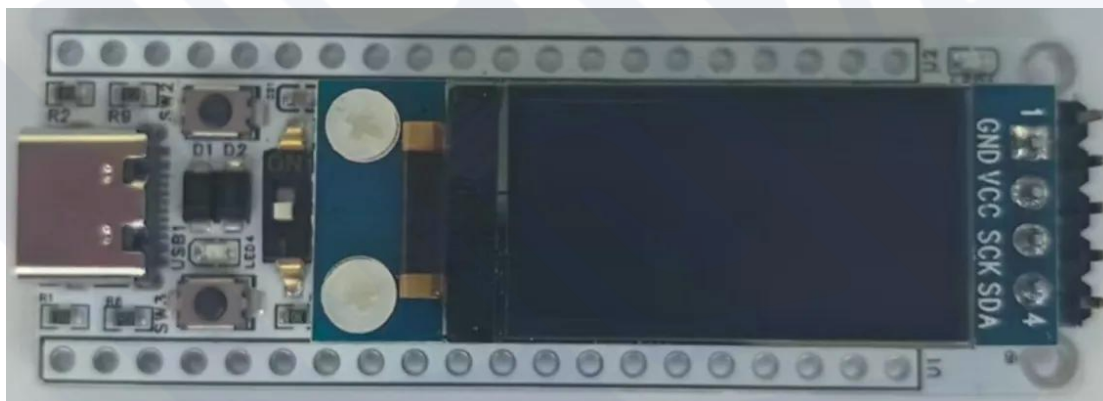


project8: GPT 定时器频率与占空比的设置

概述

GPT（通用 PWM 计时器）模块是一个高精度定时器，用于计数事件、测量外部输入信号、生成周期性中断，或输出周期性信号/PWM 信号到 GTIOC 引脚。它广泛应用于定时控制、信号测量、PWM 生成等应用，尤其适用于电机控制、LED 调光、PWM 伺服驱动和 ADC 触发等场景。

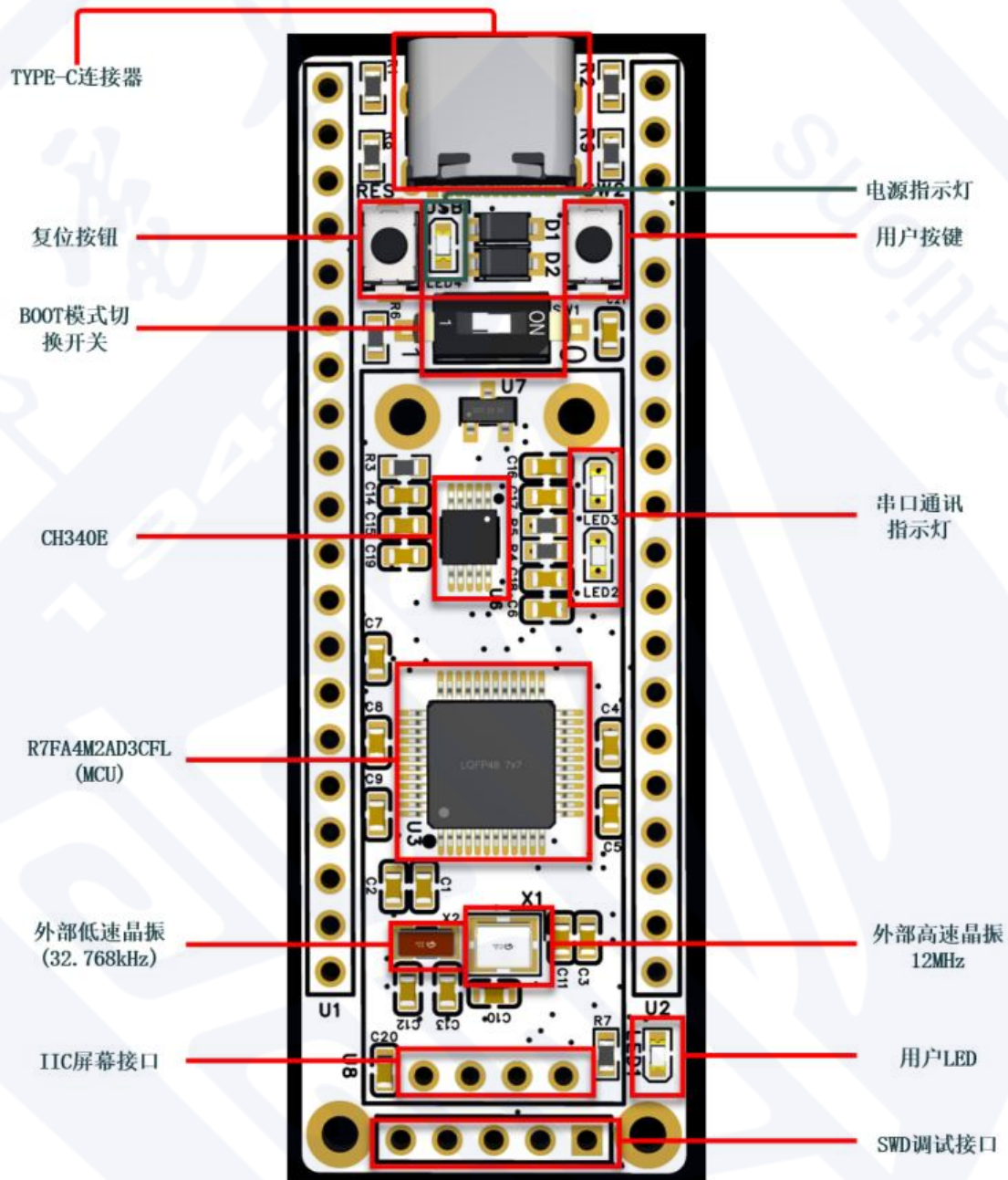
本文将详细介绍如何上一章基础上通过 GPT 定时器的 API 调整频率和占空比，并解释相关的关键概念和设置方法。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

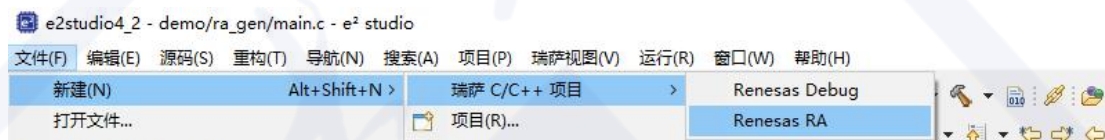


参考程序

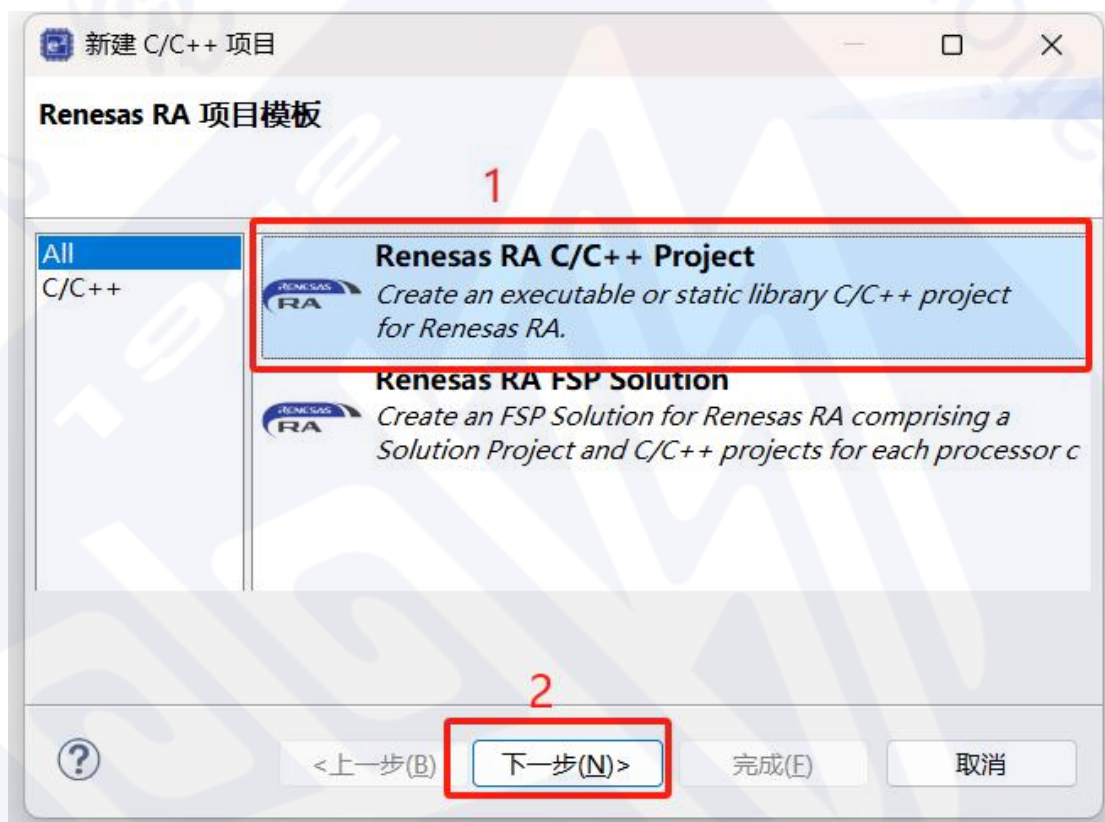
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

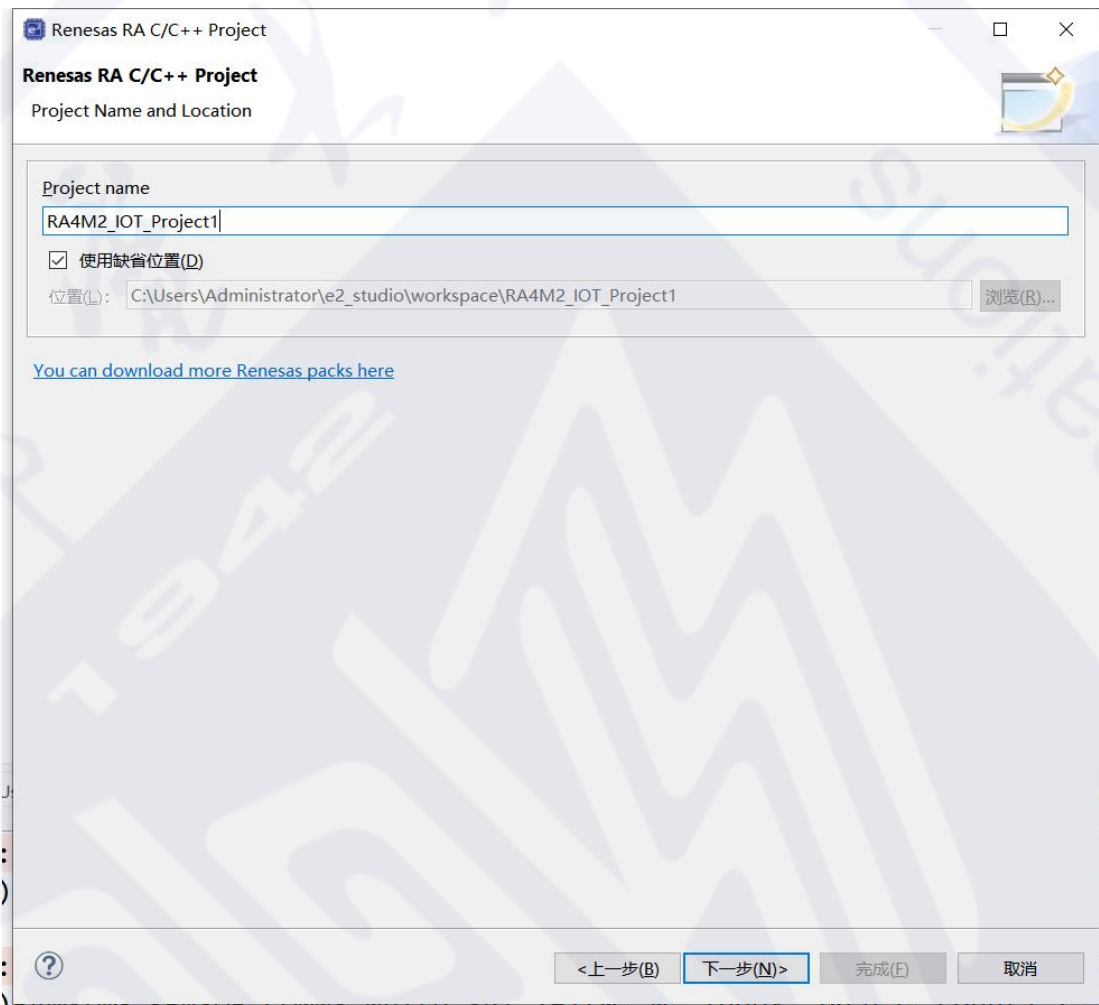
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

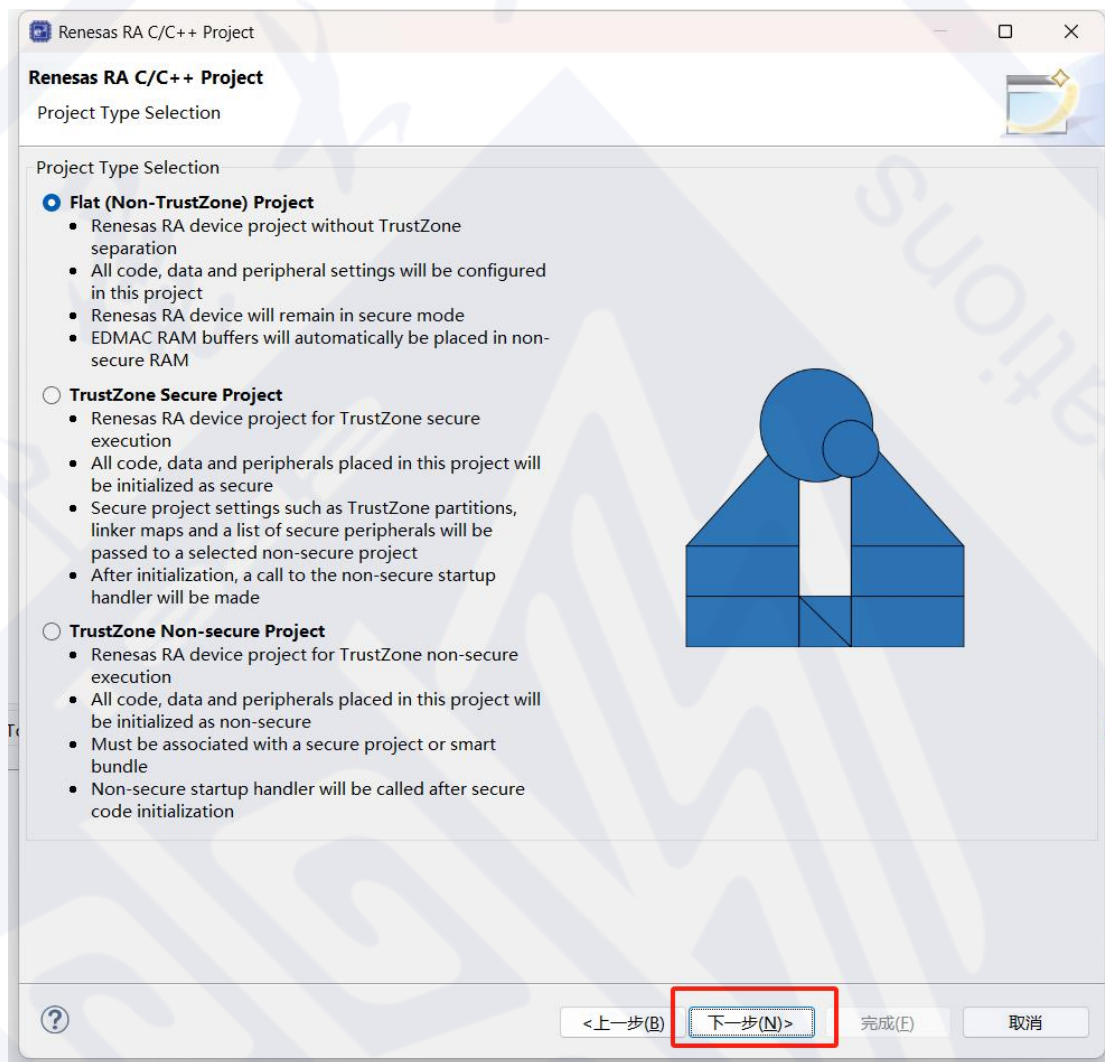
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

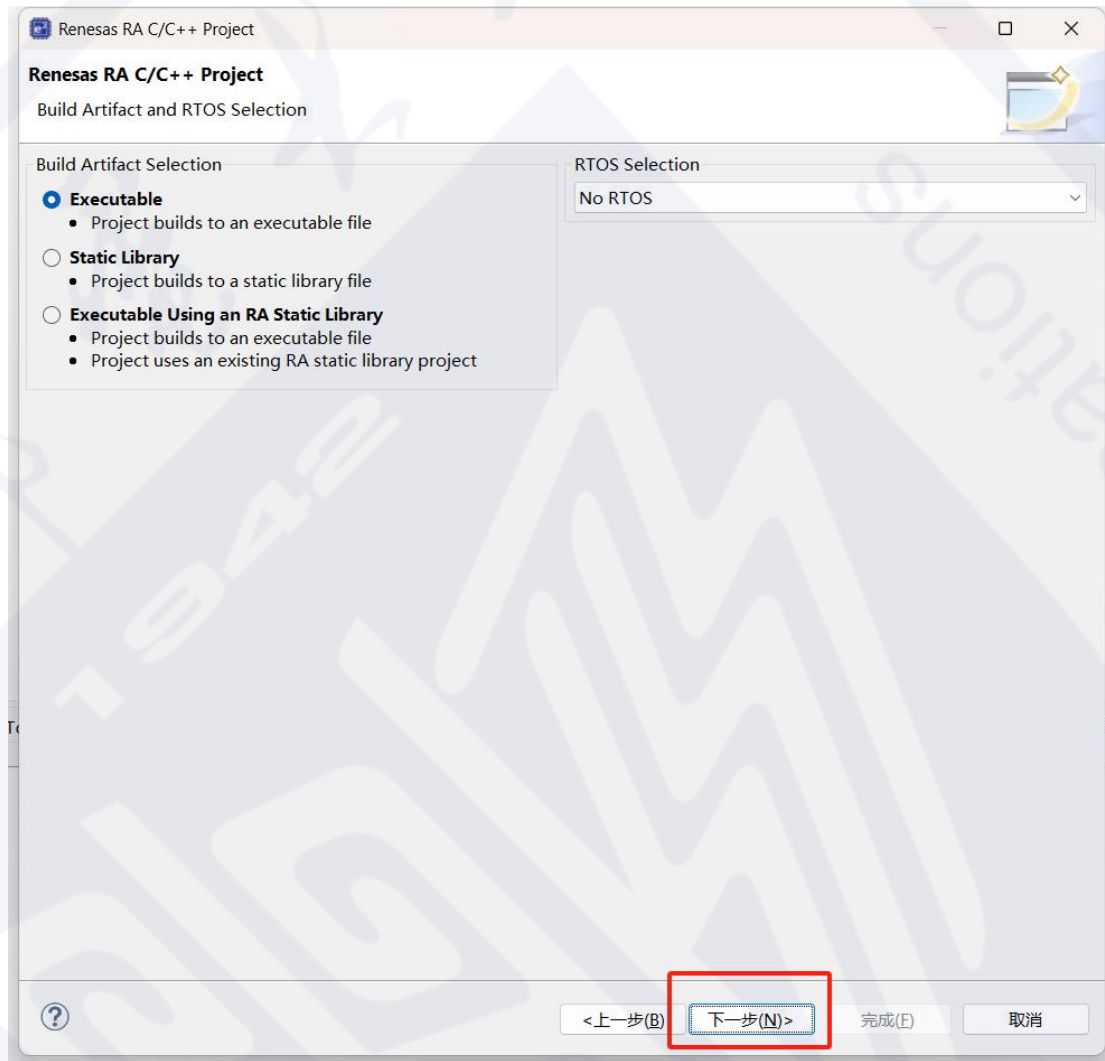
?

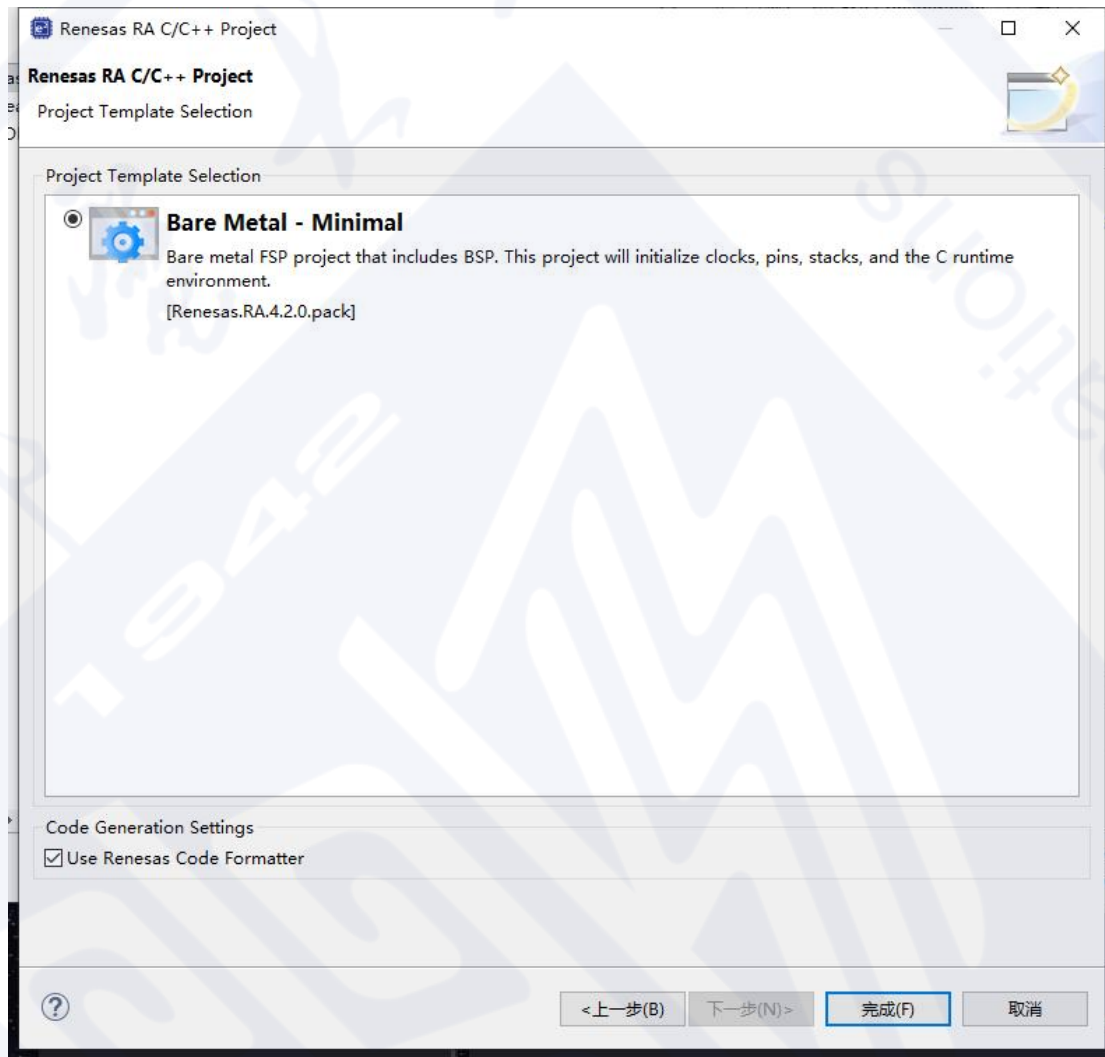
< 上一步(B)

下一步(N) >

完成(E)

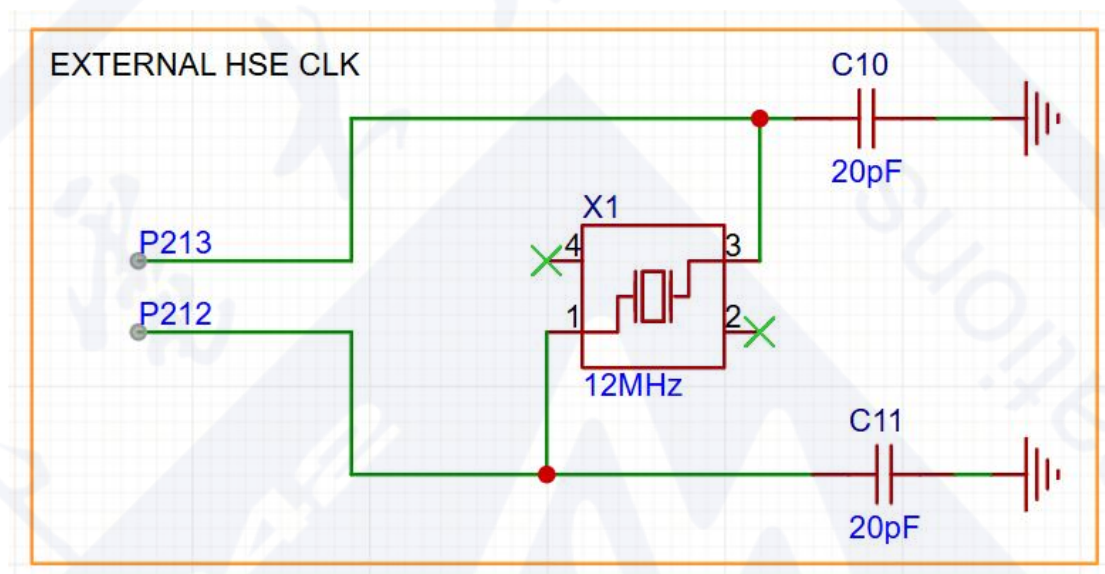
取消



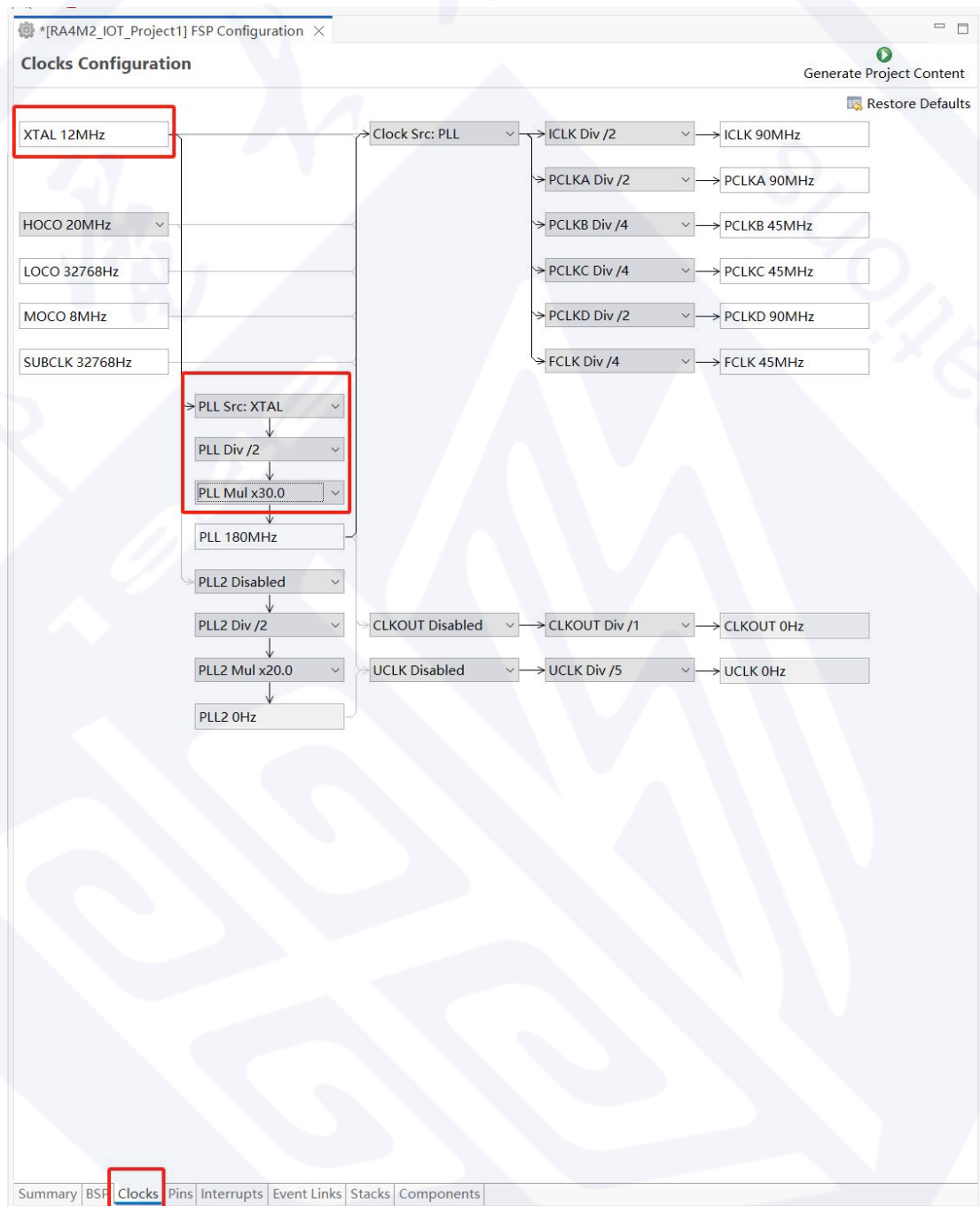


时钟设置

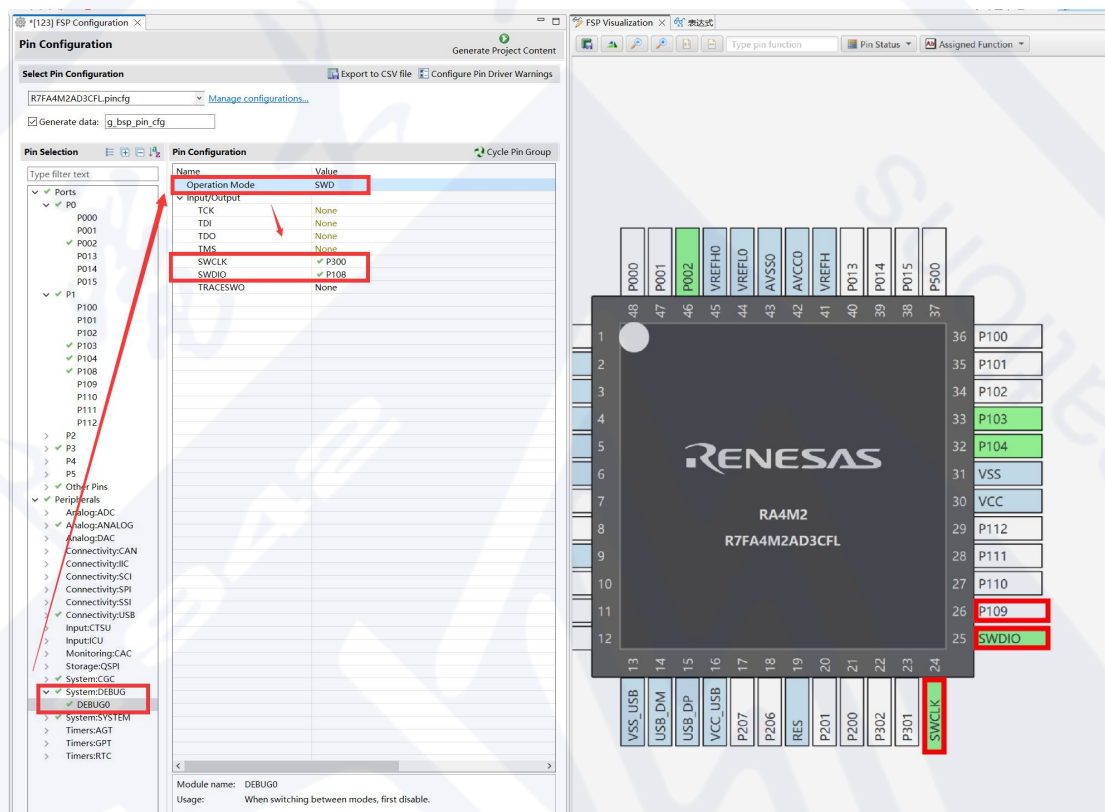
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。



SWD 调试口设置



选择计时器

RA MCU 有两个定时器外设：通用 PWM 定时器 (GPT) 和异步通用定时器 (AGT)。

	GPT	AGT
低功耗模式	GPT 可以在睡眠模式下运行。	AGT 可以在所有低功耗模式下运行。
可用频道	GPT 通道的数量因设备而异。目前所有支持的 MCU 至少都有 7 个 GPT 通道。	所有 MCU 都有 2 个 AGT 通道。
定时器分辨率	所有 MCU 均至少有一个 32 位 GPT 计时器。	AGT 计时器是 16 位计时器。
时钟源	GPT 通过可配置分频器（最高可达 1024）的 PCLKD 运行。它还可以配置为计数 ELC 事件或外部脉冲。	AGT 通过 PCLKB、LOCO 或子时钟运行。

时钟源

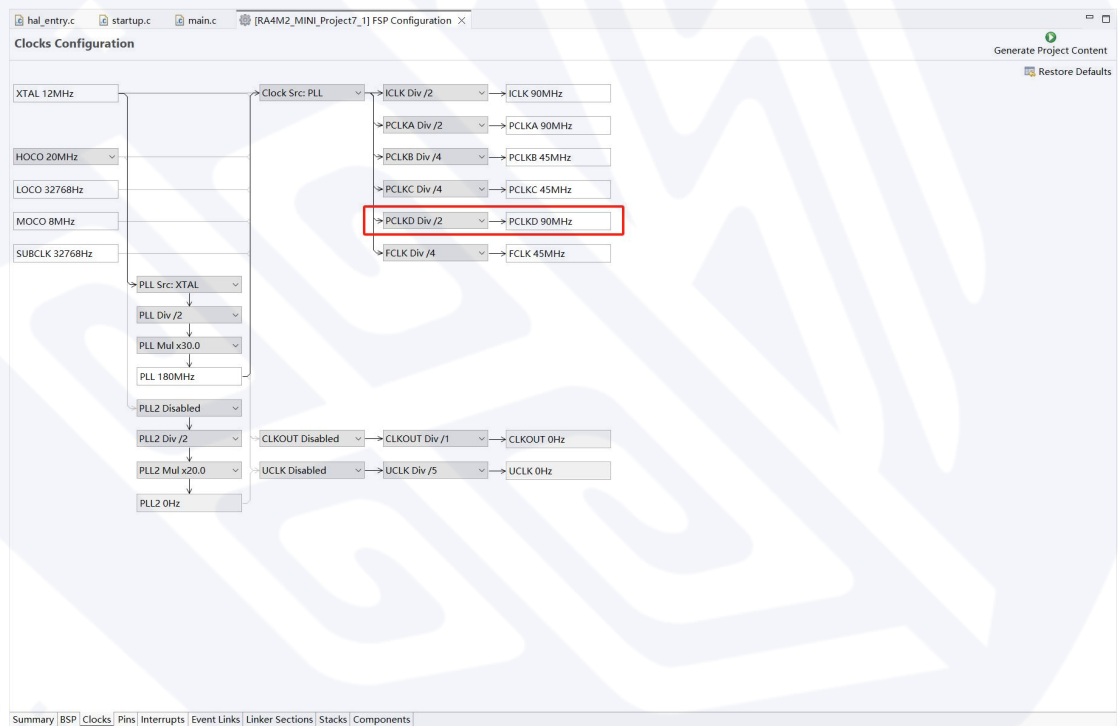
GPT 使用 PCLKD（外设时钟 D）作为主时钟源。该时钟可以通过 可配置的分频器 进行调整，最大分频因子为 1024。

Selecting a Timer

RA MCUs have two timer peripherals: the General PWM Timer (GPT) and the Asynchronous General Purpose Timer (AGT). When selecting between them, consider these factors:

	GPT	AGT
Low Power Modes	The GPT can operate in sleep mode.	The AGT can operate in all low power modes.
Available Channels	The number of GPT channels is device specific. All currently supported MCUs have at least 7 GPT channels.	All MCUs have 2 AGT channels.
Timer Resolution	All MCUs have at least one 32-bit GPT timer.	The AGT timers are 16-bit timers.
Clock Source	The GPT runs off PCLKD with a configurable divider up to 1024. It can also be configured to count ELC events or external pulses.	The AGT runs off PCLKB, LOCO, or subclock.

选择 XTAL 12MHz 作为时钟源（Clock Src: XTAL）。PCLKD 时钟的分频器设置为 Div /2，意味着 PCLKD 直接运行在 90MHz。



PWM(脉冲宽度调制)

脉冲宽度调制是一种模拟控制方式，根据相应载荷的变化来调制晶体管基极或 MOS 管栅极的偏置，来实现晶体管或 MOS 管导通时间的改变，从而实现开关稳压电源输出的改变。这种方式能使电源的输出电压在工作条件变化时保持恒定，是利用微处理器的数字信号对模拟电路进行控制的一种非常有效的技术。脉冲宽度调制是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术，广泛应用在从测量、通信到功率控制与变换的许多领域中。

在瑞萨 RA 系列 MCU 中有两种定时器，一种是通用 PWM 定时器 GPT，另外一种是异步通用定时器 AGT。

频率=主频/period
+ 占空比=cycle/period

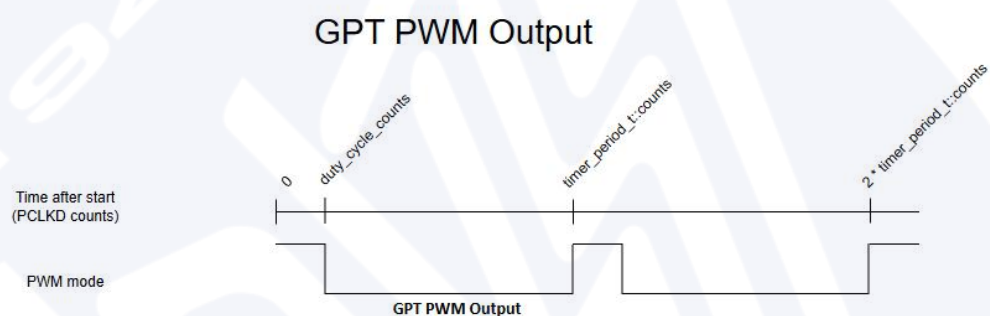
下图中展示的 PWM 输出信号，在周期开始时为高电平，周期结束时为低电平。通过调整占空比（duty cycle）和周期（timer period），可以调节 PWM 信号的行为。

- **duty_cycle_counts**: 表示占空比部分的持续时间。占空比决定了信号高电平的时间比例。
- **timer_period_counts**: 这是一个周期的时间，单位是 PCLKD 计数（即时钟频率的周期数）。

PWM Output

The PWM output signal is high at the beginning of the cycle and low at the end of the cycle.

Examples of PWM signals that can be generated by this module are shown below:



更新周期和占空比: 周期和占空比在下次定时器溢出后会更新，更新由 `R_GPT_PeriodSet()` 或 `R_GPT_DutyCycleSet()` 函数触发。如果需要在下次溢出之前强制更新这些值，可以调用 `R_GPT_Reset()`，这样定时器会被重置并开始计数。

Updating Period and Duty Cycle

The period and duty cycle are updated after the next counter overflow after calling `R_GPT_PeriodSet()` or `R_GPT_DutyCycleSet()`. To force them to update before the next counter overflow, call `R_GPT_Reset()` while the counter is running.

Note

When manually changing the timer period counts the maximum value for a 32-bit GPT is 0x100000000. This number overflows the 32-bit value for `timer_cfg_t::period_counts`. To configure the timer for the maximum period, set `timer_cfg_t::period_counts` to 0.

R_GPT_PeriodSet() 函数说明

`R_GPT_PeriodSet` 该函数用于设置定时器的周期值。若定时器正在运行，则在下次溢出时更新周期；如果定时器已停止，该函数将重置计数器并更新周期。

◆ R_GPT_PeriodSet()

```
fsp_err_t R_GPT_PeriodSet ( timer_ctrl_t *const p_ctrl,
                          uint32_t const   period_counts
                          )
```

Sets period value provided. If the timer is running, the period will be updated after the next counter overflow. If the timer is stopped, this function resets the counter and updates the period. Implements `timer_api_t::periodSet`.

Warning

If periodic output is used, the duty cycle buffer registers are updated after the period buffer register. If this function is called while the timer is running and a GPT overflow occurs during processing, the duty cycle will not be the desired 50% duty cycle until the counter overflow after processing completes.

Example:

```
/* Get the source clock frequency (in Hz). There are 3 ways to do this in FSP:
 * - If the PCLKD frequency has not changed since reset, the source clock frequency is
 *   BSP_STARTUP_PCLKD_HZ >> timer_cfg_t::source_div
 * - Use the R_GPT_InfoGet function (it accounts for the divider).
 * - Calculate the current PCLKD frequency using R_FSP_SystemClockHzGet(FSP_PRIV_CLOCK_PCLKD) and right shift
 *   by timer_cfg_t::source_div.
 * This example uses the 3rd option (R_FSP_SystemClockHzGet).
 */
uint32_t pclk_freq_hz = R_FSP_SystemClockHzGet(FSP_PRIV_CLOCK_PCLKD) >> g_timer0_cfg.source_div;

/* Calculate the desired period based on the current clock. Note that this calculation could overflow if the
 * desired period is larger than UINT32_MAX / pclk_freq_hz. A cast to uint64_t is used to prevent this. */
uint32_t period_counts =
    (uint32_t) ((uint64_t) pclk_freq_hz * GPT_EXAMPLE_DESIRED_PERIOD_MSEC / GPT_EXAMPLE_MSEC_PER_SEC);

/* Set the calculated period. */
err = R_GPT_PeriodSet(&g_timer0_ctrl, period_counts);
assert(FSP_SUCCESS == err);
```

Return values

FSP_SUCCESS Period value written successfully.
FSP_ERR_ASSERTION p_ctrl was NULL.
FSP_ERR_NOT_OPEN The instance is not opened.

频率=时钟源/period, 若设置频率为 10K, 则 period=90M/10K=9000

```
err = R_GPT_PeriodSet(&g_timer3_ctrl, 9000); //频率
assert(FSP_SUCCESS == err);

R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功
```

```
1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7
8  /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function
11 void hal_entry(void)
12 {
13     /* TODO: add your own code here */
14
15     fsp_err_t err = FSP_SUCCESS;
16     /* Initializes the module. */
17     000000d0 err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);
18     /* Handle any errors. This function should be defined by the user. */
19     000000da assert(FSP_SUCCESS == err);
20
21     /* Start the timer. */
22     000000dc (void) R_GPT_Start(&g_timer3_ctrl);
23
24     err = R_GPT_PeriodSet(&g_timer3_ctrl, 9000); //频率
25     assert(FSP_SUCCESS == err);
26     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功
27
28
29
30
31 #if BSP_TZ_SECURE_BUILD
32     /* Enter non-secure code */
33     R_BSP_NonSecureEnter();
34 #endif
35 }
36
37 /* This function is called at various points during the startup process. This implementation uses the event that is
43 void R_BSP_WarmStart(bsp_warm_start_event_t event)
44 {
45     if (BSP_WARM_START_RESET == event)
46     {
47         #if BSP_FEATURE_FLASH_LP_VERSION != 0
48             /* Enable reading from data flash. */
49
```

R_GPT_DutyCycleSet() 函数说明

R_GPT_DutyCycleSet 此函数用于设置请求引脚的占空比。占空比在缓冲寄存器中更新，更新后的占空比将在下一个计数溢出时反映出来。

◆ R_GPT_DutyCycleSet()

```
fsp_err_t R_GPT_DutyCycleSet ( timer_ctrl_t *const p_ctrl,  
                               uint32_t const duty_cycle_counts,  
                               uint32_t const pin  
                               )
```

Sets duty cycle on requested pin. Implements `timer_api_t::dutyCycleSet`.

Duty cycle is updated in the buffer register. The updated duty cycle is reflected after the next cycle end (counter overflow).

Example:

```
/* Get the current period setting. */  
timer_info_t info;  
(void) R_GPT_InfoGet(&g_timer0_ctrl, &info);  
uint32_t current_period_counts = info.period_counts;  
  
/* Calculate the desired duty cycle based on the current period. Note that if the period could be larger than  
 * UINT32_MAX / 100, this calculation could overflow. A cast to uint64_t is used to prevent this. The cast is  
 * not required for 16-bit timers. */  
uint32_t duty_cycle_counts =  
    (uint32_t) (((uint64_t) current_period_counts * GPT_EXAMPLE_DESIRED_DUTY_CYCLE_PERCENT) /  
                GPT_EXAMPLE_MAX_PERCENT);  
  
/* Set the calculated duty cycle. */  
err = R_GPT_DutyCycleSet(&g_timer0_ctrl, duty_cycle_counts, GPT_IO_PIN_GTIOCB);  
assert(FSP_SUCCESS == err);
```

Parameters

[in] p_ctrl	Pointer to instance control block.
[in] duty_cycle_counts	Duty cycle to set in counts.
[in] pin	Use gpt_io_pin_t to select GPT_IO_PIN_GTIOCA or GPT_IO_PIN_GTIOCB

Return values

FSP_SUCCESS	Duty cycle updated successfully.
FSP_ERR_ASSERTION	p_ctrl was NULL or the pin is not one of gpt_io_pin_t
FSP_ERR_NOT_OPEN	The instance is not opened.
FSP_ERR_INVALID_ARGUMENT	Duty cycle is larger than period.
FSP_ERR_INVALID_MODE	GPT_IO_PIN_TROUGH, and GPT_IO_PIN_CREST settings are invalid in the this mode.
FSP_ERR_UNSUPPORTED	GPT_CFG_OUTPUT_SUPPORT_ENABLE is 0.

+ 占空比=cycle/period，若设置 GTIOCA-占空比为 25%，所以+ 占空比为 75%，则 cycle= - 占空比* period=25%*9000=2250

```
err = R_GPT_DutyCycleSet(&g_timer3_ctrl, 2250, GPT_IO_PIN_GTIOCA); // 占空比  
assert(FSP_SUCCESS == err);  
  
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功
```



```
1 #include "hal_data.h"
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7
8 /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function
9 */
10 void hal_entry(void)
11 {
12     /* TODO: add your own code here */
13
14     fsp_err_t err = FSP_SUCCESS;
15     /* Initializes the module. */
16     err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);
17     /* Handle any errors. This function should be defined by the user. */
18     assert(FSP_SUCCESS == err);
19
20     /* Start the timer. */
21     (void) R_GPT_Start(&g_timer3_ctrl);
22
23     err = R_GPT_PeriodSet(&g_timer3_ctrl, 9000); // 频率
24     assert(FSP_SUCCESS == err);
25     R_BSP_SoftwareDelay(20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功
26
27     err = R_GPT_DutyCycleSet(&g_timer3_ctrl, 2250, GPT_IO_PIN_GTIOCA); // 占空比
28     assert(FSP_SUCCESS == err);
29     R_BSP_SoftwareDelay(20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功
30
31     #if BSP_TZ_SECURE_BUILD
32     /* Enter non-secure code */
33     R_BSP_NonSecureEnter();
34     #endif
35 }
36
37 /* * This function is called at various points during the startup process. This implementation uses the event that is
38 */
39 void R_BSP_WarmStart(bsp_warm_start_event_t event)
40 {
41 }
```

R_GPT_Reset() 函数说明

R_GPT_Reset 此函数用于将计数器值重置为 0。它实现了 `timer_api_t::reset` 操作。

注意：如果在上次调用 `R_GPT_PeriodSet()` 后未发生计数器溢出，函数将更新为新的周期值。

◆ R_GPT_Reset()

`fsp_err_t R_GPT_Reset ( timer_ctrl_t *const p_ctrl )`

Resets the counter value to 0. Implements `timer_api_t::reset`.

Note
This function also updates to the new period if no counter overflow has occurred since the last call to `R_GPT_PeriodSet()`.

Return values

<code>FSP_SUCCESS</code>	Counter value written successfully.
<code>FSP_ERR_ASSERTION</code>	<code>p_ctrl</code> was NULL.
<code>FSP_ERR_NOT_OPEN</code>	The instance is not opened.

R_GPT_Close() 函数说明

R_GPT_Close 此函数用于停止计数器、禁用输出引脚并清除内部驱动程序数据。它实现了 `timer_api_t::close` 操作。

◆ R_GPT_Close()

fsp_err_t R_GPT_Close (timer_ctrl_t *const p_ctrl)

Stops counter, disables output pins, and clears internal driver data. Implements `timer_api_t::close`.

Return values

FSP_SUCCESS Successful close.
FSP_ERR_ASSERTION p_ctrl was NULL.
FSP_ERR_NOT_OPEN The instance is not opened.

关闭定时器 GPT。

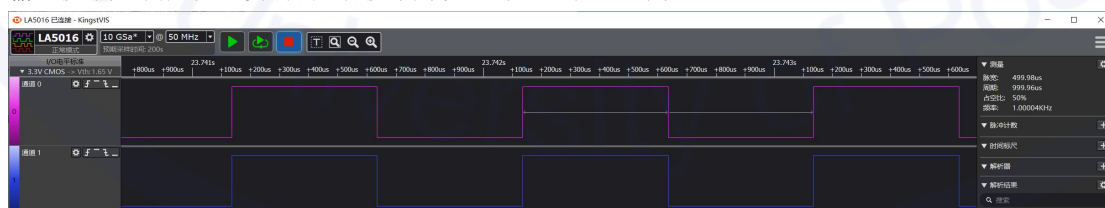
```
err = R_GPT_Close(&g_timer3_ctrl);  
  
assert(FSP_SUCCESS == err);  
  
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);
```



```
1 #include "hal_data.h"  
2  
3 FSP_CPP_HEADER  
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);  
5 FSP_CPP_FOOTER  
6  
7  
8 * * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function  
9 void hal_entry(void)  
10 {  
11     /* TODO: add your own code here */  
12  
13     fsp_err_t err = FSP_SUCCESS;  
14     /* Initializes the module. */  
15     err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);  
16     /* Handle any errors. This function should be defined by the user. */  
17     assert(FSP_SUCCESS == err);  
18  
19     /* Start the timer. */  
20     (void) R_GPT_Start(&g_timer3_ctrl);  
21  
22     err = R_GPT_PeriodSet(&g_timer3_ctrl, 9000); // 频率  
23     assert(FSP_SUCCESS == err);  
24     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功  
25  
26     err = R_GPT_DutyCycleSet(&g_timer3_ctrl, 2250, GPT_IO_PIN_GTIOCA); // 占空比  
27     assert(FSP_SUCCESS == err);  
28     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功  
29  
30     err = R_GPT_Close(&g_timer3_ctrl);  
31     assert(FSP_SUCCESS == err);  
32     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);  
33  
34     #if BSP_TZ_SECURE_BUILD  
35     /* Enter non-secure code */  
36     R_BSP_NonSecureEnter();  
37     #endif  
38  
39     * * This function is called at various points during the startup process. This implementation uses the event that is
```

波形情况

输出波形如下所示，最开始启动波形为设置的 1k，占空比为 50%。



频率设置成功后频率直接修改为 10k，占空比为 50%。



最后 A 通道正占空比成功修改为 75%，B 通道正占空比成功修改为 25%。



呼吸灯示例

LED 亮度随占空比线性升降，呈现柔和的 Breathing 灯效果。

```
void hal_entry(void)
{
    /* TODO: add your own code here */

    fsp_err_t err = FSP_SUCCESS;
    /* Initializes the module. */
    err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);
    /* Handle any errors. This function should be defined by the user. */
    assert(FSP_SUCCESS == err);

    /* Start the timer. */
    (void) R_GPT_Start(&g_timer3_ctrl);

    err = R_GPT_PeriodSet(&g_timer3_ctrl, 9000); //频率
    assert(FSP_SUCCESS == err);
    R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功

    err = R_GPT_DutyCycleSet(&g_timer3_ctrl, 2250, GPT_IO_PIN_GTIOCA); //占空比
    assert(FSP_SUCCESS == err);
}
```

```

R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功

// err = R_GPT_Close(&g_timer3_ctrl);
// assert(FSP_SUCCESS == err);
// R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);

/* -----
 * PWM “呼吸”效果核心循环
 * duty : 0-1000, 对应 0%-100% 占空比 (后面 ×9 映射到 GPT 计数)
 * rising : true 表示占空比递增, false 表示递减
 * -----*/
int duty = 1; /* 当前占空比计数 (1/1000 分辨率) */
bool rising = true; /* true=递增, false=递减 */
while(1)
{
    /* ----- 1. 计算下一步 duty ----- */
    if (rising) /* 处于递增阶段 */
    {
        duty++; /* 占空比 +1 */
        if (duty >= 1000) /* 到达上限时 */
        {
            duty = 1000; /* 饱和 */
            rising = false; /* 方向改为递减 */
        }
    }
    else /* 递减阶段 */
    {
        duty--; /* 占空比 -1 */
        if (duty <= 0) /* 到达下限时 */
        {
            duty = 0; /* 饱和 */
            rising = true; /* 方向改为递增 */
        }
    }

    /* ----- 2. 将新 duty 写入 GPT ----- */
    /* 9 × duty 把 0-1000 映射到 0-9000 的 GPT 计数值,
     * 具体比例由 PWM 频率 / 时钟设置决定。 */
    err = R_GPT_DutyCycleSet(&g_timer3_ctrl,
                             9 * duty,
                             GPT_IO_PIN_GTIOCA);
    assert(FSP_SUCCESS == err); /* 确认写入成功 */
}

```

```

/* ----- 3. 节拍延时 ----- */

/* 1ms 更新一次占空比，控制呼吸速度。

   若运行在 RTOS，可改为 vTaskDelay(1)，          */
   R_BSP_SoftwareDelay(1, BSP_DELAY_UNITS_MILLISECONDS);
}

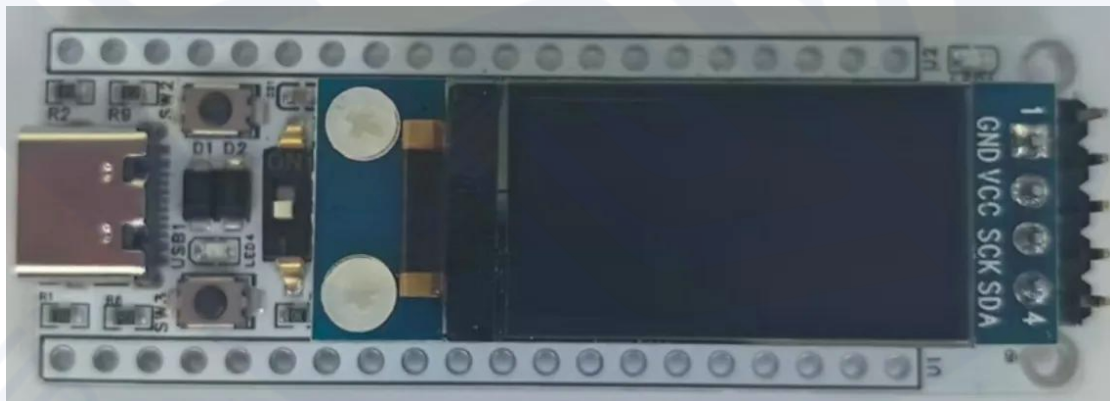
#if BSP_TZ_SECURE_BUILD
    /* Enter non-secure code */
    R_BSP_NonSecureEnter();
#endif
}

```

project9: 定时器 GPT 配置输入捕获

概述

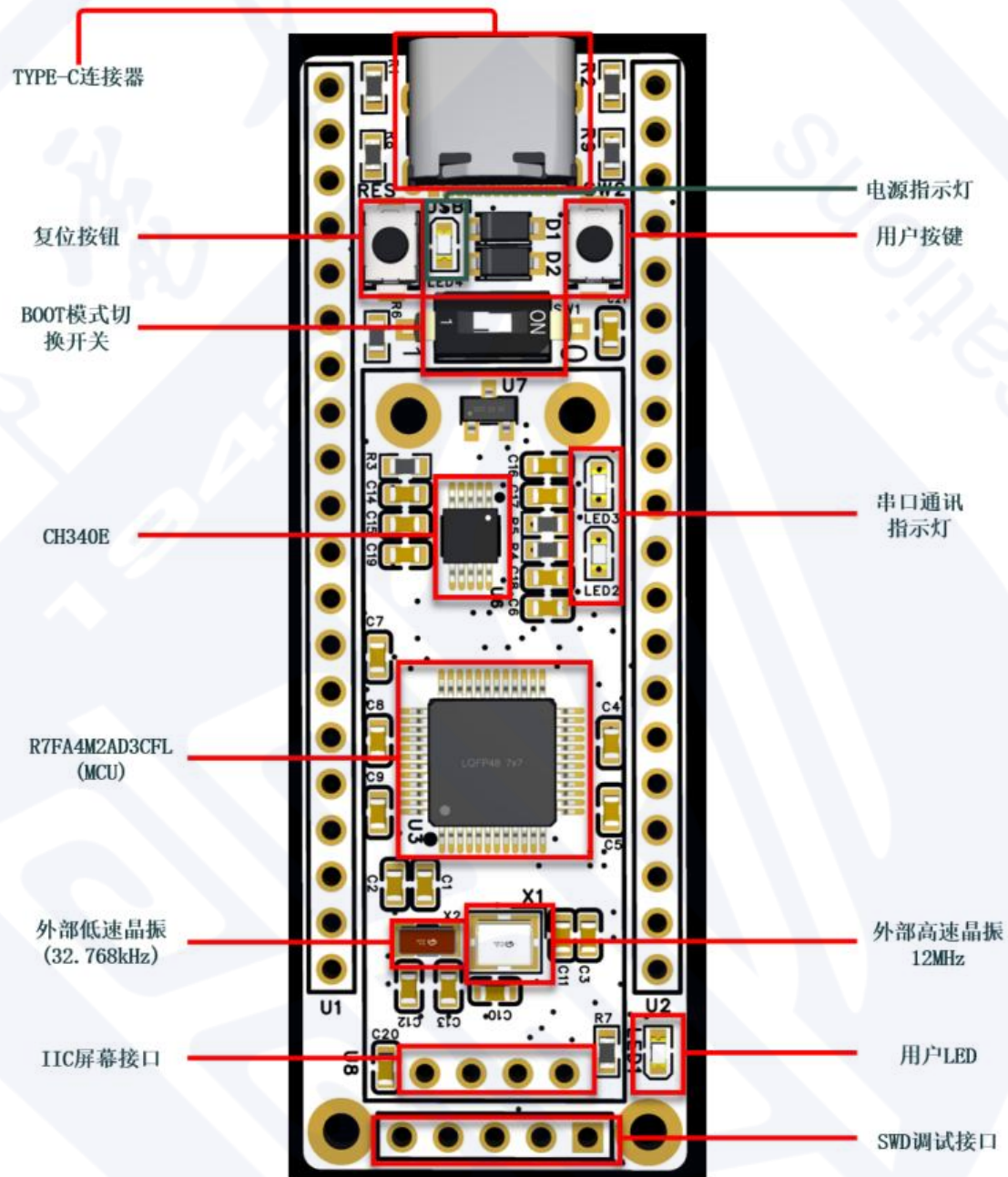
本文将探讨如何在 Renesas RA 系列微控制器上使用 GPT（通用定时器）模块来配置输入捕获功能。输入捕获是定时器的一项重要功能，它允许我们捕获外部信号（如脉冲或波形）的时间戳，广泛应用于频率计数、脉冲宽度测量以及其他需要精确时间记录的应用。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

选择计时器

RA MCU 有两个定时器外设：通用 PWM 定时器 (GPT) 和异步通用定时器 (AGT)。

	GPT	AGT
低功耗模式	GPT 可以在睡眠模式下运行。	AGT 可以在所有低功耗模式下运行。
可用频道	GPT 通道的数量因设备而异。目前所有支持的 MCU 至少都有 7 个 GPT 通道。	所有 MCU 都有 2 个 AGT 通道。
定时器分辨率	所有 MCU 均至少有一个 32 位 GPT 计时器。	AGT 计时器是 16 位计时器。
时钟源	GPT 通过可配置分频器（最高可达 1024）的 PCLKD 运行。它还可以配置为计数 ELC 事件或外部脉冲。	AGT 通过 PCLKB、LOCO 或子时钟运行。

时钟源

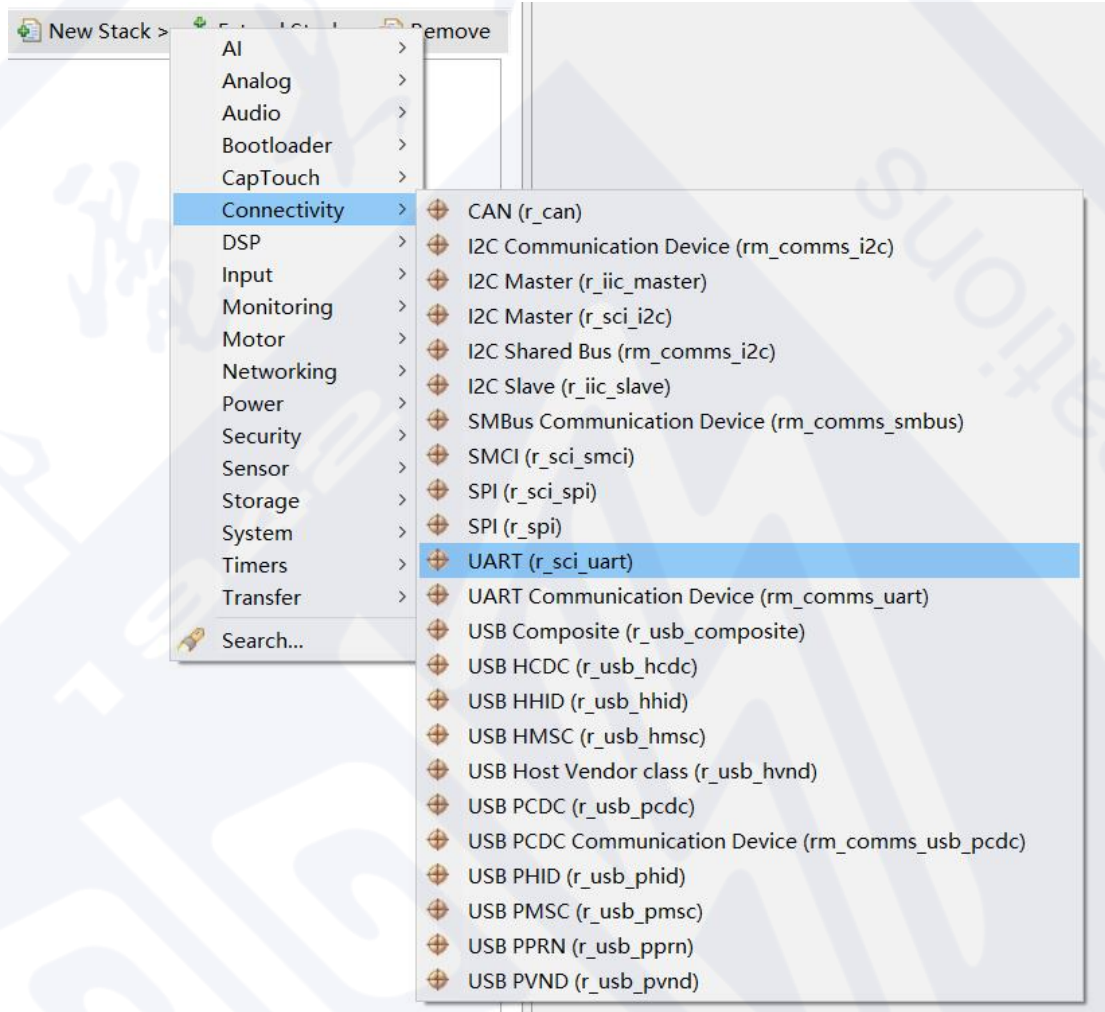
GPT 使用 PCLKD（外设时钟 D）作为主时钟源。该时钟可以通过可配置的分频器进行调整，最大分频因子为 1024。

Selecting a Timer

RA MCUs have two timer peripherals: the General PWM Timer (GPT) and the Asynchronous General Purpose Timer (AGT). When selecting between them, consider these factors:

	GPT	AGT
Low Power Modes	The GPT can operate in sleep mode.	The AGT can operate in all low power modes.
Available Channels	The number of GPT channels is device specific. All currently supported MCUs have at least 7 GPT channels.	All MCUs have 2 AGT channels.
Timer Resolution	All MCUs have at least one 32-bit GPT timer.	The AGT timers are 16-bit timers.
Clock Source	The GPT runs off PCLKD with a configurable divider up to 1024. It can also be configured to count ELC events or external pulses.	The AGT runs off PCLKB, LOCO, or subclock.

选择 XTAL 12MHz 作为时钟源（Clock Src: XTAL）。PCLKD 时钟的分频器设置为 Div /2，意味着 PCLKD 直接运行在 90MHz。



UART 属性配置

属性 ×	问题	Smart Browser	控制台	调试
g_uart9 UART (r_sci_uart)				
Settings	属性		值	
API Info	▼ Common			
	Parameter Checking		Default (BSP)	
	FIFO Support		Disable	
	DTC Support		Disable	
	Flow Control Support		Disable	
	RS-485 Support		Disable	
	IrDA Support		Disabled	
	▼ Module g_uart9 UART (r_sci_uart)			
	▼ General			
	Name		g_uart9	
	Channel		9	
	Data Bits		8bits	
	Parity		None	
	Stop Bits		1bit	
	▼ Baud			
	Baud Rate		115200	
	Baud Rate Modulation		Enabled	
	Max Error (%)		5	
	► Flow Control			
	► Extra			
	▼ Interrupts			
	Callback		user_uart_callback	
	Receive Interrupt Priority		Priority 2	
	Transmit Data Empty Interrupt Priority		Priority 2	
	Transmit End Interrupt Priority		Priority 2	
	Error Interrupt Priority		Priority 2	
	▼ Pins			
	TXD9		P109	
	RXD9		P110	
	CTS9		<unavailable>	

名称
通道
数据位
无校验位
停止位

波特率
启用波特率调制

回调函数名称

串口发送端口
串口接收端口

设置 e2studio 堆栈

printf 函数通常需要设置堆栈大小。这是因为 printf 函数在运行时需要使用栈空间来存储临时变量和函数调用信息。如果堆栈大小不足，可能会导致程序崩溃或不可预期的行为。

printf 函数使用了可变参数列表，它会在调用时使用栈来存储参数，在函数调用结束后再清除参数，这需要足够的栈空间。另外 printf 也会使用一些临时变量，如果栈空间不足，会导致程序崩溃。

因此，为了避免这类问题，应该根据程序的需求来合理设置堆栈大小。

项目资源管理器

demo

- 二进制
- Includes
- ra
- ra_gen
- src
 - hal_entry.c
 - Debug
 - ra_cfg
 - script
 - configuration.xml
 - demo Debug_FlatJlink
 - demo Debug_FlatLaunch
 - ra_cfg.txt
- Developer Assistance
 - HAL/Common
 - g_ioport I/O Port (r_ioport)

configuration.xml

demo Debug_FlatJlink

demo Debug_FlatLaunch

ra_cfg.txt

Developer Assistance

HAL/Common

g_ioport I/O Port (r_ioport)

fsp err t R IOPORT Open(ioport ctrl t *const

属性

问题

Smart Browser

控制台

Board Support Package Configuration

Generate Project Content

Restore Defaults

Device Selection

FSP version: 4.2.0

Board: Custom User Board (Any Device)

Device: R7FA4M2AD3CFP

Core: CM33

RTOS: No RTOS

Board Details

Summary

BSP

Clocks

Pins

Interrupts

Event Links

Stacks

Components

Custom User Board (Any Device)

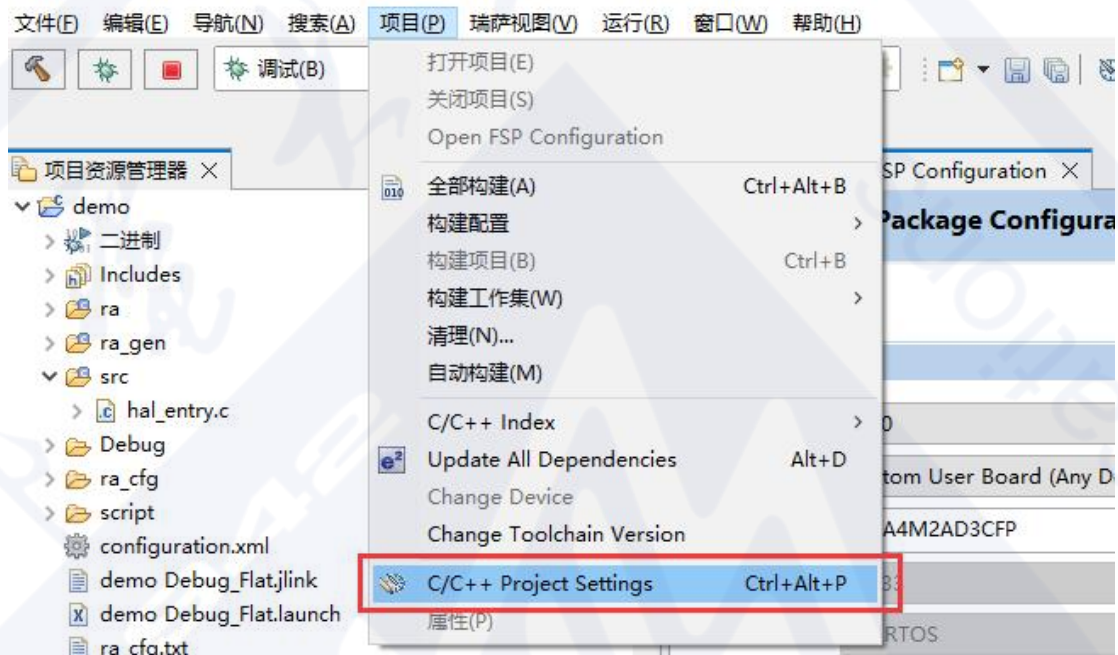
Settings

属性

属性	值
R7FA4M2AD3CFP	
part_number	R7FA4M2AD3CFP
rom_size_bytes	524288
ram_size_bytes	131072
data_flash_size_bytes	8192
package_style	LQFP
package_pins	100
RA4M2	
series	4
RA4M2 Family	
Security	
OF50 register settings	
OF51 register settings	
Block Protection Settings (BPS)	
Permanent Block Protection Settings (PBPS)	
Clocks	
Startup C-Cache Line Size	32 Bytes
Main Oscillator Wait Time	8163 cycles
RA Common	
Main stack size (bytes)	0x400
Heap size (bytes)	0x2000
MCU Vcc (mV)	3300
Parameter checking	Disabled
Assert Failures	Return FSP_ERR_ASSERTION
Error Log	No Error Log
Clock Registers not Reset Values during Startup	Disabled
Main Oscillator Populated	Populated
PFS Protect	Enabled
C Runtime Initialization	Enabled
Early BSP Initialization	Disabled
Main Oscillator Clock Source	Crystal or Resonator
Subclock Populated	Populated
Subclock Drive (Drive capacitance availability varies by MCU)	Standard/Normal mode
Subclock Stabilization Time (ms)	1000

3.设置堆栈大小

e2studio 的重定向 printf 设置



在嵌入式系统的开发中，尤其是在使用 GNU 编译器集合（GCC）时，`--specs` 参数用于指定链接时使用的系统规格（specs）文件。这些规格文件控制了编译器和链接器的行为，尤其是关于系统库和启动代码的链接。`--specs=rdimon.specs` 和 `--specs=nosys.specs` 是两种常见的规格文件，它们用于不同的场景。

`--specs=rdimon.specs`

用途：这个选项用于链接“Redlib”库，这是为裸机（bare-metal）和半主机（semihosting）环境设计的 C 库的一个变体。半主机环境是一种特殊的运行模式，允许嵌入式程序通过宿主机（如开发 PC）的调试器进行输入输出操作。

应用场景：当你需要在没有完整操作系统的环境中运行程序，但同时需要使用调试器来处理输入输出（例如打印到宿主机的终端），这个选项非常有用。

特点：它提供了一些基本的系统调用，通过调试接口与宿主机通信。

`--specs=nosys.specs`

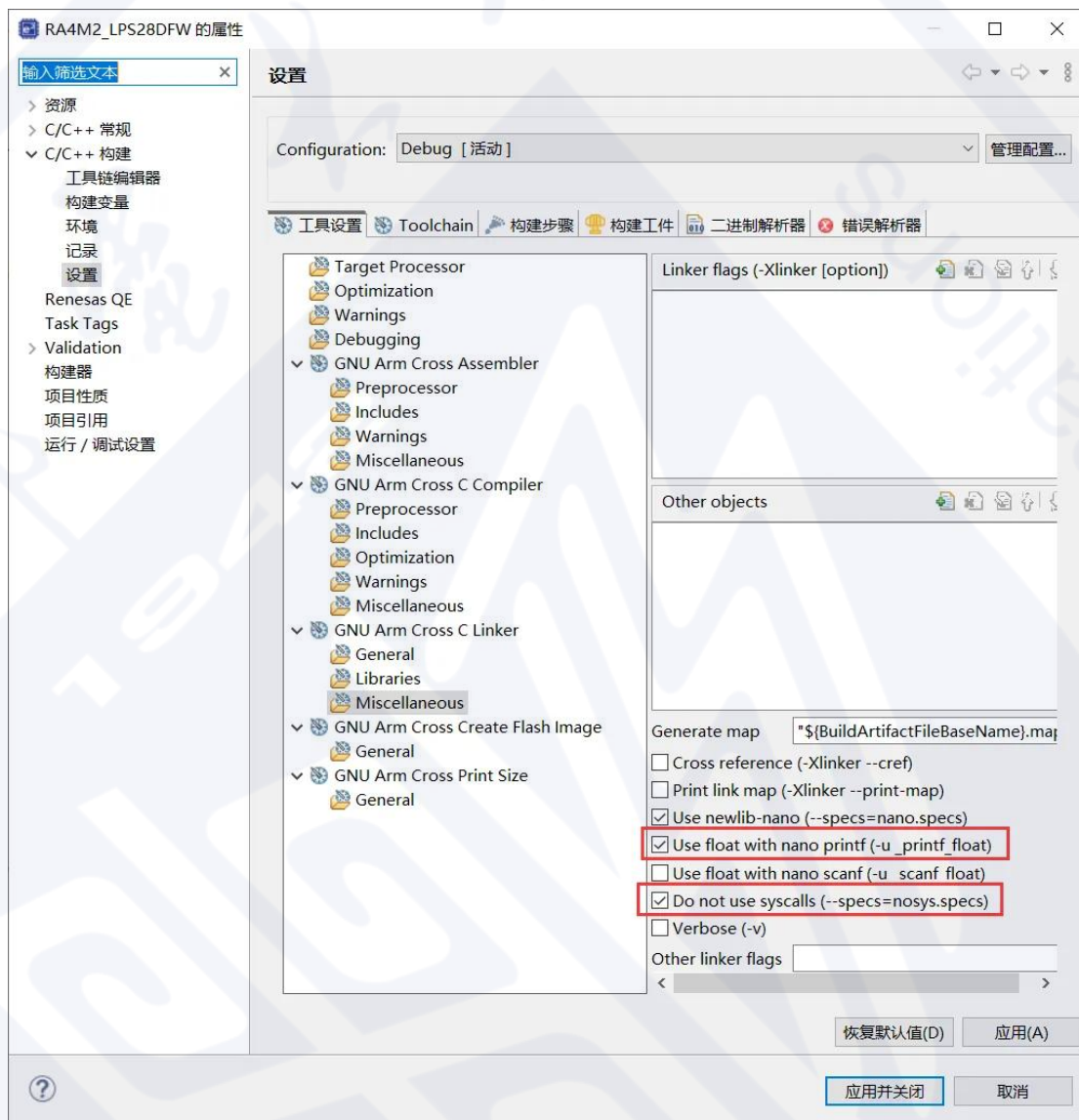
用途：这个选项链接了一个非常基本的系统库，这个库不提供任何系统服务的实现。

应用场景：适用于完全的裸机程序，其中程序不执行任何操作系统调用，比如不进行文件操作或者系统级输入输出。

特点：这是一个更“裸”的环境，没有任何操作系统支持。使用这个规格文件，程序不期望有操作系统层面的任何支持。

如果你的程序需要与宿主机进行交互（如在开发期间的调试），并且通过调试器进行基本的输入输出操作，则使用 `--specs=rdimon.specs`。

如果你的程序是完全独立的，不需要任何形式的操作系统服务，包括不进行任何系统级的输入输出，则使用 `--specs=nosys.specs`。



R_SCI_UART_Open() 函数原型

◆ R_SCI_UART_Open()

```
fsp_err_t R_SCI_UART_Open ( uart_ctrl_t *const p_api_ctrl,  
                           uart_cfg_t const *const p_cfg  
                           )
```

Configures the UART driver based on the input configurations. If reception is enabled at compile time, reception is enabled at the end of this function. Implements `uart_api_t::open`

Return values

FSP_SUCCESS	Channel opened successfully.
FSP_ERR_ASSERTION	Pointer to UART control block or configuration structure is NULL.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The requested channel does not exist on this MCU.
FSP_ERR_INVALID_ARGUMENT	Flow control is enabled but flow control pin is not defined or selected channel does not support "Hardware CTS and Hardware RTS" flow control. (or) restricted channel is selected.
FSP_ERR_ALREADY_OPEN	Control block has already been opened or channel is being used by another instance. Call <code>close()</code> then <code>open()</code> to reconfigure.
FSP_ERR_INVALID_CHANNEL	IrDA is requested for a channel that does not support IrDA.

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- `transfer_api_t::open`

故可以用 `R_SCI_UART_Open()`函数进行配置，开启和初始化 UART。

```
/* Open the transfer instance with initial configuration. */  
err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);  
assert(FSP_SUCCESS == err);
```

```

1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7
8
9
10
11
12  /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function
13  *
14  */
15  void hal_entry(void)
16  {
17      /* TODO: add your own code here */
18
19      fsp_err_t err = FSP_SUCCESS;
20      /* Initializes the module. */
21      err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);
22      /* Handle any errors. This function should be defined by the user. */
23      assert(FSP_SUCCESS == err);
24
25      /* Start the timer. */
26      (void) R_GPT_Start(&g_timer3_ctrl);
27
28      err = R_GPT_PeriodSet(&g_timer3_ctrl, 9000); // 频率
29      assert(FSP_SUCCESS == err);
30      R_BSP_SoftwareDelay(20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功
31
32      err = R_GPT_DutyCycleSet(&g_timer3_ctrl, 2250, GPT_IO_PIN_GTI0CA); // 占空比
33      assert(FSP_SUCCESS == err);
34      R_BSP_SoftwareDelay(20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功
35
36
37
38      /* Open the transfer instance with initial configuration. */
39      err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
40      assert(FSP_SUCCESS == err);
41
42
43
44
45
46
47  #if BSP_TZ_SECURE_BUILD
48      /* Enter non-secure code */
49      R_BSP_NonSecureEnter();
50  #endif
51  }
52
53  /* * This function is called at various points during the startup process. This implementation uses the event that is
54  *
55  */
56  void R_BSP_WarmStart(bsp_warm_start_event_t event)
57  {
58      if (BSP_WARM_START_RESET == event)
59      {
60          if (BSP_FEATURE_FLASH_LP_VERSION != 0

```

回调函数 user_uart_callback ()

当数据发送的时候，可以查看 UART_EVENT_TX_COMPLETE 来判断是否发送完毕。

◆ write

```

fsp_err_t(*uart_api_t::write)(uart_ctrl_t *const p_ctrl, uint8_t const *const p_src, uint32_t const bytes)

```

Write to UART device. The write buffer is used until write is complete. Do not overwrite write buffer contents until the write is finished. When the write is complete (all bytes are fully transmitted on the wire), the callback called with event **UART_EVENT_TX_COMPLETE**. The maximum transfer size is reported by [infoGet\(\)](#).

Implemented as

- R_SCI_UART_Write()
- R_SCI_B_UART_Write()

Parameters

[in] p_ctrl Pointer to the UART control block.

[in] p_src Source address to write data to.

[in] bytes Write data length.

Enumeration Type Documentation

◆ uart_event_t

enum `uart_event_t`

UART Event codes

Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

可以检查检查 "p_args" 结构体中的 "event" 字段的值是否等于 "UART_EVENT_TX_COMPLETE"。如果条件为真，那么 if 语句后面的代码块将会执行。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
}
```



```
hal_entry.c x [RA4M2_MINI_Project8_1] FSP Configuration
1 #include "hal_data.h"
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7
8 fsp_err_t err = FSP_SUCCESS;
9 volatile bool uart_send_complete_flag = false;
10 void user_uart_callback (uart_callback_args_t * p_args)
11 {
12     if(p_args->event == UART_EVENT_TX_COMPLETE)
13     {
14         uart_send_complete_flag = true;
15     }
16 }
17
18
19
20
21 * * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function
22 * *
23 void hal_entry(void)
24 {
25     /* TODO: add your own code here */
26
27     err = FSP_SUCCESS;
28     /* Initializes the module. */
29     err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);
30     /* Handle any errors. This function should be defined by the user. */
31     assert(FSP_SUCCESS == err);
32
33     /* Start the timer. */
34     (void) R_GPT_Start(&g_timer3_ctrl);
35
36     err = R_GPT_PeriodSet(&g_timer3_ctrl, 9000); //频率
37     assert(FSP_SUCCESS == err);
38     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功
39
40
41     err = R_GPT_DutyCycleSet(&g_timer3_ctrl, 2250, GPT_IO_PIN_GTIOCA); //占空比
42     assert(FSP_SUCCESS == err);
43     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功
44
45
46     /* Open the transfer instance with initial configuration. */
47     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
48     assert(FSP_SUCCESS == err);
49
50
51
52
53
54
55
56 #if BSP_TZ_SECURE_BUILD
57     /* Enter non-secure code */
58     R_BSP_NonSecureEnter();
59 #endif
60 }
```

printf 输出重定向到串口

打印最常用的方法是 printf，所以要解决的问题是将 printf 的输出重定向到串口，然后通过串口将数据发送出去。

注意一定要加上头文件#include <stdio.h>

```
#ifndef __GNUC__ //串口重定向
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
#endif

PUTCHAR_PROTOTYPE
{
    err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
}
```

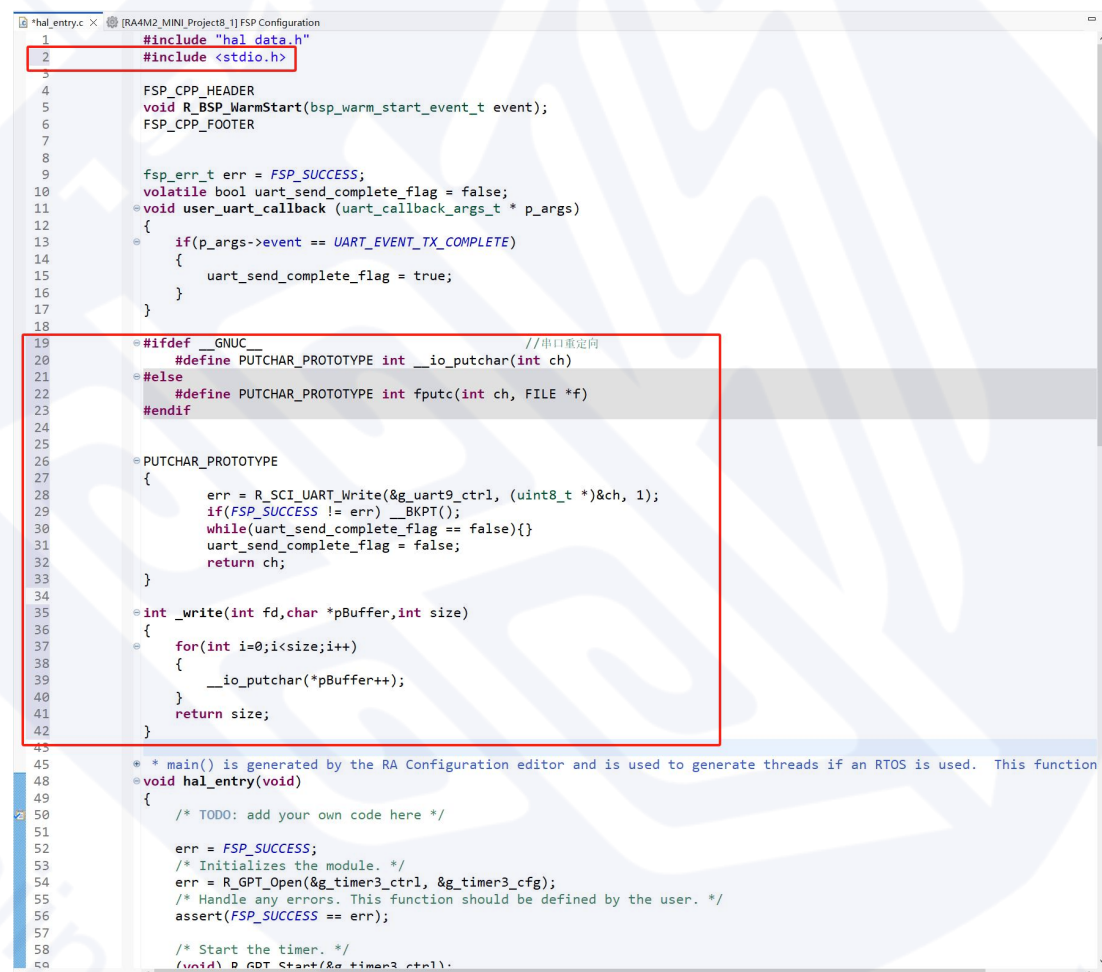
```

        return ch;
    }

    int _write(int fd, char *pBuffer, int size)
    {
        for(int i=0; i<size; i++)
        {
            __io_putchar(*pBuffer++);
        }

        return size;
    }
}

```



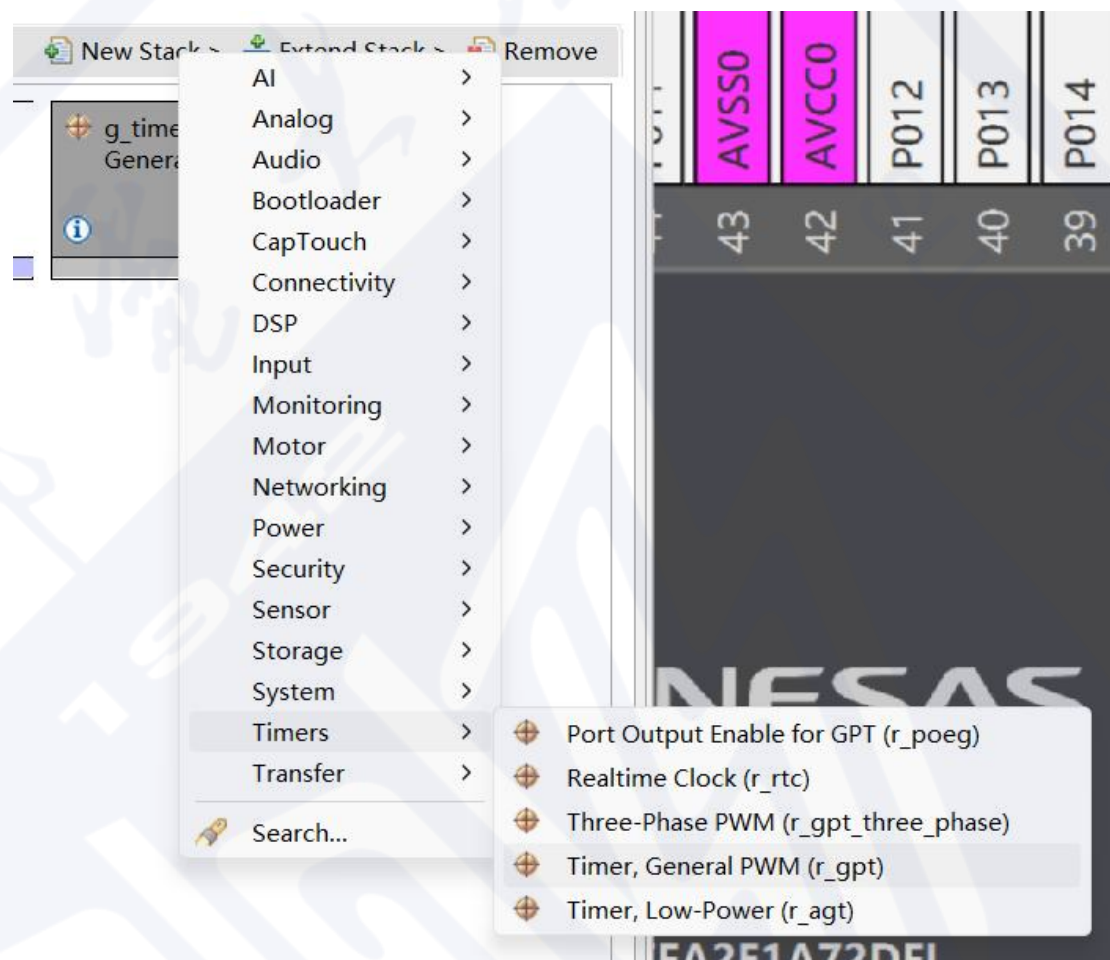
```

1  #include "hal_data.h"
2  #include <stdio.h>
3
4  FSP_CPP_HEADER
5  void R_BSP_WarmStart(bsp_warm_start_event_t event);
6  FSP_CPP_FOOTER
7
8
9  fsp_err_t err = FSP_SUCCESS;
10 volatile bool uart_send_complete_flag = false;
11 void user_uart_callback (uart_callback_args_t * p_args)
12 {
13     if(p_args->event == UART_EVENT_TX_COMPLETE)
14     {
15         uart_send_complete_flag = true;
16     }
17 }
18
19 #ifdef __GNUC__ //串口重定向
20 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
21 #else
22 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
23 #endif
24
25
26 PUTCHAR_PROTOTYPE
27 {
28     err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
29     if(FSP_SUCCESS != err) __BKPT();
30     while(uart_send_complete_flag == false){}
31     uart_send_complete_flag = false;
32     return ch;
33 }
34
35 int _write(int fd, char *pBuffer, int size)
36 {
37     for(int i=0; i<size; i++)
38     {
39         __io_putchar(*pBuffer++);
40     }
41     return size;
42 }
43
44
45 /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function
46 void hal_entry(void)
47 {
48     /* TODO: add your own code here */
49
50     err = FSP_SUCCESS;
51     /* Initializes the module. */
52     err = R_GPT_Open(&g_timer3_ctrl, &g_timer3_cfg);
53     /* Handle any errors. This function should be defined by the user. */
54     assert(FSP_SUCCESS == err);
55
56     /* Start the timer. */
57     (void) R_GPT_Start(&g_timer3_ctrl);
58 }
59

```

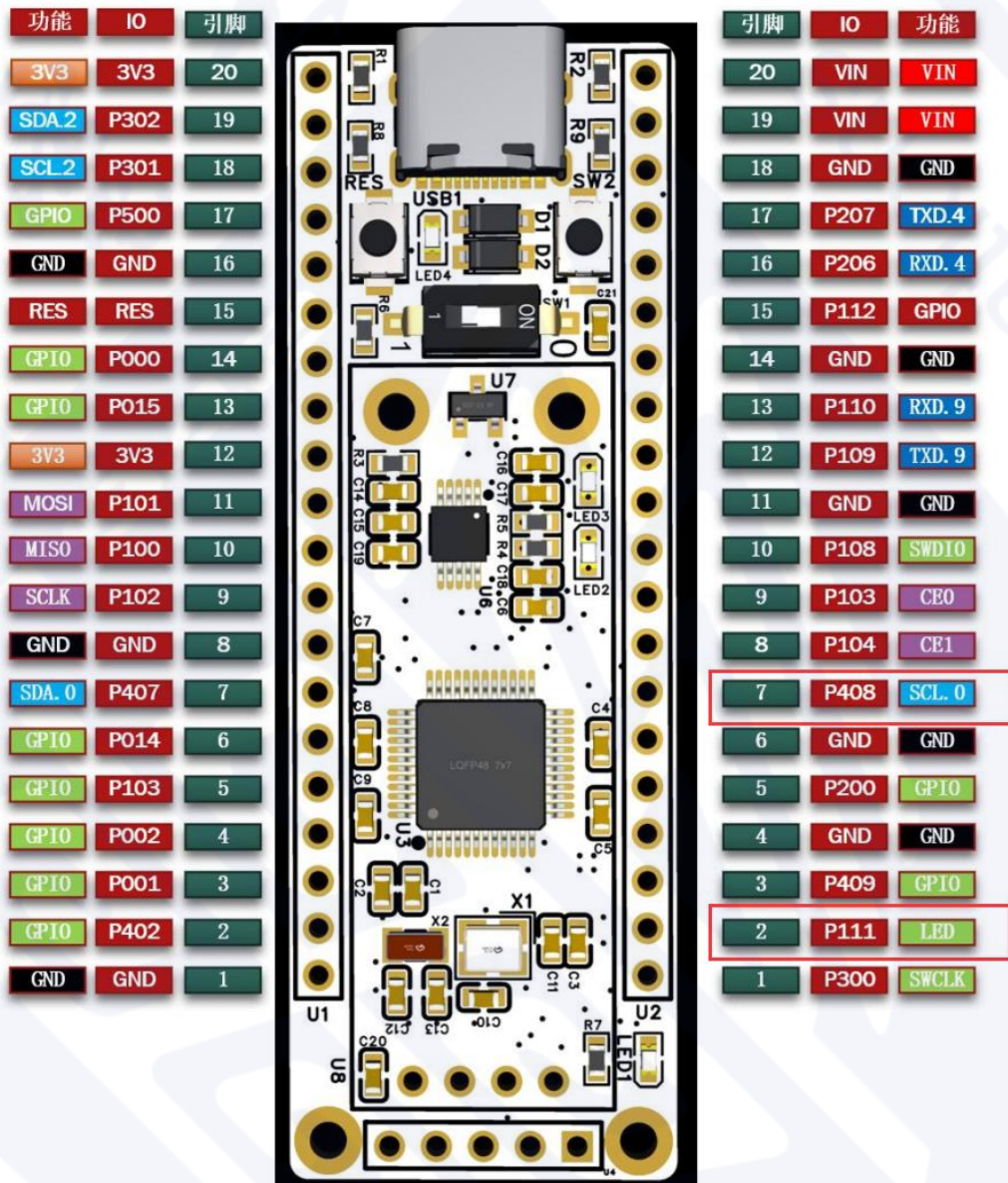
定时器输入捕获配置

操作 “New Stack > Timers > Timer, General PWM (r_gpt)” 在项目中添加 GPT 定时器。



这里设置 P408 进行捕获 PWM。

5. IO 预定义



- **Pin Output Support:** 设为 Enabled, 允许 PWM 信号输出到外部引脚。
- **Name:** 设置为 g_timer6, 这是模块的唯一标识符, 用于代码中引用此定时器实例。
- **Channel:** 设定为 6, 表示该定时器使用通道 6 进行计数和控制。
- **Mode:** 设为 Periodic, 表示该定时器工作在周期模式, 将在设定周期内重复运行。
- **Period:** 周期值设置为 0x10000, 表示定时器的周期为 65536 个时钟周期。根据时钟源频率的不同, 实际的周期时间可以转换为相应的实际时间。
- **Period Unit:** 单位选择为 Raw Counts, 表示周期单位为原始计数, 即计数器溢出的周期。
- **Capture B Source:** 设为 GPT6 CAPTURE COMPARE B, 表示捕获源为 GPT6 定时器的比较匹配 B 事件, 用于捕获输入信号的时间戳。
- **GTIOCB Rising Edge While GTIOCA Low:** 表示当 GTIOCA 为低电平时, 捕获 GTIOCB 引脚的上升沿事件。

- GTIOCB Rising Edge While GTIOCA High: 表示当 GTIOCA 为高电平时，捕获 GTIOCB 引脚的上升沿事件。
- GTIOCB Falling Edge While GTIOCA Low: 表示当 GTIOCA 为低电平时，捕获 GTIOCB 引脚的下降沿事件。
- GTIOCB Falling Edge While GTIOCA High: 表示当 GTIOCA 为高电平时，捕获 GTIOCB 引脚的下降沿事件。

g_timer6 Timer, General PWM (r_gpt)

Settings	属性	值
API Info	Common	
	Parameter Checking	Default (BSP)
	Pin Output Support	Enabled
	Write Protect Enable	Disabled
	Module g_timer6 Timer, General PWM (r_gpt)	
	General	
	Compare Match	
	Name	g_timer6
	Channel	6
	Mode	Periodic
	Period	0x10000
	Period Unit	Raw Counts
	Output	
	Input	
	Count Up Source	
	Count Down Source	
	Start Source	
	Stop Source	
	Clear Source	
	Capture A Source	
	Capture B Source	
	GTETRGARising Edge	☐
	GTETRGAFalling Edge	☐
	GTETRGBRising Edge	☐
	GTETRGBFalling Edge	☐
	GTETRGCRising Edge	☐
	GTETRGCFalling Edge	☐
	GTETRGDRising Edge	☐
	GTETRGDFalling Edge	☐
	GTIOCARising Edge While GTIOCB Low	☐
	GTIOCARising Edge While GTIOCB High	☐
	GTIOCAFalling Edge While GTIOCB Low	☐
	GTIOCAFalling Edge While GTIOCB High	☐
	GTIOCB Rising Edge While GTIOCA Low	☒
	GTIOCB Rising Edge While GTIOCA High	☒
	GTIOCB Falling Edge While GTIOCA Low	☒
	GTIOCB Falling Edge While GTIOCA High	☒
	GPT3 CAPTURE COMPARE A (Capture/Compare match A)	☐

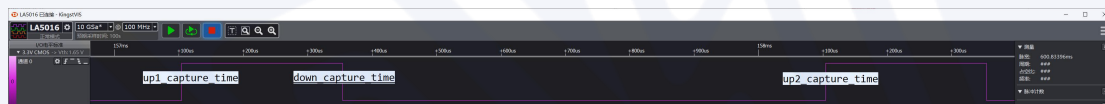
- Callback: 设置为 gpt6_callback，指定了当定时器触发中断时，调用的回调函数。在此情况下，回调函数 gpt6_callback 将在捕获事件发生时执行。
- Capture/Compare match B Interrupt Priority: 设置为 Priority 2，表示启用比较匹配 B 中断，且其优先级为 2。
- GTIOC5B: 指定 GTIOC5B 引脚为另一个定时器输入/输出引脚，连接到外部引脚 P408。

▼ Interrupts		
Callback	回调函数	gpt6_callback
Overflow/Crest Interrupt Priority		Disabled
Capture/Compare match A Interrupt Priority		Disabled
Capture/Compare match B Interrupt Priority	启用比较匹配B中断	Priority 2
Underflow/Trough Interrupt Priority		Disabled
> Extra Features		
▼ Pins		
GTIOC6A		None
GTIOC6B	对应引脚	P408

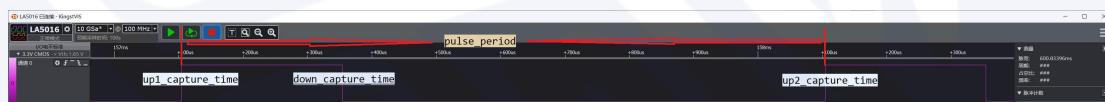
占空比与频率计算

在波形中：

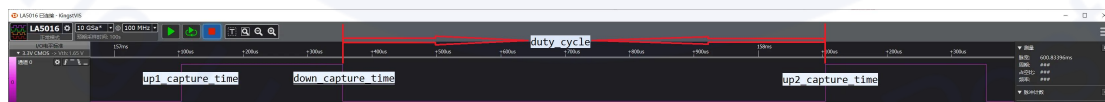
- up1_capture_time 是第一次上升沿的捕获时间。
- down_capture_time 是下降沿的捕获时间。
- up2_capture_time 是第二次上升沿的捕获时间。



脉冲周期（pulse_period）完整周期为上升沿（up1_capture_time）到第二次上升沿（up2_capture_time）之间的时间差，也就是脉冲的完整周期。



- 频率计算：我在频率计算中使用了 pulse_period，确保了计算是在第一次和第二次上升沿之间。
- 占空比计算：计算占空比时，使用了从 down_capture_time 到第二次上升沿 up2_capture_time 之间的时间差来计算脉冲宽度。



回调函数 gpt6_callback

1. 上升沿捕获:

- 在上升沿捕获时, 更新第一次和第二次上升沿的捕获时间。
- 计算 脉冲周期: 即第二次上升沿与第一次上升沿之间的时间差。
- 计算 频率: 通过时钟频率除以脉冲周期。
- 计算 占空比: 通过计算从上升沿到下降沿的脉冲宽度, 然后用脉冲宽度除以脉冲周期来计算占空比。

1. 下降沿捕获:

- 在下降沿捕获时, 更新下降沿的捕获时间。

需要注意的是, 代码中计算占空比时, 用 $(100.0f - (\text{pulse_width} * 100 / (\text{double})\text{pulse_period}))$ 来计算正占空比。

```
volatile uint32_t up1_capture_time = 0;    // 用于存储第一次上升沿捕获的时间
volatile uint32_t down_capture_time = 0;    // 用于存储第一次上升沿捕获的时间
volatile uint32_t up2_capture_time = 0;    // 用于存储第二次上升沿捕获的时间
volatile uint32_t pulse_width = 0;         // 用于存储脉冲宽度
volatile uint32_t pulse_period = 0;        // 用于存储脉冲周期
volatile double duty_cycle = 0;            // 用于存储正占空比
volatile double frequency = 0;            // 用于存储频率

void gpt6_callback(timer_callback_args_t *p_args)
{
    /* TODO: add your own code here */

    if ((TIMER_EVENT_CAPTURE_B == p_args->event)) // 捕获事件
    {
        bsp_io_level_t p_port_value_port_111;

        // 读取端口电平状态, 如果是低电平则发生的是下降沿, 高电平则是上升沿
        R_IOPORT_PinRead(&g_ioport_ctrl, BSP_IO_PORT_01_PIN_11, &p_port_value_port_111);

        // 获取当前定时器的时钟频率和周期
        timer_info_t info;
        (void) R_GPT_InfoGet(&g_timer6_ctrl, &info);
        uint64_t clock_frequency = info.clock_frequency; // 定时器时钟频率
        uint32_t current_period_counts = info.period_counts; // 定时器周期

        uint32_t current_time = p_args->capture; // 获取当前捕获时间 (计数值)

        if (p_port_value_port_111 == BSP_IO_LEVEL_HIGH) // 上升沿
        {
            // 记录第一次和第二次上升沿的时间戳
            up1_capture_time=up2_capture_time;
            up2_capture_time=current_time;
        }
    }
}
```

```

// 计算脉冲周期：从第一次到第二次上升沿的时间差
if (up2_capture_time >= up1_capture_time)
    pulse_period = (up2_capture_time - up1_capture_time);
else
    pulse_period = (current_period_counts - up1_capture_time) + up2_capture_time;
// 计算频率：频率 = 时钟频率 / 脉冲周期
frequency = (double) (clock_frequency / pulse_period);

// 计算脉冲宽度：从下降沿到第二次上升沿的时间差
if (up2_capture_time >= down_capture_time)
    pulse_width = up2_capture_time - down_capture_time;
else
    pulse_width = (current_period_counts - down_capture_time) + up2_capture_time;
// 计算占空比：占空比 = 脉冲宽度 / 脉冲周期
duty_cycle = 100.0f - (pulse_width * 100 / (double) pulse_period);

}
else // 下降沿
{
    // 更新下降沿的捕获时间
    down_capture_time = current_time;
}
}
}

```

```
43
44
45 volatile uint32_t up1_capture_time = 0; // 用于存储第一次上升沿捕获的时间
46 volatile uint32_t down_capture_time = 0; // 用于存储第一次下降沿捕获的时间
47 volatile uint32_t up2_capture_time = 0; // 用于存储第二次上升沿捕获的时间
48 volatile uint32_t pulse_width = 0; // 用于存储脉冲宽度
49 volatile uint32_t pulse_period = 0; // 用于存储脉冲周期
50 volatile double duty_cycle = 0; // 用于存储占空比
51 volatile double frequency = 0; // 用于存储频率
52
53 void gpt6_callback(timer_callback_args_t *p_args)
54 {
55     /* TODO: add your own code here */
56     if ((TIMER_EVENT_CAPTURE_B == p_args->event)) // 捕获事件
57     {
58         bsp_io_level_t p_port_value_port_102;
59         // 读取端口电平状态, 如果是低电平则发生的是下降沿, 高电平则是上升沿
60         R_IOPORT_PinRead(&g_ioport_ctrl1, BSP_IO_PORT_01_PIN_11, &p_port_value_port_102);
61
62         // 获取当前定时器的时钟频率和周期
63         timer_info_t info;
64         (void) R_GPT_InfoGet(&g_timer6_ctrl1, &info);
65         uint64_t clock_frequency = info.clock_frequency; // 定时器时钟频率
66         uint32_t current_period_counts = info.period_counts; // 定时器周期
67
68         uint32_t current_time = p_args->capture; // 获取当前捕获时间(计数值)
69
70         if (p_port_value_port_102 == BSP_IO_LEVEL_HIGH) // 上升沿
71         {
72             // 记录第一次和第二次上升沿的时间戳
73             up1_capture_time = up2_capture_time;
74             up2_capture_time = current_time;
75
76             // 计算脉冲周期: 从第一次到第二次上升沿的时间差
77             if (up2_capture_time > up1_capture_time)
78                 pulse_period = (up2_capture_time - up1_capture_time);
79             else
80                 pulse_period = (current_period_counts - up1_capture_time) + up2_capture_time;
81             // 计算频率: 频率 = 时钟频率 / 脉冲周期
82             frequency = (double) (clock_frequency / pulse_period);
83
84             // 计算脉冲宽度: 从下降沿到第二次上升沿的时间差
85             if (up2_capture_time > down_capture_time)
86                 pulse_width = up2_capture_time - down_capture_time;
87             else
88                 pulse_width = (current_period_counts - down_capture_time) + up2_capture_time;
89             // 计算占空比: 占空比 = 脉冲宽度 / 脉冲周期
90             duty_cycle = 100.0f * (pulse_width * 100) / ((double) pulse_period);
91
92         }
93         else // 下降沿
94         {
95             // 更新下降沿的捕获时间
96             down_capture_time = current_time;
97         }
98     }
99 }
100
101
102
103 /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function */
104 void hal_entry(void)
105 {
106     /* TODO: add your own code here */
107
108     fsp_err_t err = FSP_SUCCESS;
109     /* Initializes the module. */
110     err = R_GPT_Open(&g_timer3_ctrl1, &g_timer3_cfg);
111     /* Handle any errors. This function should be defined by the user. */
112     assert(FSP_SUCCESS == err);
113
114     /* Start the timer. */
115     (void) R_GPT_Start(&g_timer3_ctrl1);
116
117     err = R_GPT_PeriodSet(&g_timer3_ctrl1, 90000); // 频率
118     assert(FSP_SUCCESS == err);
119     R_BSP_SoftwareDelay(20, BSP_DELAY_UNITS_MILLISECONDS); // 不知道延时可能会设置不成功
120 }
121
122
123
```

主程序

```
void hal_entry(void)
{
    /* TODO: add your own code here */

    fsp_err_t err = FSP_SUCCESS;

    /* Initializes the module. */

    err = R_GPT_Open(&g_timer5_ctrl1, &g_timer5_cfg);

    /* Handle any errors. This function should be defined by the user. */
    assert(FSP_SUCCESS == err);

    /* Start the timer. */

    (void) R_GPT_Start(&g_timer5_ctrl1);
```



```

R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);

err = R_GPT_PeriodSet(&g_timer5_ctrl, 6000);//频率
assert(FSP_SUCCESS == err);
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);//不加延时可能会设置不成功

err = R_GPT_DutyCycleSet(&g_timer5_ctrl, 1500, GPT_IO_PIN_GTIOCA);//占空比
assert(FSP_SUCCESS == err);
err = R_GPT_DutyCycleSet(&g_timer5_ctrl, 4500, GPT_IO_PIN_GTIOCB);//占空比
assert(FSP_SUCCESS == err);
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);

// err = R_GPT_Close(&g_timer5_ctrl);
// assert(FSP_SUCCESS == err);
// R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);

/* Open the transfer instance with initial configuration. */
err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
assert(FSP_SUCCESS == err);

printf("hello\n");

/* Initializes the module. */
err = R_GPT_Open(&g_timer6_ctrl, &g_timer6_cfg);
/* Handle any errors. This function should be defined by the user. */
assert(FSP_SUCCESS == err);
/* Start the timer. */
(void) R_GPT_Start(&g_timer6_ctrl);

(void) R_GPT_Enable(&g_timer6_ctrl);
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);

while(1)
{
    printf("frequency=%.2f,duty cycle=%.2f\n",frequency,duty_cycle);
    duty_cycle = 0; // 重置占空比
    frequency = 0; // 重置频率
    R_BSP_SoftwareDelay (200, BSP_DELAY_UNITS_MILLISECONDS);
}

```

```
#if BSP_TZ_SECURE_BUILD
    /* Enter non-secure code */
    R_BSP_NonSecureEnter();
#endif
}
```

演示

接入 P111。

```
[20:47:14.837] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:15.067] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:15.297] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:15.526] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:15.755] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:15.985] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:16.214] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:16.443] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:16.672] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:16.901] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:17.131] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:17.361] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:17.589] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:17.819] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:18.048] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:18.277] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:18.508] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:18.737] 收←◆frequency=10000.00, duty cycle=75.00
[20:47:18.966] 收←◆frequency=10000.00, duty cycle=75.00
```
